TwoPhaseAMR.jl

Technical reference

The implemented equations, in the notation of the thesis

Based on Remmerswaal [19]

Contents

1 Method overview **1**
- 1.1 Scope of this reference . . . 1
- 1.2 Governing equations . . . 2
- 1.3 The discretisation at a glance . . . 4
- 1.4 One time step . . . 6
- 1.5 Methods and options . . . 10
- 1.6 Known limitations . . . 11

2 Block-forest AMR mesh **15**
- 2.1 Domain . . . 15
- 2.2 Forest, blocks and levels . . . 15
- 2.3 Faces, hanging segments and stored unknowns . . . 17
- 2.4 Halo slots . . . 18
- 2.5 Regrid and schedule rebuild . . . 20

3 Discrete setting on the composite mesh **23**
- 3.1 Continuous setting . . . 23
- 3.2 Measures and staggered control volumes . . . 23
- 3.3 Open (cut-cell) measures . . . 25
- 3.4 Function spaces and discrete integration . . . 25
- 3.5 Divergence . . . 26
- 3.6 Gradient . . . 27
- 3.7 Weighted gradient and elliptic operator . . . 29
- 3.8 Time discretisation and rebuild . . . 30

4 Interface representation, reconstruction and curvature **31**
- 4.1 Continuous description . . . 31
- 4.2 Phase fractions . . . 32
- 4.3 PLIC reconstruction . . . 33
- 4.4 Height functions . . . 35
- 4.5 Normals and choice of the plane . . . 36
- 4.6 Curvature . . . 38
- 4.7 Contact angles . . . 40

4.8 Phase apertures and interface height . . . 43
4.9 Initialisation . . . 44
4.10 Time discretisation and update schedule . . . 45

5 Volume-of-fluid and phase-scalar transport 47
5.1 Continuous model . . . 47
5.2 Discrete balance and phase fluxes . . . 48
5.3 Directional sweeps . . . 48
5.4 Donating regions and phase volumes . . . 50
5.5 Bounds, clamping and snapping . . . 53
5.6 Time-step restriction . . . 54
5.7 Phase scalars . . . 55
5.8 Two-velocity carriers . . . 57
5.9 Time-integrated balances . . . 60

6 Momentum and staggered transport 63
6.1 Continuous model . . . 63
6.2 Dual control volumes and primal–dual fluxes . . . 64
6.3 Face states . . . 68
6.4 Single-phase convection . . . 70
6.5 Conservation and kinetic energy . . . 72
6.6 Time-step restriction . . . 74
6.7 Two phases, one velocity . . . 75
6.8 Two velocities and staggered phase scalars . . . 80
6.9 Which flux on which dual face . . . 81
6.10 Cell-centred scalar transport . . . 84

7 Embedded solids and cut cells 87
7.1 Continuous setting . . . 87
7.2 Solid reconstruction and open polygons . . . 87
7.3 Face traces and shared apertures . . . 88
7.4 Open staggered volumes . . . 89
7.5 Velocity closure and operators on cut cells . . . 90
7.6 Regrid and rebuild . . . 91

8 Forces and material properties 93
8.1 Continuous model . . . 93
8.2 Material properties on the mesh . . . 94
8.3 Semi-discrete forces: the force map . . . 95
8.4 Face curvature . . . 96
8.5 Surface tension . . . 98
8.6 Gravity . . . 100
8.7 Prescribed body acceleration . . . 102
8.8 Two velocities . . . 102
8.9 Fully discrete update and time-step limits . . . 104

9 Viscous stress and implicit solve **107**
9.1 Continuous model . . . 107
9.2 Single-phase operator . . . 108
9.3 Two-phase stress operator . . . 114
9.4 Fully discrete update: implicit midpoint solves . . . 118

10 Pressure and velocity constraints **121**
10.1 Continuous model . . . 121
10.2 One velocity: the Poisson projection . . . 122
10.3 Two velocities: constraints and phase gradients . . . 125
10.4 CMDGFM projection . . . 126
10.5 Fully discrete projection stage . . . 131

11 Operators and linear solvers **139**
11.1 Cell elliptic systems . . . 139
11.2 Krylov driver and stopping rule . . . 142
11.3 Tree geometric multigrid . . . 145
11.4 Shifted viscous systems . . . 150
11.5 Projection systems . . . 152
11.6 Measured convergence . . . 153

12 Refinement criteria and conservative transfer **157**
12.1 Labels, limits and their combination . . . 157
12.2 Refinement indicators . . . 158
12.3 The regrid . . . 162
12.4 Transfer of cell fields . . . 164
12.5 Transfer of fractions, phase scalars and the solid . . . 166
12.6 Transfer of face fields . . . 168
12.7 Constraints after a regrid . . . 170
12.8 Conservation summary and measurements . . . 171

13 Stability and accuracy **173**
13.1 Time-step bounds . . . 173
13.2 Step selection and the step check . . . 176
13.3 Measured bounds and stability margins . . . 178
13.4 Known gaps between the bounds and stability . . . 179
13.5 Accuracy . . . 180

14 Verification **185**
14.1 Diagnostics . . . 185
14.2 Test suite and continuous integration . . . 187
14.3 Regression oracle . . . 189
14.4 Derivation audits . . . 190
14.5 Manufactured solutions and observed orders . . . 191
14.6 Conservation and invariants . . . 195
14.7 Benchmark flows . . . 197

14.8 The examples gallery 199
14.9 Known gaps 200

A Deviations from the thesis **203**

B Notation **205**

C API index **221**

Bibliography **223**

Reading this reference

Chapter 1 gives the governing equations, the discretisation at a glance, the stages of one time step, the selectable methods with a map of the public types to the chapters, and the known limitations. Each of the following chapters states what the code computes: the continuous model, its spatial (semi-discrete) balance and the fully discrete update, or, for static objects such as the mesh and its operators, the algebraic map. Derivations and proofs are not repeated: they are cited, first from the thesis [19] and then from the original literature. The code is normative: where it differs from the thesis the difference is stated in a box titled *Deviation from the thesis*; Chapter A collects all of them. Implementation boxes name the public API (linked to the documentation), internal functions and types by name and source file (linked to the main branch), and the tests by their `@testset` titles; no reference depends on line numbers. Each equation chapter states the commit against which it was audited; Section 1.1 describes how these commits, the names of sources and tests and the quoted numbers are checked against the code. Properties are claimed only when they are proved in a cited source, follow in one line from the stated definitions, or were checked numerically; checks give the measured numbers. Every symbol is defined in Chapter B.

— 1 —

Method overview

Equation reference, audited against commit `619ea56`.

TwoPhaseAMR.jl solves the incompressible Navier–Stokes equations of one fluid, or of two immiscible fluids separated by a sharp interface with surface tension, in a two-dimensional rectangle that may contain a fixed embedded solid, on a block-structured quadtree mesh that adapts between time steps. This chapter is the entry point of the reference. It states what the reference is and how its statements are tied to the code (Section 1.1), the governing equations (Section 1.2), the discretisation chapter by chapter (Section 1.3), the stages of one time step and of a run (Section 1.4), the selectable methods with their status and the public types of every chapter (Section 1.5), and the known limitations with the issues that track them (Section 1.6). It introduces no discretisation of its own: every formula it quotes is stated, with its implementation and its tests, in the chapter it points to, and the time step of Section 1.4 is the composition of the fully discrete updates of those chapters.

1.1 Scope of this reference

The reference states what the code computes; the code is normative. Every equation chapter describes the implementation at the commit against which it was audited, printed under its title, and where the implementation differs from the thesis of Remmerswaal [19] it states the implementation and records the difference in a box titled *Deviation from the thesis*; Chapter A lists all of them. They range from typographical errors of the thesis, confirmed by an audit (deviations 4.6 and 10.1), through methods that replace those of the thesis (deviations 8.2 and 10.5) to parts of the thesis that are not implemented (deviations 1.1 and 8.3). A known defect of the code is not described as intended behaviour: it is stated as a defect, with the issue that tracks it (Section 1.6).

The models, the notation and most methods are those of the thesis. Its Chapter 2 introduces the two-phase Navier–Stokes equations, the discrete setting, the interface description, cut cells and the refinement criteria. Part I (Chapters 3 to 5) treats the geometric volume-of-fluid method, surface tension and interface reconstruction with one velocity; Part II treats the transport of mass and momentum (Chapter 6), the pressure projections, diffusion and gravity of the one- and two-velocity formulations (Chapter 7) and their application to liquid impacts (Chapter 8). Derivations are cited from the thesis and the original literature and are not repeated. Where the package goes beyond the thesis, for the block forest and its transfer

maps (Chapters 2 and 12), the cut-cell construction (Chapter 7), the linear solvers (Chapter 11) and the selection of the time step (Chapter 13), the construction is the package's and is stated in full.

Deviation from the thesis 1.1 (Dimension and compressibility). The thesis formulates the two-phase equations in $d \in \{2, 3\}$ dimensions [19, §2.1], reports three-dimensional simulations of breaking wave impacts [19, §8.4] and outlines the generalisation to compressible flow in remarks [19, Remarks 6.1 and 7.2]. The package is two-dimensional and incompressible throughout.

How statements are tied to the code. Implementation boxes name the public API, linked to the documentation of the matching package version; internal functions and types by name together with the source file on the main branch that defines them; and tests by the exact titles of their `@testsets`. No reference uses line numbers. The manifest `coverage.toml` repeats the status and the audited commit of every chapter, classifies every source file of `src/` and `ext/` (equation, algebraic, orchestration, storage, user interface or infrastructure, with the chapters that state it) and lists the *claims*: for an equation, table or section label, the source symbols that implement it, the test sets that exercise it and, optionally, the function of the figure scripts that evaluates it independently. The script `check_links.py` fails if a named symbol is not defined in its file, a test title does not exist in its file, a source file is not classified or a classified one is missing, the status or commit of a chapter differs from the manifest, a figure file is missing, a claim names an undefined label, symbol or test, or a link from the documentation into this PDF names an undefined label. It reports, without failing, the API anchors missing from the built documentation and, for every audited chapter, the referenced source files that changed since its audited commit: such a chapter is re-read against the changes and re-pinned.

How statements are checked. Most measured numbers of the text, among them those of the paragraphs headed *Checks*, are computed from the package by the scripts of `docs/techref/figures/`, which write the figure data and the number macros that the PDF build reads, and whose assertions fail when the implementation no longer matches a statement; the build itself never runs Julia. A derivation audit checks one claim independently of the chapter that states it, symbolically and numerically against the package; each audit is archived in a folder of `proof_review/` with its steps, the errors found, the sharpness of the claim and the scripts that rerun it, and a finding is either applied to the chapter or, if the code is at fault, filed as an issue (Section 14.4). Chapter 14 defines the diagnostics and connects the statements to the test suite and its verification cases.

> *Implementation.* `docs/techref/coverage.toml` (chapter status and audited commits, source classification, claims and figures), `docs/techref/check_links.py` (checks and re-audit report; the option `–skip-api` omits the API anchors when the documentation is not built), `docs/techref/figures/generate.jl` (figure data and the number macros of `figures/generated/checks.tex`), `proof_review/README.md` (one row per audited claim). The documentation page `docs/src/method.md` links its topics into this PDF by named destinations.

1.2 Governing equations

Domain and phases. The domain Ω is a rectangle whose directions are periodic or bounded by walls (Section 2.1). A fixed solid $\Omega^{\mathrm{s}} = \{\, \boldsymbol{x} \in \Omega \mid q^{\mathrm{s}}(\boldsymbol{x}) \leq 0 \,\}$ may be embedded, and

the fluid occupies $\Omega^{\mathrm{o}} = \Omega \setminus \Omega^{\mathrm{s}}$ (Section 7.1). With two phases, Ω^{o} is split into the liquid and gas regions $\Omega^{\mathrm{l}}(t)$ and $\Omega^{\mathrm{g}}(t)$, whose common boundary is the interface $I(t)$, with unit normal $\boldsymbol{\eta}$ pointing into the gas and curvature $\kappa = \nabla \cdot \boldsymbol{\eta}$ (Section 4.1); $[\![\cdot]\!]$ is the gas minus the liquid value. There is no phase change.

Two-velocity formulation. Each phase is an incompressible Newtonian fluid with its own velocity and pressure, constant density ρ^{π} and viscosity μ^{π} [19, §2.1.1]:

$$\begin{aligned} &\nabla \cdot \boldsymbol{u}^{\pi} = 0, \qquad && \rho^{\pi}\left(\partial_t \boldsymbol{u}^{\pi} + \boldsymbol{u}^{\pi} \cdot \nabla \boldsymbol{u}^{\pi}\right) = \nabla \cdot \mathbf{T}^{\pi} + \rho^{\pi}\boldsymbol{g} + \rho^{\pi}\boldsymbol{a}^{\mathrm{b}} \quad \text{in } \Omega^{\pi}(t), \\ &[\![\boldsymbol{u} \cdot \boldsymbol{\eta}]\!] = 0, \qquad && \boldsymbol{\eta} \cdot [\![\mathbf{T}]\!] = \sigma\kappa\boldsymbol{\eta} \quad \text{on } I(t), \end{aligned} \tag{1.1}$$

with $\mathbf{T}^{\pi} = -p^{\pi}\mathsf{I} + \mu^{\pi}\left(\nabla \boldsymbol{u}^{\pi} + (\nabla \boldsymbol{u}^{\pi})^{\mathsf{T}}\right)$, the gravitational acceleration $\boldsymbol{g}$ and a prescribed body acceleration $\boldsymbol{a}^{\mathrm{b}}(\boldsymbol{x}, t)$, which the code adds to the thesis' model ((8.1) and (8.2)). The interface moves with the normal velocity, which is continuous [19, eq. (2.2)]; the tangential velocity may jump.

One-velocity formulation. If the velocity is continuous, $[\![\boldsymbol{u}]\!] = 0$, the phase balances add up to one momentum balance for one velocity $\boldsymbol{u}$ with the piecewise constant density $\rho = \rho^{\mathrm{l}}\chi^{\mathrm{l}} + \rho^{\mathrm{g}}(1 - \chi^{\mathrm{l}})$ and viscosity μ, $\chi^{\mathrm{l}}(\boldsymbol{x}, t)$ the liquid indicator [19, §2.1.2, eq. (2.9)]; surface tension is then the singular force $\sigma\kappa\nabla\chi^{\mathrm{l}} = -\sigma\kappa\boldsymbol{\eta}\delta^{I}$ (Section 8.1):

$$\nabla \cdot \boldsymbol{u} = 0, \qquad \rho\left(\partial_t \boldsymbol{u} + \boldsymbol{u} \cdot \nabla \boldsymbol{u}\right) = \nabla \cdot \mathbf{T} + \sigma\kappa\nabla\chi^{\mathrm{l}} + \rho\boldsymbol{g} + \rho\boldsymbol{a}^{\mathrm{b}}, \tag{1.2}$$

with $\mathbf{T} = -p\mathsf{I} + \mu\left(\nabla \boldsymbol{u} + (\nabla \boldsymbol{u})^{\mathsf{T}}\right)$; the phase regions move with $\boldsymbol{u}$, so that the volume of each phase in a fixed cell obeys (5.1). A single phase obeys the same equations with constant ρ and μ, without interface and without gravity (`SinglePhase` has no gravity parameter); the body acceleration can be prescribed for every model.

Boundary conditions. A wall side is a stationary or moving no-slip wall, a free-slip wall, a prescribed inflow, or an outflow or zero-gradient side (Sections 9.1 and 10.5.1); periodic directions need none. On the solid boundary $\boldsymbol{u} \cdot \boldsymbol{n} = 0$, and the tangential condition enters only the viscous stress (Chapter 7). Where the interface meets a wall or the solid, the static contact angle $\boldsymbol{\eta} \cdot \boldsymbol{n}^{\Gamma} = \cos\theta$ (4.3) enters only the curvature (Section 4.7). The VOF transport, and hence every two-phase flow, requires impermeable or periodic boundaries (Section 5.6).

Constraints in time. In the split time step the pressure enforces incompressibility: every step ends with a projection onto a discrete constraint, the open divergence for one velocity and, for two velocities, the aperture-weighted mixture divergence together with a condition on the normal velocity jump at the interface (Section 10.1).

> *Implementation.* `SinglePhase(; density, dynamic_viscosity)`, `TwoPhase(interface; density, dynamic_viscosity, surface_tension=SurfaceTension(), gravity=(0, 0), scalars=())` with `SurfaceTension`; the formulation is chosen by `FlowSimulation(mesh, physics; velocity_model=OneVelocityModel(), boundary=NoSlip(), geometry, ...)` with `OneVelocityModel` or `TwoVelocityModel`, subtypes of `AbstractVelocityModel`, and is never inferred from the initial velocity. The shared velocity is a `FaceVelocity`, the phase velocities a `TwoPhaseVelocity`; boundary conditions `NoSlip`, `FreeSlip` and `VelocityBoundaryConditions` with `Inflow`, `Outflow` and `ZeroGradientVelocity`; the solid is a `GeometryField`, the interface an `InterfaceField`. Internal: `TwoPhaseModel` (`src/two_phase/model.jl`), which also rejects gravity along a periodic direction. Tests:

`test/navier_stokes/configuration.jl` (“explicit one- and two-velocity models”) (defaults, and a liquid/gas initializer rejected with one velocity),
`test/navier_stokes/configuration.jl` (“positional configuration validates physical and policy invariants”).

1.3 The discretisation at a glance

Mesh (Chapter 2). All fields live on the leaf cells of a 2:1-graded forest of quadtrees whose nodes are blocks of a fixed number of Cartesian cells (by default 16×16). Cell values are stored once per owned cell and face-normal values once per owned face, so that the side of a coarse cell next to two finer cells carries two face segments; halo slots around every block serve local stencils. All levels advance with the same time step and every face flux is added to both of its cells, so no refluxing is needed. The forest changes only between time steps.

Discrete setting (Chapter 3). The unknowns are those of a staggered (MAC) scheme on the composite mesh: cell averages of the pressure and of the fractions, face averages of the normal velocity. The divergence is the exact flux balance of a cell; the gradient $\mathcal{G} = \mathsf{K}\mathsf{J}$ is its negative adjoint, with a sparse symmetric positive definite inverse face metric K that is diagonal on conforming patches and couples faces at level transitions; the weighted gradient and the elliptic operator of the projections and of the forces are built from both. A solid enters through the open measures of the cells and faces that it cuts.

Interface (Chapter 4). The interface is represented by the liquid volume fraction of every cell. It is reconstructed cell by cell as one straight line (PLIC) with a plane constant that matches the fraction; by default its normal is that of the local height functions where they exist, one-sided at walls, and otherwise the Weymouth–Yue normal (Table 4.1). The curvature comes from the height functions, with generalised height functions as fallback, and the static contact angle enters it at walls and at the solid. Fractions are initialised from a half-plane, a polygon or a level set.

Volume-of-fluid transport (Chapter 5). The fraction is transported by the directionally split geometric method of Weymouth and Yue [33]: two sweeps per step in alternating order, each moving the liquid volumes cut from the current reconstruction through the owned faces, with a dilation term weighted by a phase indicator frozen for the step, and with clamping and snapping to the bounds. Intrinsic phase scalars move with the same phase volumes. With two velocities the carrier is the frozen interface flux: each phase subface of the start-of-step reconstruction is extruded with its own phase velocity.

Momentum transport (Chapter 6). The velocity components are transported over staggered control volumes whose dual-face fluxes are fixed linear combinations of the primal face fluxes; at level transitions and in cut cells the combination keeps the dual divergence equal to an interpolation of the primal one. Face states are upwind, centred, Lax–Wendroff (the default) or limited three-point states. A single phase convects its velocity; the one-velocity model transports the density-weighted momentum and the two-velocity model the intrinsic phase velocities and staggered phase scalars, both inside the VOF sweeps with the interpolated phase volumes as fluxes [19, §6.4].

Embedded solid (Chapter 7). A fixed solid is embedded without changing the mesh: every cell carries its solid fraction and, if it is cut, one half-plane that approximates the solid, from which the open polygons, the face apertures and the open staggered volumes follow; the velocity on faces that the solid covers is zero. Small cut cells are not merged.

Forces (Chapter 8). Surface tension (the conservative ghost-fluid model CGFM, the default, or CSF) and gravity (the mimetic gravity model of the thesis) are integrated face jumps passed through the weighted Riesz map of the pressure gradient, so that a force that is the jump of a cell function is exactly a pressure gradient and the projection removes it (balanced force). Both are explicit increments of the predictor after the viscous step; the face curvature is interpolated from the cell curvatures. A prescribed body acceleration is added at the midpoint of the step, before the viscous step.

Viscosity (Chapter 9). The viscous step is the one implicit substep, an implicit midpoint rule. A single phase uses a componentwise staggered Laplacian of the kinematic viscosity with geometric transition contacts and, with a solid, rows for the embedded walls; two phases use the variable stress $\nabla \cdot (2\mu\mathbf{D})$ with the geometric mean of the phase viscosities, and the two-velocity model diffuses the mass-weighted mixture velocity and gives both phases its increment.

Projection (Chapter 10). Every step ends with one pressure projection: a Poisson projection for a shared velocity (one phase, or two phases with one velocity), and for the phase velocities the conservative multidimensional ghost-fluid method (CMDGFM) of the thesis. Both are incremental: the step subtracts the gradient of the previous pressure and the projection solves for the increment, so that a shortened step removes only its share of the divergence left by the previous step. After a regrid the phase velocities are projected once more.

Linear solvers (Chapter 11). The projections and the viscous step are sparse linear systems, solved by Krylov methods with a stopping rule and a round-off floor: CG for the Poisson projection and the two-phase viscous step, CG or BiCGSTAB after a symmetry test for the one-phase viscous step, and BiCGSTAB for the CMDGFM. They are preconditioned by V-cycles of a tree geometric multigrid built from the forest (Table 11.1).

Refinement and transfer (Chapter 12). After a step, refinement indicators (interface and solid bands, curvature, vorticity and interpolation error) label every leaf block *refine*, *keep* or *coarsen*. `adapt!` edits and balances the forest and transfers every registered field: cell fields by conservative restriction and limited linear prolongation, fractions and phase scalars through the reconstruction, face velocities along grid lines with corrections that keep the divergence of a shared velocity cell by cell on meshes without a solid.

Time step (Chapter 13). The explicit stages bound the step: the transport (the staggered bound in one phase, the WY bound in two) and, with two phases, the capillary and gravity-wave bounds. `simulate!` chooses steps from these bounds at CFL number 0.45, never increases the step within a run and adjusts the last step to end at the final time; `step!` rejects a step above the bounds at CFL number $\frac{1}{2}$. No order in time is claimed for the complete step.

Implementation. The equation system `NavierStokesSystem` with its `NavierStokesWorkspace`; numerical choices `NavierStokesOptions(; advection=AdvectionOptions(), pressure=LinearSolverOptions(), diffusion=LinearSolverOptions(), multigrid=MultigridOptions())`; adaptivity `Adaptivity(; criteria, fields)` with the `MeshAdaptationOptions` of the mesh. Table 1.3 lists the public types by chapter.

Table 1.1. The thesis' time step [19, §7.2] and the stages of the code, with the sections that state the differences.

Thesis	Code	Difference
T1, interface advection by unsplit donating regions	WY sweeps of the fraction and the phase scalars, alternating order	directional splitting (Section 5.3); two-velocity carriers (Section 5.8)
T2, momentum with the mass fluxes of T1	one phase: staggered convection; two phases: momentum or phase velocities in the same sweeps	face states and flux interpolant (Sections 6.3, 6.7 and 6.8)
–	body acceleration at the midpoint of the step	addition of the code (Section 8.7)
T3, implicit diffusion	implicit midpoint step	single phase: explicit half at $u^{(n)}$ (deviation 9.3)
T4, gravity	surface tension and gravity as explicit increments	surface tension moved from T5 (deviation 8.2); common increments with two velocities (deviation 8.5)
T5, pressure Poisson problem	old-pressure predictor and incremental projection (one velocity, CMDGFM)	deviation 10.5
–	projection after a regrid (two velocities)	deviation 10.6

1.4 One time step

A step advances the liquid fraction and the phase scalars, the velocity or the two phase velocities, and the pressure from $t^{(n)}$ to $t^{(n+1)} = t^{(n)} + \delta t$ by a fixed sequence of stages, the split step of (8.21). Its stages are those of the thesis' steps T1 to T5 [19, §7.2] with the changes of Table 1.1. The velocity after the transport, the viscous step and the forces is u^*, u^{**} and u^{***}, after the old-pressure predictor $u^\star$. The step takes a reference step $\delta t^{\mathrm{ref}} \geq \delta t$ ($\delta t^{\mathrm{ref}} = \delta t$ for a direct call): with the incremental projections, a shorter step keeps the share $1 - \delta t/\delta t^{\mathrm{ref}}$ of the divergence that it inherits (Section 10.5.2).

1.4.1 One velocity

`step!` with a `FaceVelocity` runs the same stages for one phase and for two phases with one velocity; the two-phase stages extend the single-phase ones by dispatch.

1. *Start.* The velocity gets its boundary values at $t^{(n)}$: prescribed faces from the boundary conditions, open faces keeping their projected values, faces covered by the solid zero (Section 10.5.1). This is $u^{(n)}$. For $\delta t < \delta t^{\mathrm{ref}}$ its open divergence is kept for the projection (10.28).

2. *Transport.* The step is checked against the bound of the transport and, with two phases, the capillary and gravity bounds (13.3). One phase: $u^{(n)}$ is convected with itself as frozen carrier by the staggered scheme (6.19), in two directional sweeps of alternating order for the default Lax–Wendroff states, with the boundary values at $t^{(n)}$. Two phases: the WY sweeps (5.6) with the carrier $u^{(n)}$ advance the fraction to $C^{\mathrm{l},(n+1)}$ and the phase scalars and, in the same sweeps, the density-weighted momentum (Section 6.7), in alternating order. The result is u^*.

3. *Body acceleration.* $u^* \leftarrow u^* + \delta t\, \boldsymbol{a}^{\mathrm{b}}(t^{(n)} + \frac{1}{2}\delta t)$ on the interior faces (8.19) (nothing is added without a body acceleration); then, in every step, the boundary values at $t^{(n)} + \frac{1}{2}\delta t$, open faces extrapolating the change since $u^{(n)}$.

4. *Viscous step.* The implicit midpoint step (9.17) (one phase) or (9.18) (two phases, with the viscosity and the density of $C^{\mathrm{l},(n+1)}$; omitted if both viscosities vanish) gives u^{**}.

5. *Phase forces* (two phases). The pressure coefficient γ is rebuilt from $C^{\mathrm{l},(n+1)}$ ((8.5)) and, for unequal densities, its preconditioner is refreshed. Then $u^{***} = u^{**} + \delta t \mathsf{R}\mathcal{J}^{\sigma} + \delta t \mathsf{R}\mathcal{J}^{\boldsymbol{g}}$, surface tension first, each term only if $\sigma > 0$ or $\boldsymbol{g} \neq \boldsymbol{0}$ (Section 8.9). With one phase $u^{***} = u^{**}$.

6. *End-of-step boundary values* at $t^{(n+1)}$, after the forces; covered faces zero.

7. *Old-pressure predictor.* The gradient of $p^{(n)}$ in the pressure map of the projection, times δt, is subtracted, giving $u^{\star}$ (10.26). This map is the weighted gradient of the refreshed coefficient, RJ on the interior faces; it vanishes on boundary faces with a prescribed flux and corrects open ones (Section 10.5.2).

8. *Projection.* The Poisson projection (10.4) of $u^{\star}$, with the retained share of the inherited divergence (10.28), gives the increment φ. The velocity is corrected by δt times the gradient of φ in the same map, open boundary faces included, and $p^{(n+1)} = p^{(n)} + \varphi$ ((10.27)).

Implementation. `step!(velocity, pressure, system, dt, workspace; time=0, force=nothing, tolerance=sqrt(eps(T)), max_cycles=nothing)` in `step!` (`src/single_phase/stepping.jl`); at most 100 (one phase) or 500 (two phases) iterations per solve by default. Stages: `_transport_substep!` (`src/single_phase/stepping.jl`) and `_transport_substep!` (`src/two_phase/stepping.jl`), with the step check `_check_transport_timestep` (`src/single_phase/stepping.jl`); `_body_force_substep!` (`src/navier_stokes/stages.jl`), `_enforce_predictor_velocity_boundary!` (`src/single_phase/stepping.jl`), `_diffusion_substep!` (`src/single_phase/stepping.jl`) and `_diffusion_substep!` (`src/two_phase/stepping.jl`), `_refresh_pressure_substep!` (`src/two_phase/stepping.jl`), `_phase_force_substep!` (`src/two_phase/stepping.jl`) with the force stage `_apply_phase_forces!` (`src/two_phase/stepping.jl`) shared by both velocity models, `_pressure_predictor!` (`src/single_phase/stepping.jl`) and `_project_prepared!` (`src/navier_stokes/projection.jl`) (with a solid `_project_prepared!` (`src/embedded/operators.jl`)). Tests: `test/navier_stokes/two_phase_navier_stokes.jl` ("One Navier-Stokes API, transported density jump"), `test/navier_stokes/body_force.jl` ("Body-force midpoint update reaches shared and intrinsic stepping"), `test/navier_stokes/two_phase_navier_stokes.jl` ("Two-phase diffusion uses the thesis split state") (the viscous stage acts on the transported state), `test/two_phase/forces.jl` ("Surface tension precedes gravity for each velocity model") (the force stage leaves boundary values and absent phases unchanged), `test/navier_stokes/navier_stokes.jl` ("increment predictor preserves open-boundary projection"), `test/navier_stokes/navier_stokes.jl` ("short step does not turn inherited divergence into pressure").

1.4.2 Two velocities

`step!` with a `TwoPhaseVelocity` advances the intrinsic phase velocities u^{l} and u^{g}, which are absent (NaN) where their phase has no staggered volume.

1. *Start.* Both phases get their boundary values at $t^{(n)}$ (Section 10.5.1), covered faces are set to zero and absent values marked with the current interface. With the CMDGFM and $\delta t < \delta t^{\mathrm{ref}}$ the mixture flux of $u^{\pi,(n)}$ on the phase apertures of the start-of-step reconstruction is saved for the projection (Section 10.5.2).

2. *Transport.* The step is checked against the bounds unless `simulate!` has just evaluated them for this state (automatic step, frozen interface fluxes); that evaluation also reconstructs the interface and freezes the phase apertures of the step start (Section 5.8.1). The WY sweeps with the frozen carrier (5.25) advance the fraction, the phase scalars, the staggered phase scalars and the phase velocities (Section 6.8), in alternating order, to $u^{\pi,*}$.

3. *Body acceleration* on both phases where they are present (8.19); then, in every step, the boundary values at $t^{(n)} + \frac{1}{2}\delta t$.

4. *Viscous step.* The mass-weighted mixture velocity is advanced by (9.18) and its increment is added to both phases (9.21), giving $u^{\pi,**}$ (omitted if both viscosities vanish).

5. *End-of-step boundary values* of both phases at $t^{(n+1)}$, here before the forces.

6. *Pressure map.* For the CMDGFM without a solid the transported interface is reconstructed and the one-velocity coefficient rebuilt, so that $\mathsf{R}^{\mathrm{tv}} = \mathsf{R}$; with a solid the CMDGFM uses the operator that it caches, with the coefficient γ^{tv} of the transported fractions (Section 8.8).

7. *Phase forces.* The common increments $\delta t \mathsf{R}^{\mathrm{tv}} \mathcal{J}^{\sigma} + \delta t \mathsf{R}^{\mathrm{tv}} \mathcal{J}^{\boldsymbol{g}}$ are added to both phases where they are present (8.20), giving $u^{\pi,***}$.

8. *Projection.* CMDGFM: the common old-pressure gradient is subtracted, $u^{\pi,\star} = u^{\pi,***} - \delta t\, \mathsf{R}^{\mathrm{tv}} \mathsf{J} p^{(n)}$ (10.26), the system (10.25) is solved for the increment φ and the gradient jump ξ, with the saved flux for a shortened step, and $p^{(n+1)} = p^{(n)} + \varphi$.

Implementation. `step!` (`src/two_phase/two_velocity/stepping.jl`) with `_enforce_phase_velocity_boundary!` (`src/two_phase/two_velocity/stepping.jl`), `_capture_two_velocity_inherited_flux!` (`src/two_phase/two_velocity/stepping.jl`), `_transport_substep!` (`src/two_phase/two_velocity/stepping.jl`) (step check `_check_transport_timestep` (`src/single_phase/stepping.jl`), prepared path `_advect_transport_substep!` (`src/two_phase/two_velocity/interface_transport.jl`)), `_diffusion_substep!` (`src/two_phase/two_velocity/stepping.jl`), `_two_velocity_pressure_operator!` (`src/two_phase/two_velocity/stepping.jl`), `_phase_force_substep!` (`src/two_phase/two_velocity/stepping.jl`) with `_apply_phase_forces!` (`src/two_phase/stepping.jl`), and `_project_two_velocity!` (`src/two_phase/two_velocity/flow_workspace.jl`). Tests:

`test/navier_stokes/two_phase_navier_stokes.jl` (“Intrinsic phase apertures are updated once per step”) (the bounds evaluated by `simulate!` prepare the transport), `test/two_phase/two_velocity/diffusion.jl` (“Two-velocity implicit diffusion identities”), `test/two_phase/forces.jl` (“Surface tension precedes gravity for each velocity model”), `test/two_phase/two_velocity/transport/two_velocity.jl` (“Two-velocity short-step reference flux”), `test/two_phase/two_velocity/projection/cmdgfm_pressure_reuse.jl` (“CMDGFM incremental projection satisfies the coupled system”).

1.4.3 A RUN

`simulate!` advances to the final time t_{end} by repeated steps; a `FlowSimulation` supplies its fields, system, workspace, time and diagnostic collection and accumulates its counters over successive calls. A call proceeds as follows.

1. *Set-up.* With refinement indicators an AMR workspace registers the fields to transfer (Section 12.3.2); dynamic adaptation excludes observers that advance copies of the state over several steps (exact keyframes). The diagnostics are recorded and the observer receives the event `:initial`; an exception there ends the call before the first step, without a `:failed` event. The observer is asked whether to stop (an exception of the poll is logged and counts as no).

2. *Step size.* Without a prescribed step (`dt=nothing`) the boundary values at $t^{(n)}$ are imposed and the step is the proposal of (13.2) from the flow bound at CFL number 0.45; otherwise it is the prescribed step. The last step is stretched or shortened to end at t_{end}, and the reference step δt^{ref} is formed with it.

3. *Step.* One `step!` as in Sections 1.4.1 and 1.4.2. A linear solve that has not converged (viscous step, projection, and the projection after a regrid) stops the run with an error; a step that fails a run-time check is not retried (Section 13.4).

4. *Regrid.* With indicators, `adapt!` is called with the number of completed steps and the time at the end of the step, and acts at most every `update_interval` steps (Section 12.3.1). If the leaf set changed, the topology-dependent operators and workspaces are rebuilt, the divergence that the transfer leaves is associated with the proposal δt^{prop}, and with two velocities the transferred velocities are projected (10.31).

5. *Events.* The diagnostics due at this step are recorded and the observer receives `:step` with the step size, the projection statistics, whether the topology changed and the wall time of the step; an exception of the observer is logged and the run continues, an interrupt propagates. The observer is asked whether to stop; if so, the run ends at this step boundary.

6. *End.* After the last step or a requested stop the diagnostics are recorded and the observer receives `:completed`; the last step and, after an automatic step, the proposal are retained for a call that continues the run (Section 13.2). If the run throws, the observer receives `:failed` with the exception (an error of the observer there is dropped), which is then rethrown.

A standalone `step!` ends the retained history, and so does `project!(flow)`, which projects an initial velocity without changing the physical pressure (Section 10.5.2).

Implementation. `simulate!(velocity, pressure, system, final_time, workspace; time=0, dt=nothing, cfl=0.45, force=nothing, diagnostics, step_offset=0, observer=NoObserver(), tolerance=sqrt(eps(T)), max_cycles=nothing)` in `simulate!` (`src/navier_stokes/simulate.jl`), for a `FlowSimulation` `simulate!` (`src/navier_stokes/flow_simulation.jl`)`(flow, final_time; dt=flow.default_dt, ...)`; rebuild `_rebuild_topology_dependent_state!` (`src/navier_stokes/workspace.jl`), projection after a regrid `_project_regridded_two_velocity!` (`src/two_phase/two_velocity/flow_workspace.jl`); diagnostics `_record_diagnostics!` (`src/simulation/diagnostics/history.jl`); events `observe!` with `SimulationState` and `StepInfo`, isolated by `_observe_safely!` (`src/observation/observer.jl`), and `stop_requested`; side steps for exact keyframes `_prepare_independent_step!` (`src/navier_stokes/independent_step.jl`); `Recorder` is the standard observer. Tests: `test/recording/recorder.jl` ("Custom observers receive completed solver states") (the events `:initial`, `:step` after every step and `:completed`, also after a requested stop), `test/recording/recorder.jl` ("Observer errors are isolated after initialization"), `test/recording/recorder.jl` ("Exact keyframes use independent side steps"), `test/mesh/adaptivity.jl` ("Navier–Stokes optionally adapts and rebuilds its operators"), `test/navier_stokes/high_level_api.jl` ("regrid defects keep the transfer reference"), `test/navier_stokes/high_level_api.jl` ("tiny final timestep is absorbed").

1.5 Methods and options

Table 1.2 lists the choices that select a method. The two-velocity method is the frozen interface fluxes with the CMDGFM projection: the frozen interface fluxes, which the thesis derives but does not pursue [19, §6.3.4], replace the projected liquid velocity of the thesis [19, §6.3.2] (Section 5.8.1). Every combination of velocity model and surface-tension method can be configured; the two-velocity transport applies to the two-velocity model only. Table 1.3 maps the public types to the chapters that state what they compute; the implementation boxes give them in context.

Implementation.
`TwoVelocityModel` (`src/two_phase/two_velocity/model.jl`),
`OneVelocityModel` (`src/two_phase/one_velocity.jl`),
`FlowSimulation` (`src/navier_stokes/flow_simulation.jl`) (which passes the projection of the velocity model to the system),
`AdvectionOptions` (`src/navier_stokes/options.jl`),
`NavierStokesOptions` (`src/navier_stokes/options.jl`),
`CMDGFMProjection` (`src/two_phase/projection/methods.jl`). Tests:
`test/navier_stokes/configuration.jl` ("explicit one- and two-velocity models") (the defaults), `test/navier_stokes/configuration.jl` ("typed two-phase choice matrix constructs") (every projection and surface-tension method),
`test/navier_stokes/configuration.jl` ("typed Navier–Stokes configuration"),
`test/navier_stokes/high_level_api.jl` ("high-level FlowSimulation API") (the

Table 1.2. Method choices, with the default first. Legacy options are kept for comparison.

Choice (set by)	Values	Section
velocity formulation (`velocity_model` of `FlowSimulation`)	`OneVelocityModel`, `TwoVelocityModel`	Section 1.2
two-velocity projection (`pressure_projection` of `TwoVelocityModel`)	`CMDGFMProjection`	Section 10.4
CMDGFM preconditioner (`preconditioner` of `CMDGFMProjection`)	`:block`, `:ilu`, `nothing`	Section 11.5
surface tension (`method` of `SurfaceTension`)	`CGFM`, `CSF`	Section 8.5
momentum face states (`scheme` of `AdvectionOptions`)	`LaxWendroff`, `Upwind`, `Centred`, `Fromm`, `Minmod`, `MonotonizedCentral`, `VanLeer`, `Superbee`	Section 6.3
states at level transitions (`amr_interface_scheme` of `AdvectionOptions`)	`MonotonizedCentral` for `LaxWendroff` and `Centred`, otherwise the scheme itself	Section 6.3
reconstruction (`reconstruction_method` of `InterfaceField`)	`:lhf`, `:lhf_ghf`, `:wy`	Section 4.5
Krylov method and preconditioner (`LinearSolverOptions` and `MultigridOptions`)	pressure projection and viscous steps: CG, or BiCGSTAB after a symmetry test for the one-phase viscous step (CG on uniform and composite meshes), with the tree geometric multigrid (`:auto`); the CMDGFM system (BiCGSTAB) has its own preconditioner (Table 11.1)	Chapter 11
time step (`dt` and `cfl` of `simulate!`)	automatic with CFL number 0.45; a prescribed step	Section 13.2
refinement (`adaptivity` of `FlowSimulation`)	none; indicators of Section 12.2	Chapter 12

default interface scheme and multigrid preconditioner).

1.6 Known limitations

Besides the scope of deviation 1.1, the implementation has these limits by design. Two-phase flows need impermeable or periodic boundaries (Section 5.6); the contact angle is static and enters only the curvature (Section 4.7); there is no phase change. No order in time is claimed for the split step, and the orders in space that the tests assert, with their known first-order spots, are those of Table 13.3. Small cut cells are not merged and reduce the step (Section 5.6), and a step that fails a run-time check is not retried (Section 13.4). Table 1.4 groups the defects and open questions that the chapters state and that are tracked by issues open at the audited commit; Section 14.9 adds the gaps of the verification itself.

Table 1.3. Public types by the chapter that states them.

Chapter	Public types
1, overview	`FlowSimulation`, `NavierStokesSystem`, `NavierStokesWorkspace`, `NavierStokesOptions`, `AbstractVelocityModel`, `OneVelocityModel`, `TwoVelocityModel`, `SimulationState`, `AbstractObserver`, `NoObserver`, `StepInfo`, `Recorder`
2, mesh	`BlockTreeMesh`, `BlockKey`, `LeafBlock`, `CellRef`, `CellField`, `FaceField`, `FaceVelocity`, `ZeroGradient`, `Dirichlet`, `MeshAdaptationOptions`, `AMRWorkspace`
3, operators	`GradientOperator`, `EllipticOperator`, `EllipticWorkspace`
4, interface	`InterfaceField`, `PolygonIndicator`
5, VOF	`WYAdvectionWorkspace`, `PhaseScalar`
6, momentum	`AdvectionOptions`, `StaggeredAdvectionWorkspace`, `StaggeredVelocityAdvectionWorkspace`, `MomentumAdvectionWorkspace`, `AdvectionWorkspace`, `PhaseFaceScalar`, `TwoPhaseVelocity`, `Upwind`, `Centred`, `LaxWendroff`, `Fromm`, `Minmod`, `MonotonizedCentral`, `VanLeer`, `Superbee`
7, solid	`GeometryField`, `PLICSolidGeometry`
8, forces	`SinglePhase`, `TwoPhase`, `SurfaceTension`, `AbstractSurfaceTensionMethod`, `CSF`, `CGFM`
9, viscosity	`StaggeredDiffusionOperator`, `VariableStaggeredDiffusionOperator`, `StaggeredEllipticOperator`, `NoSlip`, `FreeSlip`, `VelocityBoundaryConditions`, `Inflow`, `Outflow`, `ZeroGradientVelocity`
10, projection	`ProjectionWorkspace`, `CMDGFMProjection`, `CMDGFMProjectionSolver`, `CMDGFMProjectionWorkspace`, `CMDGFMProjectionStats`
11, solvers	`EllipticSolver`, `StaggeredEllipticSolver`, `LinearSolverOptions`, `LinearSolverStats`, `MultigridOptions`, `TreeGeometricMultigridPreconditioner`
12, refinement	`Adaptivity`, `AMRStats`, `AbstractRefinementIndicator`, `InterfaceRefinementIndicator`, `GeometryRefinementIndicator`, `CurvatureRefinementIndicator`, `VorticityRefinementIndicator`, `InterpolationErrorIndicator`
13, time step	`CFLTimeStep`
14, verification	`KineticEnergy`, `PotentialEnergy`, `SurfaceEnergy`, `TotalEnergy`, `LinearMomentum`, `PhaseVolume`, `PhaseMass`, `InterfaceMeasure`, `MaximumVelocity`, `DivergenceNorm`, `LeafBlockCount`, `CellCount`

Table 1.4. Known defects and open questions by area, with their issues and the sections that state them.

Limitation	Issue	Section
Mesh, operators and interface		
A periodic direction with two cells per period is resolved by the face orientation of each face, so the gradient supports it; the multigrid coarsest level keeps four cells per periodic axis	issue 36	Sections 2.2 and 11.3.1
Dirichlet halo values are imposed at the slot centre, half a cell beyond the wall	issue 43	Section 2.4
The cut-cell metric replaces a vanishing open dual volume by the number one, which depends on the unit of length	issue 42	Section 3.7
Transport and time step		
At the stable step, a cut coarse donor at a level transition can stop the WY transport or leave a fraction outside $[0, 1]$, as can the frozen phase subfaces of a cut donor; the run-time checks stop the run	issue 52, issue 56	Section 13.4
Viscosity		
Inclined embedded no-slip walls give a first-order velocity	issue 110	Section 9.2
The variable stress is first order at level transitions, and the geometric-mean viscosity at a material jump gives a first-order shear	issue 112, issue 113	Section 13.5
Linear solvers		
CG is accepted with non-symmetric multigrid cycles, and symmetric embedded viscous systems are solved by BiCGSTAB	issue 128	Sections 11.3.3 and 11.4

— 2 —

Block-forest AMR mesh

Equation reference, audited against commit `1697df9`.

Every operator of the package acts on one composite mesh: the leaf cells of a 2:1-graded forest of quadtrees whose nodes are blocks of a fixed number of Cartesian cells. This chapter defines the forest, the owned cells and face segments that carry the unknowns, the halo slots that blocks keep for local stencils, and when the mesh changes. The thesis uses the same kind of mesh (block based, graded, refined by bisection of a uniform base mesh [19, §2.5]) and refers to Van der Plas [28] for its implementation and for the operators near refinement interfaces [19, §2.2]; the definitions below are those of this package. The mesh has no continuous counterpart and no time discretisation: it is constant during a time step and changes only between steps (Section 2.5).

2.1 Domain

The domain is a rectangle,

$$\Omega = \prod_{k=1}^{2} \left(o_k,\, o_k + X_k\right), \tag{2.1}$$

and each direction k is either periodic, with the lines $x_k = o_k$ and $x_k = o_k + X_k$ identified, or bounded by two walls that form the physical boundary $\Gamma \subset \partial\Omega$. The package is two-dimensional throughout. An embedded solid does not change the mesh; its open measures are defined in Section 3.3.

> *Implementation.* `BlockTreeMesh` keywords `origin` $= (o_1, o_2)$, `extent` $= (X_1, X_2)$ (positive) and `periodic` (default `(true, true)`).

2.2 Forest, blocks and levels

Per direction k the mesh is fixed by the number of level-zero cells M_k, the number of cells per block N_k (even and at least two; M_k is a multiple of N_k) and the refinement ceiling $L \geq 0$. The cell width of level ℓ is

$$h_{\ell,k} = 2^{-\ell} h_{0,k}, \qquad h_{0,k} = X_k / M_k, \qquad 0 \leq \ell \leq L. \tag{2.2}$$

The forest $\mathcal{T}$ consists of $R_1 R_2$ quadtrees, $R_k = M_k / N_k$, whose nodes are blocks. A block $\mathcal{B}$ is addressed by its key $\mathrm{key}(\mathcal{B}) = (\ell, i, j)$, with $0 \leq i < 2^\ell R_1$ and $0 \leq j < 2^\ell R_2$, and

covers

$$\mathcal{B} = \prod_{k=1}^{2} \left[o_k + i_k N_k h_{\ell,k},\ o_k + (i_k+1) N_k h_{\ell,k} \right], \qquad (i_1, i_2) = (i, j). \tag{2.3}$$

The roots are the level-zero blocks $(0, i, j)$; they can be refined but never removed. A refined block (ℓ, i, j) has the four children $(\ell+1, 2i+\iota, 2j+\jmath)$, $\iota, \jmath \in \{0, 1\}$: refinement bisects both directions. The leaves $\mathcal{L} \subset \mathcal{T}$, the blocks that are not refined, tile Ω. A leaf $\mathcal{B}$ is divided into $N_1 \times N_2$ congruent cells $c = (\mathcal{B}, i, j)$, $1 \le i \le N_1$, $1 \le j \le N_2$, counted from its lower left corner, so that $h_{c,k} = h_{\ell(c),k}$ and $|c| = h_{c,1} h_{c,2}$. The cells are

$$\mathcal{C} = \bigcup_{\ell=0}^{L} \mathcal{C}_\ell, \qquad \mathcal{C}_\ell = \{\, (\mathcal{B}, i, j) \mid \mathcal{B} \in \mathcal{L},\ \ell(\mathcal{B}) = \ell \,\}, \tag{2.4}$$

a disjoint union that tiles Ω; $\boldsymbol{x}_c$ is the centroid of c.

Leaves that touch differ by at most one level,

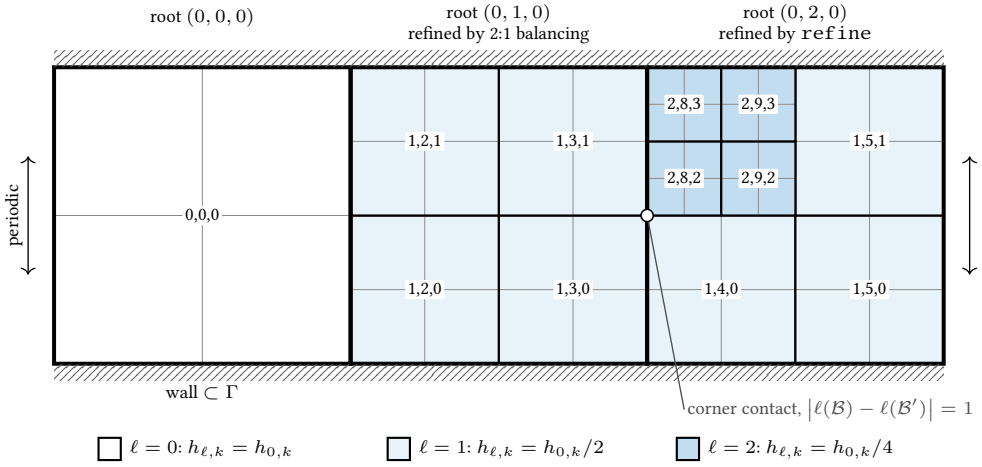
$$|\ell(\mathcal{B}) - \ell(\mathcal{B}')| \le 1 \quad \text{for all } \mathcal{B}, \mathcal{B}' \in \mathcal{L} \text{ with } \overline{\mathcal{B}} \cap \overline{\mathcal{B}'} \neq \emptyset, \tag{2.5}$$

where the closures may meet in a corner only and across a periodic seam. The code enforces (2.5) by refinement-only balancing: while a leaf touches a leaf that is more than one level finer, the coarser one is split. Balancing runs after the forest is built from the user's `refine` function and after every regrid. Figure 2.1 shows a forest in which balancing refined a root that `refine` did not mark. Contact is decided on the integer lattice of the block keys, scaled to the finer of the two levels and wrapped across periodic seams, so the balance does not depend on the rounding of block coordinates; a comparison of floating-point coordinates could miss blocks that touch when the domain extents are not exactly representable, and violated (2.5) there until issue 46 was fixed (PR #92). A periodic direction with exactly two level-0 cells is supported: the displacement does not inherit the sign tie of the half-period centre offset, because it takes its normal component from the mean cell width, positive by the face orientation, and only its tangential component from the wrap on that lattice (issue 36, issue 163; `test/mesh/topology.jl` ("periodic direction resolved by exactly two cells")).

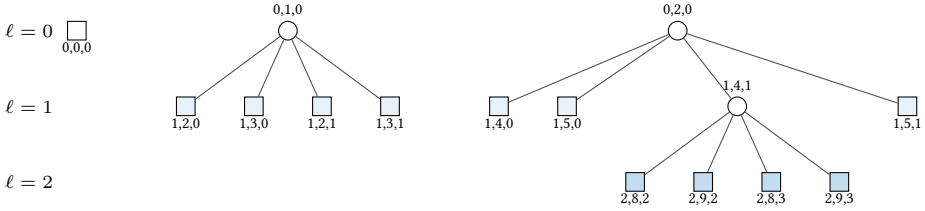
The thesis writes $h = 2^{-L} h_0$ for the width of the finest cells, with L the number of refinement levels [19, §2.5]. Here L is only a ceiling: the finest level present, L_{act}, may be lower, and widths are per direction. The thesis notation is recovered for $h_{0,1} = h_{0,2} = h_0$ and $L_{\mathrm{act}} = L$.

The leaves are stored in an array sorted by the key components (ℓ, j, i). A cell reference `CellRef(b, i, j)` uses the position `b` of its block in this array; `b` changes whenever the leaf set changes, whereas $\mathrm{key}(\mathcal{B})$ is a stable address.

> *Implementation.* `BlockTreeMesh` keywords `base_mesh` $= (M_1, M_2)$ (default: `block_size`), `block_size` $= (N_1, N_2)$ (default `(16, 16)`), `max_level` $= L$ (default 0) and `refine(key, origin, extent)`, called for every candidate block below L; `BlockKey`, `LeafBlock`, `CellRef`, `nblocks`, `ncells`, `cellcenter`, `isbalanced`. Internal: `_build_root_grid` (`src/mesh/topology/mesh.jl`), `_build_tree` (`src/mesh/topology/mesh.jl`), `_balance!` (`src/mesh/topology/mesh.jl`) (corner and periodic contact via



A. Leaf blocks $\mathcal{L}$ in Ω, labelled with their keys (ℓ, i, j); thin lines are cell boundaries.



B. The forest $\mathcal{T}$: three trees, one per root. Squares are leaves, circles refined blocks.

Figure 2.1. Forest of the code fixture `BlockTreeMesh(base_mesh=(6,2), block_size=(2,2), extent=(12,4), periodic=(true,false), max_level=2)` in which `refine` marks only the keys $(0, 2, 0)$ and $(1, 4, 1)$. Root $(0, 1, 0)$ is refined by the 2:1 balance (2.5), which also applies across corners: block $(1, 3, 0)$ touches the level-two block $(2, 8, 2)$ only at the marked vertex. Here $h_{0,k} = 2$, so the cell widths are 2, 1 and $1/2$. Drawn from `figures/generated/forest.tex`.

`_touch` (`src/mesh/topology/mesh.jl`)) and `_leaf_blocks` (`src/mesh/topology/mesh.jl`). Tests: `test/mesh/topology.jl` ("integer balance on non-dyadic extents"), `test/mesh/topology.jl` ("balance detects periodic seam violations").

2.3 Faces, hanging segments and stored unknowns

Faces are the pieces of cell sides shared by exactly one cell on either side. On a grid line normal to $\boldsymbol{e}_k$, every nonempty intersection of a side of a cell below the line with a side of a cell above it is an interior face $f \in \mathcal{F}_\Omega$, with

$$\begin{aligned} &c^-(f): \text{ the cell below}, \quad c^+(f): \text{ the cell above}, \quad \boldsymbol{n}_f = \boldsymbol{e}_k, \\ &|f|: \text{ the length of the intersection}, \quad \boldsymbol{x}_f: \text{ its midpoint}. \end{aligned} \tag{2.6}$$

In a periodic direction the lines $x_k = o_k$ and $x_k = o_k + X_k$ are one line: its faces (the seam) are interior faces with $c^-(f)$ the last and $c^+(f)$ the first cell in direction k, and $\boldsymbol{x}_f$ lies on $x_k = o_k$. A cell side on a wall is one boundary face $f \in \mathcal{F}_\Gamma$, with the single cell $c(f)$, $\boldsymbol{n}_f = \boldsymbol{e}_k$ and outward side $s_f = +1$ on $x_k = o_k + X_k$ and $s_f = -1$ on $x_k = o_k$. Boundary faces are never split. All faces are $\mathcal{F} = \mathcal{F}_\Omega \cup \mathcal{F}_\Gamma$.

Blocks are aligned with the lattice of their level and touching leaves differ by at most one level (2.5), so every cell side is one face or two. If $\bar{f}$ is the side of a coarse cell next to two cells one level finer,

$$\mathcal{H}(\bar{f}) = \{f_1, f_2\} \subset \mathcal{F}_\Omega, \qquad f_1 \cup f_2 = \bar{f}, \qquad |f_1| = |f_2| = \tfrac{1}{2}|\bar{f}|, \tag{2.7}$$

and the common endpoint of f_1 and f_2 is a *hanging vertex*. The side $\bar{f}$ only groups its segments: nothing is stored on it. A cell thus has between four and eight faces $\mathcal{F}(c)$. As in the thesis [19, §2.2.1], the orientation

$$o_{c,f} = \begin{cases} +1, & c = c^-(f), \\ -1, & c = c^+(f), \\ s_f, & f \in \mathcal{F}_\Gamma, \end{cases} \qquad \sum_{c \in \mathcal{C}(f)} o_{c,f} = 0 \quad \text{for } f \in \mathcal{F}_\Omega, \tag{2.8}$$

makes $o_{c,f}\boldsymbol{n}_f$ the outward normal of c on f.

Cell quantities are stored once per owned cell and face-normal quantities once per owned face (Figure 2.2). The stored face value is the coordinate component u_f of (3.9), also on boundary faces, where the outward component is $s_f u_f$. A quantity of the coarse side, such as the flux through $\bar{f}$, is the sum over $\mathcal{H}(\bar{f})$. Because the operators of Chapter 3 evaluate each segment flux once and add it to both adjacent cells with opposite orientation, and because all levels advance with the same time step, these operators need no flux correction between levels ("refluxing" in level-subcycled AMR [2]); none is implemented.

Implementation. `CellField`: one dense array of $(N_1 + 2H) \times (N_2 + 2H)$ values per leaf (owned cells and halo). `FaceField{k}`: the faces of $\mathcal{F}^{[k]}$, interior values in the order of the axis-k subsequence of `mesh.faces` and boundary values in that of `mesh.boundary_faces`. `FaceVelocity`: both components in those global orders. The order is fixed for one mesh generation and otherwise unspecified. `set_velocities!` samples a function at the face midpoints $\boldsymbol{x}_f$; `enforce_no_permeation!` sets every boundary value to zero. Internal: `_build_face_schedules` (`src/mesh/topology/mesh.jl`), `_match_sides!` (`src/mesh/topology/mesh.jl`) and `_push_boundary!` (`src/mesh/topology/mesh.jl`).

2.4 Halo slots

Around its owned cells each leaf block stores $H \geq 1$ layers of halo slots (default $H = 1$),

$$\mathcal{C}^{\mathrm{h}}(\mathcal{B}) = \{\, (\mathcal{B}, i, j) \mid 1 - H \leq i \leq N_1 + H,\ 1 - H \leq j \leq N_2 + H \,\} \setminus \mathcal{C}, \tag{2.9}$$

which continue the block's own lattice: a slot $\hat{c}$ has the width of the block's cells and centre $\boldsymbol{x}_{\hat{c}}$. Slots are not unknowns. `fill_halo!` sets every slot whose centre lies in Ω (after periodic wrapping) from owned cells,

$$\varphi_{\hat{c}} = \sum_{b \in \mathcal{D}(\hat{c})} w_{\hat{c},b}\,\varphi_b, \qquad w_{\hat{c},b} = \tfrac{1}{4}\#\{\, r \mid \boldsymbol{y}_{\hat{c},r} \in b \,\}, \qquad \boldsymbol{y}_{\hat{c},r} = \boldsymbol{x}_{\hat{c}} + \left(\pm\tfrac{1}{4}h_{\ell,1},\, \pm\tfrac{1}{4}h_{\ell,2}\right), \tag{2.10}$$

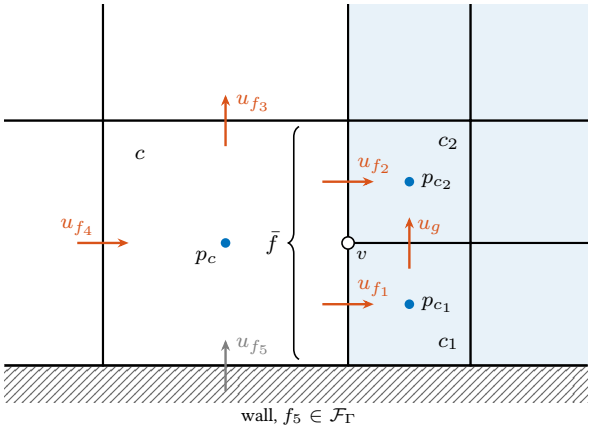


Figure 2.2. Owned faces and staggered unknowns at a 2:1 transition (the left part of Figure 2.1a, shading by level). The side $\bar{f}$ of the coarse cell c is not a face: it is partitioned into the owned segments $\mathcal{H}(\bar{f}) = \{f_1, f_2\}$, each with one velocity u_{f_r} and $c = c^-(f_r)$, $c_r = c^+(f_r)$. The cell c has the five faces $\mathcal{F}(c) = \{f_1, \dots, f_5\}$ with $o_{c,f_1} = o_{c,f_2} = o_{c,f_3} = +1$, $o_{c,f_4} = -1$ and, on the wall, $o_{c,f_5} = s_{f_5} = -1$ (grey: a prescribed boundary value, not an unknown). The fine face g between c_1 and c_2 ends at the hanging vertex v. Pressures (blue) are cell averages. Every arrow is a coordinate component $u_f = \boldsymbol{u} \cdot \boldsymbol{e}_{k(f)}$.

where $\ell = \ell(\mathcal{B})$, the four sample points are wrapped in periodic directions and $\mathcal{D}(\hat{c})$ are the leaf cells that contain them. If $H \le N_k/2$, every slot lies in a leaf that touches $\mathcal{B}$, so by (2.5) only three cases occur: a slot of a same-level neighbour copies that cell ($w_{\hat{c},b} = 1$); a slot inside a coarser cell copies the coarse value (piecewise-constant injection, $w_{\hat{c},b} = 1$); a slot covering four finer cells takes their mean ($w_{\hat{c},b} = \frac{1}{4}$), which is their area average. The fixture of Figure 2.1 has 56, 32 and 12 slots of these kinds, 44 physical slots and no other kind.

A slot whose centre lies beyond a wall is a physical slot. Its donor b is the leaf cell that contains $\Pi_{\bar{\Omega}}(\boldsymbol{x}_{\hat{c}})$, the slot centre wrapped in periodic directions and clamped to $\bar{\Omega}$, and its value is set by the boundary rule passed to `fill_halo!`,

$$\varphi_{\hat{c}} = \varphi_b \quad (\texttt{ZeroGradient}), \qquad \varphi_{\hat{c}} = \varphi^\Gamma(\boldsymbol{x}_{\hat{c}}, t) \quad (\texttt{Dirichlet}), \tag{2.11}$$

with φ^Γ the user function; a Dirichlet value is thus imposed at the slot centre, half a cell beyond the wall, not on the boundary face. Whether this is the intended semantics is under investigation (issue 43).

The rule (2.10) reproduces constants and is exact for the average of finer cells, but its coarse-to-fine values are piecewise constant, i.e. first-order accurate. The divergence, gradient and elliptic operators of Chapter 3 read owned cells through the face lists and never use halo slots. Halos are read by, among others, the regrid transfer, the interpolation-error refinement indicator, interface reconstruction and VOF transport; their chapters state how.

> *Implementation.* `fill_halo!(field, boundary=ZeroGradient(); time=0)` with `ZeroGradient` or `Dirichlet(value)`, where `value(x, y, time)` is φ^Γ; halo width: keyword `halo` of `BlockTreeMesh`. Schedule (built with the mesh and after every

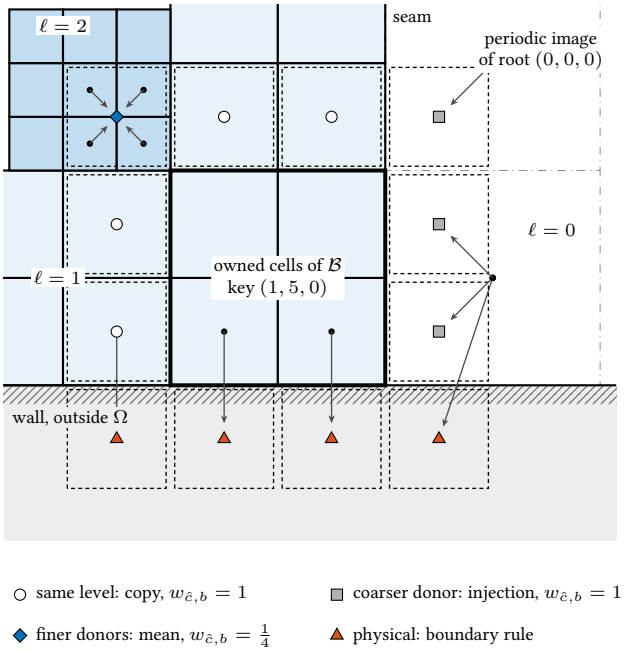


Figure 2.3. Halo of width $H = 1$ around the level-one block with key $(1, 5, 0)$ of Figure 2.1 (dashed: its twelve slots $\mathcal{C}^{\mathrm{h}}(\mathcal{B})$). An arrow runs from each donor centroid to the slot it fills by (2.10); a same-level slot coincides with its donor. On the right the slots lie in the periodic image of a level-zero root and copy its single coarse value; the corner slot at the top left averages the four level-two cells that contain its quarter-cell sample points; the four slots below the wall are set by the boundary rule of (2.11), which for `ZeroGradient` copies the marked donor, the cell that contains the clamped slot centre.

regrid): `_build_halo_schedule` (`src/mesh/topology/mesh.jl`), entries `HaloEntry` (`src/mesh/topology/mesh.jl`) with donors and weights.

2.5 Regrid and schedule rebuild

The forest changes only in `adapt!`, which receives one label per leaf (refine, keep or coarsen) from the refinement indicators of Chapter 12. Every leaf labelled *refine* with $\ell(\mathcal{B}) < L$ is split. Four sibling leaves are merged into their parent if all four are labelled *coarsen*, none of them was created in the last `coarsen_delay` steps, and the parent's level is at least `minimum_level`. Refinement-only balancing then restores (2.5); it splits a merged parent again where needed.

If the leaf set changed, every derived structure is rebuilt from the forest: the leaf array and block positions, the face lists $\mathcal{F}_\Omega$ and $\mathcal{F}_\Gamma$ with their order, the dual volumes of Section 3.2 and the halo schedule. Registered fields are transferred to the new cells and faces by the maps of Chapter 12, and topology-dependent operators (Section 3.8) are rebuilt. In `step!(flow, dt)` the forest is adapted after the physical step, at most every `update_interval` steps; time discretisation is not applicable to the mesh itself.

Implementation. `adapt!`, `AMRWorkspace`, `MeshAdaptationOptions` (`update_interval=1`, `coarsen_delay=4`, `minimum_level=0`). Tree edits: `_apply_refinement_actions!` (`src/mesh/adaptivity/workspace.jl`) and `_can_coarsen` (`src/mesh/adaptivity/workspace.jl`); order of the regrid: `_adapt_with_labels!` (`src/mesh/adaptivity/adapt.jl`); rebuild: `_rebuild_mesh_schedules!` (`src/mesh/adaptivity/field_transfer.jl`); call after the physical step in `step!` (`src/navier_stokes/flow_simulation.jl`), followed by `_rebuild_topology_dependent_state!` (`src/navier_stokes/workspace.jl`).

— 3 —

Discrete setting on the composite mesh

Equation reference, audited against commit `1697df9`.

This chapter defines the measures, function spaces and compatible operators on the cells and faces of Chapter 2: the primary divergence, the gradient derived from it, and the elliptic operator built from both. It follows the thesis' discrete setting for conforming meshes [19, §2.2] and states what changes on the composite mesh: owned face segments, unequal half-cell control volumes and a non-diagonal face metric at level transitions, and the open measures of cells cut by an embedded solid. All objects here are static algebraic maps, so time discretisation is not applicable; Section 3.8 states when they are rebuilt.

3.1 Continuous setting

A fixed solid $\Omega^{\mathrm{s}} \subset \Omega$ may be embedded; the part open for flow is $\Omega^{\mathrm{o}} = \Omega \setminus \Omega^{\mathrm{s}}$, and $\Omega^{\mathrm{o}} = \Omega$ without a solid [19, §2.4]. For a velocity $\boldsymbol{u}$ with $\boldsymbol{u} \cdot \boldsymbol{n} = 0$ on the fixed solid boundary, the divergence theorem on the open part $c^{\mathrm{o}} = c \cap \Omega^{\mathrm{o}}$ of a cell reads

$$\int_{c^{\mathrm{o}}} \nabla \cdot \boldsymbol{u} \,\mathrm{d}V = \sum_{f \in \mathcal{F}(c)} \int_{f^{\mathrm{o}}} \boldsymbol{u} \cdot o_{c,f} \boldsymbol{n}_f \,\mathrm{d}S, \tag{3.1}$$

and divergence and gradient are negative adjoints,

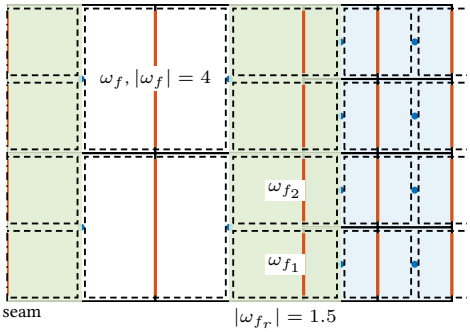
$$\int_{\Omega^{\mathrm{o}}} p \,\nabla \cdot \boldsymbol{u} \,\mathrm{d}V = -\int_{\Omega^{\mathrm{o}}} \boldsymbol{u} \cdot \nabla p \,\mathrm{d}V + \int_{\Gamma} p \,\boldsymbol{u} \cdot \boldsymbol{n} \,\mathrm{d}S, \tag{3.2}$$

with periodic contributions cancelling. The discrete divergence of Section 3.5 satisfies the analogue of (3.1) exactly, and the gradient of Section 3.6 is defined so that the analogue of (3.2) holds in a stated inner product.

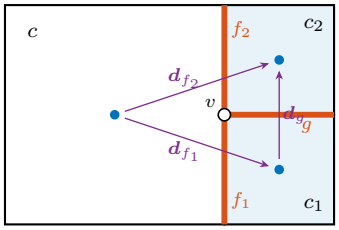
3.2 Measures and staggered control volumes

Cells and faces carry the measures $|c|$, $|f|$ and centroids $\boldsymbol{x}_c$, $\boldsymbol{x}_f$ of Chapter 2. As in the thesis, the staggered control volume ω_f of an interior face is the extrusion of f along $\boldsymbol{n}_f$ to the centroids of its two cells [19, §2.2.1]. On the composite mesh this is the union of two strips of half a cell width, one in each cell, so

$$|\omega_f| = |f| \, h_f, \qquad h_f = \tfrac{1}{2}\left(h_{c^-(f),k} + h_{c^+(f),k}\right) = \boldsymbol{n}_f \cdot \boldsymbol{d}_f, \qquad f \in \mathcal{F}_\Omega \cap \mathcal{F}^{[k]}, \tag{3.3}$$



A. Control volumes ω_f of the faces $f \in \mathcal{F}^{[1]}$ (dashed; red: the face). Green: unequal half-cell strips at level transitions.



B. Vertex patch $\mathcal{F}(v) = \{f_1, f_2, g\}$ of the hanging vertex v with the displacements $\boldsymbol{d}_f$ that enter Υ_v.

FIGURE 3.1. Staggered control volumes (3.3) and the vertex patch (3.21) on the fixture of Figure 2.1 (window $0 \le x_1 \le 6.2$; $h_{0,k} = 2$, $h_{1,k} = 1$). The volumes ω_f of one axis tile the domain; next to the 2:1 transition each consists of half a coarse and half a fine cell width. The seam faces at $x_1 = 0$ are periodic.

where $\boldsymbol{d}_f = \boldsymbol{x}_{c^+(f)} - \boldsymbol{x}_{c^-(f)}$ is taken between the nearest periodic images. Every cell side contributes the half-strips of its segments, so the control volumes of one axis tile the domain up to half strips at the walls:

$$\sum_{f \in \mathcal{F}_\Omega \cap \mathcal{F}^{[k]}} |\omega_f| = |\Omega| - \tfrac{1}{2} \sum_{f \in \mathcal{F}_\Gamma \cap \mathcal{F}^{[k]}} |f|\, h_{c(f),k}. \tag{3.4}$$

Boundary faces have no staggered control volume: their face values are boundary data, not unknowns.

Deviation from the thesis 3.1 (Staggered volume at a level transition). The thesis defines $|\omega_f| = \frac{1}{2}\sum_{c \in \mathcal{C}(f)} |c|$ and $h_f = |f|/|\omega_f|$ [19, eq. (2.15)]. The code uses the extrusion (3.3). Both agree on conforming meshes, where $|c| = |f| h_{c,k}$, but not at a 2:1 segment: in Figure 3.1 the thesis formula gives $\frac{1}{2}(|c| + |c_1|) = \frac{5}{2}$ for f_1, the code $|\omega_{f_1}| = 1.5$. The thesis' $h_f = |f|/|\omega_f|$ is a typographical error (confirmed by the author) for $|\omega_f|/|f|$, the face-projected centroid distance that the thesis describes.

Implementation. `mesh.face_dual_volumes` (uncut, in the order of `mesh.faces`): `_face_dual_volumes` (`src/mesh/topology/mesh.jl`); displacements `_face_displacement` (`src/operators/gradient.jl`). Rebuilt with the face lists after every regrid. The displacement is evaluated on the level lattice of Section 2.2, not as the difference of the two centroids: its normal component is the mean cell width of (3.3) and its tangential component twice the centre offset in finest cells, an integer, scaled by the finest level spacing. A centroid difference is a small difference of large numbers and loses the offset when the domain sits far from the origin (at origin 10^{12} a 0.15-wide cell keeps only $\approx 10^{-3}$ of its offset), whereas the lattice form is exact: the operator of a translated mesh is bitwise the operator of the untranslated one (issue 163; `test/operators/gradient.jl` ("face displacement is independent of the mesh origin")).

3.3 Open (cut-cell) measures

With an embedded solid, every cell carries the solid fraction $\alpha_c^{\mathrm{s}} \in [0,1]$ of the solid's reconstruction (one half-plane per cut cell, Chapter 7) and the open fraction

$$\alpha_c^{\mathrm{o}} = 1 - \alpha_c^{\mathrm{s}}, \qquad |c^{\mathrm{o}}| = \alpha_c^{\mathrm{o}}|c|; \tag{3.5}$$

cells with $\alpha_c^{\mathrm{o}} = 0$ are *covered*. Let $a_{c,f}^{\mathrm{o}}$ be the fraction of the segment f that is open according to the half-plane of c (1 for an uncut cell, 0 for a covered one). Each face carries one shared aperture,

$$a_f^{\mathrm{o}} = \begin{cases} 0, & \text{a cell of } \mathcal{C}(f) \text{ is covered,} \\ \min\left\{1, \tfrac{1}{2}\left(a_{c^-(f),f}^{\mathrm{o}} + a_{c^+(f),f}^{\mathrm{o}}\right)\right\}, & \text{otherwise,} \end{cases} \qquad f \in \mathcal{F}_\Omega, \tag{3.6}$$

and $a_f^{\mathrm{o}} = a_{c(f),f}^{\mathrm{o}}$ on $f \in \mathcal{F}_\Gamma$. One value per face is what makes the open divergence below conservative; the zero for a covered neighbour prevents a spurious opening where a solid boundary coincides with a grid line. With $\Lambda_{c,f}$ the strip of c that spans the full width of c normal to f and the tangential extent of f, the open staggered volume bisects every open line segment in the normal direction [31]:

$$|\omega_f^{\mathrm{o}}| = \tfrac{1}{2} \sum_{c \in \mathcal{C}(f)} |\Lambda_{c,f} \cap c^{\mathrm{o}}|, \qquad f \in \mathcal{F}_\Omega. \tag{3.7}$$

It reduces to $|\omega_f|$ in uncut cells but is in general not $a_f^{\mathrm{o}}|\omega_f|$. The thesis introduces c^{o}, f^{o}, α_c^{o} and a_f^{o} [19, §2.4]; (3.6) and (3.7) are the code's definitions.

> *Implementation.* `GeometryField` (`open_fraction`, `aperture`, `boundary_aperture`, `face_dual_volumes`), rebuilt by `refresh_geometry!`. Internal: `_cell_face_open_fraction` (`src/embedded/geometry.jl`), `_interior_open_aperture` (`src/embedded/geometry.jl`) and `_geometry_dual_volumes` (`src/embedded/geometry.jl`).

3.4 Function spaces and discrete integration

As in the thesis [19, §2.2.2], cell functions $p \in \vec{\mathcal{C}}$ and face functions $u \in \vec{\mathcal{F}}$ are averages,

$$p_c = \frac{1}{|c|} \int_c p \,\mathrm{d}V, \qquad c \in \mathcal{C}, \tag{3.8}$$

$$u_f = \frac{1}{|f|} \int_f \boldsymbol{u} \cdot \boldsymbol{n}_f \,\mathrm{d}S, \qquad f \in \mathcal{F}, \tag{3.9}$$

now on every owned segment f. Cell functions are integrated by

$$\sum_{\mathcal{C}} p = \sum_{c \in \mathcal{C}} |c|\, p_c \tag{3.10}$$

[19, eq. (2.20)], with $|c^{\mathrm{o}}|$ in place of $|c|$ on cut meshes. Face functions are paired by the inner product $\langle u, w \rangle_{\mathsf{K}^{-1}} = u^{\mathsf{T}} \mathsf{K}^{-1} w$ over $\mathcal{F}_\Omega$ with the inverse face metric K of Section 3.6. Where K is diagonal it equals $\sum_f |\omega_f| u_f w_f$, the thesis' face integral [19, eq. (2.21)]; at level transitions it couples neighbouring segments.

Implementation. `CellField` (with `integrate` for (3.10)), `FaceField`, `FaceVelocity`. Initialisation by `initialize!` or `set_velocities!` samples centroid and face midpoint values, i.e. second-order approximations of the averages.

3.5 Divergence

The primary divergence $\mathfrak{D} : \vec{\mathcal{F}} \to \vec{\mathcal{C}}$ is the thesis' operator [19, eq. (2.22)] summed over owned segments:

$$|c|\,(\mathfrak{D}u)_c = \sum_{f\in\mathcal{F}(c)} o_{c,f}\,|f|\,u_f. \tag{3.11}$$

For face averages (3.9) it equals $\int_{\partial c} \boldsymbol{u}\cdot\boldsymbol{n}\,\mathrm{d}S$ exactly. Interior faces enter two cells with opposite orientation (2.8), so

$$\sum_{\mathcal{C}} \mathfrak{D}u = \sum_{f\in\mathcal{F}_\Gamma} s_f\,|f|\,u_f, \tag{3.12}$$

the discrete divergence theorem [19, eq. (2.23)]. With an embedded solid, the open divergence is

$$|c^{\mathrm{o}}|\,(\mathfrak{D}^{\mathrm{o}}u)_c = \sum_{f\in\mathcal{F}(c)} o_{c,f}\,|f|\,a_f^{\mathrm{o}}\,u_f \quad \text{if } \alpha_c^{\mathrm{o}}>0, \qquad (\mathfrak{D}^{\mathrm{o}}u)_c = 0 \quad \text{if } \alpha_c^{\mathrm{o}}=0. \tag{3.13}$$

A face next to a covered cell has $a_f^{\mathrm{o}}=0$, so (3.12) holds for $\mathfrak{D}^{\mathrm{o}}$ with $|c^{\mathrm{o}}|$ and $|f|a_f^{\mathrm{o}}$. In uncut cells $\mathfrak{D}^{\mathrm{o}}=\mathfrak{D}$.

3.5.1 Worked sample: a hanging face

Let c be the coarse cell and c_1, c_2 the fine cells of Figure 2.2, with $c = c^-(f_r)$ and $c_r = c^+(f_r)$ for the segments $\mathcal{H}(\bar f) = \{f_1, f_2\}$. The segments contribute

$$|c^{\mathrm{o}}|\,(\mathfrak{D}^{\mathrm{o}}u)_c\,\big|_{\bar f} = |f_1|a_{f_1}^{\mathrm{o}}u_{f_1} + |f_2|a_{f_2}^{\mathrm{o}}u_{f_2}, \tag{3.14}$$

$$|c_r^{\mathrm{o}}|\,(\mathfrak{D}^{\mathrm{o}}u)_{c_r}\,\big|_{f_r} = -|f_r|a_{f_r}^{\mathrm{o}}u_{f_r}, \qquad r=1,2, \tag{3.15}$$

so the three integrated contributions cancel exactly, up to rounding. In the uncut case $a_{f_r}^{\mathrm{o}} = \alpha_c^{\mathrm{o}} = \alpha_{c_r}^{\mathrm{o}} = 1$ with $|c|=4$, $|f_r|=|c_r|=1$, $u_{f_1}=2$ and $u_{f_2}=-1$, the coarse cell receives $(\mathfrak{D}u)_c = \frac{2-1}{4} = \frac{1}{4}$ and the fine cells receive -2 and $+1$; the integrated contributions 1, -2 and 1 sum to zero.

Code check. On the fixture of Figure 2.1, whose cells c, c_1, c_2 have exactly these sizes, `divergence!` applied to the face velocity that is zero except $u_{f_1}=2$, $u_{f_2}=-1$ returns $(\mathfrak{D}u)_c = 0.25$, $(\mathfrak{D}u)_{c_1} = -2$, $(\mathfrak{D}u)_{c_2} = 1$, zero in all other cells and $\sum_{\mathcal{C}}\mathfrak{D}u = 0$ exactly. With a solid disc that cuts c and 2 hanging segments and an arbitrary interior velocity, the open divergence gives $|\sum_c |c^{\mathrm{o}}|(\mathfrak{D}^{\mathrm{o}}u)_c| = 4.4\times10^{-16}$, against $\sum_c ||c^{\mathrm{o}}|(\mathfrak{D}^{\mathrm{o}}u)_c| = 4.6\times10^{1}$ (`figures/generate.jl`). The sample involves no pressure gradient: on the composite mesh the gradient at f_r is not the two-point difference of p_c and p_{c_r} (Section 3.6).

Implementation. `divergence!(destination, velocity)` and, with open apertures, `divergence!(destination, velocity, geometry)`, implemented in `divergence!` (`src/operators/advection/cell_advection.jl`) and

`divergence!` (`src/embedded/operators.jl`). Boundary values enter with `face.side`, i.e. $s_f u_f$.

3.6 Gradient

Let J map cell values to integrated jumps on interior faces,

$$(\mathsf{J}p)_f = |f|\left(p_{c^+(f)} - p_{c^-(f)}\right) = -|f| \sum_{c \in \mathcal{C}(f)} o_{c,f}\, p_c, \qquad f \in \mathcal{F}_\Omega, \tag{3.16}$$

so that $|c|(\mathfrak{D}u)_c = -(\mathsf{J}^\mathsf{T}u)_c$ for velocities that vanish on $\mathcal{F}_\Gamma$. The gradient $\mathfrak{S} : \vec{\mathcal{C}} \to \vec{\mathcal{F}}$ is

$$(\mathfrak{S}p)_f = (\mathsf{K}\mathsf{J}p)_f \quad (f \in \mathcal{F}_\Omega), \qquad (\mathfrak{S}p)_f = 0 \quad (f \in \mathcal{F}_\Gamma), \tag{3.17}$$

with a sparse symmetric positive definite inverse face metric K, defined below. By construction it is the negative adjoint of the divergence in the inner products of Section 3.4:

$$\sum_{\mathcal{C}} p\, \mathfrak{D}u = -\langle u, \mathfrak{S}p\rangle_{\mathsf{K}^{-1}} + \sum_{f \in \mathcal{F}_\Gamma} s_f\, |f|\, u_f\, p_{c(f)}, \tag{3.18}$$

the analogue of (3.2) and of the thesis' $\mathfrak{S} = -\mathfrak{D}^\mathsf{T}$ [19, eq. (2.24)]. On a conforming patch K is diagonal with entries $1/|\omega_f|$, which turns (3.17) into the thesis' $|\omega_f|(\mathfrak{S}p)_f = -|f|\sum_c o_{c,f} p_c$ [19, eq. (2.25)]. The zero boundary value is a homogeneous Neumann condition; periodic seams are interior faces.

The metric is assembled from vertex patches. Every interior face has two endpoints on the finest lattice; with $\mathcal{F}(v)$ the interior faces that end in the vertex v,

$$\mathsf{K} = \sum_{v \in \mathcal{V}} \mathsf{K}_v, \qquad \mathsf{K}_v \text{ acting on the faces of } \mathcal{F}(v). \tag{3.19}$$

If v lies on a wall, or $\mathcal{F}(v)$ has at most two faces, the patch contributes half of the two-point metric,

$$\mathsf{K}_v = \operatorname{diag}\left(1/(2|\omega_f|)\right)_{f \in \mathcal{F}(v)}. \tag{3.20}$$

Otherwise, with the rows $|f|\boldsymbol{d}_f^\mathsf{T}$ of Y_v and $\frac{1}{2}\boldsymbol{e}_{k(f)}^\mathsf{T}$ of N_v for $f \in \mathcal{F}(v)$,

$$\mathsf{K}_v = \mathsf{N}_v\, \operatorname{sym}\left(\mathsf{Y}_v^\mathsf{T}\mathsf{N}_v\right)^{-1} \mathsf{N}_v^\mathsf{T} + \tau_v\, \mathsf{P}_v, \qquad \mathsf{P}_v = \mathsf{I} - \mathsf{Y}_v\left(\mathsf{Y}_v^\mathsf{T}\mathsf{Y}_v\right)^{-1}\mathsf{Y}_v^\mathsf{T}, \tag{3.21}$$

with $\operatorname{sym}(A) = \frac{1}{2}(A + A^\mathsf{T})$ and τ_v the trace of the first term divided by $\#\mathcal{F}(v) - 2$. For an affine pressure, $\mathsf{J}p = \mathsf{Y}_v\nabla p$ on the patch, and if $\mathsf{Y}_v^\mathsf{T}\mathsf{N}_v$ is symmetric then $\mathsf{K}_v\mathsf{Y}_v = \mathsf{N}_v$: each endpoint contributes half of $\partial_k p$. Hence $(\mathfrak{S}p)_f = \partial_k p$ for affine p on every face whose two endpoint patches are of type (3.21) with a symmetric moment tensor. The second term only acts on the patch modes that the affine condition leaves free and makes K_v positive definite; on a regular Cartesian patch the sum is again the diagonal of (3.20).

For the hanging vertex v of Figure 3.1b, $\mathcal{F}(v) = \{f_1, f_2, g\}$ with unit lengths and displacements $\boldsymbol{d}_{f_1} = (\frac{3}{2}, -\frac{1}{2})$, $\boldsymbol{d}_{f_2} = (\frac{3}{2}, \frac{1}{2})$, $\boldsymbol{d}_g = (0, 1)$ (fine cell width one). Then $\mathsf{Y}_v^\mathsf{T}\mathsf{N}_v = \operatorname{diag}\left(\frac{3}{2}, \frac{1}{2}\right)$ is symmetric, $\tau_v = \frac{5}{6}$, and the code returns, in the order (f_1, f_2, g),

$$\mathsf{K}_v = \begin{pmatrix} \frac{4}{9} & -\frac{1}{9} & \frac{5}{18} \\ -\frac{1}{9} & \frac{4}{9} & -\frac{5}{18} \\ \frac{5}{18} & -\frac{5}{18} & \frac{7}{9} \end{pmatrix} \tag{3.22}$$

instead of the two-point value $1/(2|\omega_{f_1}|) = \frac{1}{3}$ on the diagonal: the gradient at f_1 involves p_{c_2} as well.

Checks (`figures/generate.jl`). On the fixture, K has 144 off-diagonal entries for 90 interior faces and smallest eigenvalue 2.4×10^{-1}. On a wall-bounded mesh with refinement corners and square cells, all 121 patches of type (3.21) have a symmetric moment tensor, $\mathsf{K}_v\mathsf{Y}_v = \mathsf{N}_v$ holds to 1.1×10^{-16}, and the gradient of $p = 0.3x_1 - 0.7x_2$ is exact to 1.1×10^{-15} on the faces with two such patches. Stretched to $h_{\ell,1} = 2h_{\ell,2}$, the same mesh has 8 patches with a non-symmetric tensor, all at convex corners of refined blocks; there $\max|\mathsf{K}_v\mathsf{Y}_v - \mathsf{N}_v| = 3.5 \times 10^{-2}$ and the gradient error reaches 2.5×10^{-2}. The operator remains symmetric positive definite and adjoint to $\mathfrak{D}$; the symmetrisation in (3.21) gives up affine exactness only at such patches.

On square cells the tensor is symmetric on every patch of a mesh that satisfies (2.5) (only contacts across faces are used). Its off-diagonal entries $\frac{1}{2}\sum_{k(f)=2}|f|(\boldsymbol{d}_f)_1$ and $\frac{1}{2}\sum_{k(f)=1}|f|(\boldsymbol{d}_f)_2$ collect the tangential displacements, which are non-zero only on faces between cells of different levels. A vertex that is not on a wall has $\#\mathcal{F}(v) \geq 3$ and is one of two kinds. At a hanging vertex the two coarse–fine faces have the same measure h and tangential displacements $\pm\frac{h}{2}$, and the face between the two fine cells has none, so $\mathsf{Y}_v^\mathsf{T}\mathsf{N}_v = h^2\,\mathrm{diag}\left(\frac{3}{2},\frac{1}{2}\right)$ (or $h^2\,\mathrm{diag}\left(\frac{1}{2},\frac{3}{2}\right)$ for a horizontal interface). Otherwise four cells with widths $s_{\mathrm{NE}}, s_{\mathrm{NW}}, s_{\mathrm{SW}}, s_{\mathrm{SE}} \in \{h, 2h\}$ have a corner at v; the face on each of the four half-lines from v joins cells of widths a, b with $a/b \in \{\frac{1}{2}, 1, 2\}$, has measure $\min(a,b)$ and tangential displacement $\frac{1}{2}(a-b)$, and $\frac{1}{2}\min(a,b)\cdot\frac{1}{2}(a-b) = \frac{1}{12}(a^2-b^2)$. Hence

$$12\,(\mathsf{Y}_v^\mathsf{T}\mathsf{N}_v)_{12} = (s_{\mathrm{NE}}^2 - s_{\mathrm{SE}}^2) + (s_{\mathrm{SW}}^2 - s_{\mathrm{NW}}^2) = (s_{\mathrm{NE}}^2 - s_{\mathrm{NW}}^2) + (s_{\mathrm{SW}}^2 - s_{\mathrm{SE}}^2) = 12\,(\mathsf{Y}_v^\mathsf{T}\mathsf{N}_v)_{21}.$$

The eigenvalues of $\mathsf{Y}_v^\mathsf{T}\mathsf{N}_v$ lie in $[\frac{1}{2}h^2, 3h^2]$ (h the smallest cell width at v), so the positive-definiteness check never fails, the symmetrisation changes nothing and $\mathsf{K}_v\mathsf{Y}_v = \mathsf{N}_v$ holds exactly. For cells with aspect ratio $r = h_{\ell,1}/h_{\ell,2}$ the two off-diagonal entries differ by $\frac{r^2-1}{12}\left(s_{\mathrm{NE}}^2 + s_{\mathrm{SW}}^2 - s_{\mathrm{NW}}^2 - s_{\mathrm{SE}}^2\right)$ (s the cell heights), which is non-zero exactly at corners whose two diagonals carry unequally many coarse cells; hanging vertices and corners with two adjacent coarse cells stay symmetric. A single face between cells that differ by two levels already breaks the symmetry for square cells. This statement and its sharpness were verified symbolically for all vertex configurations and numerically against the code (`proof_review/eq-setting-patch-metric/`).

Deviation from the thesis 3.2 (Pressure gradient at level transitions). The thesis' gradient [19, eq. (2.25)] uses the diagonal metric $1/|\omega_f|$ and defers refinement interfaces to Van der Plas [28]. The code keeps the diagonal metric on conforming patches but couples the segments of a vertex patch at level transitions through (3.21), which reproduces affine pressures and keeps $\mathfrak{S} = -\mathfrak{D}^\mathsf{T}$. Compatible discrete operators of this kind are analysed by Bonelle and Ern [3]; the patch formula itself is the package's.

Implementation. `GradientOperator(mesh)` or `(pressure)`, `gradient!`. Internal: the jump (3.16) `_pressure_jump!` (`src/operators/gradient.jl`); the metric `_assemble_inverse_metric` (`src/operators/gradient.jl`), with vertex patches keyed by `_vertex_key` (`src/operators/gradient.jl`), `_append_boundary_patch_metric!` (`src/operators/gradient.jl`) for (3.20) and `_local_inverse_metric` (`src/operators/gradient.jl`) for (3.21) (it throws if $\mathrm{sym}\left(\mathsf{Y}_v^\mathsf{T}\mathsf{N}_v\right)$ is not positive definite); patch matrices are cached by normalised geometry in

`GradientConstructionCache` (`src/operators/gradient.jl`), and a diagonal K is stored as a vector.

3.7 Weighted gradient and elliptic operator

A non-negative face coefficient γ_f, $f \in \mathcal{F}_\Omega$, is given as a number, a vector, a function of $(\boldsymbol{x}_f, k(f))$, or interpolated from positive cell values with the distance-weighted harmonic mean

$$(\mathfrak{I}^{\mathrm{h}}\gamma)_f = \frac{h_{c^-(f),k} + h_{c^+(f),k}}{h_{c^-(f),k}/\gamma_{c^-(f)} + h_{c^+(f),k}/\gamma_{c^+(f)}}, \qquad k = k(f). \tag{3.23}$$

With $\mathsf{G} = \operatorname{diag}(\gamma)$, the weighted gradient and the elliptic operator are

$$\mathfrak{S}_\gamma p = \mathsf{G}^{1/2}\mathsf{K}\,\mathsf{G}^{1/2}\mathsf{J}p \ \text{ on } \mathcal{F}_\Omega, \qquad (\mathfrak{S}_\gamma p)_f = 0 \ \text{ on } \mathcal{F}_\Gamma, \tag{3.24}$$

$$\mathfrak{L}p = r\,p - \vartheta\,\mathfrak{D}\,(\mathfrak{S}_\gamma p), \qquad r \geq 0,\ \vartheta > 0. \tag{3.25}$$

Where K is diagonal, $(\mathfrak{S}_\gamma p)_f = \gamma_f(\mathfrak{S}p)_f$; at level transitions the symmetric placement of $\mathsf{G}^{1/2}$ keeps the adjointness. The code stores the integrated form

$$\mathsf{V}\,\mathfrak{L} = r\,\mathsf{V} + \vartheta\,\mathsf{S}, \qquad \mathsf{S} = \mathsf{J}^{\mathsf{T}}\mathsf{G}^{1/2}\mathsf{K}\,\mathsf{G}^{1/2}\mathsf{J}, \qquad \mathsf{V} = \operatorname{diag}(|c|), \tag{3.26}$$

with S symmetric positive semidefinite and $\mathsf{S}\mathbf{1} = 0$. Walls therefore carry a homogeneous Neumann condition and periodic seams none; for $r = 0$ the constants are in the null space, which the linear solver handles (Chapter 11). `PoissonOperator` is the case $r = 0$, $\vartheta = 1$.

Cut cells. For an embedded solid, the cut-cell Poisson operator uses $\mathsf{V} = \operatorname{diag}(|c^{\mathrm{o}}|)$, the coefficient $\gamma_f = \gamma_f^{\mathrm{m}}(a_f^{\mathrm{o}})^2$ with a material part γ_f^{m}, and the metric WKW with $\mathsf{W} = \operatorname{diag}\left((|\omega_f|/|\omega_f^{\mathrm{o}}|)^{1/2}\right)$, where a vanishing $|\omega_f^{\mathrm{o}}|$ is replaced by the number one; this replacement depends on the units of length and is under investigation (issue 42). Its face acceleration is

$$(\mathfrak{S}^{\mathrm{o}}p)_f = \frac{\gamma_f^{1/2}}{a_f^{\mathrm{o}}}\left(\mathsf{WKW}\,\mathsf{G}^{1/2}\mathsf{J}p\right)_f \quad \text{if } a_f^{\mathrm{o}} > 0, \qquad (\mathfrak{S}^{\mathrm{o}}p)_f = 0 \quad \text{otherwise}, \tag{3.27}$$

so that $-\mathsf{V}\mathfrak{D}^{\mathrm{o}}\mathfrak{S}^{\mathrm{o}}p = \mathsf{J}^{\mathsf{T}}\mathsf{G}^{1/2}\mathsf{WKW}\,\mathsf{G}^{1/2}\mathsf{J}p$ is again symmetric. Where K is diagonal, $(\mathfrak{S}^{\mathrm{o}}p)_f = \gamma_f^{\mathrm{m}}a_f^{\mathrm{o}}|f|(p_{c^+(f)} - p_{c^-(f)})/|\omega_f^{\mathrm{o}}|$. Covered cells are inactive: their rows and columns are replaced by those of the identity, so they decouple.

Implementation. `EllipticOperator(mesh; reaction, diffusion, face_coefficient, volumes, active_cells, inactive_condition)` with `reaction` $= r$ (default 0), `diffusion` $= \vartheta$ (default 1); `PoissonOperator`, `harmonic_face_coefficients`, `weighted_gradient!`, `apply!`, `EllipticWorkspace`. Assembly `_integrated_stiffness` (`src/operators/elliptic.jl`) and `_impose_inactive_constraints!` (`src/operators/elliptic.jl`). Cut cells: `GradientOperator(geometry)`, `PoissonOperator(geometry)` and `_cut_cell_weighted_gradient!` (`src/embedded/operators.jl`).

3.8 Time discretisation and rebuild

Time discretisation is not applicable: the operators of this chapter are algebraic maps on the current mesh and geometry. They are rebuilt when their data change. After a regrid (Section 2.5) the face lists, dual volumes and K are new; equal vertex patches are computed once, because local matrices are cached by their normalised geometry. When the solid's reconstruction changes, `refresh_geometry!` recomputes the open measures of Section 3.3 and the cut-cell metric follows. When only the face coefficient changes on a fixed mesh, as for a two-phase pressure coefficient, only S is refreshed. The pressure projection (Chapter 10), the implicit viscous step (Chapter 9) and the linear solvers (Chapter 11) use these operators.

— 4 —

Interface representation, reconstruction and curvature

Equation reference, audited against commit 8d58545.

The interface between liquid and gas is represented by cell-averaged volume fractions; its geometry is reconstructed from them, cell by cell, as one straight line (PLIC) together with a curvature estimate from height functions. This chapter states how the fractions are defined and stored, how the line, its normal and its plane constant are computed, how the curvature and its fallbacks are chosen, how contact angles enter, and how fractions are initialised. It follows the thesis' interface description [19, §2.3], reconstruction [19, §3.3] and curvature [19, §4.7] and states where the code differs. Reconstruction is an algebraic map from fractions to geometry; time discretisation is not applicable, and Section 4.10 states when the map is evaluated. Transport of the fractions is the subject of Chapter 5; the embedded solid of Chapter 7 enters through the open cell parts.

4.1 Continuous description

The open part Ω^{o} of the domain (Section 3.1) is split into the liquid region $\Omega^{\mathrm{l}}(t)$ and the gas region $\Omega^{\mathrm{g}}(t) = \Omega^{\mathrm{o}} \setminus \Omega^{\mathrm{l}}(t)$; the interface $I(t)$ is their common boundary. Its unit normal $\boldsymbol{\eta}$ points into the gas and its curvature is

$$\kappa = \nabla \cdot \boldsymbol{\eta}, \tag{4.1}$$

so a convex liquid drop has positive curvature [19, eq. (2.6)]. Where I is locally a graph over the coordinate x_t with height in the direction $k \neq t$, $x_k = H^{\pm k}(x_t)$, and the normal has a positive $(+)$ or negative $(-)$ $\boldsymbol{e}_k$-component [19, §2.3.1],

$$\boldsymbol{\eta} = \frac{\pm \boldsymbol{e}_k \mp H' \, \boldsymbol{e}_t}{\sqrt{1 + (H')^2}}, \qquad \kappa = \mp \frac{H''}{\left(1 + (H')^2\right)^{3/2}} \tag{4.2}$$

([19, eqs. (3.60) and (4.92)] in two dimensions; see deviation 4.6 for the signs). Where I meets a wall, the static contact-angle condition

$$\boldsymbol{\eta} \cdot \boldsymbol{n}^{\Gamma} = \cos\theta, \qquad 0 < \theta < \pi, \tag{4.3}$$

holds, with $\boldsymbol{n}^{\Gamma}$ the unit wall normal pointing into the fluid and θ measured in the liquid [19, eq. (2.8)]. The code imposes (4.3) only through the curvature (Section 4.7), never through the reconstructed line. A level set q with $I = \{\, \boldsymbol{x} \mid q(\boldsymbol{x}) = 0 \,\}$ and liquid where $q \le 0$ is used only to initialise the fractions (Section 4.9); it is not transported or reinitialised.

4.2 Phase fractions

For every cell the code stores the liquid volume per *full* cell volume and derives the fraction of the *open* part,

$$\alpha_c^{\pi} = \frac{|c^{\mathrm{o}} \cap \Omega^{\pi}|}{|c|}, \qquad C_c^{\pi} = \frac{\alpha_c^{\pi}}{\alpha_c^{\mathrm{o}}} \quad (\alpha_c^{\mathrm{o}} > 0), \tag{4.4}$$

so that $\alpha_c^{\mathrm{l}} + \alpha_c^{\mathrm{g}} = \alpha_c^{\mathrm{o}}$ and $C_c^{\mathrm{l}} + C_c^{\mathrm{g}} = 1$; without a solid $\alpha_c^{\mathrm{o}} = 1$ and the two coincide with the thesis' α_c^{π} [19, eq. (2.35)]. Covered cells ($\alpha_c^{\mathrm{o}} = 0$) carry $C_c^{\mathrm{l}} = 0$ and $C_c^{\mathrm{g}} = 1$. The conditional fraction C_c^{l} is what the reconstruction, the height functions, the constitutive laws and the forces read; α_c^{l} is the transported, conserved quantity.

Every reconstruction first validates and snaps the stored fraction with the *fraction floor* $\varepsilon^{\alpha} \in (0, \tfrac{1}{2})$ (default $\sqrt{\epsilon_{\mathrm{mach}}} \approx 1.5 \times 10^{-8}$): a value outside $[0, \alpha_c^{\mathrm{o}}]$ by more than $128\epsilon_{\mathrm{mach}} \max(1, \alpha_c^{\mathrm{o}})$ is an error, otherwise

$$\alpha_c^{\mathrm{l}} \leftarrow \begin{cases} 0, & C_c^{\mathrm{l}} \le \varepsilon^{\alpha}, \\ \alpha_c^{\mathrm{o}}, & C_c^{\mathrm{l}} \ge 1 - \varepsilon^{\alpha}, \\ \mathrm{clamp}(\alpha_c^{\mathrm{l}}, 0, \alpha_c^{\mathrm{o}}), & \text{otherwise}, \end{cases} \tag{4.5}$$

and C_c^{l} is recomputed. Each snap changes the liquid volume by at most $\varepsilon^{\alpha}|c^{\mathrm{o}}|$, the clamp by round-off only. The halo slots of both fields are then refilled from their donors (Section 2.4), with the zero-gradient rule on physical boundaries. A slot whose donors are finer than its block takes the ratio of the restricted liquid and open fractions with the donors' weights, $C_s^{\mathrm{l}} = \big(\sum_d w_d \alpha_d^{\mathrm{l}}\big) / \big(\sum_d w_d \alpha_d^{\mathrm{o}}\big)$ with a positive denominator, snapped by (4.5), and never the weighted mean of the per-cell ratios: a covered donor carries $C_d^{\mathrm{l}} = 0$ by construction, not gas, so the mean of ratios reads a nearly covered slot as mixed (three covered children and one liquid child give 0.25 instead of one). Every stencil that reads a cut cell across a level jump—transport fluxes, the interface reconstruction and the surface-tension force—therefore sees the same conditional fraction as the coincident coarse cell, and the refinement criteria no longer treat a nearly covered block as bordering the interface (issue 148). A cell is *mixed* if $0 < C_c^{\mathrm{l}} < 1$ after snapping. Where the code needs a phase indicator (the frozen indicator of the transport, Chapter 5, and the jump of the conservative ghost-fluid force, Chapter 8) it uses

$$\chi_c^{\mathrm{l}} = \begin{cases} 1, & C_c^{\mathrm{l}} > \tfrac{1}{2}, \\ 0, & \text{otherwise}, \end{cases} \qquad \chi_c^{\mathrm{g}} = 1 - \chi_c^{\mathrm{l}}. \tag{4.6}$$

Deviation from the thesis 4.1 (Fraction floor). The thesis treats a cell as an interface cell if $\alpha_c^{\pi} > \varepsilon^{\alpha} = 10^{-8}$ for both phases [19, §3.3]. The code instead overwrites the stored fraction by (4.5) at every reconstruction, i.e. after every directional transport sweep. Volume is therefore conserved only up to $\varepsilon^{\alpha}|c^{\mathrm{o}}|$ per snapped cell and reconstruction.

Deviation from the thesis 4.2 (Indicator at equal fractions). The thesis assigns the tie $\alpha_c^{\mathrm{l}} = \alpha_c^{\mathrm{g}}$ to the liquid [19, eq. (2.37)]; (4.6) assigns it to the gas.

Implementation. `InterfaceField(mesh; init, geometry, fraction_floor, reconstruction_method, contact_angle, embedded_contact_angle)` with fields `fraction` $= \alpha^{\mathrm{l}}$, `conditional_fraction` $= C^{\mathrm{l}}$, `normal_x`, `normal_y`, `intercept`, `curvature` and `reconstruction` (method code per cell). Snapping: `_refresh_conditional_fraction!` (`src/interface/interface_field.jl`); restricted halo: `_restrict_conditional_halo!` (`src/interface/interface_field.jl`); phase volumes `_phase_cell_fractions` (`src/two_phase/two_velocity/transport_geometry.jl`); indicator: `_advect_wy!` (`src/vof/fluxes.jl`) and `_surface_tension_jump` (`src/two_phase/forces.jl`). Tests: `test/two_phase/transport.jl` ("Global fraction floor") (floor, snapping in cut cells), `test/two_phase/two_velocity/transport/two_velocity.jl` ("Two-phase cell, staggered and face geometry partitions") ($\alpha_c^{\mathrm{l}} + \alpha_c^{\mathrm{g}} = \alpha_c^{\mathrm{o}}$ on uniform and AMR meshes) and `test/mesh/variables.jl` ("conditional fraction halos restrict numerator and denominator") (level-jump restriction with covered donors).

4.3 PLIC reconstruction

Each cell is mapped to the reference cell $\hat{Q} = [-\tfrac{1}{2}, \tfrac{1}{2}]^2$ by $\boldsymbol{x} = \boldsymbol{x}_c + \mathsf{H}_c\hat{\boldsymbol{x}}$, $\mathsf{H}_c = \operatorname{diag}(h_{c,1}, h_{c,2})$. In a mixed cell the liquid occupies one half-plane of the open part [19, eq. (3.58)],

$$\hat{Q}_c^{\mathrm{l}} = \{\, \hat{\boldsymbol{x}} \in \hat{Q}_c^{\mathrm{o}} \mid \hat{\boldsymbol{\eta}}_c \cdot \hat{\boldsymbol{x}} \le \hat{\phi}_c \,\}, \qquad \boldsymbol{\eta}_c = \frac{\mathsf{H}_c^{-1}\hat{\boldsymbol{\eta}}_c}{\left|\mathsf{H}_c^{-1}\hat{\boldsymbol{\eta}}_c\right|_2}, \quad \phi_c = \frac{\hat{\phi}_c}{\left|\mathsf{H}_c^{-1}\hat{\boldsymbol{\eta}}_c\right|_2}, \tag{4.7}$$

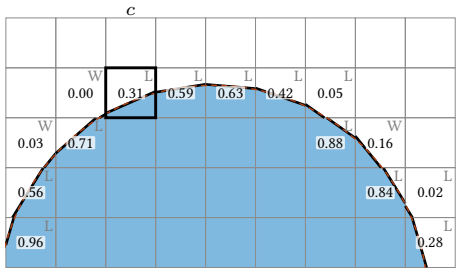
where $\hat{Q}_c^{\mathrm{o}} = \hat{Q}$ in uncut cells and the open polygon of (7.3) otherwise. The thesis' line $\boldsymbol{\eta}_c \cdot (\boldsymbol{x} - \boldsymbol{x}_c) = \phi_c$ [19, §3.3] is the image of $\hat{\boldsymbol{\eta}}_c \cdot \hat{\boldsymbol{x}} = \hat{\phi}_c$. The local normal $\hat{\boldsymbol{\eta}}_c$ is stored unnormalised (WY) or with $|(\hat{\boldsymbol{\eta}}_c)_1| + |(\hat{\boldsymbol{\eta}}_c)_2| = 1$; (4.7) is invariant under its scaling.

In uncut cells the plane constant follows from the closed-form area fraction of a rectangle $\prod_k [\hat{r}_k^-, \hat{r}_k^+] \subseteq \hat{Q}$ below the line [22]. With the projected widths $W_k = |(\hat{\boldsymbol{\eta}}_c)_k|\,(\hat{r}_k^+ - \hat{r}_k^-)$, $W_< = \min_k W_k$, $W_> = \max_k W_k$ and the plane constant measured from the lowest corner, $z = \hat{\phi}_c - \sum_k \min\big((\hat{\boldsymbol{\eta}}_c)_k \hat{r}_k^-, (\hat{\boldsymbol{\eta}}_c)_k \hat{r}_k^+\big)$,

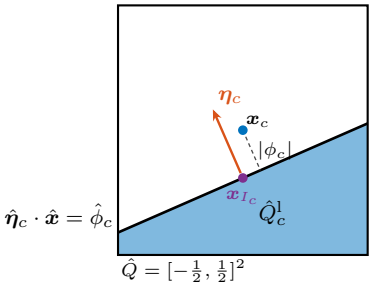
$$\mathcal{A} = \begin{cases} 0, & z \le 0, \\ F(z), & 0 < z \le \tfrac{1}{2}(W_1 + W_2), \\ 1 - F(W_1 + W_2 - z), & \tfrac{1}{2}(W_1 + W_2) < z < W_1 + W_2, \\ 1, & z \ge W_1 + W_2, \end{cases} \tag{4.8}$$

$$F(z) = \frac{z^2}{2W_<W_>} \quad (z < W_<), \qquad F(z) = \frac{z - W_</2}{W_>} \quad (z \ge W_<),$$

is the fraction of the rectangle with $\hat{\boldsymbol{\eta}}_c \cdot \hat{\boldsymbol{x}} \le \hat{\phi}_c$; for $W_< \le \epsilon_{\mathrm{mach}} W_>$ the code uses $\operatorname{clamp}(z/W_>, 0, 1)$, and the third regime is evaluated by the first through the reflection



A. Exact interface (dashed), PLIC segments I_c and liquid parts; numbers: C_c^{l}; L/W: LHF or WY normal.



B. Cell c in local coordinates; $C_c^{\mathrm{l}} = 0.31$.

Figure 4.1. PLIC reconstruction (4.7) to (4.9) on the circle fixture of `figures/generate.jl` (unit cells, radius 4.3, liquid inside; window $3 \le x_1 \le 12$, $7 \le x_2 \le 12$). Each mixed cell carries one half-plane; the segments are not connected across faces. The plane constant $\hat{\phi}_c$ is fixed by the area of $\hat{Q}_c^{\mathrm{l}}$; with square cells $\boldsymbol{\eta}_c$ is parallel to $\hat{\boldsymbol{\eta}}_c$ and $\phi_c = \hat{\phi}_c / |\hat{\boldsymbol{\eta}}_c|_2 = -0.175$ in units of the cell width. The midpoint $\boldsymbol{x}_{I_c}$ of the segment is the origin of the curvature fit of Section 4.6.

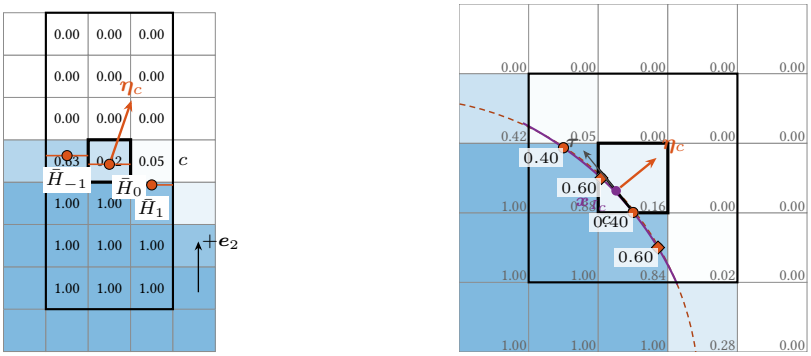
$z \mapsto W_1 + W_2 - z$ (the inclusion–exclusion form of the distribution function loses $\epsilon_{\mathrm{mach}}\, W_>/W_<$). The plane constant is the root of

$$\mathcal{A}(\hat{\boldsymbol{\eta}}_c, \hat{\phi}_c; \hat{Q}) = C_c^{\mathrm{l}} \quad \text{(uncut)}, \qquad |\hat{Q}_c^{\mathrm{l}}| = C_c^{\mathrm{l}}\, |\hat{Q}_c^{\mathrm{o}}| \quad \text{(cut)}, \tag{4.9}$$

found by 54 bisection steps on $[-r, r]$, $r = \frac{1}{2}\sum_k |(\hat{\boldsymbol{\eta}}_c)_k|$ (uncut), or on the range of $\hat{\boldsymbol{\eta}}_c \cdot \hat{\boldsymbol{x}}$ over the vertices of $\hat{Q}_c^{\mathrm{o}}$ (cut), where the area of the clipped polygon is computed by Sutherland–Hodgman clipping [23] and the shoelace formula. The PLIC segment I_c is the part of the line inside $\hat{Q}_c^{\mathrm{o}}$; its midpoint in $\hat{Q}$ (the average of the distinct points where the line meets the sides of $\hat{Q}$) is $\boldsymbol{x}_{I_c}$. Figure 4.1 shows the reconstruction on a circle.

Checks (`figures/generate.jl`). Over 1408 samples of normals and fractions the bisection root of (4.9) differs from the closed-form inverse of Scardovelli and Zaleski by at most 5.6×10^{-16} for $10^{-2} \le C_c^{\mathrm{l}} \le 1 - 10^{-2}$ (near pure cells the plane constant is ill-conditioned, $\partial\hat{\phi}_c/\partial C_c^{\mathrm{l}} \sim (C_c^{\mathrm{l}})^{-1/2}$ and $(1 - C_c^{\mathrm{l}})^{-1/2}$; the bisection evaluates $\mathcal{A}$ with an absolute error ϵ_{mach}, so its root is accurate to $\epsilon_{\mathrm{mach}}\, (1-C_c^{\mathrm{l}})^{-1/2}\, |\hat{\boldsymbol{\eta}}_c|_1$ near full cells, at most $2 \times 10^{-13}\, |\hat{\boldsymbol{\eta}}_c|_1$ for the fractions that survive the floor of (4.5), while the closed form stays within a few $10^{-16}\, |\hat{\boldsymbol{\eta}}_c|_1$ everywhere), and the area of the reconstructed polygon reproduces C_c^{l} to 2.2×10^{-16} for all samples. In the cut cells of the fixture of Figure 4.3c, $|\hat{Q}_c^{\mathrm{l}}|$ reproduces $C_c^{\mathrm{l}} |\hat{Q}_c^{\mathrm{o}}|$ to 1.1×10^{-16}.

Implementation. `_plane_fraction` (`src/interface/interface_field.jl`) (4.8) and `_plane_intercept` (`src/interface/interface_field.jl`); cut cells `_open_plane_intercept` (`src/embedded/geometry.jl`); polygons `_clip_polygon` (`src/embedded/geometry_utils.jl`) and `_polygon_area` (`src/embedded/geometry_utils.jl`); midpoint `_interface_centroid` (`src/interface/height_functions.jl`); physical segments `plic_segments`. Tests: `test/two_phase/transport.jl` ("PLIC geometry") (fraction and plane constant consistent to 5×10^{-15}), `test/indicator/phase/height_functions.jl` ("Fixed-size PLIC centroid scratch") ($\boldsymbol{x}_{I_c}$), `test/indicator/geometry.jl` ("PLIC geometry converges and produces filled solid visualization") (solid PLIC segments converge with order above 1.49).



A. LHF columns, principal direction $+\boldsymbol{e}_2$. **B.** GHF points of the 3×3 neighbourhood, weights w_j, fit (purple).

FIGURE 4.2. Height functions on the circle fixture of Figure 4.1 (shading: C_c^{l}). (a) The seven-cell columns through c and its two neighbours give the heights $\bar{H}_{-1}, \bar{H}_0, \bar{H}_1$ of (4.11) (red) and the LHF normal (4.13); no other signed direction is valid here. (b) A cell without valid LHF columns (W in Figure 4.1a): the heights of all signed directions in the outlined 3×3 block are candidate points $\boldsymbol{p}_j$; 4 are kept with the weights $w_j = \max(0, \pm\boldsymbol{\eta}_c \cdot \boldsymbol{e}_k)^2$ of (4.15), 8 are discarded (hollow). The parabola in the frame $(\boldsymbol{\eta}_c, \boldsymbol{\tau})$ at $\boldsymbol{x}_{I_c}$ of the (WY) plane gives the curvature. Dashed: the exact circle.

4.4 HEIGHT FUNCTIONS

Normals and curvature are computed from column sums of conditional fractions. For the cells of a leaf block the stencil value is C_c^{l}; four layers of halo values at the block's level are filled when the topology changes, independently of the halo of Section 2.4: a slot $\hat{c}$ takes

$$C_{\hat{c}}^{\mathrm{l}} = \sum_b \frac{|\hat{c} \cap b|}{|\hat{c}|}\, \mathcal{A}\left(\hat{\boldsymbol{\eta}}_b, \hat{\phi}_b; \mathsf{H}_b^{-1}(\hat{c} \cap b - \boldsymbol{x}_b)\right), \tag{4.10}$$

the liquid fraction of the donor PLIC over the overlap (the donor fraction for pure donors), clamped to $[0, 1]$. Slots beyond a wall, and with a solid all cells and slots that overlap a cell with $\alpha_c^{\mathrm{o}} < 1$, are *unavailable*: columns may end at a wall or at a cut cell but never integrate the solid as gas.

A column with principal direction $+\boldsymbol{e}_k$ ($-\boldsymbol{e}_k$) uses C^{l} ($C^{\mathrm{g}} = 1 - C^{\mathrm{l}}$). Through the cell $c + m\boldsymbol{e}_t$, $t \neq k$, it consists of the available cells $c + m\boldsymbol{e}_t + n\boldsymbol{e}_k$, $n_0 \le n \le n_1$, a contiguous run in $-3 \le n \le 3$ that starts at the first available cell. It is valid if the first value is one, the last is zero, the values do not increase, and no two consecutive mixed values are equal (thesis Properties 1 with three cells on either side [19, §2.3.1]). Its height, in cells of direction k and relative to the centre of $c + m\boldsymbol{e}_t$, is

$$\bar{H}_m = \sum_{n=n_0}^{n_1} C^{\mathrm{l}}_{c+m\boldsymbol{e}_t+n\boldsymbol{e}_k} + n_0 - \frac{1}{2} \qquad (\text{direction } +\boldsymbol{e}_k;\ C^{\mathrm{g}} \text{ for } -\boldsymbol{e}_k), \tag{4.11}$$

the column-sum form of the thesis' midpoint height [19, eq. (2.40)]. The equality tests use the tolerance $64\epsilon_{\mathrm{mach}}$. Figure 4.2a shows three such columns.

Implementation. Halo values:
`_height_cache!` (`src/interface/height_functions.jl`) and
`_fill_height_values!` (`src/interface/height_functions.jl`) (4.10);
column heights `_local_height` (`src/interface/height_functions.jl`) (4.11).

4.5 Normals and choice of the plane

WY normal. Every reconstruction starts from the normal of Weymouth and Yue [33] [19, §3.3.2], computed from the 3×3 block of conditional fractions (halo values of Section 2.4 at block borders). With $g_k = C^{\mathrm{l}}_{c-\boldsymbol{e}_k} - C^{\mathrm{l}}_{c+\boldsymbol{e}_k}$, the principal axis $k = 1$ if $|g_1| > |g_2|$ and $k = 2$ otherwise, $t \neq k$, the column sums $S_c = \sum_{m=-1}^{1} C^{\mathrm{l}}_{c+m\boldsymbol{e}_k}$ and the side $d = \operatorname{sgn}(g_t)\operatorname{sgn}(C^{\mathrm{l}}_c - \frac{1}{2})$, with $\operatorname{sgn}(0) = 1$ in the first and -1 in the second factor,

$$\hat{\boldsymbol{\eta}}^{\mathrm{WY}}_c = \operatorname{sgn}(g_k)\,\boldsymbol{e}_k - \frac{S_{c+d\boldsymbol{e}_t} - S_c}{d}\,\boldsymbol{e}_t \tag{4.12}$$

in local coordinates, and the plane constant follows from (4.9). It reproduces straight lines exactly. In a cell that the solid does not cut, covered neighbours enter with $C^{\mathrm{l}} = 0$; triple cells use the masked form below.

LHF normal. For each of the four signed directions $\pm\boldsymbol{e}_k$ with valid heights $\bar{H}_{-1}, \bar{H}_0, \bar{H}_1$ through $c - \boldsymbol{e}_t$, c, $c + \boldsymbol{e}_t$, central differences give $H' = h_{c,k}(\bar{H}_1 - \bar{H}_{-1})/(2h_{c,t})$ and $H'' = h_{c,k}(\bar{H}_1 - 2\bar{H}_0 + \bar{H}_{-1})/h^2_{c,t}$ and, by (4.2), a candidate normal $\boldsymbol{\eta}^{(i)}$ and curvature $\kappa^{(i)}$ [19, §3.3.1]. A candidate whose normal makes the angle $\psi_i = \arccos\left|\boldsymbol{\eta}^{(i)} \cdot \boldsymbol{e}_k\right|$ with its principal direction gets the quality and weight

$$Q_i = \mathrm{S}\left(\frac{\pi/4 + \psi^* - \psi_i}{2\psi^*}\right), \quad \psi^* = \frac{\pi}{20}, \qquad w_i = Q_i \prod_{j \neq i}(1 - Q_j), \tag{4.13}$$

over the candidates with $Q_i > 0$. The LHF result is the weighted mean [19, eq. (3.61)]: the normal $\boldsymbol{\eta} \propto \sum_i w_i \boldsymbol{\eta}^{(i)}$ and the curvature $\kappa = \sum_i w_i \kappa^{(i)} / \sum_i w_i$. Thus $Q_i = 1$ for $\psi_i \leq \pi/5$, $Q_i = 0$ for $\psi_i \geq 3\pi/10$, and one perfect candidate excludes the others.

Wall normal. In a cell adjacent to a non-periodic domain wall whose current line meets the wall side of the cell, the centred stencil lacks a column. With the wall normal along axis k, inward sign ι, $t \neq k$ and the principal direction $s\boldsymbol{e}_t$, $s = \operatorname{sgn}((\boldsymbol{\eta}_c)_t)$, the heights $\bar{H}_0$ through c and $\bar{H}_1$ through $c + \iota\boldsymbol{e}_k$ give the one-sided slope and normal

$$H' = \iota\,\frac{h_{c,t}(\bar{H}_1 - \bar{H}_0)}{h_{c,k}}, \qquad \boldsymbol{\eta}^{(w)} = \frac{s\,\boldsymbol{e}_t - s\,H'\,\boldsymbol{e}_k}{\sqrt{1 + (H')^2}}, \tag{4.14}$$

which contains no contact angle. In a corner cell the candidates with the largest quality $\max(Q, \sqrt{\epsilon_{\mathrm{mach}}})$ are averaged.

Choice of the plane. The keyword `reconstruction_method` selects which normal defines the line used by transport in a mixed cell that the solid does not cut (Table 4.1). The code first takes a candidate: the wall normal (code 5) if it applies; otherwise the LHF normal (code 1); otherwise the GHF fit of Section 4.6 (code 2); otherwise none (code 3). If the method adopts the candidate, $\hat{\boldsymbol{\eta}}_c = \mathsf{H}_c\boldsymbol{\eta} / |\mathsf{H}_c\boldsymbol{\eta}|_1$ and $\hat{\phi}_c$ is recomputed; otherwise the cell keeps its previous line, initially the WY line. Because the height values in level-transition

candidate	`:lhf` (default)	`:lhf_ghf`	`:wy`
wall normal (4.14) (5)	adopted	adopted	WY
LHF (4.13) (1)	adopted	adopted	WY
GHF (4.16) (2)	kept (curvature only)	adopted	WY
none (3)	kept	kept	WY
triple cell	guarded GHF (2), nearby LHF (1) or masked WY (3)		

Table 4.1. Line used for transport in a mixed cell. *Adopted*: the candidate normal and (4.9); *kept*: the line of the previous pass (initially WY); in all modes curvature is computed as in Section 4.6. The stored `reconstruction` code is the candidate's if adopted, otherwise 3; 0 marks pure cells. Embedded triple cells follow the same rule for every method (see the text).

halos depend on the neighbours' lines (4.10), the candidate pass is repeated three times on frozen copies of the lines (for `:wy` once without a solid; with a solid its second and third pass revisit only the triple cells); cells are independent within a pass.

Embedded triple cells. In a mixed cell that the solid cuts, $0 < \alpha_c^{\mathrm{s}} < 1$, the height values are unavailable (Section 4.4) and every method uses one rule. Its initial line and seed is the *masked* WY normal $\hat{\boldsymbol{\eta}}_c^{\mathrm{M}}$, which is (4.12) with three changes: the conditional fractions are α^1/α°, also in halo slots, so that covered donors of a coarse slot do not count; a difference g_1 or g_2 with one covered neighbour is twice the one-sided difference to c, and zero with two; and the column difference $S_{c+d\boldsymbol{e}_t} - S_c$ sums only over the pairs $c + m\boldsymbol{e}_k$, $c + m\boldsymbol{e}_k + d\boldsymbol{e}_t$, $|m| \le 1$, of open cells, scaled by three over their number, on the side $-d$ if side d has no such pair, and is replaced by $-dg_t/\,|g_k|$ if neither has (zero for $g_k = 0$). Without covered neighbours $\hat{\boldsymbol{\eta}}_c^{\mathrm{M}}$ is the WY normal. In every pass the code then fits the GHF of (4.15) without contact row, with height points from the 7×7 block around c and the frame of the cell's current line, and adopts it (code 2) unless its normal opposes the seed, $\boldsymbol{\eta}^{\mathrm{GHF}} \cdot \mathsf{H}_c^{-1}\hat{\boldsymbol{\eta}}_c^{\mathrm{M}} \le 0$. Otherwise it takes, among the fully open mixed cells of the 5×5 block whose LHF normal $\boldsymbol{\eta}'$ exists and lies within 45° of the seed, $\boldsymbol{\eta}' \cdot \mathsf{H}_c^{-1}\hat{\boldsymbol{\eta}}_c^{\mathrm{M}} \ge \left|\mathsf{H}_c^{-1}\hat{\boldsymbol{\eta}}_c^{\mathrm{M}}\right|_2/\sqrt{2}$, the normal of the cell whose segment midpoint lies nearest to that of the seed line in c (code 1), and if none qualifies the seed itself (code 3); the plane constant follows from (4.9) on the open polygon. The contact angle enters none of these steps, only the curvature (Section 4.7).

Checks (`triple_check`, `triple_accuracy`). On the sloped-wall fixture of Figure 4.3c with four wall slopes and four contact points, for every method and the prescribed angles 60°, 90° and 120° (144 reconstructions), an independent evaluation of the rule reproduces the stored lines and codes of all triple cells to 1.8×10^{-14} (137 cells over the fixtures and methods: 127 GHF, 0 neighbouring LHF and 10 masked WY lines); no line changes with the prescribed angle or opposes the exact normal. With 16×8, 32×16 and 64×32 cells the largest deviation $|\boldsymbol{\eta}_c - \boldsymbol{\eta}|_2$ of the triple-cell normals from the exact normal is at most 0.801, 0.138 and 0.020 over the three methods, and the largest curvature error 0.160, 0.049 and 0.009 for the exact $1/4.2$.

Checks (`figures/generate.jl`). On the circle fixture of Figure 4.1 the normals and curvatures of the 28 LHF cells, evaluated independently from (4.11) and (4.13), differ from the code's by at most 0 and 0, and the WY normals (4.12) of all mixed cells by 0.

Deviation from the thesis 4.3 (Reconstruction family). The thesis falls back to ELVIRA [15] where no LHF normal exists [19, §3.3.1], reconstructs triple points with LVIRA or MOF [19, §3.3.4] and also studies MOF and parabolic reconstructions (PLVIRA, PROST, PMOF; [19, §3.3.3 and Ch. 5]). The code implements the WY, LHF, one-sided wall and GHF normals of this section only; it advects no centroid, and in triple cells it uses the guarded GHF, a neighbouring LHF or the masked WY line as stated above.

Deviation from the thesis 4.4 (Height-function quality). With $z = \boldsymbol{\eta}^{(i)} \cdot \boldsymbol{e}_k / \cos(\pi/4 + \psi^*)$ the thesis prints the quality as $3z^3 - 2z^2$ for $0 < z < 1$ and one for $z \geq 1$ [19, eq. (3.62)], which is neither monotone on that interval nor equal to one for $\psi_i < \pi/4 - \psi^*$ as the text requires. The code uses the smoothstep in the angle (4.13), which has the stated limits; this correction is approved by the author.

Implementation. `_wy_normal` (`src/interface/height_functions.jl`) (4.12); `_height_quality` (`src/interface/height_functions.jl`) and `_lhf_geometry` (`src/interface/height_functions.jl`) (4.13); `_one_sided_wall_normal` (`src/interface/height_functions.jl`) and `_plic_touches_wall` (`src/interface/height_functions.jl`) (4.14); choice `_reconstruct_interface_cell!` (`src/interface/interface_field.jl`), passes `_reconstruct_interface!` (`src/interface/interface_field.jl`), called by `reconstruct!`; triple cells `_reconstruct_embedded_triple_cell!` (`src/interface/interface_field.jl`) with the seed `_solid_aware_wy_normal` (`src/interface/height_functions.jl`) and the fallback `_nearby_lhf_normal` (`src/interface/interface_field.jl`). Tests: `test/indicator/phase/height_functions.jl` ("Selectable PLIC reconstruction") (dispatch of Table 4.1), `test/indicator/phase/height_functions.jl` ("WY fallback exactly reconstructs linear interfaces"), `test/indicator/phase/height_functions.jl` ("LHF planar normals and curvature") (exact for lines on uniform, refined and adapted meshes), `test/indicator/phase/height_functions.jl` ("Wall PLIC normals use a contact-angle-free two-point stencil") (exact for lines), `test/indicator/phase/anisotropic_cells.jl` ("anisotropic reconstruction refinement") (normals converge on cells with aspect ratio two).

4.6 Curvature

LHF curvature. Where the LHF normal of Section 4.5 exists, its weighted curvature is used [19, eq. (4.92)].

GHF fit. Otherwise the generalised height function of Popinet [16] with the weights of the thesis [19, §4.7.1] is fitted. Let $\boldsymbol{\eta} = \boldsymbol{\eta}_c$ be the current normal of the cell, $\boldsymbol{\tau} = (-(\boldsymbol{\eta})_2, (\boldsymbol{\eta})_1)$, $\boldsymbol{x}_{I_c}$ the midpoint of its segment and $h = \min_k h_{c,k}$. Every valid height of every signed direction $\pm\boldsymbol{e}_k$ through a cell c' of the 3×3 block around c (the 7×7 block in a triple cell) gives the point $\boldsymbol{p}_j = \boldsymbol{x}_{c'} + h_{c,k}\bar{H}_0\boldsymbol{e}_k$, $\bar{H}_0$ that column's height (4.11), with weight $w_j = \max(0, \pm\boldsymbol{\eta} \cdot \boldsymbol{e}_k)^2$; points with zero weight or closer than $h/2$ to an earlier point are discarded (order: rows, columns, axis, sign). In the frame $x_\tau = \boldsymbol{\tau} \cdot (\boldsymbol{p}_j - \boldsymbol{x}_{I_c})/h$,

$x_\eta = \boldsymbol{\eta} \cdot (\boldsymbol{p}_j - \boldsymbol{x}_{I_c})/h$ the parabola minimises

$$\sum_j w_j \left(A_2 x_\tau^2 + A_1 x_\tau + A_0 - x_\eta\right)^2 \tag{4.15}$$

[19, eq. (4.93)], with the coordinates of $\boldsymbol{p}_j$ and, in a triple cell, the contact row (4.20) added with weight 10^3, provided there are at least three rows and the smallest singular value of the matrix exceeds 10^{-10} times the largest. It yields

$$\boldsymbol{\eta}^{\mathrm{GHF}} = \frac{\boldsymbol{\eta} - A_1 \boldsymbol{\tau}}{\sqrt{1 + A_1^2}}, \qquad \kappa^{\mathrm{GHF}} = -\frac{2A_2}{h\,(1 + A_1^2)^{3/2}}, \tag{4.16}$$

evaluated at $\boldsymbol{x}_{I_c}$. In a triple cell both are negated if the components of $\boldsymbol{\eta}^{\mathrm{GHF}}$ and $\boldsymbol{\eta}$ along the wall tangent have opposite signs. If the height points do not determine the fit, the thesis uses the segment midpoints of the neighbouring mixed cells with unit weights; the code tries the 3×3, 5×5 and 7×7 blocks in turn, with the previous-pass lines of the block's cells and WY lines of halo cells, Figure 4.2b shows a height-point fit.

Choice and bounds. The curvature of a mixed cell is, in this order: the domain-wall contact curvature of Section 4.7; in a triple cell, the fit with the contact row if an embedded contact angle is set and that fit exists, otherwise the unconstrained fit of the plane rule (also when its plane was rejected); in other cells the candidate's curvature (LHF or GHF) and, for a wall-normal cell without one, the unconstrained GHF fit. The result is clipped to the mesh,

$$\kappa_c \leftarrow \mathrm{clamp}\left(\kappa_c,\, -\frac{1}{2\max_k h_{c,k}},\, \frac{1}{2\max_k h_{c,k}}\right), \tag{4.17}$$

and is undefined (NaN) where no estimate exists and in pure cells; the force of Chapter 8 raises an error on a face that needs it.

Checks (`figures/generate.jl`). On the circle fixture (exact curvature 0.233) the LHF curvatures deviate by at most 1.1×10^{-2} and the GHF curvatures of the 6 remaining mixed cells by 4.0×10^{-2}; an independent evaluation of (4.15) and (4.16), including the centroid fallback used in 1 of them, agrees with the code to 5.6×10^{-17}.

Deviation from the thesis 4.5 (GHF centroid fallback). The thesis' fallback fits the interface centroids of the 3×3 neighbourhood only [19, §4.7.1]; the code enlarges the block to 5×5 and 7×7 before giving up, so that under-resolved arcs beside walls still receive a curvature.

Implementation. `_ghf_fit` (`src/interface/height_functions.jl`), `_ghf_geometry` (`src/interface/height_functions.jl`) and `_orient_ghf_geometry` (`src/interface/height_functions.jl`) (4.15) and (4.16); the contact row `_embedded_ghf_constraint` (`src/interface/height_functions.jl`) (4.20); choice and clipping `_reconstruct_interface_cell!` (`src/interface/interface_field.jl`) and `_clip_curvature_to_mesh` (`src/interface/interface_field.jl`) (4.17). Tests: `test/indicator/phase/height_functions.jl` ("Circular LHF curvature converges") (order at least 1.7),

`test/indicator/phase/height_functions.jl` ("Curvature is bounded by the local mesh resolution"), `test/indicator/phase/height_functions.jl` ("Expanded GHF resolves a shallow wall arc") (finite curvature; no accuracy assertion), `test/indicator/phase/levelset_initialization.jl` ("interface reconstruction is independent of base-root partitioning").

4.7 Contact angles

The contact angle enters only the curvature of the cells at the wall; the lines of Section 4.5 do not depend on it. The defaults are $\theta = \pi/2$ on every non-periodic domain wall and on an embedded solid; `contact_angle` takes one angle or a tuple (left, right, bottom, top) whose entries may be `nothing`, and `embedded_contact_angle` the solid's.

Domain walls. For a mixed cell next to a wall with prescribed θ whose line meets the wall side (wall normal along axis k, inward sign ι, $t \neq k$), two branches follow the thesis [19, §4.7.2], Figure 4.3. If $|\theta - \pi/2| \leq \pi/4$, the heights along $s\boldsymbol{e}_t$, $s = \operatorname{sgn}((\boldsymbol{\eta}_c)_t)$, through the wall cell and its inward neighbour, $H_m = h_{c,t}\bar{H}_m$ for $m = 0, 1$, are extended by a ghost value that imposes (4.3) on the height function, whose derivative along $\iota\boldsymbol{e}_k$ at the wall is $-\cos\theta/(s\sin\theta)$ [1],

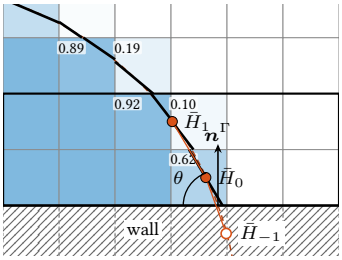
$$H_{-1} = H_0 + h_{c,k}\frac{\cos\theta}{s\sin\theta}, \qquad \kappa_c = -s\,\frac{(H_1 - 2H_0 + H_{-1})/h_{c,k}^2}{\left(1 + ((H_1 - H_{-1})/(2h_{c,k}))^2\right)^{3/2}}. \tag{4.18}$$

If $|\theta - \pi/2| > \pi/4$, the phase of the wedge at the wall, $s = \operatorname{sgn}(\cos\theta)$ (liquid for acute, gas for obtuse angles), is measured by heights from the wall: $H_m = h_{c,k}\bar{H}_m$, $m = 1, 2$, in the next two columns away from the contact point, each the sum over the four cells from the wall (non-increasing values, last one empty). With x_t measured along the wall from the first of these columns, away from the contact point, the parabola $H = A_2 x_t^2 + A_1 x_t + H_1$ through both heights that vanishes with slope $|\tan\theta|$ at an unknown point is

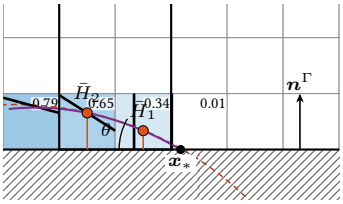
$$A_2 = \frac{(H_2 - H_1)^2 - h_{c,t}^2\tan^2\theta}{h_{c,t}^2\left(H_1 + H_2 + \sqrt{4H_1H_2 + h_{c,t}^2\tan^2\theta}\right)}, \qquad A_1 = \frac{H_2 - H_1}{h_{c,t}} - A_2 h_{c,t},$$
$$\kappa_c = \frac{-2s\,A_2}{(1 + (A_1 - 2A_2h_{c,t})^2)^{3/2}}, \tag{4.19}$$

required $H_2 > H_1 > 0$. The parabolas through both heights with the slope $|\tan\theta|$ at an unknown zero have $A_2 = (H_1 + H_2 \pm \sqrt{D})/h_{c,t}^2$, $D = 4H_1H_2 + h_{c,t}^2\tan^2\theta$ [19, §4.7.2]; (4.19) is the minus sign, written without the subtraction of nearly equal numbers. It is the thesis' root with $B\tan\theta > 0$ [19, eq. (4.94)], where the slope of the wedge phase is $|\tan\theta|$, i.e. the root with $A_1 > 0$: its zero lies at $x_t = -2H_1/(A_1 + |\tan\theta|) < 0$, before the first column, whereas the other root has it beyond. The slope in the curvature is evaluated at the wall cell, $x_t = -h_{c,t}$ (deviation 4.7). The first wall (left, right, bottom, top) whose branch has valid heights determines the curvature.

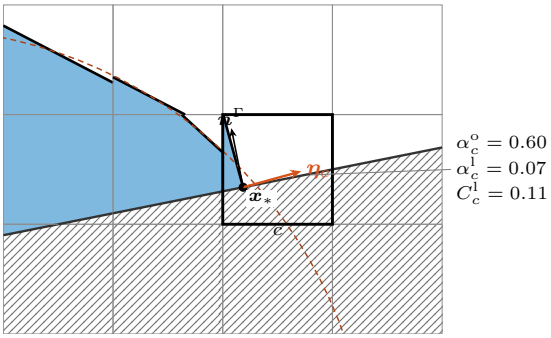
Embedded walls. In a triple cell the curvature is the GHF fit (4.15) with one more row [19, eq. (4.97)]. The triple point $\boldsymbol{x}_*$ is the intersection of the interface and solid lines of the cell, if it lies in the cell; with the solid normal into the fluid, $\boldsymbol{n}^\Gamma = \mathsf{H}_c^{-1}\hat{\boldsymbol{\eta}}_c^{\mathrm{s}} / \left|\mathsf{H}_c^{-1}\hat{\boldsymbol{\eta}}_c^{\mathrm{s}}\right|_2$ (Chapter 7),



A. $\theta = 70°$: rows parallel to the wall, ghost value $\bar{H}_{-1}$ beyond it.



B. $\theta = 30°$: heights from the wall and the contact parabola (purple).



C. Embedded wall (hatched: solid parts of the cut cells; covered cells), liquid parts of the open polygons and the triple point $\boldsymbol{x}_*$ of the two planes in cell c; the prescribed $\theta = 60°$ enters only the curvature fit, the angle between $\boldsymbol{\eta}_c$ and $\boldsymbol{n}^\Gamma$ is 86°.

FIGURE 4.3. Contact-angle geometry, $\boldsymbol{\eta} \cdot \boldsymbol{n}^\Gamma = \cos\theta$ with $\boldsymbol{n}^\Gamma$ into the fluid (dashed: exact interfaces of the fixtures in `figures/generate.jl`; numbers: C_c^{l}). (a), (b): a drop of radius 6 on the bottom wall, the two branches (4.18) and (4.19) at the right contact cell. (c): a drop on a sloped embedded wall (16×8 unit cells, default reconstruction), the contact row (4.20) of the constrained fit.

and $x_\tau^* = \boldsymbol{\tau} \cdot (\boldsymbol{x}_* - \boldsymbol{x}_{I_c})/h$, the linearised condition (4.3) on the fitted normal is

$$-2x_\tau^*(\boldsymbol{\tau} \cdot \boldsymbol{n}^\Gamma)\, A_2 - (\boldsymbol{\tau} \cdot \boldsymbol{n}^\Gamma)\, A_1 = \cos\theta - \boldsymbol{\eta} \cdot \boldsymbol{n}^\Gamma, \tag{4.20}$$

added to (4.15) as a row multiplied by 10^3 (weight 10^6 in the sum, the height rows being multiplied by $\sqrt{w_j}$). It is the unnormalised fitted normal $\boldsymbol{\eta} - (A_1 + 2A_2x_\tau^*)\boldsymbol{\tau}$ inserted in (4.3), i.e. the exact condition with the factor $\sqrt{1 + (A_1 + 2A_2x_\tau^*)^2}$ on its right-hand side replaced by one: the fitted unit normal at $\boldsymbol{x}_*$ makes the angle θ' with $\boldsymbol{n}^\Gamma$ where $\cos\theta' = \cos\theta/\sqrt{1 + (A_1 + 2A_2x_\tau^*)^2}$, an error $\approx \frac{1}{2}(A_1 + 2A_2x_\tau^*)^2 \cot\theta$ of the angle. If the two lines do not meet in the cell the fit is computed without this row; if the fit fails, the curvature of the unconstrained fit of the plane rule is used. Figure 4.3c shows a triple cell.

Checks (`figures/generate.jl`). Independent evaluations of (4.18) (sessile drop, $\theta = 70°$, 2 contact cells), (4.19) ($\theta = 30°$, 3 cells) and of the triple-cell curvature of Section 4.6 (constrained fit with fallback; 94 cells with a contact row) agree with the code to 5.6×10^{-17}, 5.6×10^{-17} and 2.3×10^{-14}. With radius 6 cells (exact curvature 0.167) the contact curvatures deviate by at most 5.0×10^{-3} and 3.6×10^{-2}.

Deviation from the thesis 4.6 (Signs of the height-function formulas). With $\boldsymbol{\eta}$ pointing into the gas, $\kappa = \nabla \cdot \boldsymbol{\eta}$ [19, eq. (2.6)] and $\boldsymbol{n}^\Gamma$ pointing into the fluid with θ measured in the liquid, (4.2) carries $\mp$ for $H^{\pm k}$, the right-angle branch has $\mathrm{d}H^{+2}/\mathrm{d}x_1 = -1/\tan\theta$ at a wall with $\boldsymbol{n}^\Gamma = \boldsymbol{e}_1$, and the acute branch has $\kappa = -2A_2/(1+(H')^2)^{3/2}$. The thesis prints [19, eq. (4.92)] $\kappa = \pm(\dots)/W^3$, whereas the preceding line, $\kappa = \nabla \cdot \boldsymbol{\eta}$ with $\boldsymbol{\eta} = \pm(-H_x, -H_y, 1)/W$, gives $\mp$; in §4.7.2 it prints $\mathrm{d}H^{+y}/\mathrm{d}x = 1/\tan\theta$ right after $\cos\theta = -H_x/\sqrt{1+H_x^2}$, which gives $-1/\tan\theta$; and it prints $\kappa = 2A/\sqrt{1+B^2}^3$ for its acute example, whose positive principal direction gives $-2A/(1+B^2)^{3/2}$ (the printed sign is correct for the negative direction, $s=-1$, obtuse angles). Each of the three results contradicts a premise printed in the same paragraph (the three results are those obtained with $\boldsymbol{\eta}$ pointing into the liquid, the premises those with $\boldsymbol{\eta}$ into the gas), so they are typographical errors (pending the author's confirmation). The code follows the definitions above: its curvature converges to $1/R > 0$ for sessile drops at 20°, 55°, 70°, 90°, 125° and 160°, and the ghost value (4.18) makes the discrete wall slope of the height function exactly $-\cos\theta/(s\sin\theta)$.

Deviation from the thesis 4.7 (Acute and obtuse contact curvature). The thesis evaluates the curvature of the contact parabola with the slope at the first height column, $\kappa = 2A_2/(1+A_1^2)^{3/2}$ in the notation used here (sign as printed, see deviation 4.6) [19, §4.7.2]; the code evaluates the slope at the column of the wall cell, one cell closer to the contact point (4.19). The two values differ by the relative amount $6h_{c,t}A_2A_1/(1+A_1^2)$, of order $h_{c,t}\,|\kappa|$. Both are first-order accurate: on sessile drops with $R/h = 96$ at $\theta = 10°, 20°, 30°, 150°, 160°, 170°$ the largest curvature errors over the contact cells are 5.6×10^{-4} to 3.6×10^{-3} (code) and 1.1×10^{-3} to 2.2×10^{-3} (thesis) for $1/R = 0.167$, neither uniformly smaller.

Implementation. `contact_angle`, `embedded_contact_angle` of `InterfaceField`; domain walls `_wall_height` (`src/interface/height_functions.jl`) and `_contact_curvature` (`src/interface/height_functions.jl`) (4.18) and (4.19); embedded walls `_embedded_ghf_constraint` (`src/interface/height_functions.jl`) (4.20). Tests: `test/indicator/phase/height_functions.jl` ("Contact-angle curvature converges at triple points") (sessile drop at 20°, 55°, 90°, 125°, 160°: order at least 0.7, at least 1.8 for 90°), `test/indicator/phase/height_functions.jl` ("Contact angle changes curvature but not the PLIC interface"), `test/indicator/geometry.jl` ("Embedded contact angles do not change PLIC planes"), `test/indicator/geometry.jl` ("Embedded GHF contact curvature converges on sloped walls") (median curvature error, order above 0.8, `:lhf_ghf`), `test/indicator/geometry.jl` ("Triple-cell normals converge with matching and mismatched angles") (median normal errors decrease on 32×16, 64×32, 128×64 cells for every method and angles of 60°, 90°, 120°), `test/indicator/geometry.jl` ("Sliver triple-cell orientation does not reverse"), `test/indicator/geometry.jl` ("Solid-aware WY differences and masked columns"), `test/indicator/geometry.jl` ("Triple-cell fit failure excludes covered WY neighbours"), `test/indicator/geometry.jl` ("Triple-cell curvature falls back to the unconstrained fit"), `test/indicator/phase/levelset_initialization.jl` ("Default right-angle wall contact").

4.8 Phase apertures and interface height

The phase apertures of a face are the averages of the one-sided traces of the two adjacent reconstructions [19, eq. (2.41)]. With $\hat{f}_c$ the local image of the segment f in the coordinates of c, the liquid trace is

$$a^{\mathrm{l}}_{c,f} = \frac{|\hat{f}_c \cap \hat{Q}^{\mathrm{l}}_c|}{|\hat{f}_c|} \tag{4.21}$$

(the solid half-plane is applied first, Section 7.3; pure cells give 0 or the open trace $a^{\mathrm{o}}_{c,f}$). On an interior face,

$$a^{\mathrm{l}}_f = \begin{cases} 0, & \text{no open cell of } \mathcal{C}(f) \text{ has } C^{\mathrm{l}}_c > 0, \\ a^{\mathrm{o}}_f, & \text{no open cell of } \mathcal{C}(f) \text{ has } C^{\mathrm{l}}_c < 1, \\ \mathrm{clamp}\left(\tfrac{1}{2}\sum_{c\in\mathcal{C}(f)} a^{\mathrm{l}}_{c,f},\ 0,\ a^{\mathrm{o}}_f\right), & \text{otherwise,} \end{cases} \tag{4.22}$$

on a boundary face $a^{\mathrm{l}}_f = \mathrm{clamp}(a^{\mathrm{l}}_{c(f),f}, 0, a^{\mathrm{o}}_f)$, and on every face $a^{\mathrm{g}}_f = a^{\mathrm{o}}_f - a^{\mathrm{l}}_f$. The pure-phase cases keep an absent phase exactly closed. The apertures belong to the reconstructed state: they are computed once per reconstruction and read by the CFL bound, the two-velocity transport and the projection.

The public `interface_height` returns, for a column in direction k at transverse position x_t (wrapped if periodic), the height obtained by integrating the reconstructed phase indicator along the column,

$$H(x_t) = \begin{cases} o_k + \sum_c h_{c,k}\,\lambda_c, & \text{liquid at the low end,} \\ o_k + X_k - \sum_c h_{c,k}\,\lambda_c, & \text{liquid at the high end,} \end{cases} \tag{4.23}$$

where λ_c is the liquid fraction of the line $x_t = \text{const}$ in cell c according to its PLIC. The two ends must be in opposite phases and no cell of the column may be cut by a solid; otherwise it throws.

Checks (`figures/generate.jl`). On the 36 faces with both phases of the sloped-wall fixture, an independent evaluation of (4.21) and (4.22) agrees with the code to 5.6×10^{-17}; $a^{\mathrm{l}}_f + a^{\mathrm{g}}_f = a^{\mathrm{o}}_f$ holds to 0.

> *Implementation.* `_face_aperture` (`src/two_phase/forces.jl`) (4.21) and `_clip_line_interval` (`src/two_phase/forces.jl`); `_phase_face_apertures` (`src/two_phase/two_velocity/transport_geometry.jl`), `_current_phase_face_apertures!` (`src/two_phase/two_velocity/transport_geometry.jl`) and `_phase_boundary_face_apertures` (`src/two_phase/two_velocity/transport_geometry.jl`) (4.22); `interface_height` (4.23) via `_interface_height` (`src/interface/height_functions.jl`). Tests: `test/two_phase/two_velocity/transport/two_velocity.jl` ("Two-phase cell, staggered and face geometry partitions") (aperture partition on uniform and AMR meshes), `test/two_phase/two_velocity/transport/two_velocity.jl` ("Phase apertures are computed once per reconstructed state"), `test/indicator/phase/height_functions.jl` ("Physical interface height").

4.9 Initialisation

Fractions are initialised from a prescribed liquid region Ω_0^{l}; the fields are then reconstructed. Three descriptions are supported.

Half-plane $\Omega_0^{\mathrm{l}} = \{\, \boldsymbol{x} \mid \boldsymbol{n} \cdot \boldsymbol{x} \le d \,\}$ (`initialize!` with a normal and a constant): in uncut cells $\alpha_c^{\mathrm{l}} = \mathcal{A}(\mathsf{H}_c\boldsymbol{n},\, d - \boldsymbol{n}\cdot\boldsymbol{x}_c; \hat{Q})$ exactly by (4.8); in cut cells $\alpha_c^{\mathrm{l}} = |\{\, \hat{\boldsymbol{x}} \in \hat{Q}_c^{\mathrm{o}} \mid \mathsf{H}_c\boldsymbol{n}\cdot\hat{\boldsymbol{x}} \le d - \boldsymbol{n}\cdot\boldsymbol{x}_c \,\}|$.

Polygon (`PolygonIndicator`, a simple polygon in either orientation): $\alpha_c^{\mathrm{l}} = |\Omega_0^{\mathrm{l}} \cap c|/|c|$ by Sutherland–Hodgman clipping against the four sides of c and, with a solid, against the solid half-plane in local coordinates, clamped to $[0, \alpha_c^{\mathrm{o}}]$. No quadrature tolerance is involved.

Level set ($\Omega_0^{\mathrm{l}} = \{\, \boldsymbol{x} \mid q(\boldsymbol{x}) \le 0 \,\}$, any finite real function): per cell the area is computed recursively to the absolute tolerance $\varepsilon^q|c|$ ($\varepsilon^q = 10^{-10}$, depths at most `maxdepth` $= 16$). For a rectangle $\mathcal{Q}$,

1. if a Lipschitz bound Λ_q is given and $|q(\boldsymbol{x}_{\mathcal{Q}})| \ge \Lambda_q \operatorname{diam}(\mathcal{Q})/2$, $\mathcal{Q}$ is pure;
2. q is sampled at the corners, edge midpoints and centre; if all nine samples have one strict sign (and no Λ_q is given, depth at least `min_depth`), $\mathcal{Q}$ is pure;
3. if the samples change sign or vanish and q is strictly monotone in direction k on the three sample lines, with one-sided finite differences (step 10^{-5} of the width) of the same sign at all nine points, the area is the integral over x_t of the length of $\{\, x_k \mid q(\boldsymbol{x}) \le 0 \,\}$ on the line through x_t, found by bisection; the integral is split where q changes sign on the two transverse sides and evaluated by adaptive 8-point Gauss–Legendre quadrature with interval halving;
4. otherwise, if a Lipschitz bound Λ_q is given and the area that $\mathcal{Q}$ could misattribute to its declared phase is at most $\varepsilon^q|c|$, $\mathcal{Q}$ is accepted with the sign of its centre. A sample s with that sign rules the other phase out of the ball of radius $|q(s)|/\Lambda_q$ about s, because Λ_q bounds how fast q crosses zero; the other phase therefore lies outside the union of the nine balls, hence outside the largest one alone, and the misattributed area is at most $|\mathcal{Q}|$ minus that ball's overlap with $\mathcal{Q}$, which is evaluated in closed form. Accepting $\mathcal{Q}$ adds at most ε^q to the cell's fraction, and $\varepsilon^q|c|$ does not shrink with $\mathcal{Q}$;
5. otherwise $\mathcal{Q}$ is split into four quarters, each with a quarter of its quadrature tolerance; at depth `maxdepth` initialisation fails.

Fractions within $64\epsilon_{\mathrm{mach}}$ of zero or one are set to zero or one. Failures leave the field unchanged. Without Λ_q, features that fall between the samples of item 2 are not detected; `min_depth` forces a finer search. Item 4 is what resolves a tangential contact: with Λ_q given, a level set that touches a side of $\mathcal{Q}$ at a point that is never sampled is not certified pure by item 1 at any depth, item 3 does not apply, and the largest ball of item 4 covers all of $\mathcal{Q}$ except a small remainder near the contact. A circle of radius 0.27 centred at $(0.52, 0.47)$ in the unit square with 16×16 cells and `lipschitz=1` touches the grid line $x_1 = 0.25$ and initialises with a volume error of 6×10^{-17}, the value the same circle gets without Λ_q; `maxdepth=14` and $\varepsilon^q = 10^{-13}$ still throw for it, so it is the tolerance and not the depth limit that accepts the contact.

With a solid, the level set is first initialised and reconstructed on a field without solid (same `reconstruction_method`); then $\alpha_c^{\mathrm{l}} = |\hat{Q}_c^{\mathrm{o}} \cap \{\, \hat{\boldsymbol{x}} \mid \hat{\boldsymbol{\eta}}_c \cdot \hat{\boldsymbol{x}} \le \hat{\phi}_c \,\}|$ with that field's line, the exact intersection of two PLIC reconstructions rather than of the exact regions.

> *Implementation.* `initialize!` and `InterfaceField(mesh, levelset; atol=1e-10, maxdepth=16, min_depth=0, lipschitz=nothing)` with the recursion `_levelset_area` (`src/interface/levelset_initialization.jl`) and the area bound `_levelset_disk_area` (`src/interface/levelset_initialization.jl`); polygons `PolygonIndicator`; half-plane `initialize!(field, normal, intercept)`. Tests: `test/indicator/phase/levelset_initialization.jl` ("Level-set cell fractions on uniform, mixed and AMR meshes") (half-planes, circle, ellipse and wave areas to 2×10^{-11}), `test/indicator/phase/levelset_initialization.jl` ("Polygon indicators use exact geometric clipping") (polygons to round-off), `test/indicator/phase/levelset_initialization.jl` ("Level-set pure cells are exactly pure"), `test/indicator/phase/levelset_initialization.jl` ("Level set tangent to a grid line at a Lipschitz bound") (tangential contact, and the disk-area bound against independent values), `test/indicator/phase/levelset_initialization.jl` ("Interior components and initialization failures") (enclosed components, failures leave the field unchanged), `test/indicator/geometry.jl` ("Geometry-aware level-set initialization converges in open volume") (bounded by α_c^{o}, second order in open volume).

4.10 Time discretisation and update schedule

Time discretisation is not applicable: the reconstruction is a map from the current fractions and solid to lines, curvatures and apertures. It is evaluated by `reconstruct!` after initialisation, after every directional sweep of the transport (Chapter 5), after a regrid (the transfer of Chapter 12 prolongs fractions by clipping the parent's lines and then reconstructs) and after the solid's geometry changes. A call is a no-op if the fractions, the leaf set and the solid's revision are unchanged.

> *Implementation.* Calls: `_advect_wy!` (`src/vof/fluxes.jl`) (after each sweep), `_finish_prolongation!` (`src/interface/interface_field.jl`) (after a regrid); caching `_reconstruction_is_current` (`src/interface/interface_field.jl`) and `_mark_reconstruction_current!` (`src/interface/interface_field.jl`).

— 5 —

Volume-of-fluid and phase-scalar transport

Equation reference, audited against commit `a2ac51b`.

The liquid fraction of Section 4.2 is transported by a directionally split geometric volume-of-fluid method of Weymouth and Yue [33] (WY) [19, §3.2.3]: one sweep per coordinate direction, each moving time-integrated phase volumes through the owned faces, with a dilation term weighted by a phase indicator that is frozen for the time step, and a reconstruction of the interface between the sweeps. Intrinsic phase scalars are carried by the same phase volumes. For the two-velocity model the phase volumes come from frozen phase subfaces, each extruded with its own phase velocity. This chapter states the continuous balance, the discrete balance and the geometric fluxes, the update of a sweep, the bounds, snapping and time-step restriction, the phase scalars, the frozen carrier, and the time-integrated balances with the exact conditions under which they conserve volume. The transport is a nonlinear map of the reconstruction (Chapter 4) and is not derived from a method-of-lines semi-discretisation; the face integrals are approximated in space and time at once. The thesis transports mass and momentum with dimensionally unsplit donating regions [19, §§6.3–6.4]; the deviations are stated where they arise. Staggered phase scalars and phase momentum, which are advanced in the same sweeps, are the subject of Chapter 6.

5.1 Continuous model

The open domain Ω^{o} (Section 3.1) is split into the liquid and gas regions $\Omega^{\pi}(t)$, whose common boundary $I(t)$ moves with the normal velocity $\boldsymbol{u}^{\pi} \cdot \boldsymbol{\eta}$; there is no phase change, so this normal velocity is continuous across I [19, eq. (2.2)]. The one-velocity model has $\boldsymbol{u}^{\mathrm{l}} = \boldsymbol{u}^{\mathrm{g}} = \boldsymbol{u}$; the two-velocity model has a velocity per phase. With $c^{\pi}(t) = c^{\mathrm{o}} \cap \Omega^{\pi}(t)$ and $f^{\pi}(t) = f^{\mathrm{o}} \cap \Omega^{\pi}(t)$ the open parts of a cell and of a face occupied by phase π, the volume of each phase in a fixed cell obeys, over one time step,

$$|c^{\pi}(t^{(n+1)})| - |c^{\pi}(t^{(n)})| + \int_{t^{(n)}}^{t^{(n+1)}} \sum_{f \in \mathcal{F}(c)} \int_{f^{\pi}(t)} \boldsymbol{u}^{\pi} \cdot o_{c,f} \boldsymbol{n}_f \, \mathrm{d}S \, \mathrm{d}t = 0 \tag{5.1}$$

[19, eqs. (3.2) and (6.14)]; the solid boundary carries no flux ($\boldsymbol{u} \cdot \boldsymbol{n} = 0$, Chapter 7). A cell-averaged intrinsic phase quantity φ^{π} (a phase density, a tracer concentration) is carried by

its phase,

$$\int_{c^\pi(t^{(n+1)})} \varphi^\pi \,\mathrm{d}V - \int_{c^\pi(t^{(n)})} \varphi^\pi \,\mathrm{d}V + \int_{t^{(n)}}^{t^{(n+1)}} \sum_{f\in\mathcal{F}(c)} \int_{f^\pi(t)} \varphi^\pi \,\boldsymbol{u}^\pi \cdot o_{c,f}\boldsymbol{n}_f \,\mathrm{d}S\,\mathrm{d}t = 0, \quad (5.2)$$

which is the thesis' conservation law [19, eq. (6.2)] with the constant ρ^π divided out; (5.1) is the case $\varphi^\pi = 1$. Adding (5.1) for both phases and letting the step tend to zero gives the mixture constraint $\sum_{f\in\mathcal{F}(c)} \sum_\pi \int_{f^\pi} \boldsymbol{u}^\pi \cdot o_{c,f}\boldsymbol{n}_f \,\mathrm{d}S = 0$ [19, eqs. (2.5b) and (6.15)]. On impermeable or periodic boundaries, summing (5.1) over all cells shows that the volume of each phase is constant.

5.2 Discrete balance and phase fluxes

The stored unknown is the liquid fraction per full cell α_c^{l}; the transport works with the conditional fraction $C_c^{\mathrm{l}} = \alpha_c^{\mathrm{l}}/\alpha_c^{\mathrm{o}}$ and the phase volume $|c^{\mathrm{o}}|C_c^\pi = |c|\alpha_c^\pi$, $C_c^{\mathrm{g}} = 1 - C_c^{\mathrm{l}}$ ((4.4)). Writing the space–time integral over f in (5.1) as the time-integrated phase volume $\delta t|f|v_f^\pi$ that crosses f in the direction $\boldsymbol{n}_f$, the balance of every open cell is the exact finite-volume statement

$$|c^{\mathrm{o}}|C_c^{\pi,(n+1)} = |c^{\mathrm{o}}|C_c^{\pi,(n)} - \sum_{f\in\mathcal{F}(c)} o_{c,f}\,\delta t|f|v_f^\pi, \quad (5.3)$$

and the scheme consists entirely in the geometric approximation of these face volumes from the reconstruction (Section 5.4) and in the directional splitting (Section 5.3). Every owned segment is evaluated once and enters its two cells with opposite orientation, so the face terms cancel in the sum over cells exactly, also at hanging segments and periodic seams ((3.12)); physical boundary faces carry no volume (Section 5.6). A time discretisation separate from the spatial one does not exist: the flux is a space–time integral evaluated at once, and no method-of-lines form of the scheme is implemented.

> *Implementation.* `InterfaceField` (`fraction` $= \alpha^{\mathrm{l}}$, `conditional_fraction` $= C^{\mathrm{l}}$) and `WYAdvectionWorkspace(interface; ctu_threshold=0.5)`, whose vectors `open_flux`, `liquid_flux`, `gas_flux` hold the signed time-integrated volumes $\delta t|f|v_f^{\mathrm{o}}$, $\delta t|f|v_f^{\mathrm{l}}$, $\delta t|f|v_f^{\mathrm{g}}$ in the order of `mesh.faces`, each face from the sweep along its axis; the incidence $o_{c,f}$ (`cell_face_orientations`) is the negated `_cell_face_incidence` (`src/operators/elliptic.jl`).

5.3 Directional sweeps

A time step consists of two sweeps $j = 1, 2$ along the axes $k_1 \neq k_2$, $(k_1, k_2) \in \{(1,2),(2,1)\}$. Before the first sweep the fractions are reconstructed (Chapter 4) and the indicator (4.6) is frozen,

$$\chi_c^{\mathrm{l}} = \begin{cases} 1, & C_c^{\mathrm{l},(n)} > \tfrac{1}{2}, \\ 0, & \text{otherwise}, \end{cases} \qquad \chi_c^{\mathrm{g}} = 1 - \chi_c^{\mathrm{l}}, \quad (5.4)$$

for both sweeps. With one velocity, the carrier volume of a face is

$$\delta t|f|v_f^{\mathrm{o}} = \delta t|f|a_f^{\mathrm{o}} u_f, \quad (5.5)$$

the same in both sweeps ($a_f^{\mathrm{o}} = 1$ without a solid); the two-velocity carriers replace it in Section 5.8. Sweep j moves the faces $f \in \mathcal{F}^{[k_j]}$ with the liquid volumes $\delta t|f|v_f^{\mathrm{l},[j]}$ cut from the reconstruction after sweep $j-1$ (Section 5.4) and updates every open cell, $C_c^{\mathrm{l},[0]} = C_c^{\mathrm{l},(n)}$,

$$|c^{\mathrm{o}}|C_c^{\mathrm{l},[j]} = |c^{\mathrm{o}}|C_c^{\mathrm{l},[j-1]} - \sum_{f\in\mathcal{F}(c)\cap\mathcal{F}^{[k_j]}} o_{c,f}\,\delta t|f|\left(v_f^{\mathrm{l},[j]} - \chi_c^{\mathrm{l}} v_f^{\mathrm{o}}\right) + |c^{\mathrm{o}}|\delta C_c^{[j]}, \tag{5.6}$$

where $\delta C_c^{[j]}$ is the change by clamping and snapping (Section 5.5); covered cells keep $C_c^{\mathrm{l}} = 0$. Then $\alpha_c^{\mathrm{l}} = \alpha_c^{\mathrm{o}} C_c^{\mathrm{l},[j]}$ and the interface is reconstructed. This is the thesis' sweep [19, eq. (3.46)] with the open volumes of the cut cells, the owned segments of the composite mesh and the time-integrated volumes in place of $\delta t|f|v_f$: the term $\chi_c^{\mathrm{l}}\sum_f o_{c,f}\delta t|f|v_f^{\mathrm{o}}$ is the dilation of the one-dimensional carrier, which is not divergence free. Because the liquid and gas volumes of a face partition its carrier volume,

$$\delta t|f|v_f^{\mathrm{l},[j]} + \delta t|f|v_f^{\mathrm{g},[j]} = \delta t|f|v_f^{\mathrm{o}} \tag{5.7}$$

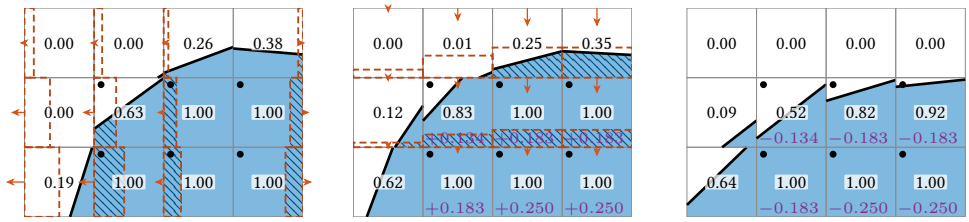
(exactly, by the construction of (5.13)), the gas fraction $1 - C_c^{\mathrm{l}}$ obeys (5.6) with l and g exchanged: in a cell with $\chi_c^{\mathrm{l}} = 0$ the liquid is moved conservatively, in a cell with $\chi_c^{\mathrm{l}} = 1$ the gas, which is the thesis' geometric reading of the dilation term [19, eqs. (3.48)–(3.49)]. Summing the two sweeps gives, with $v_f^{\mathrm{l}} = v_f^{\mathrm{l},[j]}$ for $f \in \mathcal{F}^{[k_j]}$ and $|c|(\mathfrak{D}v^{\mathrm{o}})_c = \sum_{f\in\mathcal{F}(c)} o_{c,f}|f|v_f^{\mathrm{o}}$ ((3.11)),

$$|c^{\mathrm{o}}|C_c^{\mathrm{l},(n+1)} = |c^{\mathrm{o}}|C_c^{\mathrm{l},(n)} - \sum_{f\in\mathcal{F}(c)} o_{c,f}\,\delta t|f|v_f^{\mathrm{l}} + \chi_c^{\mathrm{l}}\,\delta t|c|(\mathfrak{D}v^{\mathrm{o}})_c + |c^{\mathrm{o}}|\sum_{j=1}^{2}\delta C_c^{[j]}, \tag{5.8}$$

the discrete balance (5.3) plus the dilation of the full carrier and the snapped volume [19, §3.2.3]. With $|c|(\mathfrak{D}v^{\mathrm{o}})_c = |c^{\mathrm{o}}|(\mathfrak{D}^{\mathrm{o}}u)_c$ for one velocity, the dilation vanishes in every cell in which the transport velocity is discretely divergence free (Section 5.9). Figure 5.1 shows a step; there the dilation of a sweep reaches 0.25 of a cell volume and cancels over the two sweeps.

In a flow simulation the transport velocity is the projected velocity of the previous step with the boundary values enforced, and the order alternates, $(1,2)$ in the first step, $(2,1)$ in the second, and so on; `advect!` takes the order as a keyword (default $(1,2)$). Before each sweep the halos of the phase scalars are filled (zero gradient at walls, Section 2.4); the fractions need no halo for the fluxes, which read only the two cells of a face.

Deviation from the thesis 5.1 (Directional splitting). The thesis transports mass and momentum of both formulations with dimensionally unsplit donating regions [19, §§3.2.2, 6.3–6.4], the projected liquid velocity included [19, eq. (6.20)], and describes the WY sweeps [19, §3.2.3] as an alternative on rectilinear meshes. The code uses WY sweeps for every carrier, with the alternating order above.



A. $C_c^{\mathrm{l},(n)}$; sweep 1 along $\boldsymbol{e}_1$. **B.** $C_c^{\mathrm{l},[1]}$; sweep 2 along $\boldsymbol{e}_2$. **C.** $C_c^{\mathrm{l},(n+1)} = C_c^{\mathrm{l},[2]}$.

Figure 5.1. One time step of (5.6) on the uniform fixture of `figures/vof.jl` (unit cells, the streamfunction velocity of Section 5.9, $\delta t = \delta t_{\mathrm{WY}} = 0.524$, order $(k_1, k_2) = (1, 2)$; window $3 \le x_1 \le 7, 6 \le x_2 \le 9$). Numbers: conditional fractions; dots: frozen indicator $\chi_c^{\mathrm{l}} = 1$, fixed at the start of the step; dashed red: the donating rectangles of the next sweep, hatched their liquid; arrows: the velocity of the next sweep; purple (b, c): the dilation volume of the preceding sweep per cell volume, $\chi_c^{\mathrm{l}} \sum_f o_{c,f} \delta t |f| v_f^{\circ} / |c|$, which reaches 0.25 in a sweep but cancels over the two sweeps to 2.8×10^{-17} because $\mathfrak{D}u = 0$; in full cells it balances the divergence of the one-dimensional liquid flux, so they stay full. The PLIC lines are reconstructed after each sweep.

Deviation from the thesis 5.2 (First moments). The thesis also advects interface centroids, among others for split methods [19, §3.4]; the code transports no first moments, so no moment-of-fluid reconstruction is possible (see also the reconstruction family of Section 4.5).

Implementation. `advect!(interface, velocity, dt, work; scalars=(), staggered_scalars=(), momentum=nothing, density=nothing, momentum_workspace=nothing, sweep_order=(1,2), scheme=LaxWendroff(), amr_interface_scheme, check_timestep=true)` calls `_advect_wy!` (`src/vof/fluxes.jl`), which freezes the indicator and the dilation values, runs the sweeps with the update (5.6) and reconstructs after each; the alternation is in `_transport_substep!` (`src/two_phase/stepping.jl`) and `_transport_substep!` (`src/two_phase/two_velocity/stepping.jl`). Tests: `test/two_phase/transport.jl` ("WY volume and phase constants on uniform, mixed and regridded meshes") (bounded fractions, volume to 2×10^{-14} and constant phase scalars, alternating order), `test/two_phase/transport.jl` ("WY dimensional dilation preserves phase constants") (solenoidal composite face velocities from a streamfunction: volume to 3×10^{-14} over a reversing flow), `test/two_phase/transport.jl` ("WY transport is independent of base-root partitioning") (to 4×10^{-14}), `test/integration/two_phase_vortex.jl` ("Classical T=1 vortex refinement study").

5.4 Donating regions and phase volumes

Let $f \in \mathcal{F}^{[k]}$ be swept with carrier velocity $u_f \neq 0$. Its donor is the upwind cell, $c^{\mathrm{u}}(f) = c^-(f)$ if $u_f \ge 0$ and $c^{\mathrm{u}}(f) = c^+(f)$ otherwise, so that $o_{c^{\mathrm{u}}(f),f} = \operatorname{sgn}(u_f)$. In the local coordinates $\hat{\boldsymbol{x}}$ of the donor (Section 4.3), f occupies the side $(\hat{\boldsymbol{x}})_k = \frac{1}{2} o_{c^{\mathrm{u}}(f),f}$ over its tangential extent $\hat{r}_t^- \le (\hat{\boldsymbol{x}})_t \le \hat{r}_t^+$, $t \neq k$ (a half side if the donor is the coarse cell at a level transition, (2.7)). The donating rectangle of depth $\hat{d}$ is the strip of the donor along that side,

$$\hat{R}_f(\hat{d}) = \{\, \hat{\boldsymbol{x}} \mid \frac{1}{2} - \hat{d} \le o_{c^{\mathrm{u}}(f),f} (\hat{\boldsymbol{x}})_k \le \frac{1}{2},\ \hat{r}_t^- \le (\hat{\boldsymbol{x}})_t \le \hat{r}_t^+ \,\}, \qquad 0 \le \hat{d} \le 1, \tag{5.9}$$

and its depth is fixed by the carrier volume,

$$\hat{d}_f = \begin{cases} \delta t\,|u_f|\,/h_{c^{\mathrm{u}}(f),k}, & \text{no solid,} \\ a^{\mathrm{o}}_f \delta t\,|u_f|\,/h_{c^{\mathrm{u}}(f),k}, & \text{solid present, } \alpha^{\mathrm{s}}_{c^{\mathrm{u}}(f)} = 0, \\ \text{root of } |\hat{R}_f(\hat{d}) \cap \hat{Q}^{\mathrm{o}}_{c^{\mathrm{u}}(f)}||c^{\mathrm{u}}(f)| = \left|\delta t|f|v^{\mathrm{o}}_f\right|, & 0 < \alpha^{\mathrm{s}}_{c^{\mathrm{u}}(f)} < 1. \end{cases} \tag{5.10}$$

In the first two cases the area of $\hat{R}_f(\hat{d}_f)$ times $|c^{\mathrm{u}}(f)|$ is the carrier volume (the second is clamped to $\hat{d} \le 1$); in a pure-gas or pure-liquid donor the rectangle is not needed. In a mixed cut donor the depth is found by 54 bisection steps on $[0, 1]$, the open area being evaluated with the solid half-plane ((7.2)); if the carrier volume exceeds the open area of the whole strip $\hat{R}_f(1)$, both in units of $|c^{\mathrm{u}}(f)|$, by more than $256\epsilon_{\mathrm{mach}}$ times the larger of one and that area, the transport stops with an error. The donating region is the open part of the rectangle in physical coordinates,

$$\Delta_f = \{\, \boldsymbol{x}_{c^{\mathrm{u}}(f)} + \mathsf{H}_{c^{\mathrm{u}}(f)}\hat{\boldsymbol{x}} \mid \hat{\boldsymbol{x}} \in \hat{R}_f(\hat{d}_f) \cap \hat{Q}^{\mathrm{o}}_{c^{\mathrm{u}}(f)} \,\}, \tag{5.11}$$

and the liquid fraction of the swept volume is that of the donor's reconstruction in it,

$$C^{\Delta}_f = \begin{cases} C^{\mathrm{l}}_{c^{\mathrm{u}}(f)}, & C^{\mathrm{l}}_{c^{\mathrm{u}}(f)} \in \{0, 1\}, \\ \mathcal{A}\left(\hat{\boldsymbol{\eta}}_{c^{\mathrm{u}}(f)}, \hat{\phi}_{c^{\mathrm{u}}(f)}; \hat{R}_f(\hat{d}_f)\right), & \alpha^{\mathrm{s}}_{c^{\mathrm{u}}(f)} = 0, \\ \mathrm{clamp}\left(|\hat{R}_f(\hat{d}_f) \cap \hat{Q}^{\mathrm{l}}_{c^{\mathrm{u}}(f)}||c^{\mathrm{u}}(f)|/\left|\delta t|f|v^{\mathrm{o}}_f\right|, 0, 1\right), & 0 < \alpha^{\mathrm{s}}_{c^{\mathrm{u}}(f)} < 1, \end{cases} \tag{5.12}$$

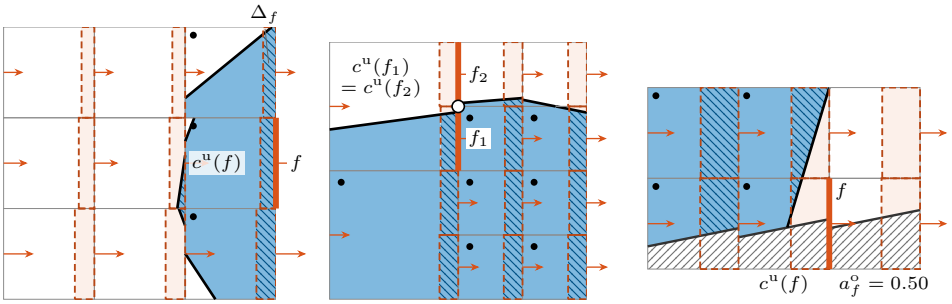
with the rectangle-area function (4.8) in uncut donors and Sutherland–Hodgman clipping of the rectangle by the solid and the liquid half-planes in cut donors ($\hat{Q}^{\mathrm{l}}_c$ of (4.7)). The phase volumes of the face are

$$\delta t|f|v^{\mathrm{l},[j]}_f = C^{\Delta}_f\, \delta t|f|v^{\mathrm{o}}_f, \qquad \delta t|f|v^{\mathrm{g},[j]}_f = \delta t|f|v^{\mathrm{o}}_f - \delta t|f|v^{\mathrm{l},[j]}_f, \tag{5.13}$$

both zero if $u_f = 0$. In an uncut donor $\delta t|f|v^{\mathrm{l}}_f$ is the liquid volume of the reconstruction inside Δ_f, the thesis' flux [19, eq. (3.45)]. On the composite mesh each owned segment has its own rectangle: a coarse donor whose side carries two segments has two rectangles, each over half of the side, with the depths of the two segment velocities, whereas a fine donor sweeps its whole side (Figure 5.2b). The tangential offset of the segment is wrapped across periodic seams.

Cut donor strips. On a hanging side the open part of the strip behind one segment can be much smaller than the whole open cell. The time-step restriction therefore includes the actual open strip of each outgoing segment and the open portions available to pairs of opposing segments (Section 5.6). In the fixture of `strip_limitation_check` the former whole-cell bound returned a step at which the segment had to sweep more than its strip could hold (on `origin/main`: 0.0267 against 0.0253 at $\delta t = 0.0594$); the revised bound limits that segment before the sweep, so the returned step $\delta t = 0.0563$ keeps the segment volume 0.0253 within the open strip 0.0253 (issue 52). The same restriction applies to each frozen phase subface, including in a pure donor.

Checks (`figures/vof.jl`, `one_velocity_flux_check`). An independent evaluation of (5.9) to (5.13) by polygon clipping with its own bisection, on four fixtures at



A. Uniform faces. **B.** Coarse donor, two segments. **C.** Cut donor and cut face.

FIGURE 5.2. Donating regions (5.9) to (5.11) of the first sweep, along $\boldsymbol{e}_1$, at $\delta t = \delta t_{\mathrm{WY}}$. Dashed red: the rectangles $\hat{R}_f(\hat{d}_f)$ of all faces with $u_f \neq 0$, mapped to physical coordinates; arrows: u_f; hatched: the liquid part, whose volume is $\delta t|f|v_f^{\mathrm{l}}$; dots: cells with the frozen indicator $\chi_c^{\mathrm{l}} = 1$. (a, b) Shear flow $u = 1 + 0.3\sin(\pi x_2/4)$ around a drop on a composite mesh (`vof_shear_fixture` of `figures/vof.jl`; unit coarse cells, the columns $4 \leq x_1 \leq 8$ refined once); the depth is $\delta t\,|u_f|$ in units of the donor width, at most $\nu^\dagger = \frac{1}{2}$ ((5.15)), so the rectangles of the coarse cells are half as deep as those of the fine cells for equal velocities. (b) Each of the two segments of the coarse side has its own rectangle over half of the coarse donor, with its own velocity and depth; open circle: the hanging vertex. (c) Uniform flow $u = 1$ over a sloped wall (hatched), PLIC lines of `:lhf_ghf`; the rectangle spans the whole face and its depth $\hat{d}_f = 0.423$ is set by the open area (shaded), whereas $\delta t\,|u_f|\,/h_{c^{\mathrm{u}}(f),1} = 0.417$.

$\delta t = \delta t_{\mathrm{WY}}$, reproduces `liquid_flux` on 1026 faces between cells of one level without solid (95 with a mixed donor) to $1.1\times10^{-16}|c^{\mathrm{u}}(f)|$, on 86 faces at level transitions (24 mixed donors) to $5.6\times10^{-17}|c^{\mathrm{u}}(f)|$ and on 200 faces with $a_f^{\mathrm{o}} < 1$ or a cut donor (29 mixed cut donors) to $8.3\times10^{-17}|c^{\mathrm{u}}(f)|$; the partition (5.7) holds to 6.9×10^{-18}.

Deviation from the thesis 5.3 (Donating regions in cut cells and at level transitions). The thesis' split donating region is the extrusion of the face by $\delta t\,|u_f|$ on a conforming mesh [19, eq. (3.45)]. The code extrudes each owned segment in units of its donor's width and, with a solid, chooses the depth so that the open part of the rectangle has the carrier volume (5.10): in cut donors by inverting the clipped area, in uncut donors by shortening the rectangle by a_f^{o}. This is the thesis' enforcement of donating-region volumes [19, eq. (6.38)] applied to the solid; it makes the phase volumes a bounded partition of the carrier volume (5.7).

Implementation. `_swept_fraction` (`src/vof/fluxes.jl`) (5.10) and (5.12) and `_compute_directional_wy_fluxes!` (`src/vof/fluxes.jl`) (5.5) and (5.13); open areas `_open_subcell_area` (`src/embedded/geometry.jl`); clipping `_clip_polygon` (`src/embedded/geometry_utils.jl`) and `_polygon_area` (`src/embedded/geometry_utils.jl`). Tests: `test/indicator/geometry.jl` ("Cut-cell WY swept flux is a bounded phase partition") (swept liquid and gas within the donor's to $512\epsilon_{\mathrm{mach}}$), `test/indicator/geometry.jl` ("Cut-cell WY reverses wall-tangent contact-line translation") (order at least 1.8 on a sloped wall, `:lhf_ghf`), `test/integration/anisotropic_cells.jl` ("anisotropic geometric VOF advection") (cells with aspect ratio two).

5.5 Bounds, clamping and snapping

After the update of a sweep every open cell is checked and the fraction is clamped and snapped with the fraction floor ε^α ((4.5)). With $\check{C}_c^{\mathrm{l},[j]}$ the right-hand side of (5.6) without $\delta C_c^{[j]}$, the code raises an error if the liquid volume $|c^{\mathrm{o}}|\check{C}_c^{\mathrm{l},[j]}$ leaves $[0, |c^{\mathrm{o}}|]$ by more than $128\epsilon_{\mathrm{mach}}|c|$ (`_wy_fraction_in_bounds` (`src/vof/fluxes.jl`)): the round-off of the clipped and summed volumes is measured in units of the full cell, not amplified by a small open fraction (since issue 55), while genuine violations still raise, including those produced by an excessive step. Otherwise it clamps $\check{C}_c^{\mathrm{l},[j]}$ to $[0, 1]$ and snaps it, which together give

$$C_c^{\mathrm{l},[j]} = \begin{cases} 0, & \check{C}_c^{\mathrm{l},[j]} \le \varepsilon^\alpha, \\ 1, & \check{C}_c^{\mathrm{l},[j]} \ge 1 - \varepsilon^\alpha, \\ \check{C}_c^{\mathrm{l},[j]}, & \text{otherwise}, \end{cases} \qquad \delta C_c^{[j]} = C_c^{\mathrm{l},[j]} - \check{C}_c^{\mathrm{l},[j]}. \tag{5.14}$$

The reconstruction that follows applies (4.5) again, which leaves these values unchanged up to rounding. Each snap changes the liquid volume by at most $\varepsilon^\alpha|c^{\mathrm{o}}|$, each clamp by at most $128\epsilon_{\mathrm{mach}}|c|$, and both enter the balance (5.8) (deviation 4.1).

Boundedness. The argument of the thesis' boundedness lemma [19, Lemma 3.3] carries over when, in the sweep, the open portions of the rectangles through which a cell donates are pairwise disjoint: the outflow of the phase that the cell moves conservatively (liquid if $\chi_c^{\mathrm{l}} = 0$, gas otherwise) is then at most the volume of that phase in the reconstruction, so that phase fraction stays non-negative, and its inflow over both sweeps is at most $\nu_c^{\mathrm{in}} \le \nu^\dagger \le \frac{1}{2}$ of the cell ((5.15)), so it stays at most $\frac{1}{2} + \nu_c^{\mathrm{in}} \le 1$; up to rounding and to the at most ε^α added by a snap after the first sweep, $0 \le C \le 1$ before clamping. The rectangles are disjoint in uncut donors, where $\hat{d}_f \le \nu^\dagger \le \frac{1}{2}$ by the extrusion bound. In a cut donor the corresponding *open parts* are disjoint: outgoing regions from one side have disjoint tangential intervals, while each pair from opposing sides is separated within their common open support by the geometric bound of Section 5.6. This also applies to hanging sides and frozen subfaces. The check remains active to detect genuine violations, including steps supplied with the check disabled. The separation plane need not be the cell midpoint: a cut strip may have no open area on the half adjacent to the face but have positive open area farther inside. The adaptive separation still permits a positive step whenever the required strip areas are positive. The bound $\nu_c^{\mathrm{in}} \le \frac{1}{2}$ cannot be relaxed: a cell with $C_c^{\mathrm{l},(n)} = \frac{1}{2}$ that receives liquid from both sides reaches $\check{C}_c^{\mathrm{l}} = \frac{1}{2} + \nu_c^{\mathrm{in}}$.

> *Implementation.* Check, clamp and snap: `_advect_wy!` (`src/vof/fluxes.jl`); the reconstruction's snap `_refresh_conditional_fraction!` (`src/interface/interface_field.jl`). Tests: `test/two_phase/transport.jl` ("Global fraction floor") (a displacement below ε^α leaves fractions and scalars unchanged).

5.6 Time-step restriction

For a carrier with face rates $|f|a_f^{\mathrm{o}}u_f$ the code bounds, in every open cell and on every face,

$$\nu_c^{\mathrm{in}} = \frac{\delta t}{|c^{\mathrm{o}}|}\sum_{f\in\mathcal{F}(c)}\left[-o_{c,f}|f|a_f^{\mathrm{o}}u_f\right]^+, \qquad \nu_c^{\mathrm{out}} = \frac{\delta t}{|c^{\mathrm{o}}|}\sum_{f\in\mathcal{F}(c)}\left[o_{c,f}|f|a_f^{\mathrm{o}}u_f\right]^+,$$
$$\nu_f^{\mathrm{ext}} = \frac{\delta t\,|u_f|}{h_{c^{\mathrm{u}}(f),k(f)}}, \tag{5.15}$$

summed over the faces of both axes, by $\nu^\dagger$. For every outgoing face of a cut donor let $q_f = |f|a_f^{\mathrm{o}}\,|u_f|$ and let A_f be the physical open area of its complete tangential strip. For each pair of outgoing faces on opposite sides whose tangential intervals overlap in the open part of the donor, choose a plane halfway between the minimum and maximum normal coordinates of that common open part. Each face of the pair also receives the area A of its own strip on its side of this plane. All such full-strip and pairwise areas define constraints

$$\nu^{\mathrm{geom}} = \frac{\delta t q_f}{2A} \le \nu^\dagger. \tag{5.16}$$

If the common open part is empty, no pairwise restriction is needed. A positive rate through a strip whose open area is zero, or below the sweep's area tolerance $256\epsilon_{\mathrm{mach}}a_f^{\mathrm{o}}$, cannot be represented by a donating region in that donor: the bound raises an `ArgumentError` naming the face, the donor cell and the strip area instead of returning an unusable step. Phase apertures within $64\epsilon_{\mathrm{mach}}$ of a pure phase are snapped to zero (the traces are fractions of the whole face, so their rounding is absolute), and a face whose own open extent is at that level is left out of the strip constraints because it carries no resolvable flux. The sweep's area tolerance is widened by the slack of the step check ((13.3)), so a step that passes the check is not rejected by the sweep. Otherwise `stable_timestep` returns

$$\delta t_{\mathrm{WY}} = \nu^\dagger \Big/ \max\Big\{\max_c \frac{\nu_c^{\mathrm{in}}}{\delta t}, \max_c \frac{\nu_c^{\mathrm{out}}}{\delta t}, \max_f \frac{\nu_f^{\mathrm{ext}}}{\delta t}, \max \frac{\nu^{\mathrm{geom}}}{\delta t}\Big\}, \qquad 0 < \nu^\dagger \le \frac{1}{2}\ (\text{default } \frac{1}{2}). \tag{5.17}$$

where the last maximum is zero without geometric constraints. It returns ∞ for a vanishing velocity; covered cells do not enter. It also requires a finite velocity with $u_f = 0$ on every physical boundary face ("impermeable or periodic boundaries"); inflow and outflow boundaries are not supported by the VOF transport. `advect!` checks $\delta t \le (1 + 10^{-6} + 32\epsilon_{\mathrm{mach}})\delta t_{\mathrm{WY}}$ with $\nu^\dagger = \frac{1}{2}$ unless `check_timestep=false`. In an uncut donor the extrusion bound confines every rectangle to the half of the donor next to its face. In a cut donor, (5.16) gives $\delta t q_f \le A$ and hence disjoint open donating regions even when their rectangles extend beyond the cell midpoint.

In a flow simulation, `step!` checks $\delta t \le (1+10^{-6}+32\epsilon_{\mathrm{mach}})\min\{\delta t_{\mathrm{WY}}, \delta t_\sigma, \delta t_{\mathrm{grav}}\}$ ((13.3); the 10^{-6} admits the stretched last step of issue 98) with $\nu^\dagger = \frac{1}{2}$ for the velocity it transports (the frozen two-velocity carrier skips this check when `simulate!` has just

evaluated the bound for the same state), where, with $h = \min_{c,k} h_{c,k}$ (Chapter 13),

$$\delta t_\sigma = \sqrt{\frac{(\rho^{\mathrm{l}} + \rho^{\mathrm{g}})h^3}{4\pi\sigma}} \ (\sigma > 0), \qquad \delta t_{\mathrm{grav}} = \sqrt{\frac{(\rho^{\mathrm{l}} + \rho^{\mathrm{g}})h}{2\,|\rho^{\mathrm{l}} - \rho^{\mathrm{g}}|\,|\boldsymbol{g}|}} \tag{5.18}$$

($|\boldsymbol{g}| > 0$, $\rho^{\mathrm{l}} \neq \rho^{\mathrm{g}}$), and ∞ otherwise. Without a prescribed step, `simulate!` evaluates these bounds with $\nu^\dagger = 0.45$ (keyword `cfl`) for the boundary-enforced velocity at the start of every step and takes $\delta t^{(n)} = \min\left\{\delta t^{(n-1)}, \delta t_{\mathrm{WY}}, \delta t_\sigma, \delta t_{\mathrm{grav}}\right\}$: the step never grows; an infinite first bound gives a step of $1/128$ of the requested interval, and the last step is shortened to end at the final time.

Checks (`cfl_check`). An independent evaluation of (5.15) and (5.17) equals `stable_timestep` to 0 (relative) on the uniform, composite and cut fixtures, `advect!` accepts δt_{WY} and rejects $(1 + 2 \times 10^{-6})\delta t_{\mathrm{WY}}$, and the deepest uncut rectangle at δt_{WY} has $\hat{d}_f = 0.500$. On the cut fixture with the plain streamfunction velocity of Section 5.9, whose smallest open fraction is 0.113, the full bound gives $\delta t_{\mathrm{WY}} = 0.091$, whereas the extrusion bound alone would allow 0.316: small cut cells can reduce the step, since no cells are merged.

Deviation from the thesis 5.4 (Time-step bound). The thesis imposes the incoming bound [19, eqs. (3.50)–(3.51)] with $\nu^\dagger = \frac{1}{2}$ and full cell volumes, and the incoming and outgoing bounds for compressible flow [19, eq. (3.55)], from which the containment of every donating region in half of its cell follows on a uniform mesh [19, eq. (3.56)]. The code always imposes both bounds with open volumes, adds the extrusion bound $\nu_f^{\mathrm{ext}} \leq \nu^\dagger$ for coarse-donor segments, and applies the geometric strip and opposing-pair constraints in cut donors ((5.16)); a flow simulation selects steps with $\nu^\dagger = 0.45$.

Implementation. `stable_timestep(velocity, work; cfl=0.5)` (5.17) with the boundary check `_check_wy_workspace` (`src/vof/workspace.jl`); the check in `advect!` is in `_advect_wy!` (`src/vof/fluxes.jl`); flow bounds (5.18) in `_two_phase_timestep_bound` (`src/two_phase/stepping.jl`); `stable_timestep(velocity, solver, workspace; cfl=0.45)`; step selection in `simulate!`. Tests: `test/two_phase/transport.jl` ("WY volume and phase constants on uniform, mixed and regridded meshes") (a step of $2\delta t_{\mathrm{WY}}$ and $\nu^\dagger = 0.6$ are rejected).

5.7 Phase scalars

A `PhaseScalar` stores intrinsic values φ_c^{l} and φ_c^{g}; the conservative amounts $|c^{\mathrm{o}}|C_c^\pi\varphi_c^\pi$ exist only during an update, and `integrate(q, phase)` returns their sum. The values weighting the dilation are frozen at the start of the step, $\chi_c^\pi\varphi_c^{\pi,(n)}$. Sweep j updates the amount with the phase volumes of the fraction update,

$$|c^{\mathrm{o}}|(C_c^\pi\varphi_c^\pi)^{[j]} = |c^{\mathrm{o}}|(C_c^\pi\varphi_c^\pi)^{[j-1]} - \sum_{f \in \mathcal{F}(c) \cap \mathcal{F}^{[k_j]}} o_{c,f}\,\delta t |f| \left(\upsilon_f^{\pi,[j]}(\mathfrak{F}\varphi^\pi)_f - \chi_c^\pi\varphi_c^{\pi,(n)}\upsilon_f^{\mathrm{o}}\right) \tag{5.19}$$

($C^{\mathrm{g}} = 1 - C^{\mathrm{l}}$ and v^{g} of (5.13)). The code stores it in fluctuation form: with the snapped fraction $C_c^{\pi,[j]}$,

$$\varphi_c^{\pi,[j]} = \varphi_c^{\pi,[j-1]} + \frac{1}{|c^{\mathrm{o}}|C_c^{\pi,[j]}} \sum_{f\in\mathcal{F}(c)\cap\mathcal{F}^{[k_j]}} o_{c,f}\,\delta t|f|\Big(v_f^{\pi,[j]}\left(\varphi_c^{\pi,[j-1]} - (\mathfrak{F}\varphi^{\pi})_f\right) + \chi_c^{\pi} v_f^{\mathrm{o}}\left(\varphi_c^{\pi,(n)} - \varphi_c^{\pi,[j-1]}\right)\Big) \tag{5.20}$$

if $C_c^{\pi,[j]} > \varepsilon^{\alpha}$, and $\varphi_c^{\pi,[j]} = \varphi_c^{\pi,[j-1]}$ otherwise. Without snapping (5.20) equals (5.19): it is an algebraic rearrangement. Let $\delta C_c^{\pi,[j]} := C_c^{\pi,[j]} - \check{C}_c^{\pi,[j]}$ be the change of the phase fraction by clamping and snapping; it is $\delta C_c^{[j]}$ for the liquid, $-\delta C_c^{[j]}$ for the gas (the complement of the liquid). If the snapped fraction exceeds ε^{α}, the amount after the sweep exceeds (5.19) by $|c^{\mathrm{o}}|\,\delta C_c^{\pi,[j]}\,\varphi_c^{\pi,[j-1]}$, so a snap to a positive fraction leaves the snapped volume with the value $\varphi_c^{\pi,[j-1]}$. A snap to zero removes the amount that (5.19) assigns to the cell; since all faces of such a cell use upwind states, this is of order $\varepsilon^{\alpha}|c^{\mathrm{o}}|\max|\varphi|$. A constant φ^{π} gives a zero increment, so constants are preserved exactly (bitwise), also across level transitions, in cut cells and when fractions snap.

The face value is the upwind value near the interface and the selected scheme in the bulk. Let c' be the cell of f upwind of the phase volume v_f^{π} ($c^-(f)$ if it is non-negative), and φ^{u}, φ^{d} and φ^{uu} the values of c', of the other cell of f and of the slot $c' - o_{c',f}\boldsymbol{e}_k$ beyond c' (a halo slot at block borders). The face is *near the interface* if a cell $c \in \mathcal{C}(f)$ has $\min\{C_c^{\pi,[j-1]}, \check{C}_c^{\pi,[j]}\} < 1 - 64\epsilon_{\mathrm{mach}}$; then

$$(\mathfrak{F}\varphi^{\pi})_f = \begin{cases} \varphi^{\mathrm{u}}, & \text{near the interface or } \texttt{Upwind},\\ \tfrac{1}{2}(\varphi^{\mathrm{u}} + \varphi^{\mathrm{d}}), & \texttt{Centred},\\ \varphi^{\mathrm{LW}} = \varphi^{\mathrm{u}} + \tfrac{1}{2}(1-\nu_f)(\varphi^{\mathrm{d}} - \varphi^{\mathrm{u}}), & \texttt{LaxWendroff},\\ \varphi^{\mathrm{u}} + \Psi^{\mathrm{M}}(\Theta_f)(\varphi^{\mathrm{LW}} - \varphi^{\mathrm{u}}), & \text{three-point scheme M}, \end{cases} \tag{5.21}$$

where $\check{C}_c^{\pi,[j]}$ is the fraction after the sweep before clamping (5.14), $\nu_f = \left|\delta t|f|v_f^{\pi}\right| / \min_{c\in\mathcal{C}(f)}|c^{\mathrm{o}}|$ and

$$\Theta_f = \frac{h_{c^-(f),k} + h_{c^+(f),k}}{2h_{c',k}}\,\frac{\varphi^{\mathrm{u}} - \varphi^{\mathrm{uu}}}{\varphi^{\mathrm{d}} - \varphi^{\mathrm{u}}}, \tag{5.22}$$

with $\Theta_f = 1$ if both differences are at most $16\epsilon_{\mathrm{mach}}\max(1, |\varphi^{\mathrm{uu}}|, |\varphi^{\mathrm{u}}|, |\varphi^{\mathrm{d}}|)$ and $\Theta_f = 0$ if only the downstream one is. The limiters are those of Table 5.1 [24]. Faces between levels use the scheme `amr_interface_scheme`, by default `MonotonizedCentral` for `LaxWendroff` and `Centred` and the scheme itself otherwise; an upwind interface scheme requires `Upwind`. `advect!` defaults to `LaxWendroff`; a flow simulation passes its momentum scheme (Chapter 6). Staggered phase scalars and phase momentum are advanced in the same sweeps with interpolants of these phase volumes (Chapter 6).

Checks (`step_balance_check`). Constant phase scalars ($\varphi^{\mathrm{l}} = 3$, $\varphi^{\mathrm{g}} = 7$) are reproduced bitwise after a step on all fixtures and carriers of this chapter.

scheme	$\Psi^{\mathrm{M}}(\Theta)$	thesis
`Fromm`	$(1+\Theta)/2$	Ψ^{Fromm}
`Minmod`	$\max\{0, \min\{1, \Theta\}\}$	–
`MonotonizedCentral`	$\max\{0, \min\{2\Theta, (1+\Theta)/2, 2\}\}$	Ψ^{MC}
`VanLeer`	$2\Theta/(1+\Theta)$ for $\Theta > 0$, 0 otherwise, 2 at $\Theta = \infty$	–
`Superbee`	$\max\{0, \min\{2\Theta, 1\}, \min\{\Theta, 2\}\}$	–

TABLE 5.1. Flux limiters of the three-point face states (5.21); the thesis' Lax–Wendroff and upwind limiters are the cases $\Psi^{\mathrm{M}} \equiv 1$ and $\Psi^{\mathrm{M}} \equiv 0$ [19, §6.4.2].

Deviation from the thesis 5.5 (Phase-scalar face states). The thesis' advection operator evaluates a flux interpolant at the centroid of unsplit donating regions [19, eqs. (6.18) and (6.47)–(6.48)] and switches to its corner-transport-upwind interpolant where a control volume becomes less than $\frac{1}{2}$ full [19, eq. (6.51)]. For cell-centred phase scalars the code uses the directional states (5.21): upwind at every face of a cell that is not completely filled by the phase before or after the sweep.

Implementation. `PhaseScalar(interface; liquid=1, gas=1)` (values or functions of (x, y)) and `integrate(q, phase)`; face states `_phase_flux_state` (`src/vof/fluxes.jl`) (5.21) with `_three_point_state` (`src/operators/advection/staggered_schedules.jl`) and `_flux_limiter` (`src/operators/advection/staggered_schedules.jl`); schemes `Upwind`, `Centred`, `LaxWendroff`, `Fromm`, `Minmod`, `MonotonizedCentral`, `VanLeer`, `Superbee` and `default_amr_interface_scheme`; the update (5.20) is in `_advect_wy!` (`src/vof/fluxes.jl`). Tests: `test/two_phase/transport.jl` ("WY volume and phase constants on uniform, mixed and regridded meshes") (constants and tracer amounts to 2×10^{-14}), `test/two_phase/transport.jl` ("All centred flux interpolants in two-phase flow").

5.8 TWO-VELOCITY CARRIERS

With intrinsic phase velocities $u_f^{\mathrm{l}}, u_f^{\mathrm{g}}$, absent (NaN) where a phase has no staggered volume, the two-velocity transport uses the update (5.6) for the physical liquid fraction with the frozen interface fluxes of Section 5.8.1 and the CMDGFM projection of Chapter 10. A face is *known* for a phase if its staggered conditional fraction

$$\widetilde{C}_f^{\mathrm{l}} = \frac{\sum_{c \in \mathcal{C}(f)} |\Lambda_{c,f} \cap c^{\mathrm{o}}| C_c^{\mathrm{l}}}{\sum_{c \in \mathcal{C}(f)} |\Lambda_{c,f} \cap c^{\mathrm{o}}|} \quad \left(= \frac{h_{c^-(f),k} C_{c^-(f)}^{\mathrm{l}} + h_{c^+(f),k} C_{c^+(f)}^{\mathrm{l}}}{h_{c^-(f),k} + h_{c^+(f),k}} \text{ without solid} \right), \tag{5.23}$$

zero if the open staggered volume vanishes (7.5) ([19, eq. (2.36)]), and if $\widetilde{C}_f^{\mathrm{l}} > 0$ or < 1 respectively.

5.8.1 FROZEN INTERFACE FLUXES

The phase apertures $a_f^{\mathrm{l}}, a_f^{\mathrm{g}}$ of the reconstruction at the start of the step ((4.22)) are frozen for both sweeps. With a solid the phase-subface ordering is also frozen, so the sweep uses the intervals checked by the geometric bound. In a flow simulation the apertures are those of

the preceding physical projection, or of the projection after an intervening regrid. Each face is divided into an open interval and two phase subfaces. In the local tangential coordinate of the donor, with $w = \hat{r}_t^+ - \hat{r}_t^-$, the open interval lies at the end towards which the averaged tangential solid normal points, and the liquid subface at the end towards which the averaged tangential interface normal does not point,

$$f^{\mathrm{o}} = \begin{cases} [\hat{r}_t^+ - a_f^{\mathrm{o}} w, \hat{r}_t^+], & (\hat{\boldsymbol{\eta}}^{\mathrm{s}}_{c^-(f)} + \hat{\boldsymbol{\eta}}^{\mathrm{s}}_{c^+(f)})_t > 0, \\ [\hat{r}_t^-, \hat{r}_t^- + a_f^{\mathrm{o}} w], & \text{otherwise}, \end{cases} \qquad f^\pi \subseteq f^{\mathrm{o}}, \\ |f^\pi| = \min\left\{a_f^\pi, a_f^{\mathrm{o}}\right\} w, \\ f^{\mathrm{l}} \text{ at the lower end of } f^{\mathrm{o}} \text{ iff } (\hat{\boldsymbol{\eta}}_{c^-(f)} + \hat{\boldsymbol{\eta}}_{c^+(f)})_t \geq 0, \qquad f^{\mathrm{g}} \text{ at the other end}, \tag{5.24}$$

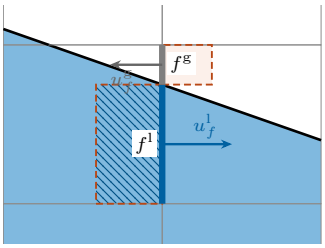
where $f^\pi = f^{\mathrm{o}}$ if $a_f^\pi \geq a_f^{\mathrm{o}}$. The stored local normals of both cells enter (in pure cells those of the WY stencil); the solid's are those of (7.2). With a solid the ordering is frozen at the start of the step; without a solid it follows the current reconstruction. Every phase with $a_f^\pi > 0$ and $u_f^\pi \neq 0$ sweeps the rectangle (5.9) over f^π, upwind of u_f^π, whose open part has the volume $\delta t |f| a_f^\pi \left|u_f^\pi\right|$ (bisection whenever a solid is present, also in pure donors, with the error of Section 5.4 if the strip over the subface is too small; otherwise $\hat{d}_f = \delta t \left|u_f^\pi\right| / h_{c^{\mathrm{u}}(f),k}$); its liquid fraction $C^\Delta_{f,\pi}$ follows from (5.12), and

$$\delta t |f| v_f^{\mathrm{o}} = \sum_\pi \delta t |f| a_f^\pi u_f^\pi, \qquad \delta t |f| v_f^{\mathrm{l}} = \sum_\pi C^\Delta_{f,\pi}\, \delta t |f| a_f^\pi u_f^\pi, \tag{5.25}$$

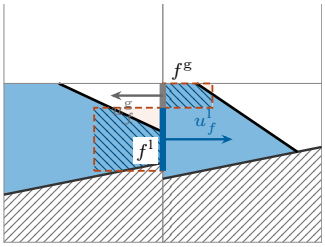
with v_f^{g} from (5.7). The liquid volume of a face thus includes liquid swept through the gas subface and vice versa (Figure 5.3). The fractions then follow (5.6) with this carrier, and the gas is the complement. The dilation of a step in a cell c with $\chi_c^{\mathrm{l}} = 1$ is $\delta t \sum_f o_{c,f} |f| (a_f^{\mathrm{l}} u_f^{\mathrm{l}} + a_f^{\mathrm{g}} u_f^{\mathrm{g}})$, δt times the discrete mixture divergence of c on the frozen apertures [19, eq. (6.19)]: it vanishes exactly when the constraint holds in c; in a cell with $\chi_c^{\mathrm{l}} = 0$ there is no dilation whatever the divergence. Of the two sweeps only the sum vanishes. The two-velocity projections impose the constraint, up to their tolerance, on the phase apertures of the reconstruction at the time of the projection (Chapter 10). In a flow simulation, if AMR changes the topology after that projection, the transferred velocities are projected again on the new apertures before the next transport step; the transferred physical pressure is kept. Standalone `adapt!` does not project, so a passive transport caller must impose the constraint. The time-step bound is (5.15) and (5.16) with the rate and tangential interval of each phase subface treated separately,

$$\nu_c^{\mathrm{in}} = \frac{\delta t}{|c^{\mathrm{o}}|} \sum_{f \in \mathcal{F}(c)} \sum_\pi \left[-o_{c,f} |f| a_f^\pi u_f^\pi\right]^+, \qquad \nu_f^{\mathrm{ext}} = \max_{\pi:\, a_f^\pi > 0} \frac{\delta t \left|u_f^\pi\right|}{h_{c^{\mathrm{u}}(f),k(f)}}, \tag{5.26}$$

ν_c^{out} likewise; it validates only active phase velocities and requires $u_f^\pi = 0$ on the boundary faces of cells that contain the phase. Without a solid and with $u_f^{\mathrm{l}} = u_f^{\mathrm{g}}$ on every face the two subface rectangles tile the one-velocity rectangle: their widths add up to the segment because $a_f^{\mathrm{l}} + a_f^{\mathrm{g}} = a_f^{\mathrm{o}} = 1$ (to rounding), both have the depth $\delta t \, |u_f| / h_{c^{\mathrm{u}}(f),k}$, and in the



A. Without a solid: $a_f^{\mathrm{l}} = 0.75$, $a_f^{\mathrm{g}} = 0.25$.



B. With a solid: $a_f^{\mathrm{l}} = 0.40$, $a_f^{\mathrm{g}} = 0.15$, $a_f^{\mathrm{o}} = 0.55$.

FIGURE 5.3. Frozen phase subfaces (5.24) of the face f at $x_1 = 2$ in the first sweep of the two-velocity transport (`vof_frozen_fixture` of `figures/vof.jl`: unit cells, liquid below a line, $u^{\mathrm{l}} = 0.8$, $u^{\mathrm{g}} = -0.6$, δt the frozen bound (5.26); PLIC lines of `:lhf_ghf`, poor in the triple cells of (b), Section 4.5). The liquid subface (blue) lies at the end of the open interval indicated by the averaged tangential normal and is extruded upwind of u_f^{l}, the gas subface (grey) upwind of u_f^{g}; dashed red: the two donating rectangles, each of the area $\delta t|f|a_f^{\pi}\left|u_f^{\pi}\right|$ in its open part (shaded), hatched: their liquid. The liquid flux of the face is the sum of both hatched areas, so in (b) the gas subface also carries liquid.

second sweep the widths remain frozen while their positions follow the current reconstruction. The carrier, the phase volumes and the time-step bound then coincide with those of Section 5.4 up to rounding. With a solid the subfaces lie in the open interval of the segment and each has the depth that holds its own volume in its own open strip, whereas the one-velocity rectangle spans the whole segment with one depth, so the fluxes differ.

Checks (`frozen_flux_check`, `equal_velocity_reduction_check`). An independent evaluation of (5.24) and (5.25) reproduces `liquid_flux` on 799 faces of the uniform, composite and cut fixtures (47 with two subfaces) to $8.3 \times 10^{-17}|c^{\mathrm{u}}(f)|$; (5.26) reproduces `stable_timestep` to 0 (relative); with equal phase velocities the frozen and one-velocity fluxes agree to $5.6 \times 10^{-17}|c|$ on the uniform and composite fixtures.

Deviation from the thesis 5.6 (Frozen interface fluxes). The thesis derives frozen-in-time cut faces with phase-volume-enforced donating regions for an unsplit method without dilation term [19, eqs. (6.34)–(6.38)], places the cut face with the interpolated interface normal projected on the face [19, §6.3.4], and does not pursue the method because a Kelvin–Helmholtz instability at density ratio 10^{-3} did not converge under refinement. The code uses it, in WY sweeps with the dilation term (5.6) of the open carrier (5.25), and orders the subfaces by the sign of the averaged stored normals.

Implementation. `_freeze_phase_apertures!` (`src/two_phase/two_velocity/interface_transport.jl`), `_frozen_subface_interval` (`src/two_phase/two_velocity/interface_transport.jl`) (5.24), `_frozen_subface_swept_fraction` (`src/two_phase/two_velocity/interface_transport.jl`) and `_compute_directional_wy_fluxes!` (`src/two_phase/two_velocity/interface_transport.jl`) (5.25); bound `_frozen_interface_stable_timestep` (`src/two_phase/two_velocity/transport_geometry.jl`) (5.26); public and prepared paths `advect!`, `stable_timestep` and `_advect_transport_substep!` (`src/two_phase/two_velocity/interface_transport.jl`). Tests:

`test/two_phase/two_velocity/transport/two_velocity.jl` ("Frozen-subface WY tangential transport") (a tangential jump leaves a planar interface and constants unchanged, with and without solid, uniform and AMR), `test/two_phase/two_velocity/transport/two_velocity.jl` ("Frozen-subface WY common-normal translation"), `test/two_phase/two_velocity/transport/two_velocity.jl` ("Prepared transport dispatch matches public advection"), `test/two_phase/two_velocity/transport/two_velocity_wy_reference.jl` ("Two-velocity directional WY reference equations"), `test/integration/two_velocity_transport.jl` ("Two-velocity split-WY reversal convergence").

5.9 Time-integrated balances

Summing (5.8) over all cells, the liquid volume $V^{\mathrm{l}} = \sum_c |c^{\mathrm{o}}| C_c^{\mathrm{l}} = \sum_c |c| \alpha_c^{\mathrm{l}}$ of a step changes by

$$V^{\mathrm{l},(n+1)} - V^{\mathrm{l},(n)} = \delta t \sum_c \chi_c^{\mathrm{l}} |c| (\mathfrak{D} v^{\mathrm{o}})_c + \sum_{j=1}^{2} \sum_c |c^{\mathrm{o}}| \delta C_c^{[j]}, \tag{5.27}$$

because the liquid volumes of the interior faces cancel and the boundary faces carry none; the gas volume, the complement, changes by the opposite amount. This identity holds for both carriers, with v^{o} the respective carrier, up to rounding. Each phase volume is therefore conserved to rounding if, besides impermeable or periodic boundaries,

- the carrier is discretely divergence free in every cell with $\chi_c^{\mathrm{l}} = 1$: $(\mathfrak{D}^{\mathrm{o}} u)_c = 0$ for one velocity, $\sum_f o_{c,f} |f| (a_f^{\mathrm{l}} u_f^{\mathrm{l}} + a_f^{\mathrm{g}} u_f^{\mathrm{g}}) = 0$ on the frozen apertures; and
- no fraction is clamped or snapped ($\delta C_c^{[j]} = 0$).

A velocity produced by a projection is divergence free only to its solver tolerance, so in a simulation the volume changes per step by the dilation of that residual. For a phase scalar, summing (5.19) over the cells and the two sweeps gives, if no fraction is clamped or snapped,

$$\sum_c |c^{\mathrm{o}}| (C_c^{\pi} \varphi_c^{\pi})^{(n+1)} - \sum_c |c^{\mathrm{o}}| (C_c^{\pi} \varphi_c^{\pi})^{(n)} = \delta t \sum_c \chi_c^{\pi} \varphi_c^{\pi,(n)} |c| (\mathfrak{D} v^{\mathrm{o}})_c, \tag{5.28}$$

so the frozen dilation values make the dilation of a scalar vanish together with that of the volume. A snap to a positive fraction adds $|c^{\mathrm{o}}| \, \delta C_c^{\pi,[j]} \, \varphi_c^{\pi,[j-1]}$ with $\delta C_c^{\mathrm{l},[j]} = \delta C_c^{[j]}$ and $\delta C_c^{\mathrm{g},[j]} = -\delta C_c^{[j]}$, and a snap to zero removes the cell's amount. The intrinsic values are not bounded by these identities; near-empty cells are excluded by the floor in (5.20).

Checks (`step_balance_check` for one velocity and `frozen_step_check` for two velocities). An independent sweep sequence (fluxes, update (5.6), clamp and snap, then the code's reconstruction) reproduces the fractions of `advect!` after a full step to 2.2×10^{-16} for all fixtures, and the identity (5.27) holds to 7.6×10^{-15} in volume. With face velocities from a streamfunction, which are exactly divergence free on the composite mesh (on the cut fixture the open volumes are differences of a streamfunction that vanishes on solid vertices, $\max |\mathfrak{D}^{\mathrm{o}} u| = 2.2 \times 10^{-15}$), no fraction is snapped and the liquid volume changes relatively by 0 (uniform mesh), 1.7×10^{-16} (composite) and 0 (cut cells). With the

plain streamfunction velocity on the cut fixture, which is not divergence free in the open cells, the volume changes by 8.1×10^{-4}, equal to the dilation term of (5.27).

— 6 —

Momentum and staggered transport

Equation reference, audited against commit `afcce97`.

The velocity components live on the faces, so their convection is a finite-volume transport over the staggered control volumes of Section 3.2. The code builds, once per mesh, a schedule of *dual faces* that join two staggered control volumes, and obtains the volume flux through every dual face as a fixed linear combination of the fluxes through the owned primal faces [19, §6.4.1]: on a uniform mesh the average of two of them, at level transitions and in cut cells a factorisation that keeps the dual divergence equal to an interpolation of the primal one. Face states on the dual faces are upwind, centred, Lax–Wendroff or limited three-point states; the update is explicit, in two directional sweeps for the second-order states. The same schedule carries three kinds of transport: the convection of the single-phase velocity, the density-weighted momentum of the one-velocity two-phase model, and the intrinsic phase velocities and staggered phase scalars of the two-velocity model, the latter two inside the volume-of-fluid sweeps of Chapter 5 and with the interpolated phase volumes as their fluxes. This chapter states the continuous balance, the dual schedule and its divergence identity, the face states, the three updates with their time levels, the time-step bound and the exact conditions under which momentum, the mass–momentum consistency and, for the centred operator, kinetic energy are conserved, and it collects which face state each dual face uses; the last section gives the cell-centred upwind transport of a passive scalar. Derivations are in the thesis [19, §§6.2, 6.4 and App. 6.C].

6.1 Continuous model

For one phase of constant density, the convective part of the balance of the velocity component $\widetilde{\varphi} = \boldsymbol{u} \cdot \boldsymbol{e}_k$ over the staggered control volume ω_f of a face $f \in \mathcal{F}^{[k]}$ is

$$\int_{\omega_f} \widetilde{\varphi}(t^{(n+1)})\,\mathrm{d}V - \int_{\omega_f} \widetilde{\varphi}(t^{(n)})\,\mathrm{d}V + \int_{t^{(n)}}^{t^{(n+1)}} \int_{\partial\omega_f} \widetilde{\varphi}\,\boldsymbol{u}\cdot\boldsymbol{n}\,\mathrm{d}S\,\mathrm{d}t = 0; \tag{6.1}$$

pressure, viscosity and forces are separate stages of the time step (Chapter 1). In two phases each phase carries its own amount over the part $\omega_f^\pi(t) = \omega_f \cap \Omega^\mathrm{o} \cap \Omega^\pi(t)$,

$$\int_{\omega_f^\pi(t^{(n+1)})} \rho^\pi \widetilde{\varphi}^\pi\,\mathrm{d}V - \int_{\omega_f^\pi(t^{(n)})} \rho^\pi \widetilde{\varphi}^\pi\,\mathrm{d}V + \int_{t^{(n)}}^{t^{(n+1)}} \int_{\partial\omega_f^\pi(t)\setminus I(t)} \rho^\pi \widetilde{\varphi}^\pi\,\boldsymbol{u}^\pi\cdot\boldsymbol{n}\,\mathrm{d}S\,\mathrm{d}t = 0 \tag{6.2}$$

[19, eq. (6.39)], the staggered counterpart of (5.2): the interface carries no flux because there is no phase change. The one-velocity model transports the mixture momentum, the sum of (6.2) over both phases with $\boldsymbol{u}^{\mathrm{l}} = \boldsymbol{u}^{\mathrm{g}} = \boldsymbol{u}$ and $\widetilde{\varphi}^{\pi} = \boldsymbol{u} \cdot \boldsymbol{e}_k$; the two-velocity model transports each phase velocity, $\widetilde{\varphi}^{\pi} = \boldsymbol{u}^{\pi} \cdot \boldsymbol{e}_k$; a staggered phase scalar is an intrinsic quantity with ρ^{π} divided out. With $\widetilde{\varphi}^{\pi} = 1$, (6.2) is the balance of the phase mass of ω_f. If $\nabla \cdot \boldsymbol{u} = 0$ and the boundary is periodic or impermeable, $\int_\Omega \widetilde{\varphi}\, \boldsymbol{u} \cdot \nabla \widetilde{\varphi}\, \mathrm{d}V = 0$, so (6.1) conserves $\int_\Omega \widetilde{\varphi}\, \mathrm{d}V$ and $\int_\Omega \frac{1}{2} \widetilde{\varphi}^2\, \mathrm{d}V$, the kinetic energy of the component.

6.2 Dual control volumes and primal–dual fluxes

Unknowns and volumes. The unknowns of component k are the values $\widetilde{\varphi}_f$ on the interior faces $f \in \mathcal{F}^{[k]} \cap \mathcal{F}_\Omega$. The phase transport of a bounded domain or of a mesh with a solid adds the boundary faces $f \in \mathcal{F}^{[k]} \cap \mathcal{F}_\Gamma$ as unknowns with a half control volume in their cell. Every unknown has the transport volume

$$\widetilde{V}_f = \frac{1}{2} \sum_{c \in \mathcal{C}(f)} |\Lambda_{c,f} \cap c^{\mathrm{o}}| = \begin{cases} |\omega_f|, & \text{no solid,} \\ |\omega_f^{\mathrm{o}}|, & \text{solid present,} \end{cases} \qquad f \in \mathcal{F}_\Omega, \tag{6.3}$$

with the strip $\Lambda_{c,f}$ of (3.7), the single term of $c(f)$ for a boundary unknown, and $\widetilde{V}_f = 1$ where this measure vanishes (no flux reaches such a closed unknown). The volume-weighted interpolation of a cell function to the faces is

$$(\mathfrak{H}\varphi)_f = \frac{1}{2\widetilde{V}_f} \sum_{c \in \mathcal{C}(f)} |\Lambda_{c,f} \cap c^{\mathrm{o}}|\, \varphi_c, \tag{6.4}$$

$\varphi_{c(f)}$ on a boundary unknown; without solid it is the thesis' interpolant $\mathfrak{H}$ [19, eq. (2.30)] with the extrusion volumes (3.3), and the staggered fractions $\widetilde{C}_f^{\pi} = (\mathfrak{H} C^{\pi})_f$ of (5.23) use it.

Sides. For a cell c and the transported component k, the faces of $\mathcal{F}^{[k]}$ in $\mathcal{F}(c)$ lie on the low or the high side of c, and each covers a fraction of its side ($t \neq k$),

$$\mathcal{F}_s(c) = \{\, f \in \mathcal{F}(c) \cap \mathcal{F}^{[k]} \mid o_{c,f} = s \,\}, \qquad \varpi_{c,f} = \frac{|f|}{h_{c,t}}, \qquad \sum_{f \in \mathcal{F}_s(c)} \varpi_{c,f} \in \{0, 1\}, \tag{6.5}$$

with $\varpi_{c,f} = \frac{1}{2}$ for a segment of a hanging coarse side ((2.7)) and the sum zero for a wall side whose boundary face is not an unknown.

Dual faces. A dual face $g \in \mathcal{G}$ joins two unknowns, its tail $f^-(g)$ and head $f^+(g)$, with $\widetilde{o}_{f,g} = +1$ for $f = f^-(g)$ and -1 for $f = f^+(g)$, and carries the primal fluxes of one axis $k(g)$ with weights $\widetilde{w}_{g,f}$; there is one dual face per pair of unknowns and axis, and reversing its orientation changes the sign of its weights. With (f', f) the dual face with tail f' and head f, of axis k in (N) and $t \neq k$ in (T), the weights are the sums of the contributions

$$\begin{aligned} &\text{(N)} \quad \widetilde{w}_{(f',f),f} \mathrel{+}= \tfrac{1}{2}\, o_{c,f}\, \varpi_{c,f'} && \text{for } f \in \mathcal{F}^{[k]},\ c \in \mathcal{C}(f), \\ &\qquad\qquad && \quad f' \in \mathcal{F}(c) \cap \mathcal{F}^{[k]},\ f' \neq f, \\ &\text{(T)} \quad \widetilde{w}_{(f',f''),f} \mathrel{+}= \tfrac{1}{2}\, \varpi_{c^-(f),f'}\, \varpi_{c^+(f),f''} && \text{for } f \in \mathcal{F}^{[t]},\ s = \pm 1, \\ &\qquad\qquad && \quad f' \in \mathcal{F}_s(c^-(f)),\ f'' \in \mathcal{F}_s(c^+(f)). \end{aligned} \tag{6.6}$$

Rule (N) routes the flux of a face normal to $\boldsymbol{e}_k$ from the other faces of its two cells into the face itself; rule (T) couples the faces on the same side of the two cells of a transverse face, a transport plan whose marginals are the fractions $\varpi_{c,f}/2$. The integrated volume flux of a dual face is

$$\widetilde{\Upsilon}_g = \sum_{f\in\mathcal{F}} \widetilde{w}_{g,f}\,|f|\,v_f, \quad v_f = a_f^{\mathrm{o}} u_f, \qquad \delta t\widetilde{\Upsilon}_g^{\pi} = \sum_{f\in\mathcal{F}} \widetilde{w}_{g,f}\,\delta t|f|v_f^{\pi}, \tag{6.7}$$

the first for the single-phase convection and the second for the phases, with the time-integrated phase volumes of Chapter 5, which carry their apertures already, and $a_f^{\mathrm{o}} = 1$ without solid. On a conforming patch every $\varpi_{c,f}$ is one and a dual face carries two primal fluxes with weight $\frac{1}{2}$: the dual face through the centre of the cell between f' and f and the one through the common vertex of f and f' along $\boldsymbol{e}_t$ have

$$\widetilde{\Upsilon}_g = \frac{1}{2}\left(|f'|v_{f'} + |f|v_f\right), \qquad \widetilde{\Upsilon}_g = \frac{1}{2}\left(|h|v_h + |h'|v_{h'}\right), \tag{6.8}$$

$h, h' \in \mathcal{F}^{[t]}$ the two faces with that vertex as end point, the thesis' area-weighted interpolant $\widetilde{\mathfrak{H}}$ [19, eq. (6.10)] (Figure 6.1a). On a uniform fully periodic mesh the code stores exactly these two-flux faces; on every other mesh it builds (6.6), which reduces to them away from level transitions. At a hanging side the fine segment f_1 of Figure 6.1c has 8 dual faces, among them $\widetilde{\Upsilon}_g = \frac{1}{4}|f_2|v_{f_2} - \frac{1}{4}|f_1|v_{f_1}$ to the other segment of the same side and couplings with weight $\frac{1}{8}$ to the segments of the coarse sides above and below. With a solid the uncut weights multiply the open fluxes $|f|a_f^{\mathrm{o}}u_f$ and the volumes are the open ones (Figure 6.1b), the averaging of volume fluxes of Dröge and Verstappen [8] that the thesis follows [19, §6.4.1]; only the one-velocity momentum replaces the fractions $\varpi_{c,f}$ of a cut cell by the open-strip fractions (6.24). Walls add the boundary dual faces of Section 6.4.

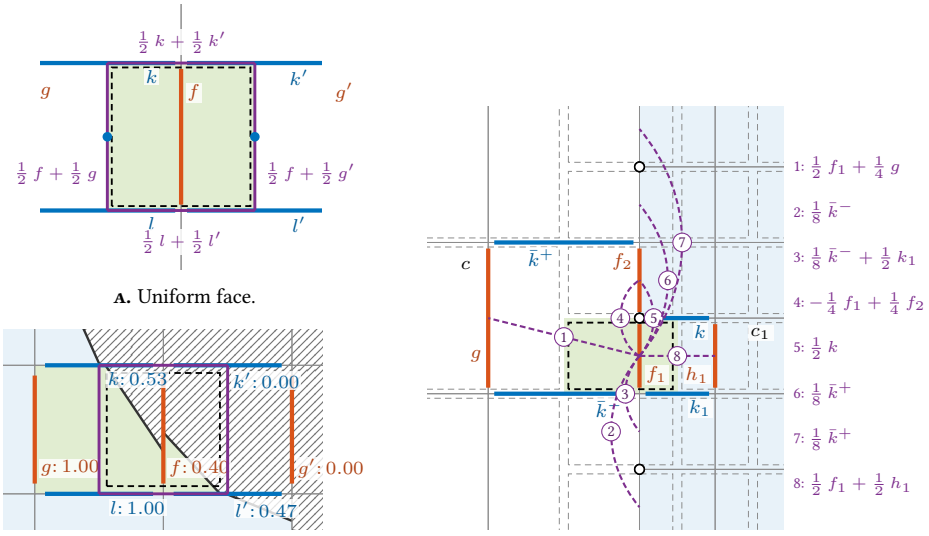
Divergence identity. The staggered divergence of the dual faces normal to $\boldsymbol{e}_t$ is

$$\widetilde{V}_f\,(\widetilde{\mathfrak{D}}_t\widetilde{\Upsilon})_f = \sum_{g\in\mathcal{G}(f),\ k(g)=t} \widetilde{o}_{f,g}\,\widetilde{\Upsilon}_g, \qquad \widetilde{\mathfrak{D}} = \widetilde{\mathfrak{D}}_1 + \widetilde{\mathfrak{D}}_2, \tag{6.9}$$

the thesis' staggered divergence [19, eq. (6.5)]. For every unknown $f \in \mathcal{F}^{[k]}$ whose cells have unknowns on both sides ($\sum_{f'\in\mathcal{F}_s(c)} \varpi_{c,f'} = 1$ for $s = \pm 1$ and $c \in \mathcal{C}(f)$) and no flux through a boundary face, the weights (6.6) give

$$\widetilde{V}_f\,(\widetilde{\mathfrak{D}}_t\widetilde{\Upsilon})_f = \sum_{c\in\mathcal{C}(f)} \frac{\varpi_{c,f}}{2} \sum_{f'\in\mathcal{F}(c)\cap\mathcal{F}^{[t]}} o_{c,f'}\,|f'|v_{f'}, \qquad t = 1, 2. \tag{6.10}$$

For $t = k$ the flux of a face $f' \neq f$ of c contributes $\frac{1}{2}\varpi_{c,f}o_{c,f'}|f'|v_{f'}$ to the left side through the dual face joining f and f', as on the right, while the flux of f itself contributes $-\frac{1}{2}o_{c,f}(S_c - \varpi_{c,f})|f|v_f$, with $S_c = \sum_{s=\pm1}\sum_{f'\in\mathcal{F}_s(c)} \varpi_{c,f'}$; the difference to the right side is $-\frac{1}{2}S_c o_{c,f}|f|v_f$, which cancels between the two cells of f for $S_c = 2$. For $t \neq k$ the flux of a face $h \in \mathcal{F}(c) \cap \mathcal{F}^{[t]}$ reaches f through the dual faces between f and the unknowns f'' on the same side s of the neighbour c' across h, with weights summing to $\frac{1}{2}\varpi_{c,f}o_{c,h}\sum_{f''\in\mathcal{F}_s(c')} \varpi_{c',f''}$, which is the right side when that sum is one. On a 2:1-balanced mesh the hypothesis implies this: a side of c' without unknown is a wall, so either



A. Uniform face.

B. Face between two cut cells.

C. Fine segment f_1 of a hanging side.

FIGURE 6.1. Staggered control volumes of the component u on faces $f \in \mathcal{F}^{[1]}$ (dashed; green: the one of the face considered) and their dual faces (purple) with the weights $\widetilde{w}_{g,f'}$ of the primal fluxes $|f'|v_{f'}$, read from the code's schedules (labels: weight times face name; red: faces of $\mathcal{F}^{[1]}$, blue: $\mathcal{F}^{[2]}$). (a) Uniform periodic mesh: each dual face averages two primal fluxes (6.8). (b) Disc fixture (cells of width $\frac{1}{2}$, solid hatched): the weights are $\frac{1}{2}a^{\circ}_{f'}$, labels give $a^{\circ}_{f'}$; green: the open parts of the two strips, half of whose area is $|\omega^{\circ}_f| = 0.119$ against $|\omega_f| = \frac{1}{4}$. (c) Composite mesh of Figure 5.2b: all 8 dual faces of the fine segment f_1, drawn as edges between the unknowns they join (dashed: non-regular; arcs join unknowns on the same line, bent right for $\mathcal{F}^{[1]}$-fluxes and left for $\mathcal{F}^{[2]}$-fluxes; open circles: hanging vertices); $\widetilde{V}_{f_1} = \frac{3}{8}$.

f lies on that wall or c' is coarser than c and the other cell of f reaches the wall. At a wall the two sides differ by exactly the missing terms: a wall cell ($S_c = 1$) with the boundary face $b \in \mathcal{F}^{[k]}$ lacks $\frac{1}{2}s_b|f|(v_b + v_f)$, a boundary face $b \in \mathcal{F}(c) \cap \mathcal{F}^{[t]}$ lacks $\frac{1}{2}\varpi_{c,f}s_b|b|v_b$, and an unknown whose neighbour c' has a wall on the side s, which happens where a level transition meets a wall normal to $\boldsymbol{e}_k$, lacks $\frac{1}{2}\varpi_{c,f}o_{c,h}|h|v_h$; the boundary dual faces of Section 6.4 supply exactly these terms ((6.18)). In a cell that is uncut or in which f is the single face of its side, $\frac{1}{2}\varpi_{c,f}|c^{\circ}| = \frac{1}{2}|\Lambda_{c,f} \cap c^{\circ}|$, so the right-hand side is $\widetilde{V}_f$ times the interpolation (6.4) of the directional primal divergence: the thesis' operator connection $\widetilde{\mathfrak{D}}\widetilde{\mathfrak{H}} = \mathfrak{H}\mathfrak{D}$ [19, Lemma 6.1, eq. (6.13)], on the composite mesh and per sweep direction. In a cut cell whose side carries two segments the open part of the half strip is in general not half the open cell: the right side then differs from $\widetilde{V}_f(\mathfrak{H}\mathfrak{D}^{\circ}_t u)_f$ by $\frac{1}{2}\sum_c (\varpi_{c,f}|c^{\circ}| - |\Lambda_{c,f} \cap c^{\circ}|)\,(\mathfrak{D}^{\circ}_t u)_c$, $\mathfrak{D}^{\circ}_t$ the part of $\mathfrak{D}^{\circ}$ through the faces of $\mathcal{F}^{[t]}$, which vanishes for $\mathfrak{D}^{\circ}_t u = 0$; the one-velocity momentum avoids the difference with the fractions (6.24).

Non-regular dual faces. A dual face is *regular* unless it is in

$$\mathcal{G}^* = \Big\{\, g \in \mathcal{G} \;\Big|\; \#\{\, f \mid \widetilde{w}_{g,f} \neq 0 \,\} \neq 2 \text{ or } |\widetilde{w}_{g,f}| \neq \frac{1}{2} \text{ for some } f \\ \text{or } |\omega_{f^-(g)}| \neq |\omega_{f^+(g)}| \text{ or } \big\{f^-(g), f^+(g)\big\} \cap \mathcal{F}_\Gamma \neq \emptyset \,\Big\}, \quad (6.11)$$

decided once, when the schedule is built, with the uncut weights and volumes to $64\epsilon_{\text{mach}}$ (relative), so that a dual face between cut cells is regular when its uncut counterpart is. The last condition concerns the boundary unknowns of the phase schedule of a bounded domain or of a mesh with a solid, whose uncut volume is half their cell. Every dual face of Figure 6.1c is non-regular; Section 6.9 collects which face state each kind of dual face uses.

Checks (`figures/momentum.jl`, `convection_flux_check`). An independent construction of (6.6) reproduces the weights of the code's schedules exactly on the uniform, composite and disc fixtures (the cut weights after multiplication by the apertures), and (6.10) holds to 2.2×10^{-16} for a velocity that is not solenoidal.

Deviation from the thesis 6.1 (Primal–dual fluxes on composite meshes and in cut cells). The thesis defines the dual faces and their area-weighted interpolant for conforming meshes, with $|g| = \frac{1}{2}\sum_f |f|$ over the two primal faces of the same normal [19, eqs. (6.5)–(6.13)]. The code generalises the interpolant to hanging sides by the factorisation (6.6), whose dual faces need not have a geometric face, may join unknowns on one line, and are kept separate per axis so that a directional sweep uses only fluxes of its own direction; with a solid it multiplies the weights by the apertures and uses the open volumes (6.3), and for the one-velocity momentum it replaces the fraction of a segment of a hanging side of a cut cell by its open-strip fraction (6.24). At walls the single-phase schedule adds the boundary dual faces (6.17).

Implementation. `StaggeredAdvectionWorkspace(field)` and `StaggeredVelocityAdvectionWorkspace(velocity)`, and with a `GeometryField` as second argument for cut cells. Schedules: `_uniform_dual_schedule` (`src/operators/advection/staggered_schedules.jl`) (uniform periodic), `_amr_dual_schedule` (`src/operators/advection/staggered_schedules.jl`) (6.6) with the incidences `_component_incidences` (`src/operators/advection/staggered_schedules.jl`) (6.5) and the classification `_assemble_amr_dual_schedule` (`src/operators/advection/staggered_schedules.jl`) (6.11), `_cut_cell_dual_schedule` (`src/embedded/operators.jl`) (apertures, open volumes); phase schedules `_bounded_phase_schedule` (`src/two_phase/phase_face_transport.jl`) and `_geometry_phase_schedule` (`src/embedded/operators.jl`); interpolation (6.4) `_phase_face_average!` (`src/two_phase/phase_face_fields.jl`); dual rates (6.7) `_amr_dual_flux` (`src/operators/advection/staggered_fluxes.jl`) and `_phase_dual_flux` (`src/two_phase/phase_face_transport.jl`); divergence (6.9) `_directional_flux_divergence!` (`src/operators/advection/staggered_fluxes.jl`). The schedule is built when the workspace is constructed and after every regrid (Section 2.5). Tests: `test/operators/advection/staggered_advection.jl` ("centred-to-staggered

divergence compatibility"), `test/operators/advection/staggered_advection.jl` ("AMR staggered flux compatibility and stability") (identity (6.10) at hanging faces to 8×10^{-14}), `test/operators/advection/staggered_advection.jl` ("staggered transport is independent of uniform block partitioning"), `test/two_phase/transport.jl` ("bounded phase schedule classifies connections") (boundary unknowns non-regular, regular faces with upstream unknowns).

6.3 Face states

Let g be a dual face with rate $\widetilde{\Upsilon}_g$ (time-integrated volume $\delta t \widetilde{\Upsilon}^{\pi}_g$ in the phase transport), upwind unknown $f^{\mathrm{u}}_g = f^-(g)$ if the rate is non-negative and $f^+(g)$ otherwise, downwind unknown f^{d}_g the other one, and Courant number $\nu_g = \delta t \left|\widetilde{\Upsilon}_g\right| / \min(\widetilde{V}_{f^-(g)}, \widetilde{V}_{f^+(g)})$. The face state of a staggered quantity is

$$(\widetilde{\mathfrak{F}}\widetilde{\varphi})_g = \begin{cases} \widetilde{\varphi}_{f^{\mathrm{u}}_g}, & \texttt{Upwind}, \\ \frac{1}{2}\left(\widetilde{\varphi}_{f^-(g)} + \widetilde{\varphi}_{f^+(g)}\right), & \texttt{Centred}, \\ \widetilde{\varphi}^{\mathrm{LW}}_g, & \texttt{LaxWendroff}, \\ \text{(6.13)}, & \text{three-point scheme M}, \end{cases} \tag{6.12}$$

$$\widetilde{\varphi}^{\mathrm{LW}}_g = \frac{1}{2}(1+\nu_g)\widetilde{\varphi}_{f^{\mathrm{u}}_g} + \frac{1}{2}(1-\nu_g)\widetilde{\varphi}_{f^{\mathrm{d}}_g} - \tfrac{1}{4}\delta t \sum_{f \in \{f^-(g), f^+(g)\}} \widetilde{\varphi}_f (\widetilde{\mathfrak{D}}_{k_j} \widetilde{\Upsilon})_f,$$

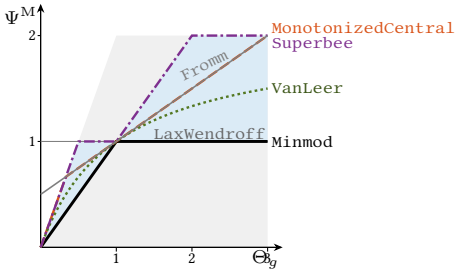
where the last term of the Lax–Wendroff state, the directional dual divergence (6.9) of the transporting flux in the current sweep j, belongs to the single-phase transport only; when a boundary condition is passed, the boundary dual faces of the sweep are added to it, which makes it the closed divergence $\widetilde{\mathfrak{D}}^{\Gamma}_{k_j}$ of (6.18). The variable-coefficient predictor $\widetilde{\varphi} - \frac{1}{2}\delta t\, \partial_k(u_k \widetilde{\varphi})$ splits into an advective part, the first two terms, and a compressive part, whose directional parts cancel only after both sweeps for a solenoidal velocity. The phase transport uses the first two terms. On the uniform schedule $\nu_g = \delta t \left|\widetilde{\Upsilon}_g\right| / \widetilde{V}_f$ is the classical $\delta t\, |u| \,/h$ of the dual face. The three-point schemes `Fromm`, `Minmod`, `MonotonizedCentral`, `VanLeer` and `Superbee` limit the Lax–Wendroff correction,

$$(\widetilde{\mathfrak{F}}\widetilde{\varphi})_g = \widetilde{\varphi}_{f^{\mathrm{u}}_g} + \Psi^{\mathrm{M}}(\Theta_g)\left(\widetilde{\varphi}^{\mathrm{LW}}_g - \widetilde{\varphi}_{f^{\mathrm{u}}_g}\right), \tag{6.13}$$

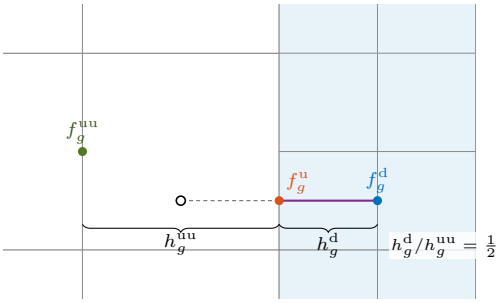
with the limiters of Table 5.1 (Figure 6.2a) and the smoothness ratio

$$\Theta_g = \frac{h^{\mathrm{d}}_g}{h^{\mathrm{uu}}_g} \frac{\widetilde{\varphi}_{f^{\mathrm{u}}_g} - \widetilde{\varphi}_{f^{\mathrm{uu}}_g}}{\widetilde{\varphi}_{f^{\mathrm{d}}_g} - \widetilde{\varphi}_{f^{\mathrm{u}}_g}}, \qquad h^{\mathrm{d}}_g = \left|(\boldsymbol{x}_{f^{\mathrm{d}}_g} - \boldsymbol{x}_{f^{\mathrm{u}}_g})_t\right|, \quad h^{\mathrm{uu}}_g = \left|(\boldsymbol{x}_{f^{\mathrm{u}}_g} - \boldsymbol{x}_{f^{\mathrm{uu}}_g})_t\right|, \tag{6.14}$$

$t = k(g)$, $\boldsymbol{x}_f$ the face centroids and periodic displacements to the nearest image; the tolerance rule of (5.22) sets $\Theta_g = 1$ or 0 for vanishing differences. The second upwind unknown is chosen once, when the schedule is built: among the unknowns joined to f^{u}_g by another dual face of axis t and lying beyond f^{u}_g along $\boldsymbol{e}_t$, it is the one nearest to the reflection of f^{d}_g



A. Limiters $\Psi^{\mathrm{M}}(\Theta_g)$ (shaded: the TVD region, blue: its second-order part).



B. Upstream neighbour of the fine segment f_1 of Figure 6.1c for a flux towards $+\boldsymbol{e}_1$.

FIGURE 6.2. Three-point face states (6.13). (a) The limiters of Table 5.1, evaluated by the code at $\Theta_g \in [0, 3]$; Fromm and Lax–Wendroff leave the TVD region of Sweby [24]. (b) Composite mesh of Figure 6.1: the upstream neighbour f_g^{uu} of the dual face $g = (f_1, h_1)$ is the unknown joined to $f_g^{\mathrm{u}} = f_1$ by a dual face normal to $\boldsymbol{e}_1$ that lies nearest to the reflection of $f_g^{\mathrm{d}} = h_1$ through f_1 (open circle), here the coarse face g of Figure 6.1c; the segment f_2 lies on the line of f_1 and does not qualify. The spacing ratio enters the smoothness ratio (6.14); for the opposite flux direction the code selects h_1 with ratio 2.

through f_g^{u},

$$f_g^{\mathrm{uu}} = \arg\min_f \left| \boldsymbol{x}_f - \left(2\boldsymbol{x}_{f_g^{\mathrm{u}}} - \boldsymbol{x}_{f_g^{\mathrm{d}}}\right)\right|_2, \qquad \operatorname{sgn}\left((\boldsymbol{x}_{f_g^{\mathrm{d}}} - \boldsymbol{x}_{f_g^{\mathrm{u}}})_t\right)\,(\boldsymbol{x}_f - \boldsymbol{x}_{f_g^{\mathrm{u}}})_t < -\tau, \tag{6.15}$$

ties broken by the lower index, $\tau = 64\epsilon_{\mathrm{mach}} \max(1, X_t)$. If no unknown qualifies (as beyond an unknown next to a wall of the single-phase schedule, whose wall face is not an unknown), or if $\left|(\boldsymbol{x}_{f_g^{\mathrm{d}}} - \boldsymbol{x}_{f_g^{\mathrm{u}}})_t\right| \le \tau$ (two segments of one side), the state is upwind; so is a limited state at a local extremum ($\Theta_g \le 0$). On a uniform mesh this is the classical second upwind neighbour with $h_g^{\mathrm{d}} = h_g^{\mathrm{uu}}$; at a level transition the spacing ratio keeps $\Theta_g = 1$ for a linear profile (Figure 6.2b), the non-uniform-grid correction of Zeng [34].

Scheme selection. The keyword `scheme` (default `LaxWendroff`) selects the state of the regular dual faces, the keyword `amr_interface_scheme` that of the non-regular ones $g \in \mathcal{G}^*$ and the states at walls; Section 6.9 collects the rules. The default interface scheme, `default_amr_interface_scheme(scheme)`, is the bulk scheme if that is `Upwind` or one of the three-point schemes; for the bulk schemes `Centred` and `LaxWendroff` it is `MonotonizedCentral`. An upwind interface scheme with another bulk scheme is rejected. On a uniform fully periodic mesh $\mathcal{G}^*$ is empty and the keyword has no effect. The reason for the default is measurable: on a composite mesh with a unit value at the most-coupled interface unknown and zeros elsewhere, the spatial operator of Lax–Wendroff (and of Fromm) at the interface makes a zero unknown grow, i.e. creates a new extremum for every positive step, whereas the four limiters do not (test below).

Deviation from the thesis 6.2 (Directional face states). The thesis interpolates to the centroid of an unsplit donating region of the dual face, along a line through the downwind node with upwind values $\widetilde{\mathfrak{U}}_1\widetilde{\varphi}$, $\widetilde{\mathfrak{U}}_2\widetilde{\varphi}$ from (bi)linear interpolation, and limits with $\Psi(\iota)$ of

a spacing-weighted slope ratio [19, eqs. (6.47)–(6.48)]. The code uses the one-dimensional states of every dual face in directional sweeps: the Lax–Wendroff state (6.12) with Courant number per dual face and, in the single-phase transport, the compressive term; the upstream unknown of the dual graph (6.15); and a separate scheme on non-regular dual faces and at walls, for which the thesis has no counterpart.

Implementation. Schemes `Upwind`, `Centred`, `LaxWendroff`, `Fromm`, `Minmod`, `MonotonizedCentral`, `VanLeer`, `Superbee` (or the symbols `:upwind`, `:centred`, `:lw`, `:fromm`, `:minmod`, `:mc`, `:van_leer`, `:superbee`) and `default_amr_interface_scheme`; states `_dual_state` (`src/operators/advection/staggered_schedules.jl`), `_amr_dual_state` (`src/operators/advection/staggered_fluxes.jl`) and `_phase_bulk_state` (`src/two_phase/phase_face_transport.jl`) (6.12); `_three_point_state` (`src/operators/advection/staggered_schedules.jl`) (6.13) and (6.14) with `_flux_limiter` (`src/operators/advection/staggered_schedules.jl`); upstream unknowns `_three_point_stencils` (`src/operators/advection/staggered_schedules.jl`) (6.15). Tests: `test/operators/advection/staggered_advection.jl` (“staggered advection schemes”) (one-dimensional Lax–Wendroff stencil, the split two-dimensional state and the compressive term), `test/operators/advection/staggered_advection.jl` (“stationary AMR-interface extremum probe”), `test/operators/advection/staggered_advection.jl` (“mixed-level staggered convection convergence”) (rate above 1.3 with `MonotonizedCentral` at the interface), `test/operators/advection/staggered_advection.jl` (“staggered scheme canonicalization”).

6.4 Single-phase convection

Spatial operator. With the rates $\widetilde{\Upsilon}_g$ of an advecting velocity u, the staggered convection operator is

$$\widetilde{V}_f\,(\widetilde{\mathfrak{A}}[u]\widetilde{\varphi})_f = \sum_{g\in\mathcal{G}(f)} \widetilde{o}_{f,g}\,\widetilde{\Upsilon}_g\,(\widetilde{\mathfrak{F}}\widetilde{\varphi})_g + \sum_{b\in\mathcal{G}_\Gamma(f)} \widetilde{o}_{f,b}\,\widetilde{\Upsilon}_b\,\widetilde{\varphi}_b, \tag{6.16}$$

the thesis’ $\widetilde{\mathfrak{A}}[m]\widetilde{\varphi} = \widetilde{\mathfrak{D}}\left((\widetilde{\mathfrak{H}}m)(\widetilde{\mathfrak{F}}\widetilde{\varphi})\right)$ [19, eq. (6.44)] with a constant density. The boundary dual faces $\mathcal{G}_\Gamma(f)$ close the control volumes next to a wall, whose boundary faces are not unknowns of the single-phase schedule. They are built with the schedule of every mesh with walls and enter the operator when a boundary condition is passed (the Navier–Stokes step does). There are three kinds: (B1) for the boundary face $b \in \mathcal{F}^{[k]}$ of the wall cell $c = c(b)$ and an unknown f on the opposite side of c; (B2) for a boundary face $b \in \mathcal{F}^{[t]}$, $t \neq k$, of a cell $c \in \mathcal{C}(f)$; (B3) for a face $h \in \mathcal{F}(c) \cap \mathcal{F}^{[t]}$ of a cell $c \in \mathcal{C}(f)$ whose other cell c' has, on the side of f in c, the boundary face $b \in \mathcal{F}^{[k]}$: the partner that rule (T) misses

where a level transition meets a wall normal to $\boldsymbol{e}_k$. Their rates, orientations and states are

$$\begin{aligned}
&\text{(B1)} \quad \widetilde{\Upsilon}_b = \frac{1}{2}|f|\,(\upsilon_b + \upsilon_f)\,, && \widetilde{o}_{f,b} = s_b,\\
&\text{(B2)} \quad \widetilde{\Upsilon}_b = \frac{1}{2}\varpi_{c,f}|b|\,\upsilon_b, && \widetilde{o}_{f,b} = s_b,\\
&\text{(B3)} \quad \widetilde{\Upsilon}_b = \frac{1}{2}\varpi_{c,f}\varpi_{c',b}|h|\,\upsilon_h, && \widetilde{o}_{f,b} = o_{c,h},
\end{aligned}$$

$$\widetilde{\varphi}_b = \begin{cases} \widetilde{\varphi}^{\Gamma}_b \text{ if } \widetilde{o}_{f,b}\widetilde{\Upsilon}_b < 0, \text{ else } \widetilde{\varphi}_f, & \text{(B1), (B3) with an } \texttt{Upwind} \text{ interface scheme,}\\ \frac{1}{2}\left(\widetilde{\varphi}_f + \widetilde{\varphi}^{\Gamma}_b\right), & \text{(B1), (B3) otherwise,}\\ \boldsymbol{u}^{\Gamma}(\boldsymbol{x}_b, t)\cdot\boldsymbol{e}_k, & \text{(B2) at } \texttt{NoSlip} \text{ and } \texttt{Inflow} \text{ boundaries,}\\ \widetilde{\varphi}_f, & \text{(B2) otherwise,} \end{cases} \tag{6.17}$$

with $\upsilon = a^{\mathrm{o}}u$ as in (6.7) (u_b the normal velocity and a^{o}_b the open aperture of the boundary face), and $\widetilde{\varphi}^{\Gamma}_b$ the stored boundary value of the transported component on the face $b \in \mathcal{F}^{[k]}$, which for a velocity component at an impermeable wall is its normal velocity, zero. At walls $u_b = 0$: (B2) carries no flux, (B1) carries $\frac{1}{2}|f|\upsilon_f$ through the centre of the wall cell, and (B3) the share $\frac{1}{2}\varpi_{c,f}|h|\upsilon_h$ of a transverse flux ($\varpi_{c',b} = 1$). The kind (B3) and the apertures were added for issue 60 and issue 61; before, a constant velocity was not a steady state at the unknowns concerned.

Closed identity. The boundary dual faces supply exactly the terms that (6.10) lacks at a wall (Section 6.2): for every unknown f and $t = 1, 2$ the closed divergence satisfies

$$\begin{aligned}
\widetilde{V}_f\,(\widetilde{\mathfrak{D}}^{\Gamma}_t\widetilde{\Upsilon})_f &:= \widetilde{V}_f\,(\widetilde{\mathfrak{D}}_t\widetilde{\Upsilon})_f + \sum_{b\in\mathcal{G}_{\Gamma}(f),\ k(b)=t} \widetilde{o}_{f,b}\,\widetilde{\Upsilon}_b\\
&= \sum_{c\in\mathcal{C}(f)} \frac{\varpi_{c,f}}{2} \sum_{f'\in\mathcal{F}(c)\cap\mathcal{F}^{[t]}} o_{c,f'}\,|f'|\upsilon_{f'},
\end{aligned} \tag{6.18}$$

with $\mathcal{F}(c)$ including the boundary faces of c and $k(b)$ the axis of the primal flux that a boundary dual face carries (k for (B1), t for (B2) and (B3)). At $\delta t = 0$ every state of a constant whose boundary values are that constant is the constant (at walls (B2) carries no flux), so $\widetilde{\mathfrak{A}}[u]1 = \widetilde{\mathfrak{D}}^{\Gamma}\widetilde{\Upsilon}$: for a discretely solenoidal velocity, $\mathfrak{D}^{\mathrm{o}}u = 0$ ((3.13)), a constant is a steady state at every unknown, also where a level transition meets a wall and in cut cells next to a wall.

Time discretisation. `advect!` advances with forward Euler. If the scheme is `LaxWendroff` or a three-point scheme, or, on any schedule but the uniform periodic one, the interface scheme is one of them, it makes two directional sweeps along the axes $k_1 \neq k_2$,

$$\widetilde{\varphi}^{[j]} = \widetilde{\varphi}^{[j-1]} - \delta t\,\widetilde{\mathfrak{A}}[u]\widetilde{\varphi}^{[j-1]}\big|_{k_j}, \quad j = 1, 2, \qquad \widetilde{\varphi}^{[0]} = \widetilde{\varphi}^{(n)}, \quad \widetilde{\varphi}^{(n+1)} = \widetilde{\varphi}^{[2]}, \tag{6.19}$$

where $\widetilde{\mathfrak{A}}[u]\cdot|_{k_j}$ keeps the dual faces with $k(g) = k_j$ and the boundary dual faces with $k(b) = k_j$, and the Lax–Wendroff term uses $\widetilde{\mathfrak{D}}^{\Gamma}_{k_j}$ (without a boundary condition $\widetilde{\mathfrak{D}}_{k_j}$); the order $(1, 2)$ or $(2, 1)$ alternates between successive calls. `Upwind` and `Centred` (with an upwind or centred interface scheme) make one unsplit step $\widetilde{\varphi}^{(n+1)} = \widetilde{\varphi}^{(n)} - \delta t\,\widetilde{\mathfrak{A}}[u]\widetilde{\varphi}^{(n)}$.

For a `FaceVelocity` both components are evaluated from the same input in each sweep and published together, a coordinate split and not a split between components. The advecting velocity is fixed during the step unless the field advects itself (`advect!(velocity, dt, workspace)`), in which case each sweep uses the published intermediate. The single-phase Navier–Stokes step transports $u^{(n)}$ with itself frozen as carrier, after enforcing its boundary values at $t^{(n)}$, and passes its boundary condition and $t^{(n)}$ to the boundary dual faces. `convection!` returns $(\widetilde{\varphi} - \widetilde{\varphi}^{(n+1)})/\delta t$ for a split scheme without advancing the order, and $\widetilde{\mathfrak{A}}[u]\widetilde{\varphi}$ with all dual faces and $\nu_g = 0$ for $\delta t = 0$.

Checks (`boundary_identity_check`). On a composite mesh with walls, a composite mesh whose refined root touches a wall (with 2 dual faces of kind (B3)) and the sloped-wall fixture of Chapter 5, (6.18) holds to 1.4×10^{-16} (relative) for a velocity that is neither solenoidal nor zero on the boundary, and for a discretely solenoidal velocity $\left|\widetilde{\mathfrak{A}}[u]1\right| \le 4.5 \times 10^{-17}$ relative to the largest rate of (6.23) at every unknown.

Implementation. `advect!(field, velocity, dt, workspace; scheme=LaxWendroff(), amr_interface_scheme, boundary=nothing, time=0)` for a `FaceField` with a `StaggeredAdvectionWorkspace`, and `advect!(velocity, advecting, dt, workspace; ...)` or `advect!(velocity, dt, workspace; ...)` with a `StaggeredVelocityAdvectionWorkspace`; `convection!` with the same keywords (6.16). Internal: `_convection_schedule!` (`src/operators/advection/staggered_fluxes.jl`), `_split_convection_rate!` (`src/operators/advection/staggered_fluxes.jl`), boundary dual faces `_boundary_dual_faces` (`src/operators/advection/staggered_schedules.jl`) (the three kinds), their rates `_boundary_dual_flux` (`src/operators/advection/staggered_fluxes.jl`) and states `_add_boundary_convection!` (`src/operators/advection/staggered_fluxes.jl`) (6.17), closed divergence `_add_boundary_flux_divergence!` (`src/operators/advection/staggered_fluxes.jl`) (6.18); Navier–Stokes stage `_transport_substep!` (`src/single_phase/stepping.jl`); options `AdvectionOptions(; scheme, amr_interface_scheme)`. Tests: `test/operators/advection/staggered_advection.jl` ("simultaneous staggered velocity transport"), `test/operators/advection/staggered_advection.jl` ("Lax-Wendroff convergence") (order above 1.9 for a translation), `test/operators/advection/staggered_advection.jl` ("staggered convection uses current boundary configuration"), `test/operators/advection/staggered_advection.jl` ("wall and cut-wall staggered constant identity") ((6.18) for every scheme, with and without solid, also per sweep direction), `test/operators/advection/staggered_advection.jl` ("staggered CFL includes wall-transition flux") (states of (B3)).

6.5 Conservation and kinetic energy

Momentum. Every dual face enters its two control volumes with opposite orientation, so summing (6.19) over the unknowns,

$$\sum_f \widetilde{V}_f \widetilde{\varphi}_f^{[j]} = \sum_f \widetilde{V}_f \widetilde{\varphi}_f^{[j-1]} - \delta t \sum_f \sum_{b \in \mathcal{G}_\Gamma(f),\ k(b)=k_j} \widetilde{o}_{f,b}\, \widetilde{\Upsilon}_b\, \widetilde{\varphi}_b, \tag{6.20}$$

for every scheme, interface scheme and mesh: on a periodic mesh the momentum $\sum_f \widetilde{V}_f \widetilde{\varphi}_f$ of each component is conserved up to rounding, also at level transitions and in cut cells (the unsplit step likewise); with walls it changes by the fluxes through the boundary dual faces. `integrate(field, workspace)` returns it.

Constants. At $\delta t = 0$ a constant $\widetilde{\varphi}$ gives $\widetilde{\mathfrak{A}}[u]1 = \widetilde{\mathfrak{D}}^{\Gamma}\widetilde{\Upsilon}$, which by (6.18) vanishes at every unknown for a velocity that is discretely solenoidal, $\mathfrak{D}^{\mathrm{o}}u = 0$ ((3.13)), on periodic meshes, at walls and in cut cells. The Lax–Wendroff state of a constant is the constant plus the compressive term, which cancels over the two sweeps only for a solenoidal velocity.

Kinetic energy. For the all-centred operator, with `Centred` as both `scheme` and `amr_interface_scheme`, the state is the mean of the two unknowns and (6.16) gives, on a periodic mesh,

$$\langle \psi, \widetilde{\mathfrak{A}}[u]\widetilde{\varphi}\rangle_{\widetilde{V}} + \langle \widetilde{\varphi}, \widetilde{\mathfrak{A}}[u]\psi\rangle_{\widetilde{V}} = \sum_f \widetilde{V}_f\, \widetilde{\varphi}_f \psi_f\, (\widetilde{\mathfrak{D}}\widetilde{\Upsilon})_f, \qquad 2\,\frac{\partial(\widetilde{\mathfrak{A}}[u]\widetilde{\varphi})_f}{\partial \widetilde{\varphi}_f} = (\widetilde{\mathfrak{D}}\widetilde{\Upsilon})_f : \tag{6.21}$$

the operator is skew-adjoint in $\langle\cdot,\cdot\rangle_{\widetilde{V}}$ up to half its dual divergence on the diagonal. The arithmetic mean is the only two-point state with this property: $\lambda\widetilde{\varphi}_{f^-(g)} + (1-\lambda)\widetilde{\varphi}_{f^+(g)}$ adds $(1-2\lambda)\widetilde{\Upsilon}_g$ to the off-diagonal symmetric part of $\widetilde{V}\widetilde{\mathfrak{A}}[]$ for the pair $f^-(g), f^+(g)$. With walls and vanishing boundary values $\widetilde{\varphi}^{\Gamma}_b$ the same identities hold with the closed divergence $\widetilde{\mathfrak{D}}^{\Gamma}\widetilde{\Upsilon}$ of (6.18) in place of $\widetilde{\mathfrak{D}}\widetilde{\Upsilon}$: the boundary dual faces (B1) and (B3) have the state $\frac{1}{2}\widetilde{\varphi}_f$ and add $\frac{1}{2}\widetilde{o}_{f,b}\widetilde{\Upsilon}_b$ to the diagonal, which the interior dual faces miss, and (B2) carries no flux at a wall (for non-vanishing boundary values the left side gains $\frac{1}{2}\sum_f \sum_b \widetilde{o}_{f,b}\widetilde{\Upsilon}_b\,(\widetilde{\varphi}^{\Gamma}_b\psi_f + \psi^{\Gamma}_b\widetilde{\varphi}_f)$ over the boundary dual faces of kinds (B1) and (B3)). The energy of the semi-discrete system $\mathrm{d}\widetilde{\varphi}/\,\mathrm{d}t = -\widetilde{\mathfrak{A}}[u]\widetilde{\varphi}$ therefore changes at the rate $-\frac{1}{2}\sum_f \widetilde{V}_f\widetilde{\varphi}_f^2\,(\widetilde{\mathfrak{D}}^{\Gamma}\widetilde{\Upsilon})_f$, and for a discretely solenoidal velocity it conserves $\frac{1}{2}\langle\widetilde{\varphi},\widetilde{\varphi}\rangle_{\widetilde{V}}$ on uniform, composite and cut meshes, with or without walls; this is the conservation of quadratic invariants of the thesis [19, App. 6.C, eqs. (6.78)–(6.81) and footnote 2], where the Lax–Wendroff state reduces to the centred one as $\delta t \to 0$, and the symmetry-preserving property of Verstappen and Veldman [32]. The default `Centred()` uses `MonotonizedCentral` on non-regular dual faces and is not skew on a composite mesh. The fully discrete forward-Euler step with the centred operator adds energy,

$$\tfrac{1}{2}\langle\widetilde{\varphi}^{(n+1)},\widetilde{\varphi}^{(n+1)}\rangle_{\widetilde{V}} - \tfrac{1}{2}\langle\widetilde{\varphi}^{(n)},\widetilde{\varphi}^{(n)}\rangle_{\widetilde{V}} = \tfrac{1}{2}\delta t^2\,\langle\widetilde{\mathfrak{A}}[u]\widetilde{\varphi}^{(n)},\widetilde{\mathfrak{A}}[u]\widetilde{\varphi}^{(n)}\rangle_{\widetilde{V}} \;\geq 0 \quad (\widetilde{\mathfrak{D}}^{\Gamma}\widetilde{\Upsilon} = 0), \tag{6.22}$$

so centred transport is not energy-stable in time; no energy statement is claimed for the split Lax–Wendroff or limited updates, which dissipate energy in the checks below.

Checks (`convection_conservation_check`). On the uniform, composite and disc fixtures with velocities from a streamfunction (on the disc fixture one that vanishes on solid vertices, so $\mathfrak{D}^{\mathrm{o}}u = 0$): one step of `Upwind`, `LaxWendroff`, `MonotonizedCentral` and `Superbee` changes the momentum by at most 5.9×10^{-16} (relative); the all-centred operator assembled from `convection!` at $\delta t = 0$ satisfies (6.21) with $\max\left|\widetilde{V}\widetilde{\mathfrak{A}}[] + (\widetilde{V}\widetilde{\mathfrak{A}}[])^{\mathsf{T}}\right| \leq 6.0 \times 10^{-15}$ relative to its largest entry and twice its diagonal equals the dual divergence to 4.3×10^{-16}; one centred forward-Euler step satisfies (6.22) to 3.8×10^{-8} of the growth term (the cut fixture, whose small step makes the

growth tiny); one default Lax–Wendroff step decreases the energy by 4.2×10^{-4} (uniform) and 4.2×10^{-4} (composite) of its value. With walls (`boundary_identity_check`, the three bounded meshes of Section 6.4) the all-centred operator with zero boundary values satisfies the closed form of (6.21), $\max \left| \widetilde{V}\widetilde{\mathfrak{A}}[] + (\widetilde{V}\widetilde{\mathfrak{A}}[])^{\intercal} - \operatorname{diag}(\widetilde{V}\widetilde{\mathfrak{D}}^{\Gamma}\widetilde{\Upsilon}) \right| \le 2.0 \times 10^{-16}$ relative to its largest entry, for a velocity that is not solenoidal.

Implementation. `integrate(field, workspace)` and `staggered_inner_product(a, b, workspace)` weight with $\widetilde{V}_f$. Tests: `test/operators/advection/staggered_advection.jl` ("centred convection matrix"), `test/operators/advection/staggered_advection.jl` ("centred staggered convection is skew-symmetric"), `test/operators/advection/staggered_advection.jl` ("AMR staggered flux compatibility and stability") (momentum to 2×10^{-13} at hanging faces for both orders, weighted skew symmetry for a constant velocity), `test/indicator/geometry.jl` ("Cut-cell projection and centred convection preserve symmetry") (energy rate after a cut-cell projection to 2×10^{-11}).

6.6 Time-step restriction

For the single-phase convection the code bounds the sum of the absolute rates of every control volume,

$$\delta t_{\mathrm{stag}} = \nu^{\mathrm{s}} \Big/ \max_{k=1,2} \max_{f} \frac{1}{\widetilde{V}_f} \Big(\sum_{g \in \mathcal{G}(f)} \left| \widetilde{\Upsilon}_g \right| + \sum_{b \in \mathcal{G}_{\Gamma}(f)} \left| \widetilde{\Upsilon}_b \right| \Big), \qquad 0 < \nu^{\mathrm{s}} \le 1, \tag{6.23}$$

over the dual faces of both axes and the boundary dual faces (6.17), with $\delta t_{\mathrm{stag}} = \infty$ for a vanishing velocity. `stable_timestep(velocity, workspace; cfl=0.9)` returns it; `step!` requires $\delta t \le (1 + 10^{-6} + 32\epsilon_{\mathrm{mach}})\delta t_{\mathrm{stag}}$ ((13.3), issue 98) with $\nu^{\mathrm{s}} = 1$, the top of the range (one phase; two phases bound the step as in Section 5.6), and `simulate!` chooses steps with $\nu^{\mathrm{s}} = 0.45$. The check at $\nu^{\mathrm{s}} = 1$ accepts every step that `stable_timestep` returns for `cfl` in $(0, 1]$ (issue 121): the sum in (6.23) bounds $\delta t \left| \widetilde{\Upsilon}_g \right|$ by $\nu^{\mathrm{s}} \min \left\{ \widetilde{V}_{f^-(g)}, \widetilde{V}_{f^+(g)} \right\}$ for every dual face g, because g belongs to both adjacent control volumes and each term of the sum is bounded by the sum. Hence $\nu_g \le \nu^{\mathrm{s}} \le 1$, and the Lax–Wendroff face state of (6.12) is a convex combination of its two upwind neighbours. Small open volumes reduce the bound, as no cells are merged. Under the bound the unsplit upwind update (`Upwind` as both schemes) of a discretely solenoidal velocity is a convex combination of old and boundary values: by (6.18) the outflow of a control volume through its dual and boundary dual faces equals its inflow, so the coefficient of the old value, $1 - \delta t \sum \left| \widetilde{\Upsilon} \right| / \widetilde{V}_f$ over the outflow faces, is at least $1 - \frac{1}{2}\nu^{\mathrm{s}}$, and the inflow terms carry upwind neighbours or, through (B1) and (B3), boundary values; $\nu^{\mathrm{s}} \le 2$ would suffice for this. The smallest coefficient at $\nu^{\mathrm{s}} = 1$ was $\frac{1}{2}$ on walled composite meshes (derivation audit, `proof_review/sec-momentum-cfl/`). No stability or TVD statement is claimed for the split second-order updates on composite meshes: the classical limiter regions of Figure 6.2a are one-dimensional.

The two-phase transport of Sections 6.7 and 6.8 has no bound of its own: the step is bounded by the volume-of-fluid bound, capillary and gravity limits of Section 5.6. On a

uniform mesh without solid this bounds the dual Courant numbers: a dual face through a cell centre carries half of the volume through each of two faces of that cell, each at most $\nu^\dagger|c|$ (the volume through a face is at most the volume entering or leaving the cell), and a transverse dual face half of a face volume of each of two cells, so $\nu_g \le \nu^\dagger \le \frac{1}{2}$ with $\widetilde{V}_f = |c|$; a uniform translation attains $\nu^\dagger$. The same bound holds on every 2:1-balanced mesh without solid, periodic or with walls: a hanging fine control volume is smaller, but the volumes through the faces of its small cell are capped by its own volume (checked as exact linear programs over every local configuration of random meshes, not proved here). With a solid the open volumes are small, and the argument above fails. The cut-donor bound of Section 5.6 restores it: it limits every outgoing flux by its donor's open strip, the same strip whose half the staggered volume of the receiving unknown interpolates, and the phase schedules weigh the fluxes by those strips ((6.24)). Over 25 random 2:1 cut meshes with a solid and a level transition, swept with the `Upwind`, `LaxWendroff` and `MonotonizedCentral` states and both directional sweeps, at $\nu^\dagger = \frac{1}{2}$ and $\nu^\dagger = 0.3$, $\nu_g/\nu^\dagger$ stayed at 1.000000000000000, and `test/two_phase/transport.jl` ("Dual Courant number next to a solid") holds the bound on the cut fixture; the supremum over the states the older per-cell bound admits reached 3.3 before that bound was in place (`proof_review/sec-momentum-cfl/`, issue 72). So the selected scheme's Lax–Wendroff or limited state stays a convex combination of its two neighbours next to a solid as well.

Implementation. `stable_timestep(velocity, workspace; cfl=0.9)` for a `StaggeredAdvectionWorkspace` or `StaggeredVelocityAdvectionWorkspace` (minimum over the components) via `_accumulate_staggered_rates!` (`src/operators/advection/staggered_update.jl`) with the boundary rates of `_boundary_dual_flux` (`src/operators/advection/staggered_fluxes.jl`); the step check in `_transport_substep!` (`src/single_phase/stepping.jl`). Tests: `test/operators/advection/staggered_advection.jl` ("staggered advection schemes") (upwind bounded at $\nu^{\mathrm{s}} = 0.8$; limited schemes bounded for a step), `test/operators/advection/staggered_advection.jl` ("directionally split AMR amplification") (spectral radius of the linear update at most one for the tested meshes), `test/operators/advection/staggered_advection.jl` ("staggered CFL includes wall-transition flux").

6.7 Two phases, one velocity

The one-velocity model transports the momentum $\widetilde{V}_f\widetilde{\rho}_f u_f$ of every staggered control volume inside the sweeps of Section 5.3, with the phase volumes of the fraction update interpolated to the dual faces (6.7), so that the staggered mass follows the cell mass algebraically [19, §6.4.1]. Its schedule is the phase schedule of Section 6.2 except that the rules (N) and (T) take, for a segment of a hanging side of a cut cell, the open-strip fraction

$$\varpi^{\mathrm{o}}_{c,f} = \frac{|\Lambda_{c,f} \cap c^{\mathrm{o}}|}{|c^{\mathrm{o}}|} \qquad (0 < \alpha^{\mathrm{o}}_c < 1,\ \varpi_{c,f} < 1) \tag{6.24}$$

in place of $\varpi_{c,f}$, and $\varpi^{\mathrm{o}}_{c,f} = \varpi_{c,f}$ otherwise, which equals the quotient in an uncut cell and for a single face. The strips of the segments of a side partition the cell, so these fractions

still sum to one per side, (6.10) holds with them, and $\frac{1}{2}\varpi^{\mathrm{o}}_{c,f}|c^{\mathrm{o}}| = \frac{1}{2}|\Lambda_{c,f}\cap c^{\mathrm{o}}|$ is the weight of the interpolation (6.4) in every open cell. The staggered density is the mixture density of the interpolated fractions (6.4),

$$\widetilde{\rho}_f = \rho^{\mathrm{l}}\widetilde{C}^{\mathrm{l}}_f + \rho^{\mathrm{g}}\widetilde{C}^{\mathrm{g}}_f, \qquad \widetilde{C}^{\mathrm{g}}_f = 1 - \widetilde{C}^{\mathrm{l}}_f. \tag{6.25}$$

In sweep j along k_j the carrier dilation per open volume of a cell, the quantity frozen in (5.6), is

$$\mathcal{D}^{[j]}_c = \frac{1}{|c^{\mathrm{o}}|}\sum_{f\in\mathcal{F}(c)\cap\mathcal{F}^{[k_j]}} o_{c,f}\,\delta t|f|v^{\mathrm{o}}_f \quad (\alpha^{\mathrm{o}}_c > 0), \tag{6.26}$$

and zero in covered cells, and the staggered fractions predicted for the end of the sweep interpolate the clamped fractions before snapping ($\check{C}^{\mathrm{l},[j]}_c$ of (5.14)),

$$\widetilde{\check{C}}^{\mathrm{l},[j]}_f = \Big(\mathfrak{H}\,\mathrm{clamp}(\check{C}^{\mathrm{l},[j]},0,1)\Big)_f, \qquad \widetilde{\check{C}}^{\mathrm{g},[j]}_f = 1-\widetilde{\check{C}}^{\mathrm{l},[j]}_f, \qquad \widetilde{\rho}^{[j]}_f = \textstyle\sum_\pi \rho^\pi \widetilde{\check{C}}^{\pi,[j]}_f. \tag{6.27}$$

Each component is updated in fluctuation form, as stored by the code:

$$u^{[j]}_f = u^{[j-1]}_f + \frac{1}{\widetilde{\rho}^{[j]}_f}\Bigg(\frac{1}{\widetilde{V}_f}\sum_\pi \rho^\pi \sum_{g\in\mathcal{G}(f),\ k(g)=k_j} \widetilde{o}_{f,g}\,\delta t\widetilde{\Upsilon}^\pi_g\left(u^{[j-1]}_f - (\widetilde{\mathfrak{F}}u^{[j-1]})^\pi_g\right) + \left(\mathfrak{H}\Big[\textstyle\sum_\pi \rho^\pi\chi^\pi\mathcal{D}^{[j]}\Big]\right)_f\left(u^{(n)}_f - u^{[j-1]}_f\right)\Bigg), \tag{6.28}$$

with the frozen indicator (5.4) and the start values $u^{(n)}$ in the dilation term, the analogue of (5.20); the face state of phase π is that of the modified flux interpolant below. A constant u gives a zero increment whatever the fractions, so constants are preserved exactly. Multiplying by $\widetilde{V}_f\widetilde{\rho}^{[j]}_f$ shows the conservative form

$$\widetilde{V}_f\widetilde{\rho}^{[j]}_f u^{[j]}_f = \widetilde{V}_f\widetilde{\rho}^{[j-1]}_f u^{[j-1]}_f - \sum_\pi \rho^\pi\sum_g \widetilde{o}_{f,g}\,\delta t\widetilde{\Upsilon}^\pi_g\,(\widetilde{\mathfrak{F}}u)^\pi_g + \widetilde{V}_f\left(\mathfrak{H}\Big[\textstyle\sum_\pi\rho^\pi\chi^\pi\mathcal{D}^{[j]}\Big]\right)_f u^{(n)}_f \tag{6.29}$$

provided the staggered mass obeys the interpolated cell mass update,

$$\widetilde{V}_f\left(\widetilde{\rho}^{[j]}_f - \widetilde{\rho}^{[j-1]}_f\right) = -\sum_\pi\rho^\pi\sum_g\widetilde{o}_{f,g}\,\delta t\widetilde{\Upsilon}^\pi_g + \widetilde{V}_f\left(\mathfrak{H}\Big[\textstyle\sum_\pi\rho^\pi\chi^\pi\mathcal{D}^{[j]}\Big]\right)_f, \tag{6.30}$$

the thesis' mass–momentum consistency $\widetilde{\mathfrak{A}}[m]1 = \mathfrak{H}(\mathfrak{A}[m]1)$ [19, eq. (6.45)] with the dilation of the split update, where $\widetilde{\rho}^{[j-1]}$ is the density of the fractions stored at the start of the sweep (snapped, (5.14)) and $\widetilde{\rho}^{[j]}$ that of the predicted fractions (6.27). By (6.10) for the phase volumes and the fraction update (5.6), for a schedule built with the side fractions $\varpi_{c,f}$ the left side of (6.30) minus its right side is

$$\frac{1}{2}\sum_{c\in\mathcal{C}(f)}\left(\varpi_{c,f}|c^{\mathrm{o}}| - |\Lambda_{c,f}\cap c^{\mathrm{o}}|\right)\mathcal{M}^{[j]}_c + \frac{1}{2}\sum_{c\in\mathcal{C}(f)}|\Lambda_{c,f}\cap c^{\mathrm{o}}|\,(\rho^{\mathrm{l}}-\rho^{\mathrm{g}})\left(\mathrm{clamp}(\check{C}^{\mathrm{l},[j]}_c,0,1) - \check{C}^{\mathrm{l},[j]}_c\right), \tag{6.31}$$

with $\mathcal{M}_c^{[j]} = |c^{\mathrm{o}}|^{-1} \sum_\pi \rho^\pi \sum_{f' \in \mathcal{F}(c) \cap \mathcal{F}^{[k_j]}} o_{c,f'}\, \delta t |f'| v^\pi_{f'}$ the net phase-mass outflow of c in the sweep (the dilation terms cancel). The first sum vanishes when (b) every cell $c \in \mathcal{C}(f)$ is uncut or has f as the single face of its side, so that $|\Lambda_{c,f} \cap c^{\mathrm{o}}| = \varpi_{c,f} |c^{\mathrm{o}}|$. The one-velocity schedule has $\varpi^{\mathrm{o}}_{c,f}$ in place of $\varpi_{c,f}$ in (6.31), so its first sum vanishes in every cell, and (6.30) holds exactly, up to rounding, when (a) no fraction is clamped in the sweep. The code stops if the liquid volume $|c^{\mathrm{o}}| \check{C}_c^{\mathrm{l},[j]}$ of an open cell leaves $[0, |c^{\mathrm{o}}|]$ by more than $128 \epsilon_{\mathrm{mach}} |c|$ (since issue 55), so the clamp changes the liquid volume by at most $128 \epsilon_{\mathrm{mach}} |c|$ and the second sum is rounding in units of the cell volume. Before issue 63 the one-velocity schedule used the side fractions, and where (b) failed, in a cut coarse cell whose side carries two segments, the staggered mass and momentum drifted from the cell mass by the first sum; the schedule of Section 6.8 still uses them. A snap of a fraction after the sweep (5.14) changes $\widetilde{\rho}$ but not u, so it changes the momentum by $\widetilde{V}_f\, \delta \widetilde{\rho}_f u_f$, as for the phase scalars of Section 5.7. Summing (6.29) over the unknowns and the two sweeps, the momentum $\sum_f \widetilde{V}_f \widetilde{\rho}_f u_f$ of each component changes by $\sum_j \sum_f \widetilde{V}_f (\mathfrak{H}[\sum_\pi \rho^\pi \chi^\pi \mathcal{D}^{[j]}])_f u_f^{(n)}$, which vanishes for a carrier that is solenoidal in every cell, since the frozen values make the two dilations of a cell cancel; it is therefore conserved up to rounding under (a), without snapping, and with periodic or impermeable boundaries. This is the thesis' exact conservation of the sum of the phase momenta [19, eq. (6.53)]: the update divides the sum of the phase momenta by the sum of the phase masses.

Modified flux interpolant. The state of phase π on the dual face g is the bulk state (6.12) of the selected scheme, without the compressive term and with $\nu_g = \left|\delta t \widetilde{\Upsilon}^\pi_g\right| / \min \widetilde{V}$, unless an adjacent predicted fraction is small, in which case the flux is upwinded term by term:

$$\delta t \widetilde{\Upsilon}^\pi_g (\widetilde{\mathfrak{F}} \widetilde{\varphi})^\pi_g = \begin{cases} \displaystyle\sum_f \widetilde{w}_{g,f}\, \delta t |f| v^\pi_f\ \widetilde{\varphi}_{f^{\mathrm{u}}_{g,f}}, & \min\left(\widetilde{\check{C}}^{\pi,[j]}_{f^-(g)}, \widetilde{\check{C}}^{\pi,[j]}_{f^+(g)}\right) < \alpha^\dagger, \\ \delta t \widetilde{\Upsilon}^\pi_g\ (\widetilde{\mathfrak{F}} \widetilde{\varphi})_g, & \text{otherwise}, \end{cases} \tag{6.32}$$

where $f^{\mathrm{u}}_{g,f} = f^-(g)$ if the partial volume $\widetilde{w}_{g,f} \delta t |f| v^\pi_f$ is non-negative and $f^+(g)$ otherwise, and $\alpha^\dagger$ is `ctu_threshold` (default $\frac{1}{2}$, in $(0, 1]$). Keeping the partial interpolated volumes separate upwinds each of them with the value of its own upwind unknown; with P_g and N_g the sums of the positive partial volumes and of the magnitudes of the negative ones,

$$\sum_f \widetilde{w}_{g,f}\, \delta t |f| v^\pi_f\ \widetilde{\varphi}_{f^{\mathrm{u}}_{g,f}} = \delta t \widetilde{\Upsilon}^\pi_g \widetilde{\varphi}_{f^{\mathrm{u}}_g} + \min(P_g, N_g) \left(\widetilde{\varphi}_{f^-(g)} - \widetilde{\varphi}_{f^+(g)}\right), \tag{6.33}$$

the upwind flux of the net volume plus a counter-flow exchange between the two unknowns, which is diffusive and vanishes unless partial volumes of both signs meet at the dual face. This is the corner-transport-upwind (CTU) flux of the thesis [19, eqs. (6.49)–(6.51)] restricted to the two unknowns of the dual face (the thesis attributes partial volumes to their donor cells over $2 \times 3^{d-1}$ faces): the own flux of an unknown enters its control volume at one dual face with the value of the upwind neighbour and leaves at another with its own value, so no fluid bypasses the control volume. On a conforming patch the volume of a phase leaving a control volume is half the volume leaving each of its two cells, hence at most the volume of that phase in it, and the coefficient of the old value in the update stays non-negative: the update of a draining control volume is bounded by the cell-wise outflow bound, as in the

thesis lemmas [19, Lemmas 6.2 and 6.3], and not by the switch. At a segment of a hanging side the cell-wise bound does not reach the code's factorization and the coefficient of (6.34) can be negative; the transport limits the outflow there (deviation 6.4).

Boundaries and solids. In a bounded domain and with a solid the phase schedule (6.3) includes the boundary unknowns (whose primal volumes are zero, Section 5.6), so the identity holds there without boundary dual faces; the transported boundary values are overwritten when the Navier–Stokes step enforces its boundary condition. The dual faces of this schedule are classified by (6.11) with the uncut weights and volumes, and its three-point states find their upstream unknowns among all unknowns, the boundary unknowns included: a regular dual face, also one between cut cells, uses the selected scheme, a non-regular one or one incident to a boundary unknown the interface scheme (Section 6.9). Before issue 62 every dual face of this schedule was non-regular and had no upstream unknown, so that the transport of a bounded domain or of a mesh with a solid was first-order upwind away from the interface.

Checks (`one_velocity_momentum_check`). An independent replica of (6.25) to (6.28) and (6.32) with its own schedule (fractions (6.24), classification (6.11), upstream unknowns (6.15)) and states and the code's primal phase volumes reproduces the velocity after `advect!` to 2.2×10^{-15} (relative) on the uniform, composite, cut and walled fixtures (drop and streamfunction velocity of Chapter 5; on the cut fixture a gas bubble crosses a level transition and the disc of Figure 6.1 lies in the liquid). With `LaxWendroff` on these four fixtures and with `MonotonizedCentral` in the order $(2,1)$ on the composite one, no fraction is clamped or snapped, the mass identity (6.30) holds to 5.4×10^{-15} in every unknown and sweep, also in the 16 unknown-sweeps of the cut fixture where (b) fails (with the side fractions it would miss there by up to $0.078\,\rho^{\mathrm{l}}$ per unit volume), and the momentum changes by at most 2.8×10^{-17} (relative). A constant transported velocity is reproduced exactly on all fixtures (`constant_velocity_check`).

Deviation from the thesis 6.3 (Modified flux interpolant in directional sweeps). The thesis switches to the CTU interpolant when the staggered fraction at the end of the unsplit step falls below $\alpha^\dagger = \frac{1}{2}$, with partial volume fluxes of the unsplit donating regions over the $2\times3^{d-1}$ neighbouring faces [19, eqs. (6.49)–(6.51)]. The code applies the switch per directional sweep with the predicted fractions (6.27) and upwinds the partial interpolated volumes of the weights (6.6) between the two unknowns of the dual face only, a restriction of the thesis' CTU flux (6.33); the bulk state is the directional one of Section 6.3, and a dilation term with frozen values completes the split update.

Deviation from the thesis 6.4 (Outflow limiter at hanging segments). The thesis' lemmas bound the CTU update of a draining staggered control volume [19, Lemmas 6.2 and 6.3] through the cell-wise outflow bound. At a segment f of a hanging side that bound does not reach the code's factorization (rule (N), (6.6)). Let the own flux of f, from $c^-(f)$ into $c^+(f)$, be the only non-zero flux, all of the phase of $c^-(f)$ lie in its donating region and $c^+(f)$ be empty: rule (N) passes $\frac{1}{2}(2-\varpi_{c^+(f),f})$ of this flux out of the control volume of f, which holds only $\frac{1}{2}\varpi_{c^-(f),f}$ of it, and the coefficient of the old value in the update of f would be

$$\gamma_f = -\frac{2-\varpi_{c^-(f),f}-\varpi_{c^+(f),f}}{\varpi_{c^+(f),f}}, \tag{6.34}$$

zero on a conforming patch but $-\frac{1}{2}$ from a coarse into a fine cell and -1 from a fine into a coarse cell, for every face state including the net-volume upwind one. Left alone the update leaves the range of the old values: in one step a staggered phase scalar 0 among neighbours 1 became 1.98, and a one-velocity momentum 1.95 at the density ratio 1000 (derivation audit, `proof_review/eq-momentum-ctu/`). The transport limits the outflow instead of the state (`_phase_drain_limit!` (`src/two_phase/phase_face_transport.jl`)). Where the phase volume O_f leaving an unknown exceeds the phase volume $\widetilde{V}_f\widetilde{C}^\pi_f$ its staggered content holds at the start of the sweep, the excess $O_f - \widetilde{V}_f\widetilde{C}^\pi_f$ leaves at the volume-weighted mean of the values that enter the unknown, over the I_f that enter, rather than at its own value. Every partial volume carries one value, for its sender and its receiver alike, so the update stays the conservative one of (6.28) (the excess redistributes the value the flux carries, not the flux), and with the excess fraction $\xi_f = \max(0, 1 - \widetilde{V}_f\widetilde{C}^\pi_f/O_f)$ the coefficients of the old value, $\max(0, \widetilde{V}_f\widetilde{C}^\pi_f - O_f)$, and of the entering values, $I_f - \xi_f O_f$, are both non-negative and sum to the new content $\widetilde{V}_f\widetilde{C}^\pi_f - O_f + I_f$. The mean is taken over the values the neighbouring unknowns carry at the start of the sweep, not over their limited ones, so where a neighbour is itself limited the mean is a convex combination of the raw values: the coefficients above stay non-negative, but no new extremum appeared in 3000 random hanging configurations only up to 5×10^{-14}, i.e. as observed behaviour, not as a proved property of that case. A limited unknown takes the term-wise flux of (6.33) in both branches of the switch: the bulk state moves the net volume of a face at one value, while the limiter counts the partial volumes by sign, so the two agree on the outflow of a limited unknown only when every partial is upwinded. On a conforming patch the cell-wise bound keeps $O_f \le \widetilde{V}_f\widetilde{C}^\pi_f$, so $\xi_f = 0$ and the transport is unchanged; the limiter acts only at level transitions, where `test/two_phase/transport.jl` ("Staggered transport stays bounded at a hanging segment") holds the range of the old values and the phase amount for both the staggered phase scalar and the one-velocity momentum (issue 70).

Implementation. `MomentumAdvectionWorkspace(interface)` and `advect!(interface, velocity, dt, work; momentum=destination, density=(rho_l, rho_g), momentum_workspace, scheme, amr_interface_scheme)` with a `WYAdvectionWorkspace(interface; ctu_threshold=0.5)`; `mixture_density` (6.25). Schedule `_open_strip_phase_schedule` (`src/two_phase/phase_face_transport.jl`) (6.24), shared by the one-velocity momentum, the two-velocity momentum and the staggered phase scalars; update `_advect_momentum_component!` (`src/two_phase/momentum_workspace.jl`) (6.28), modified interpolant `_phase_transport` (`src/two_phase/phase_face_transport.jl`) and `_ctu_phase_transport` (`src/two_phase/phase_face_transport.jl`) (6.32); called per sweep from `_advect_wy!` (`src/vof/fluxes.jl`), whose bound check is `_wy_fraction_in_bounds` (`src/vof/fluxes.jl`); flow stage `_transport_substep!` (`src/two_phase/stepping.jl`). Tests: `test/two_phase/transport.jl` ("All staggered flux interpolants transport one-velocity momentum") (momentum to $512\epsilon_{\mathrm{mach}}$, uniform and composite), `test/two_phase/transport.jl` ("All staggered flux interpolants in two-phase flow") (bulk equal to single-phase transport, bounded values for the limiters), `test/two_phase/transport.jl` ("Cut coarse-fine mass-momentum consistency")

(identity with the open-strip fractions, momentum to $128\epsilon_{\mathrm{mach}}$ with a cut coarse cell with two segments on a side), `test/two_phase/transport.jl` ("WY sliver-cell bound distinguishes round-off from violations").

6.8 Two velocities and staggered phase scalars

The two-velocity model stores intrinsic phase velocities, and a `PhaseFaceScalar` stores intrinsic liquid and gas values on the faces of one axis; both are transported by the same update, per phase, with the phase volumes of the carrier of Section 5.8, on the phase schedule of Section 6.2 with the open-strip fractions of (6.24), as for the one-velocity momentum: the divergence of that factorization commutes with the staggered fractions $\widetilde{C}_f^{\pi}$ of (6.4) also in a cut coarse cell whose side carries two segments, which the geometric side fractions do not (issue 71). Before the sweeps, a value is *absent* where its phase has no open staggered volume ($\widetilde{V}_f$ of (6.3) zero, or $\mathrm{clamp}(\widetilde{C}_f^{\mathrm{l}},0,1)=0$ for the liquid, $=1$ for the gas); absent values are extended in two passes over the dual faces, each taking the mean of the present values of the unknowns joined to it, and set to zero beyond,

$$\widetilde{\varphi}_f^{\mathrm{e},\pi} = \begin{cases} \widetilde{\varphi}_f^{\pi}, & \text{present,} \\ \frac{1}{\#N(f)} \sum_{f'\in N(f)} \widetilde{\varphi}_{f'}^{\mathrm{e},\pi}, & N(f)\neq\emptyset \text{ in one of the two passes,} \\ 0, & \text{otherwise,} \end{cases} \tag{6.35}$$

$N(f)$ the unknowns with a value that share a dual face with f at the start of the pass. These working values are also the frozen start values $\widetilde{\varphi}^{\pi,(n)}$ of the dilation term. In sweep j, with the predicted fraction $\widetilde{\widetilde{C}}_f^{\pi,[j]}$ of (6.27),

$$\widetilde{\varphi}_f^{\pi,[j]} = \widetilde{\varphi}_f^{\pi,[j-1]} + \frac{1}{\widetilde{\widetilde{C}}_f^{\pi,[j]}} \left(\frac{1}{\widetilde{V}_f} \sum_{g\in\mathcal{G}(f),\ k(g)=k_j} \widetilde{o}_{f,g}\, \delta t \widetilde{\Upsilon}_g^{\pi} \left(\widetilde{\varphi}_f^{\pi,[j-1]} - (\widetilde{\mathfrak{F}}\widetilde{\varphi}^{\pi,[j-1]})_g \right) \right.$$
$$\left. + \left(\mathfrak{H}\left[\chi^{\pi}\mathcal{D}^{[j]}\right] \right)_f \left(\widetilde{\varphi}_f^{\pi,(n)} - \widetilde{\varphi}_f^{\pi,[j-1]} \right) \right) \quad \text{if } \widetilde{\widetilde{C}}_f^{\pi,[j]} > \varepsilon^{\alpha}, \tag{6.36}$$

and $\widetilde{\varphi}_f^{\pi,[j]} = \widetilde{\varphi}_f^{\pi,[j-1]}$ otherwise, with the modified interpolant (6.32). There is no density: for constant ρ^{π} it cancels, and the conserved amount of phase π is $\rho^{\pi}\sum_f \widetilde{V}_f \widetilde{C}_f^{\pi}\widetilde{\varphi}_f^{\pi}$ (`integrate(q, phase)` for a phase scalar). The phase volumes of (5.25) partition the carrier, $\widetilde{\widetilde{C}}^{\mathrm{g},[j]} = 1 - \widetilde{\widetilde{C}}^{\mathrm{l},[j]}$, and both phases follow (6.36) with their indicator. After the step the velocities of phases without open staggered volume in the final state are stored as NaN (6.35). As for (6.29), the update conserves the phase amount up to the frozen dilation,

$$\sum_f \widetilde{V}_f \widetilde{C}_f^{\pi} \widetilde{\varphi}_f^{\pi} \Big|_{(n)}^{(n+1)} = \sum_{j=1}^{2} \sum_f \widetilde{V}_f \left(\mathfrak{H}\left[\chi^{\pi}\mathcal{D}^{[j]}\right] \right)_f \widetilde{\varphi}_f^{\pi,(n)} \tag{6.37}$$

under condition (a) of Section 6.7, without snapping and with every updated unknown above the floor; the dilation sum vanishes for a carrier that is solenoidal in every cell (the mixture constraint on the frozen apertures).

Checks (`two_velocity_momentum_check`). An independent replica of (6.32), (6.35) and (6.36) reproduces both phase velocities after `advect!` to 2.2×10^{-16} (relative) on the uniform, composite and walled fixtures and the NaN pattern exactly; the balance (6.37) holds to 2.5×10^{-16} (relative), where the second streamfunction used as gas velocity makes the frozen carrier non-solenoidal in mixed cells and the dilation sum non-zero.

Deviation from the thesis 6.5 (Two-velocity momentum). The thesis transports the liquid momentum with the liquid mass fluxes of the projected-liquid method and the gas momentum with the gas mass fluxes of its auxiliary carrier, divided by the auxiliary gas mass and reset to the complement after every time step [19, eqs. (6.54)–(6.55)]. The code uses the frozen carrier (Section 5.8.1) instead, with which both phase amounts are balanced by (6.37); the projected carrier of the thesis is not implemented.

Implementation. `MomentumAdvectionWorkspace(interface, velocity)` for a `TwoPhaseVelocity` and `PhaseFaceScalar\{Axis\}(interface; liquid=1, gas=1)` (values or functions of (x, y)), passed to `advect!` as `momentum`, `momentum_workspace` and `staggered_scalars`; `integrate(q::PhaseFaceScalar, phase)`. Schedule `_phase_advection_workspace` (`src/two_phase/phase_face_transport.jl`) with `_open_strip_phase_schedule` (`src/two_phase/phase_face_transport.jl`) ((6.24)); preparation `_prepare_phase_momentum_values!` (`src/two_phase/two_velocity/momentum_workspace.jl`) with `_extend_phase_face_values!` (`src/two_phase/two_velocity/velocity_fields.jl`) (6.35); update `_advect_phase_faces_workspace!` (`src/two_phase/phase_face_transport.jl`) (6.36); marking `_mark_inactive_phase_velocities!` (`src/two_phase/two_velocity/velocity_fields.jl`). Tests: `test/two_phase/two_velocity/transport/two_velocity.jl` ("Compatible separate phase-momentum transport") (constant phase velocities, with solid and composite mesh), `test/two_phase/transport.jl` ("Compatible staggered phase constants"), `test/two_phase/transport.jl` ("bounded two-velocity staggered schemes") and `test/two_phase/transport.jl` ("bounded staggered schemes with both phases present") (selected scheme on bounded and cut schedules, equal to the single-phase transport away from the interface), `test/two_phase/transport.jl` ("two-velocity transport near a sloped solid"), `test/two_phase/transport.jl` ("Cut coarse-fine phase-schedule consistency") (the divergence of the phase schedule commutes with the open-strip interpolation of the staggered fraction at a split coarse cut side, to 2×10^{-13}; the geometric side fractions miss by 1×10^{-1}).

6.9 Which flux on which dual face

The face state of a dual face depends on the transport, on the kind of the dual face and, in two phases, on the staggered fractions next to it. Table 6.1 collects the rules and Figure 6.3 shows them on a walled composite mesh; in detail:

1. *Regular dual faces*, $g \notin \mathcal{G}^*$, take the state of `scheme` (6.12), by default `LaxWendroff`, in the single-phase convection (`_convection_schedule!` (`src/operators/advection/staggered_fluxes.jl`)) and in the two-phase transport (`_phase_connection_scheme` (`src/`

`two_phase/phase_face_transport.jl`)); every dual face of the uniform periodic schedule counts as regular (`_dual_state` (`src/operators/advection/staggered_schedules.jl`)). The classification (6.11) is made once, with the uncut weights and volumes (`_assemble_amr_dual_schedule` (`src/operators/advection/staggered_schedules.jl`)), and the schedules with a solid keep it (`_cut_cell_dual_schedule` (`src/embedded/operators.jl`), `_geometry_phase_schedule` (`src/embedded/operators.jl`)): a dual face between cut cells is regular when its uncut counterpart is, also in the phase schedules.

2. *Non-regular dual faces*, $g \in \mathcal{G}^*$, take the state of `amr_interface_scheme`: the dual faces at level transitions and, in the phase schedule of a bounded domain or of a mesh with a solid, those incident to a boundary unknown (`_bounded_phase_schedule` (`src/two_phase/phase_face_transport.jl`)). The default, `default_amr_interface_scheme`, is `Upwind` for `Upwind`, `MonotonizedCentral` for `Centred` and `LaxWendroff`, and the scheme itself for `Fromm` and the four limited schemes; an `Upwind` interface scheme with another bulk scheme is rejected (`_validate_amr_interface_scheme` (`src/operators/advection/staggered_schemes.jl`)).

3. *Three-point states without upstream unknown* are upwind: no unknown qualifies in (6.15), as beyond an unknown next to a wall normal to $\boldsymbol{e}_k$ in the single-phase schedule, which has no unknown on the wall, or the two unknowns lie on one line normal to $\boldsymbol{e}_{k(g)}$, as two segments of one side (`_three_point_stencils` (`src/operators/advection/staggered_schedules.jl`), `_amr_dual_state` (`src/operators/advection/staggered_fluxes.jl`), `_phase_bulk_state` (`src/two_phase/phase_face_transport.jl`)). The phase schedules search the upstream unknown among all unknowns, the boundary unknowns included.

4. *Boundary dual faces* (6.17) exist in the single-phase convection only and enter when a boundary condition is passed; their rates carry the open apertures of their primal faces (`_boundary_dual_flux` (`src/operators/advection/staggered_fluxes.jl`)). (B1) and (B3) take the upwind state with the stored boundary value on inflow if `amr_interface_scheme` is `Upwind`, and the centred state $\frac{1}{2}(\widetilde{\varphi}_f + \widetilde{\varphi}_b^{\Gamma})$ otherwise; (B2) takes the prescribed velocity at `NoSlip` and `Inflow` boundaries and $\widetilde{\varphi}_f$ otherwise (`_add_boundary_convection!` (`src/operators/advection/staggered_fluxes.jl`)). (B3) joins a fine unknown to the wall face of a coarse cell where a level transition meets a wall normal to $\boldsymbol{e}_k$ (`_boundary_dual_faces` (`src/operators/advection/staggered_schedules.jl`)).

5. *In the interface band* of the two-phase transport (the one-velocity momentum, the phase velocities of the two-velocity model and the staggered phase scalars) a phase takes, before all of the above, the CTU flux (6.32) on a dual face where its predicted staggered fraction (6.27), the volume-weighted average over the staggered

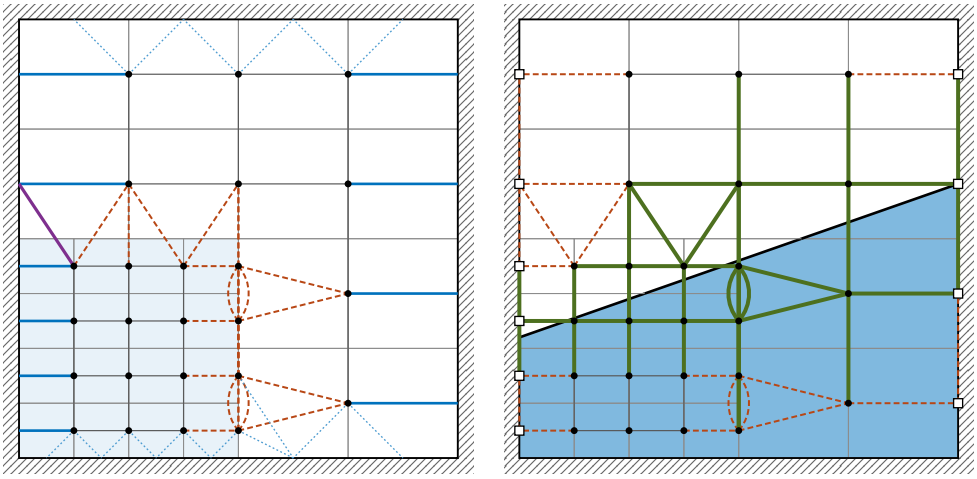
dual face	single-phase convection	two-phase transport, phase π
regular, $g \notin \mathcal{G}^*$ (also between cut cells; all dual faces of the uniform periodic schedule)	`scheme`	`scheme`
level transition, $g \in \mathcal{G}^*$	`amr_interface_scheme`	`amr_interface_scheme`
incident to a boundary unknown, $g \in \mathcal{G}^*$	— (walls are not unknowns)	`amr_interface_scheme`
boundary dual face (B1), (B3)	upwind with $\widetilde{\varphi}_b^\Gamma$ for an `Upwind` interface scheme, else $\frac{1}{2}(\widetilde{\varphi}_f + \widetilde{\varphi}_b^\Gamma)$	—
boundary dual face (B2)	prescribed velocity (`NoSlip`, `Inflow`), else $\widetilde{\varphi}_f$	—
three-point state without upstream unknown	upwind	upwind
$\min(\widetilde{C}^{\pi,[j]}_{f^-(g)}, \widetilde{C}^{\pi,[j]}_{f^+(g)}) < \alpha^\dagger$	—	CTU (6.32), before all rows above

Table 6.1. Face state of each kind of dual face, as implemented (Section 6.9); `amr_interface_scheme` defaults to `default_amr_interface_scheme(scheme)`.

control volume of the cell fractions predicted for the end of the sweep, is below the threshold at one of the two unknowns, $\min(\widetilde{C}^{\pi,[j]}_{f^-(g)}, \widetilde{C}^{\pi,[j]}_{f^+(g)}) < \alpha^\dagger$ (`_phase_transport` (`src/two_phase/phase_face_transport.jl`)). The threshold is the keyword `ctu_threshold` of `WYAdvectionWorkspace`, default $\frac{1}{2}$, in $(0, 1]$ (`WYAdvectionWorkspace` (`src/vof/workspace.jl`)). The two phases are switched separately; a phase without volume at a dual face carries no flux there.

Checks (`phase_classification_check`). On the walled composite fixture of Section 6.7 the phase schedules of the two components have 578 regular dual faces, 204 non-regular ones at level transitions and 92 incident to a boundary unknown; on the cut fixture 105 regular dual faces next to a cut cell use the selected scheme. The weights, the classification and the 7000 upstream unknowns of the code's phase-scalar and one-velocity schedules on both fixtures equal those of the independent construction, which the replicas of Sections 6.7 and 6.8 use.

Implementation. Tests: `test/two_phase/transport.jl` ("bounded phase schedule classifies connections"), `test/two_phase/transport.jl` ("bounded two-velocity staggered schemes") (selected scheme on the regular dual faces of a bounded mesh and of a cut row), `test/operators/advection/staggered_advection.jl` ("stationary AMR-interface extremum probe"), `test/operators/advection/staggered_advection.jl` ("staggered CFL includes wall-transition flux") (states of (B3) for `Upwind` and `Centred`).



A. Single-phase convection. **B.** Two-phase transport.

FIGURE 6.3. Face state of every dual face of the component u on $\mathcal{F}^{[1]}$, read from the code's schedules on a 4×4 mesh of unit cells with walls on all sides whose lower-left root is refined once (`momentum_rules_data`); every dual face is drawn as an edge between the two unknowns it joins (dots; squares: boundary unknowns of the phase schedule), arcs where two dual faces of different axes join the same pair. Thin grey: regular, the state of `scheme`; ---- non-regular or incident to a boundary unknown, the state of `amr_interface_scheme`; (a) boundary dual faces, drawn to their boundary face: — (B1), ····· (B2), without flux at a wall, — (B3), where the level transition meets the wall; (b) liquid below $x_2 = 1.1 + 0.35x_1$ (blue, PLIC lines), and — CTU: a phase with volume at one of the two unknowns has a staggered fraction below $\alpha^\dagger = \frac{1}{2}$ at one of them (here the fractions at the start of a sweep, which are the predicted fractions of a vanishing step).

6.10 CELL-CENTRED SCALAR TRANSPORT

A passive `CellField` φ is transported by the conservative law $\partial_t \varphi + \nabla \cdot (\varphi \boldsymbol{u}) = 0$ with the explicit upwind finite-volume update

$$
\begin{aligned}
|c^{\mathrm{o}}|\varphi_c^{(n+1)} &= |c^{\mathrm{o}}|\varphi_c^{(n)} - \delta t \sum_{f \in \mathcal{F}(c)} o_{c,f}\, |f| a_f^{\mathrm{o}} u_f\, \varphi_f^{\mathrm{u}}, \\
\varphi_f^{\mathrm{u}} &= \begin{cases} \varphi_{c^-(f)} \text{ if } u_f \geq 0,\ \varphi_{c^+(f)} \text{ otherwise}, & f \in \mathcal{F}_\Omega, \\ \varphi_{c(f)} \text{ if } s_f u_f \geq 0,\ \varphi_f^\Gamma \text{ otherwise}, & f \in \mathcal{F}_\Gamma, \end{cases}
\end{aligned}
\tag{6.38}
$$

where the inflow value φ_f^Γ is the cell value (`ZeroGradient`) or the prescribed function at $(\boldsymbol{x}_f, t)$ (`Dirichlet`); without solid $a_f^{\mathrm{o}} = 1$ and $|c^{\mathrm{o}}| = |c|$, and covered cells are not updated. Every owned segment is evaluated once, so the update is conservative on the composite mesh (Section 3.5.1); a constant is preserved where $\mathfrak{D}^{\mathrm{o}} u = 0$. The bound

$$
\delta t_{\mathrm{cell}} = \nu^{\mathrm{s}} \Big/ \max_{c,\ |c^{\mathrm{o}}|>0} \frac{1}{|c^{\mathrm{o}}|} \sum_{f \in \mathcal{F}(c)} |f| a_f^{\mathrm{o}} |u_f|, \qquad 0 < \nu^{\mathrm{s}} \leq 1 \text{ (default } 0.9), \tag{6.39}
$$

sums over all faces of the cell, boundary faces included. With a solid it uses the open apertures and volumes when the workspace is built with the `GeometryField` or the geometry is passed to `stable_timestep` (since issue 64); covered cells do not restrict it. Under the bound the coefficient of the old value in (6.38) is at least $1 - \nu^{\mathrm{s}} \geq 0$, so non-negative data stay non-negative; for $\mathfrak{D}^{\mathrm{o}} u = 0$ the coefficients sum to one, and the update is a convex combination of upwind and inflow values, also in small cut cells. The transport is not used by the flow solvers, whose cell quantities are the phase fractions and phase scalars of Chapter 5.

Implementation. `AdvectionWorkspace(scalar[, geometry])`, `advect!(scalar, velocity, dt, workspace; boundary=ZeroGradient(), time=0)`, which uses the open measures of a workspace built with a geometry, and the cut-cell method with a `GeometryField` as fifth argument; `stable_timestep(velocity, workspace[, geometry]; cfl=0.9)`: the update in `advect!` (`src/operators/advection/cell_advection.jl`) and `advect!` (`src/embedded/operators.jl`), the bound in `stable_timestep` (`src/operators/advection/cell_advection.jl`). Tests: `test/operators/advection/cell_advection.jl` ("conservative centred advection") (constants and mass on uniform and composite meshes, Dirichlet inflow), `test/operators/advection/cell_advection.jl` ("centred transport is independent of base-root partitioning"), `test/operators/advection/cell_advection.jl` ("Cut-cell scalar CFL uses open face and cell measures") (values within $[0, 1]$ at the bound with a sloped solid, where the step of the uncut measures left the range).

— 7 —

Embedded solids and cut cells

Equation reference, audited against commit `8d58545`.

A fixed solid is embedded in the block-forest mesh without changing it: every cell carries the volume fraction of the solid and, if it is cut, one half-plane that approximates the solid inside it. This chapter states how the solid is represented and reconstructed, how the open polygons, the one-sided face traces, the shared apertures and the open staggered volumes are computed from the half-planes, how the velocity is closed on faces that the solid covers, and how the geometry is transferred when the mesh changes. The open measures themselves and the operators built on them are defined in Sections 3.3, 3.5 and 3.7; phase fractions in cut cells are those of Section 4.2. The thesis introduces the open parts of cells and faces [19, §2.4] and refers to the ComFLOW cut-cell method of Kleefsman et al. and Dröge and Verstappen [31] for the discretisation; the construction below is the package's. The geometry is fixed between regrids, so time discretisation is not applicable (Section 7.6).

7.1 Continuous setting

The solid occupies the fixed region

$$\Omega^{\mathrm{s}} = \{\, \boldsymbol{x} \in \Omega \mid q^{\mathrm{s}}(\boldsymbol{x}) \le 0 \,\}, \qquad \Omega^{\mathrm{o}} = \Omega \setminus \Omega^{\mathrm{s}}, \tag{7.1}$$

given by a level set q^{s} (negative inside the solid; it need not be a distance function). On the solid boundary the velocity satisfies the no-penetration condition $\boldsymbol{u} \cdot \boldsymbol{n} = 0$; the tangential condition (no slip or free slip) enters only the viscous stresses of Chapter 9, and the contact angle of an interface that meets the solid enters the curvature (Section 4.7). The divergence theorem on open cell parts (3.1) then involves only the open parts of the cell faces.

7.2 Solid reconstruction and open polygons

The solid is an `InterfaceField` without geometry whose "liquid" is the solid: its fractions α_c^{s} are initialised from q^{s} by the level-set quadrature of Section 4.9 and reconstructed by Sections 4.3 and 4.5 (default `:lhf`: LHF normals with the WY fallback, one-sided normals at domain walls). Its fraction floor (4.5) closes cells whose open fraction is at most ε^{α} and removes solid slivers of at most ε^{α}; no other small-cell treatment is applied. In a cut

cell, $0 < \alpha_c^{\mathrm{s}} < 1$, the solid occupies the half-plane

$$\hat{Q}_c^{\mathrm{s}} = \{\, \hat{\boldsymbol{x}} \in \hat{Q} \mid \hat{\boldsymbol{\eta}}_c^{\mathrm{s}} \cdot \hat{\boldsymbol{x}} \le \hat{\phi}_c^{\mathrm{s}} \,\}, \qquad \mathcal{A}(\hat{\boldsymbol{\eta}}_c^{\mathrm{s}}, \hat{\phi}_c^{\mathrm{s}}; \hat{Q}) = \alpha_c^{\mathrm{s}}, \tag{7.2}$$

so $\hat{\boldsymbol{\eta}}_c^{\mathrm{s}}$ points into the fluid. The open polygon and the open fraction are

$$\hat{Q}_c^{\mathrm{o}} = \begin{cases} \hat{Q}, & \alpha_c^{\mathrm{s}} = 0, \\ \{\, \hat{\boldsymbol{x}} \in \hat{Q} \mid \hat{\boldsymbol{\eta}}_c^{\mathrm{s}} \cdot \hat{\boldsymbol{x}} \ge \hat{\phi}_c^{\mathrm{s}} \,\}, & 0 < \alpha_c^{\mathrm{s}} < 1, \\ \emptyset, & \alpha_c^{\mathrm{s}} = 1, \end{cases} \qquad \alpha_c^{\mathrm{o}} = 1 - \alpha_c^{\mathrm{s}} = |\hat{Q}_c^{\mathrm{o}}|, \tag{7.3}$$

the last equality holding to the tolerance of the plane constant. Cells with $\alpha_c^{\mathrm{o}} > 0$ are active; covered cells ($\alpha_c^{\mathrm{o}} = 0$) are inactive in the operators of Chapter 3. The polygon is obtained by clipping $\hat{Q}$ with the half-plane (Sutherland–Hodgman [23]); the PLIC segments of the solid and its triangulated reconstruction are available for visualisation. Because each cut cell has its own half-plane, the reconstructed boundary is in general discontinuous across faces (Figure 7.1a).

Check (`figures/generate.jl`). On the cylinder fixture of Figure 7.1 the area of $\hat{Q}_c^{\mathrm{o}}$ equals α_c^{o} to 1.9×10^{-16} in all 20 cut cells.

> *Implementation.* `GeometryField(mesh, levelset; kwargs...)` (keywords of `InterfaceField`) or `GeometryField(solid)`; fields `solid`, `open_fraction` $= \alpha^{\mathrm{o}}$, `active_cells`; `refresh_geometry!`; open polygons `_open_cell_polygon` (`src/embedded/geometry.jl`) by `_clip_polygon` (`src/embedded/geometry_utils.jl`); `plic_segments`, `plic_solid_geometry` and `PLICSolidGeometry`. Tests: `test/indicator/geometry.jl` ("PLIC geometry converges and produces filled solid visualization") (solid PLIC segments converge with order above 1.49; triangulated area equals $\sum_c \alpha_c^{\mathrm{s}} |c|$), `test/indicator/geometry.jl` ("embedded geometry is independent of base-root partitioning") (open fractions and apertures), `test/two_phase/transport.jl` ("Global fraction floor") (floor in cut cells).

7.3 Face traces and shared apertures

Let $\hat{f}_c$ be the image of a face segment $f \in \mathcal{F}(c)$ in the local coordinates of c: $(\hat{\boldsymbol{x}})_k = \pm\frac{1}{2}$ on the side of c that contains f ($k = k(f)$, sign $+$ for $c = c^-(f)$), and $\hat{r}_t^- \le (\hat{\boldsymbol{x}})_t \le \hat{r}_t^+$, $t \ne k$, the tangential extent of f (a half side at a level transition, (2.7)). The one-sided open fraction of f according to the half-plane of c is

$$a_{c,f}^{\mathrm{o}} = \frac{|\hat{f}_c \cap \hat{Q}_c^{\mathrm{o}}|}{|\hat{f}_c|}, \tag{7.4}$$

one for an uncut and zero for a covered cell. On the segment the solid condition $\hat{\boldsymbol{\eta}}_c^{\mathrm{s}} \cdot \hat{\boldsymbol{x}} \le \hat{\phi}_c^{\mathrm{s}}$ is linear in $(\hat{\boldsymbol{x}})_t$, so its solid part is an interval, computed in closed form (the whole segment or nothing if $|(\hat{\boldsymbol{\eta}}_c^{\mathrm{s}})_t| \le 64\epsilon_{\mathrm{mach}}$). The shared aperture of an interior face is the average of the two traces, zero if either cell is covered (3.6); a boundary face carries the trace of its cell, $a_f^{\mathrm{o}} = a_{c(f),f}^{\mathrm{o}}$. One value per face is what makes the open divergence conservative and the cut-cell gradient its negative adjoint (Sections 3.5 and 3.7); the covered rule closes faces on

a grid-aligned solid boundary, where both cells are pure and the average would otherwise be $\frac{1}{2}$. The phase apertures of Section 4.8 split a_f^{o} into liquid and gas parts.

Checks (`figures/generate.jl`). On the cylinder fixture (112 interior faces, 25 of them partially open), an independent evaluation of (3.6) and (7.4) from the stored half-planes agrees with `geometry.aperture` to 1.1×10^{-16} and with the boundary apertures to 0.

Deviation from the thesis 7.1 (Open apertures). The thesis defines the open fraction and aperture as the sums of the phase quantities, $\alpha^{\mathrm{o}} = \{\!\{\alpha\}\!\}$ and $a^{\mathrm{o}} = \{\!\{a\}\!\}$ [19, §2.4], with each phase aperture the average of two one-sided traces [19, eq. (2.41)]. The code builds the open geometry from the solid alone: $\alpha_c^{\mathrm{o}} = 1 - \alpha_c^{\mathrm{s}}$, a_f^{o} the average of the two solid traces with the covered rule, and the phase apertures split it, $a_f^{\mathrm{g}} = a_f^{\mathrm{o}} - a_f^{\mathrm{l}}$. On a face without a covered cell, a_f^{o} equals the sum of the thesis' phase apertures, $\frac{1}{2}\sum_{c\in\mathcal{C}(f)}(a_{c,f}^{\mathrm{l}} + a_{c,f}^{\mathrm{g}}) = \frac{1}{2}\sum_{c\in\mathcal{C}(f)} a_{c,f}^{\mathrm{o}}$ (the clamp in (4.22) is then inactive up to round-off). If a cell of $\mathcal{C}(f)$ is covered, the thesis' sum is half of the open trace of the other cell (one half at a grid-aligned solid boundary), the code's is zero.

Implementation. `_solid_line_fraction` (`src/embedded/geometry.jl`), `_cell_solid_line_fraction` (`src/embedded/geometry.jl`), `_face_local_tangent_interval` (`src/embedded/geometry.jl`), `_cell_face_open_fraction` (`src/embedded/geometry.jl`) (7.4), `_interior_open_aperture` (`src/embedded/geometry.jl`) and `_boundary_open_aperture` (`src/embedded/geometry.jl`); stored as `aperture` (order of `mesh.faces`) and `boundary_aperture`. Tests: `test/indicator/geometry.jl` ("Grid-aligned solid faces are closed"), `test/indicator/geometry.jl` ("Geometry and phase PLIC transfer conserve physical volumes on AMR") (liquid aperture bounded by a_f^{o} through two regrids).

7.4 Open staggered volumes

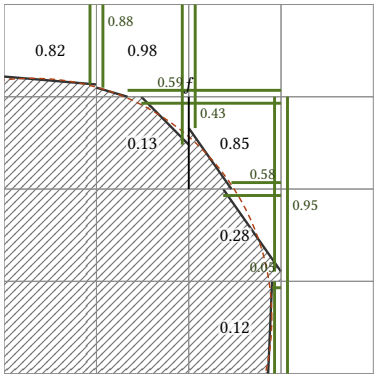
The open staggered volume (3.7) bisects every open line segment normal to the face [31]. With $\Lambda_{c,f}$ the strip of c over the full width of c normal to f and the tangential extent of f, whose local image is the rectangle $[-\frac{1}{2}, \frac{1}{2}]\times[\hat{r}_t^-, \hat{r}_t^+]$ (axes ordered k, t), each cell contributes half of the open area of its strip,

$$|\omega_f^{\mathrm{o}}| = \frac{1}{2}\sum_{c\in\mathcal{C}(f)} |c|\,\left(\hat{r}_t^+ - \hat{r}_t^-\right)\left(1 - \mathcal{A}\left(\hat{\boldsymbol{\eta}}_c^{\mathrm{s}}, \hat{\phi}_c^{\mathrm{s}}; [-\frac{1}{2}, \frac{1}{2}]\times[\hat{r}_t^-, \hat{r}_t^+]\right)\right), \tag{7.5}$$

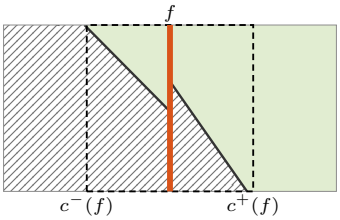
with $\mathcal{A} \equiv 0$ for uncut and the contribution zero for covered cells. A boundary face gets the half strip of its single cell; it is stored but is not an unknown. Without a solid $|\omega_f^{\mathrm{o}}| = |\omega_f|$, but in general $|\omega_f^{\mathrm{o}}| \neq a_f^{\mathrm{o}}|\omega_f|$: for the face of Figure 7.1b, $|\omega_f^{\mathrm{o}}| = 0.490$ and $a_f^{\mathrm{o}}|\omega_f| = 0.429$. The cut-cell elliptic operator of Section 3.7 rescales its metric by $(|\omega_f|/|\omega_f^{\mathrm{o}}|)^{1/2}$.

Check (`figures/generate.jl`). Evaluating (7.5) by clipping the strips independently reproduces `geometry.face_dual_volumes` on all faces of the cylinder fixture to 1.1×10^{-16}.

Implementation. `_open_subcell_area` (`src/embedded/geometry.jl`), `_geometry_dual_volumes` (`src/embedded/geometry.jl`) and `_geometry_boundary_dual_volumes` (`src/embedded/geometry.jl`); stored



A. Solid half-planes (hatched) and open fractions α_c^{o}; green: open part of each face seen from either side, number: a_f^{o}.



B. Open staggered volume of f: half of the open parts (green) of the two strips $\Lambda_{c,f}$; dashed: ω_f.

Figure 7.1. Cut-cell geometry on the cylinder fixture of `figures/generate.jl` (solid disc of radius 2.6 in 8×8 unit cells; window $[4, 8] \times [3, 7]$; dashed red: the exact boundary). Each cut cell carries its own solid half-plane (7.2), so the two traces (7.4) of a face differ; the shared aperture (3.6) averages them and is zero next to a covered cell. (b) The face f marked in (a): the open staggered volume (3.7) is not $a_f^{\mathrm{o}}|\omega_f|$.

per axis in `dual_volumes` (operator order), `face_dual_volumes` (order of `mesh.faces`) and `boundary_dual_volumes`.

7.5 Velocity closure and operators on cut cells

Faces that the solid closes carry the stationary-wall value,

$$u_f \leftarrow 0 \quad \text{for every } f \in \mathcal{F} \text{ with } a_f^{\mathrm{o}} = 0, \tag{7.6}$$

applied wherever the time step enforces the velocity boundary conditions (Chapter 1) and before and after the cut-cell projection (for the two phase velocities, values marked absent are kept). On a partially open face u_f is the velocity of the open part: the solid part of ∂c^{o} carries no flux in the open divergence (3.13), which is the discrete form of $\boldsymbol{u} \cdot \boldsymbol{n} = 0$ there. The cut-cell operators use only the quantities of this chapter: the open divergence and pressure gradient (Sections 3.5 and 3.7), the upwind transport of a cell scalar with fluxes $\delta t\,|f| a_f^{\mathrm{o}} u_f$ and updates divided by $|c^{\mathrm{o}}|$ (Chapter 6), the geometric VOF fluxes of Chapter 5 and the wall stresses of Chapter 9. Open cells remain active however small they are; there is no merging or flux redistribution; the geometric transport bounds its time step with the open volumes (Chapters 5 and 13).

Implementation. `enforce_geometry_no_slip!(velocity, geometry; preserve_absent=false)` (7.6), `divergence!` and `advect!` with a `GeometryField` (`divergence!` (`src/embedded/operators.jl`), `advect!` (`src/embedded/operators.jl`)); calls in the time step `_enforce_current_velocity_boundary!` (`src/single_phase/stepping.jl`) and `_enforce_phase_velocity_boundary!` (`src/two_phase/two_velocity/stepping.jl`), and in the cut-cell projection `project!` (`project!` (`src/embedded/operators.jl`)); open-volume CFL bound of the transport

| `stable_timestep` (5.17).

7.6 Regrid and rebuild

Time discretisation is not applicable: the geometry depends on the mesh only. When the leaf set changes, the solid fractions are transferred geometrically. A child c_r, $r = 1, \dots, 4$, of a refined cut cell c receives the solid area of the parent's half-plane over its quadrant $\hat{Q}_r \subset \hat{Q}$ of the parent's local coordinates,

$$\alpha^{\mathrm{s}}_{c_r} = 4\,|\hat{Q}_r \cap \hat{Q}^{\mathrm{s}}_c|, \qquad \alpha^{\mathrm{s}}_c = \tfrac{1}{4}\sum_{r=1}^{4} \alpha^{\mathrm{s}}_{c_r} \ \text{(coarsening)}, \tag{7.7}$$

which conserves the solid volume exactly up to rounding. Then the solid is reconstructed and every quantity of Sections 7.2 to 7.4 is recomputed; a revision counter makes the phase interface reconstruct its cut cells again. The level set q^{s} is not evaluated again, so refining near the solid does not make its representation more accurate than the half-planes from which the children inherit their fractions; construct the `GeometryField` on a mesh that already resolves the solid.

Deviation from the thesis 7.2 (ComFLOW boundary features). The thesis' ComFLOW-specific free-surface and wave-generation boundary conditions are not implemented.

Implementation. `GeometryProlongation` (`src/embedded/geometry.jl`) (the interface prolongation applied to the solid, then `refresh_geometry!`) and `register_adaptive_field!`; the child clipping is the call of `InterfaceProlongation` (`src/interface/interface_field.jl`), the coarsening average `_restrict_parent!` (`src/interface/interface_field.jl`). Tests: `test/indicator/geometry.jl` ("Geometry and phase PLIC transfer conserve physical volumes on AMR") (solid and phase volumes conserved to 5×10^{-14} through refinement and coarsening), `test/indicator/geometry.jl` ("Geometry hierarchy is rebuilt consistently after AMR").

— 8 —

Forces and material properties

Equation reference, audited against commit 497a401.

Besides pressure, convection and viscosity, three forces act on the velocity: a prescribed body acceleration, surface tension and gravity. The body acceleration is added to the transported velocity before the viscous step; surface tension and gravity are added after it, as explicit increments of the predictor that the projection of Chapter 10 then makes solenoidal, the order of the thesis [19, §7.2, Remark 7.1]. Both phase forces are written as integrated face jumps that pass through the same weighted Riesz map as the pressure gradient of Section 3.7: a force that is the jump of a cell function is therefore exactly the gradient of a pressure, and the projection removes it (balanced force). This chapter states the continuous model and its sign conventions, the material properties on the mesh, the force map and the exact conditions under which it is balanced, the face curvature with its interpolation, extension and treatment of unresolved faces, the CSF and CGFM surface-tension jumps, the gravity model, the body acceleration and the two-velocity variant (the semi-discrete forces), and finally the stage of the time step at which each force acts and the time-step limits (the fully discrete update). The cell curvature is that of Chapter 4, the viscous stress is in Chapter 9. Derivations are in the thesis [19, §§4.3, 4.5, 4.7.1 and 7.6].

8.1 Continuous model

In each phase the momentum balance with the Newtonian stress is [19, eqs. (2.1b), (2.7)]

$$\rho^\pi \left(\partial_t \boldsymbol{u}^\pi + \boldsymbol{u}^\pi \cdot \nabla \boldsymbol{u}^\pi\right) = \nabla \cdot \mathbf{T}^\pi + \rho^\pi \boldsymbol{g} + \rho^\pi \boldsymbol{a}^{\mathrm{b}}, \qquad \mathbf{T}^\pi = -p^\pi \mathsf{I} + \mu^\pi \left(\nabla \boldsymbol{u}^\pi + (\nabla \boldsymbol{u}^\pi)^{\mathsf{T}}\right) \tag{8.1}$$

in $\Omega^\pi(t)$, where the prescribed body acceleration $\boldsymbol{a}^{\mathrm{b}}(\boldsymbol{x}, t)$ is an addition of the code. At the interface the stresses balance the surface tension [19, eq. (2.6)],

$$\boldsymbol{\eta} \cdot [\![\mathbf{T}]\!] = \sigma\kappa\boldsymbol{\eta}, \qquad \text{without viscosity} \qquad [\![p]\!] = -\sigma\kappa \tag{8.2}$$

(Young–Laplace [19, eq. (4.5)]), with $[\![\cdot]\!]$ the gas minus the liquid value, $\boldsymbol{\eta}$ pointing into the gas and $\kappa = \nabla \cdot \boldsymbol{\eta}$ (4.1): a convex drop, $\kappa > 0$, has the higher pressure inside, $p^{\mathrm{l}} - p^{\mathrm{g}} = \sigma\kappa$. The one-velocity model sums (8.1) over the phases with ρ and μ piecewise constant [19, eq. (2.9)]; surface tension then is the singular force $\sigma\kappa\nabla\chi^{\mathrm{l}} = -\sigma\kappa\boldsymbol{\eta}\delta^I$, the sharp limit of the continuum force $\sigma\kappa\nabla\alpha^\epsilon$ of Brackbill et al. [4] [19, eqs. (4.6), (4.8)], and gravity the force

$\rho\boldsymbol{g}$ [19, eq. (7.53)], with $\chi^{\mathrm{l}}(\boldsymbol{x},t)$ the liquid indicator and δ^I the interface delta function. In each phase gravity is a gradient, $\rho^\pi\boldsymbol{g}=\nabla(\rho^\pi\boldsymbol{g}\cdot\boldsymbol{x})$. The discretisation is built to keep two steady states:

$$\text{(a) } \boldsymbol{u}=\boldsymbol{0},\ \kappa=\kappa_0,\ p=p_0+\sigma\kappa_0\chi^{\mathrm{l}}; \qquad \text{(b) } \boldsymbol{u}=\boldsymbol{0},\ p=\rho^\pi\boldsymbol{g}\cdot\left(\boldsymbol{x}-\boldsymbol{x}^I\right) \text{ in } \Omega^\pi, \tag{8.3}$$

a drop of constant curvature without gravity, and a planar interface normal to $\boldsymbol{g}$ through $\boldsymbol{x}^I$, whose pressure is continuous [19, eq. (7.55)].

The material constants are validated when the model is built: finite positive densities $(\rho^{\mathrm{l}},\rho^{\mathrm{g}})$; dynamic viscosities $(\mu^{\mathrm{l}},\mu^{\mathrm{g}})$ that are finite and either both positive or both zero (a scalar sets both); $\sigma\geq 0$; and a finite $\boldsymbol{g}$ whose component along every periodic direction vanishes, because the potential $\boldsymbol{g}\cdot\boldsymbol{x}$ is not periodic. A single phase has one finite positive density and a positive viscosity.

Implementation. `TwoPhase(interface; density, dynamic_viscosity, surface_tension=SurfaceTension(), gravity=(0,0))` with `SurfaceTension(coefficient; method=CGFM())` (a number σ is shorthand for the CGFM default), `SinglePhase`; the checks are in `TwoPhase` (`src/two_phase/physics.jl`), `SurfaceTension` (`src/two_phase/physics.jl`) and `TwoPhaseModel` (`src/two_phase/model.jl`) (periodic gravity). Test: `test/two_phase/forces.jl` ("Conservative surface-tension method") (the method types and their rejection of invalid arguments).

8.2 Material properties on the mesh

The cell quantities are the conditional fractions C^{l}_c of (4.4). On every interior face the staggered liquid fraction and the mixture density are

$$\widetilde{\rho}_f=\rho^{\mathrm{l}}\,\hat{C}_f+\rho^{\mathrm{g}}\left(1-\hat{C}_f\right), \qquad \hat{C}_f=\operatorname{clamp}\left(\widetilde{C}^{\mathrm{l}}_f,0,1\right), \quad \widetilde{C}^{\mathrm{l}}_f=(\mathfrak{H}C^{\mathrm{l}})_f, \tag{8.4}$$

with the volume-weighted interpolation $\mathfrak{H}$ of (6.4): without solid $\widetilde{C}^{\mathrm{l}}_f=(h_{c^-(f),k}C^{\mathrm{l}}_{c^-(f)}+h_{c^+(f),k}C^{\mathrm{l}}_{c^+(f)})/(h_{c^-(f),k}+h_{c^+(f),k})$, $k=k(f)$; with a solid the weights are the open parts $|\Lambda_{c,f}\cap c^{\mathrm{o}}|$ of the two strips, and $\hat{C}_f=0$ where both vanish. This is the density of the one-velocity momentum (6.25), of the viscous mass matrix (Chapter 9) and of the pressure coefficient

$$\gamma_f=\begin{cases}1/\widetilde{\rho}_f, & \text{no solid},\\ (a^{\mathrm{o}}_f)^2/\widetilde{\rho}_f, & \text{solid},\end{cases} \qquad \gamma^{\mathrm{tv}}_f=\begin{cases}1/\widetilde{\rho}_f, & \text{no solid},\\ a^{\mathrm{o}}_f/\widetilde{\rho}_f, & \text{solid},\end{cases} \tag{8.5}$$

where γ is the one-velocity coefficient of the elliptic operator of Section 3.7 (material part $\gamma^{\mathrm{m}}_f=1/\widetilde{\rho}_f$ of the cut-cell coefficient) and γ^{tv} the common pressure coefficient of the two-velocity projections, which minimise the correction in the kinetic norm of the open staggered volumes (Chapter 10). With a solid the two coefficients differ by the factor a^{o}_f on every cut face, the two force maps by the congruence of Section 8.8. A single phase of density ρ has $\gamma_f=1/\rho$ (times $(a^{\mathrm{o}}_f)^2$ with a solid), so that the pressure carries physical units. The two-phase coefficient is rebuilt from the transported fractions once per step, before the force stage (Section 8.9); only the coefficient changes, overwritten in place in the

operator of the step, and the metric K is kept. The cell viscosity is the geometric mean $\mu_c = (\mu^{\mathrm{l}})^{C_c^{\mathrm{l}}}(\mu^{\mathrm{g}})^{1-C_c^{\mathrm{l}}}$ ((9.11)).

Implementation. `mixture_density(interface, density[, geometry])` (8.4) via `_staggered_liquid_fraction` (`src/two_phase/forces.jl`) and `_mixture_density!` (`src/two_phase/forces.jl`); `PoissonOperator(interface[, geometry]; density)` (8.5) in `PoissonOperator` (`src/two_phase/forces.jl`); the two-velocity coefficient in `_projection_pressure_operator` (`src/two_phase/projection/common.jl`); refresh `_refresh_pressure_substep!` (`src/two_phase/stepping.jl`). Test: `test/two_phase/forces.jl` ("Variable-density balanced CSF and CGFM projection") ($\gamma = 1/\widetilde{\rho}$ bitwise), `test/indicator/geometry.jl` ("Cut-cell AMR forces retain balanced pressure cancellation") ($\gamma = (a^{\mathrm{o}})^2/\widetilde{\rho}$ bitwise).

8.3 Semi-discrete forces: the force map

Both phase forces are given by integrated jumps $\mathcal{J}_f$ on the interior faces, the counterpart of $(\mathsf{J}p)_f = |f|(p_{c^+(f)} - p_{c^-(f)})$ of (3.16), and act through the force map

$$(\mathsf{R}\mathcal{J})_f = \begin{cases} \left(\mathsf{G}^{1/2}\mathsf{K}\,\mathsf{G}^{1/2}\mathcal{J}\right)_f, & \text{no solid,} \\ \dfrac{\gamma_f^{1/2}}{a_f^{\mathrm{o}}}\left(\mathsf{W}\mathsf{K}\mathsf{W}\,\mathsf{G}^{1/2}\mathcal{J}\right)_f \text{ if } a_f^{\mathrm{o}} > 0, \quad 0 \text{ if } a_f^{\mathrm{o}} = 0, & \text{solid,} \end{cases} \qquad f \in \mathcal{F}_\Omega, \tag{8.6}$$

with $\mathsf{G} = \mathrm{diag}\,(\gamma)$, the one-velocity coefficient (8.5), the inverse face metric K of (3.19) and its cut-cell scaling W; boundary faces get zero. It is exactly the map of the pressure gradient,

$$\mathfrak{S}_\gamma p = \mathsf{R}(\mathsf{J}p) \quad ((3.24)), \qquad \mathfrak{S}^{\mathrm{o}} p = \mathsf{R}(\mathsf{J}p) \quad ((3.27)), \tag{8.7}$$

evaluated by the same products with the same operator. On a conforming patch, i.e. at the faces whose row of K has the single entry $1/|\omega_f|$,

$$(\mathsf{R}\mathcal{J})_f = \frac{\mathcal{J}_f}{\widetilde{\rho}_f\,|\omega_f|} \quad \text{(no solid)}, \qquad (\mathsf{R}\mathcal{J})_f = \frac{a_f^{\mathrm{o}}\mathcal{J}_f}{\widetilde{\rho}_f\,|\omega_f^{\mathrm{o}}|} \quad (\text{solid}, a_f^{\mathrm{o}} > 0): \tag{8.8}$$

an integrated force per staggered mass. At a level transition K couples the segments of a vertex patch, so a face without a jump can carry an acceleration (Figure 8.1); on the composite drop fixture below these are the 78 of 1240 faces within one coarse cell of the transition (derivation audit, `proof_review/eq-forces-map/`).

Balanced force. Let the jumps be those of a cell function φ:

$$\mathcal{J}_f = (\mathsf{J}\varphi)_f \quad \text{for every } f \text{ with } \gamma_f > 0 \quad \Longleftrightarrow \quad \mathsf{R}\mathcal{J} = \mathfrak{S}_\gamma\varphi \ \ (\text{or } \mathfrak{S}^{\mathrm{o}}\varphi). \tag{8.9}$$

"$\Rightarrow$" holds since faces with $\gamma_f = 0$ do not enter (8.6); "$\Leftarrow$" since K (and WKW) is symmetric positive definite and the factors $\gamma_f^{1/2}$ (and $\gamma_f^{1/2}/a_f^{\mathrm{o}}$) are positive on the faces with $\gamma_f > 0$, so that R is injective on them. Consequently, if the velocity u before the force is discretely solenoidal ($\mathfrak{D}u = 0$, or $\mathfrak{D}^{\mathrm{o}}u = 0$ with a solid) and satisfies the boundary conditions, the one-velocity projection of $u + \delta t\mathsf{R}\mathcal{J}$ (Chapter 10) returns u and the pressure φ, up to a function that is constant on each connected component of the graph whose edges

are the faces with $\gamma_f > 0$ (a constant without a solid): $p = \varphi$ makes the corrected velocity u, which is solenoidal, and the projected velocity is unique. The statement is exact up to the tolerance of the linear solve. It needs both conditions: the jumps must be those of one cell function on every open face, which by "⇐" is also necessary, and the projection must use the operator of (8.6), which the one-velocity step guarantees by refreshing it once and using it for both (Section 8.9). A projection with another coefficient leaves a velocity of the size of the force: 0.32 of $\delta t \max_f |(\mathsf{R}\mathcal{J})_f|$ with the coefficient of a displaced drop, 1.45 with the two-velocity coefficient γ^{tv} on the cut mesh (derivation audit, `proof_review/eq-forces-balance/`). No balance is claimed for a curvature that varies from face to face, for a face with unresolved curvature (Section 8.4), and for gravity where the hydrostatic condition (8.18) fails (Section 8.6).

Checks (`figures/forces.jl`, `surface_tension_check`). With $\mathcal{J}$ the independently evaluated jumps of Section 8.5, K (and W) read from the code and an independent $\widetilde{\rho}$ (8.4), (8.6) reproduces the code's acceleration to 1.3×10^{-16} (relative) on the uniform, composite and cut drop fixtures (walled box of 16×16 level-zero cells; the composite mesh refines the right half, the cut one adds a sloped solid below the drop).

> *Implementation.* Internal: `_weighted_jump_force!` (`src/two_phase/forces.jl`) (8.6), called by `surface_tension!` and `gravity!` with the pressure operator of the step; it writes the jumps into the operator's scratch `gradient_operator.pressure_jump`, which `weighted_gradient!` and `_cut_cell_weighted_gradient!` (`src/embedded/operators.jl`) also use. Tests: `test/two_phase/forces.jl` ("Variable-density balanced CSF and CGFM projection") (a constant-curvature force equals `weighted_gradient!` of $\sigma\kappa_0\chi^\sigma$ to 5×10^{-13} and is projected to below 10^{-9} on uniform, composite and adapted meshes, density ratios 1 and 1000), `test/indicator/geometry.jl` ("Cut-cell AMR forces retain balanced pressure cancellation") (the same on a cut composite mesh, below 10^{-10}).

8.4 Face curvature

Surface tension needs a curvature on every face with a phase jump. The cell curvature κ_c of Chapter 4 is the LHF or GHF curvature, at domain walls the contact-angle formulas of Section 4.7, and in a triple cell the GHF fit with the contact row (4.20); it is clipped by (4.17) and undefined (NaN) in pure cells and where no estimate exists. With the weight $w_c^\kappa = C_c^{\mathrm{l}}(1 - C_c^{\mathrm{l}})$, the face curvature is the weighted average over the two cells of the face,

$$\widetilde{\kappa}_f = \frac{\sum_c w_c^\kappa \kappa_c}{\sum_c w_c^\kappa}, \qquad c \in \mathcal{C}(f),\ w_c^\kappa > 0,\ \kappa_c \text{ finite}, \tag{8.10}$$

the thesis' interpolant $\mathfrak{I}_\kappa$ [19, §4.7.1, "Interpolation"], if this set is not empty. Otherwise the curvature is extended from the cells around the face: with $r_f = 3 \max_{c \in \mathcal{C}(f)} \bar{h}_c$, $\bar{h}_c = \max_k h_{c,k}$, and the distance $|\boldsymbol{x}_{c'} - \boldsymbol{x}_f|_2$ to the nearest periodic image,

$$\widetilde{\kappa}_f = \frac{\sum_{c'} \tilde{w}_{c'} \kappa_{c'}}{\sum_{c'} \tilde{w}_{c'}}, \qquad \tilde{w}_{c'} = \frac{w_{c'}^\kappa}{\max\left(|\boldsymbol{x}_{c'} - \boldsymbol{x}_f|_2^2, (\bar{h}_{c'}/4)^2\right)}, \tag{8.11}$$

$$\text{over the cells } c' \text{ with } |\boldsymbol{x}_{c'} - \boldsymbol{x}_f|_2 \le r_f + \bar{h}_{c'},\ w_{c'}^\kappa > 0,\ \kappa_{c'} \text{ finite},$$

summed over all leaf cells in block and cell order (the code first discards blocks whose bounding box lies beyond $r_f + \bar{h}_{c'}$, which changes nothing). Both rules are convex combinations of cell curvatures. The reach is a hard cut, not a decaying weight: a cell curvature just beyond $r_f + \bar{h}_{c'}$ does not count at all, so the face curvature, and whether it exists, change discontinuously with the cell data (in the derivation audit, `proof_review/eq-forces-curvature/`, a CSF jump face whose nearest cell curvature lies at the distance 1.01 for the reach 1.00 is unresolved). The extension serves the faces between two pure cells of different phases, which occur at cut-cell conditional fractions and when the interface lies on a grid line, and the faces whose mixed cells have no curvature. If it is empty too, the face is *unresolved*:

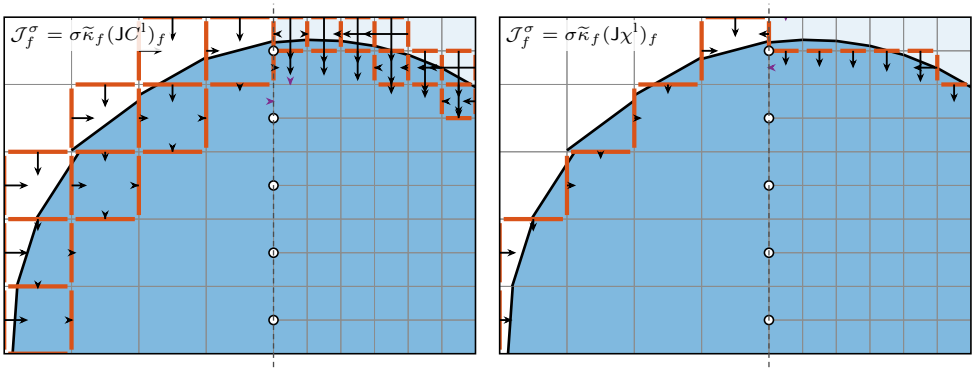
$$\mathcal{F}_\emptyset^\kappa = \{\, f \in \mathcal{F}_\Omega \mid \sigma > 0,\ (\mathrm{J}\chi^\sigma)_f \neq 0,\ a_f^\mathrm{o} > 0,\ \text{no curvature by (8.10) and (8.11)} \,\},$$
$$\mathcal{J}_f^\sigma = 0 \quad (f \in \mathcal{F}_\emptyset^\kappa), \tag{8.12}$$

with the jump of Section 8.5: such a face gets zero curvature, hence no tension, and is counted. The field-level `step!` sets `workspace.unresolved_curvature_faces[]` to zero at its start and to $\#\mathcal{F}_\emptyset^\kappa$ when it evaluates the tension. Each full `step!(flow, dt)` adds this count to `flow.unresolved_curvature_faces`; `simulate!` returns the sum over its completed steps as `unresolved_curvature_faces` and the flow's cumulative count as `total_unresolved_curvature_faces`. `print_simulation_summary` prints both when they are positive. Until issue 58 (fixed by PR #65) the force raised an error instead. With the keyword `curvature=` κ_0 (for tests) every face has $\widetilde{\kappa}_f = \kappa_0$ and none is unresolved. The embedded contact angle enters the tension only through the curvature of the triple cells, the domain-wall angle through the wall formulas, both via (8.10) and (8.11).

Checks (`face_curvature_check`, `unresolved_check`). An independent evaluation of (8.10) and (8.11) over all leaf cells reproduces the code on all 2960 faces of the three drop fixtures to 0 (relative), also on the faces without a jump, where the code never asks for a curvature: there 1508 faces take the extension and 1009 have none, exactly where the code returns none. All 168 faces of the composite fixture whose conditional fractions differ take (8.10): a resolved drop needs neither the extension nor the zero rule. On the thin film of the test `test/two_phase/forces.jl` ("Unresolved surface-tension curvature"), whose mixed cells have no curvature, 5 faces with a CGFM jump are unresolved; their jumps are zero and the count equals the code's.

Deviation from the thesis 8.1 (Face curvature). The thesis interpolates the cell curvature with the weights $\alpha_c^\mathrm{l}\alpha_c^\mathrm{g}$ over the cells of the face [19, §4.7.1]. The code uses the conditional fractions (the same in uncut cells), extends the curvature from the cells within $r_f + \bar{h}_{c'}$ with inverse-square distance weights (8.11) when neither cell of the face has one, and gives the remaining faces zero curvature while counting them (8.12).

Implementation. `_face_curvature` (`src/two_phase/forces.jl`) (8.10) and (8.11); the count in `surface_tension!` (`src/two_phase/forces.jl`) (keyword `unresolved_curvature_faces`, a `Ref\{Int\}`), reset in `step!` (`src/single_phase/stepping.jl`) and `step!` (`src/two_phase/two_velocity/stepping.jl`), accumulated in `step!` (`src/navier_stokes/flow_simulation.jl`), summed per call in `_simulate!` (`src/navier_stokes/simulate.jl`), printed by



A. CSF B. CGFM

Figure 8.1. Surface-tension jumps and accelerations on the composite drop fixture of Section 8.5 (level transition dashed, hanging vertices open circles; liquid shaded). Red: faces with $\mathcal{J}_f^\sigma \neq 0$ (8.13), 168 for CSF and 57 for CGFM over the whole mesh; arrows: the face acceleration $(\mathsf{R}\mathcal{J}^\sigma)_f$ (8.6), scaled by its maximum. CSF spreads the jump over the faces between cells with different conditional fractions, CGFM puts it on the faces where the indicator changes. Purple: faces without a jump whose acceleration is not zero because the metric K couples the segments of a vertex patch at the level transition (7 for CSF, 5 for CGFM); the pressure gradient couples them in the same way.

`print_simulation_summary` (`src/navier_stokes/flow_diagnostics.jl`). Tests: `test/two_phase/forces.jl` ("Unresolved surface-tension curvature") (zero jumps and force on the unresolved faces, per-step, per-run and total counts, summary line, no unresolved face on a resolved drop), `test/two_phase/forces.jl` ("Curvature extension agrees with exhaustive search") (the block prefilter, periodic and walled composite meshes).

8.5 Surface tension

With the phase function $\chi^\sigma = C^{\mathrm{l}}$ for CSF and $\chi^\sigma = \chi^{\mathrm{l}}$, $\chi_c^{\mathrm{l}} = 1$ iff $C_c^{\mathrm{l}} > \frac{1}{2}$ (4.6), for CGFM (the default), the integrated surface-tension jumps are

$$\mathcal{J}_f^\sigma = \sigma\,\widetilde{\kappa}_f\,(\mathsf{J}\chi^\sigma)_f = \sigma\,\widetilde{\kappa}_f\,|f|\left(\chi^\sigma_{c^+(f)} - \chi^\sigma_{c^-(f)}\right), \qquad f \in \mathcal{F}_\Omega, \tag{8.13}$$

and $\mathcal{J}_f^\sigma = 0$ if $\sigma = 0$, $(\mathsf{J}\chi^\sigma)_f = 0$, $a_f^{\mathrm{o}} = 0$ or $f \in \mathcal{F}_\emptyset^\kappa$. The force is the acceleration $\mathsf{R}\mathcal{J}^\sigma$ of (8.6), added at the stage of Section 8.9. At a face whose low cell is liquid and high cell gas, $\mathcal{J}_f^\sigma = -\sigma\widetilde{\kappa}_f|f|$: for a convex drop the force points into the liquid. On a conforming patch without solid (8.8) gives $(\mathsf{R}\mathcal{J}^\sigma)_f = \sigma\widetilde{\kappa}_f(\mathfrak{S}_\gamma\chi^\sigma)_f$, the thesis' CSF force $S^{\mathrm{CSF}} = \sigma\widetilde{\kappa}\mathfrak{S}\alpha$ [19, eq. (4.29)] and CGFM force $S^{\mathrm{CGFM}} = \sigma\widetilde{\kappa}\mathfrak{S}\chi^{\mathrm{l}}$ [19, eq. (4.55)] divided by the staggered mass $\widetilde{\rho}$. CSF spreads the jump over the faces between cells of different fractions, CGFM puts all of it on the faces where the indicator changes (Figure 8.1). At a level transition the curvature multiplies the jump before K couples the faces of a vertex patch, so $(\mathsf{R}\mathcal{J}^\sigma)_f$ equals $\sigma\widetilde{\kappa}_f(\mathfrak{S}_\gamma\chi^\sigma)_f$ only at the faces whose row of K has a single entry or where $\widetilde{\kappa}$ is constant over the faces that this row couples (at the 78 coupled faces of the composite drop fixture the two differ by 2–6 % of the largest value; derivation audit, `proof_review/eq-forces-st-balance/`).

Balance and pressure jump. If $\widetilde{\kappa}_f = \kappa_0$ on every face with $(\mathsf{J}\chi^\sigma)_f \neq 0$ and $a_f^{\mathrm{o}} > 0$, and none of them is unresolved, then $\mathcal{J}^\sigma = \mathsf{J}(\sigma\kappa_0\chi^\sigma)$ and (8.9) applies:

$$\mathsf{R}\mathcal{J}^\sigma = \mathfrak{S}_\gamma\,(\sigma\kappa_0\chi^\sigma) \quad (\text{or } \mathfrak{S}^{\mathrm{o}}), \qquad p = \sigma\kappa_0\chi^\sigma + \text{const}, \tag{8.14}$$

so the one-velocity projection leaves a solenoidal predictor unchanged on uniform, composite and cut meshes; for CGFM the pressure jumps by $\sigma\kappa_0$ across the indicator, the discrete Young–Laplace condition (8.2) of the static drop (8.3)(a), for CSF it follows the conditional fraction. The pressure solve itself imposes no value jump: the Young–Laplace jump enters only through the explicit increment and is carried by the pressure that balances it. The curvature of a discrete circle varies from face to face, so the computed force of a drop is not exactly balanced; the remaining (parasitic) velocity is a property of the curvature estimate, not of the map. In the drop fixtures one projection of δt times the force with the computed curvature leaves 0.8–$3\,\%$ of $\delta t \max_f |(\mathsf{R}\mathcal{J}^\sigma)_f|$ on the uniform and composite meshes, and $6\,\%$ (CGFM) and $15\,\%$ (CSF) on the cut mesh, although the solid does not touch the drop; the Laplace pressure of the projected CGFM force is $2\,\%$ (uniform) and $1\,\%$ (composite) above $\sigma\kappa_0$ for the curvature κ_0 of the circle (derivation audits, `proof_review/eq-forces-st-balance/` and `proof_review/eq-forces-continuous/`).

Checks (`surface_tension_check`). On the three drop fixtures ($\sigma = 0.7$, densities $(1000, 1)$), an independent evaluation of (8.13) with the independent face curvature equals the code's jumps to 0 for both formulations; with a prescribed constant curvature the force equals the weighted gradient of $\sigma\kappa_0\chi^\sigma$ (8.14) to 2.7×10^{-16} (relative), and one projection of δt times it leaves at most 1.3×10^{-6} of $\delta t \max_f |(\mathsf{R}\mathcal{J}^\sigma)_f|$ at the projection tolerance 10^{-12}.

Deviation from the thesis 8.2 (Surface tension as a predictor force). The thesis imposes the Young–Laplace jump inside the scaled pressure gradient of the projection, $\bar{\mathfrak{g}}(p) = (\mathfrak{S}p - \sigma\widetilde{\kappa}\mathfrak{S}\chi^{\mathrm{l}})/\left\{\!\!\left\{\widetilde{\phi}\rho\right\}\!\!\right\}$ [19, eq. (4.53)], in the last step T5 of the time step [19, §7.2]. The code adds $\delta t\mathsf{R}\mathcal{J}^\sigma$ to the predictor before the projection. For the one-velocity model and the conservative distance $\widetilde{\phi} = \widetilde{\alpha}$ [19, eq. (4.54)] both give the same velocity where K is diagonal; at level transitions the code multiplies the jump by $\widetilde{\kappa}_f$ before applying K.

Deviation from the thesis 8.3 (Surface-tension models). The thesis also derives the mimetic continuum model (MCSF, [19, §4.4]), which the code does not implement, the GFM with a geometric, nonconservative distance $\widetilde{\phi} \neq \widetilde{\alpha}$ [19, §4.5.2] and the sharp surface-tension model based on the first variation of the area [19, §4.6]. The code implements CSF and CGFM only.

Implementation. `surface_tension!(acceleration, interface, pressure_operator, workspace; sigma, formulation=CGFM(), curvature=nothing, unresolved_curvature_faces=nothing)` overwrites `acceleration` with $\mathsf{R}\mathcal{J}^\sigma$ (boundary values zero); `CSF`, `CGFM`, `AbstractSurfaceTensionMethod`. Jump `_surface_tension_jump` (`src/two_phase/forces.jl`) (8.13). Tests: `test/two_phase/forces.jl` ("Variable-density balanced CSF and CGFM projection") (jumps bitwise, CGFM is the default), `test/two_phase/forces.jl` ("Surface-tension strength is linear in sigma"), `test/indicator/geometry.jl` ("Cut-cell AMR forces retain

| balanced pressure cancellation").

8.6 Gravity

Gravity is the gravity model of the thesis [19, §7.6.2]: an aperture density times the gradient of the potential at the interface centroids. The cell potential is

$$\Phi_c = \boldsymbol{g}\cdot\boldsymbol{x}_c^I, \qquad \boldsymbol{x}_c^I = \begin{cases} \boldsymbol{x}_c + \mathsf{H}_c\,\hat{\boldsymbol{m}}_c, & 0 < C_c^{\mathrm{l}} < 1, \\ \boldsymbol{x}_c, & \text{otherwise}, \end{cases} \tag{8.15}$$

with $\hat{\boldsymbol{m}}_c$ the midpoint of the segment $\{\,\hat{\boldsymbol{x}}\in\hat{Q} \mid \hat{\boldsymbol{\eta}}_c\cdot\hat{\boldsymbol{x}} = \hat{\phi}_c\,\}$ of the PLIC line in the whole reference cell, also in a triple cell, where the segment is not clipped by the solid; pure cells use the cell centre, also when cut. The aperture density is the phase mixture over the open part of the face,

$$\rho_f^a = \frac{\rho^{\mathrm{l}}a_f^{\mathrm{l}} + \rho^{\mathrm{g}}a_f^{\mathrm{g}}}{a_f^{\mathrm{o}}} \quad (a_f^{\mathrm{o}} > 0), \tag{8.16}$$

and the integrated jumps are

$$\mathcal{J}_f^{\boldsymbol{g}} = \rho_f^a\,(\mathsf{J}\Phi)_f = \rho_f^a\,|f|\,\big(\Phi_{c^+(f)} - \Phi_{c^-(f)}\big) \quad (a_f^{\mathrm{o}} > 0), \qquad \mathcal{J}_f^{\boldsymbol{g}} = 0 \quad (a_f^{\mathrm{o}} = 0), \tag{8.17}$$

with the phase apertures $a_f^{\mathrm{l}} + a_f^{\mathrm{g}} = a_f^{\mathrm{o}}$ of (4.22) ($a_f^{\mathrm{o}} = 1$ without solid), and the force is $\mathsf{R}\mathcal{J}^{\boldsymbol{g}}$. A gravity component along a periodic direction is rejected. On a face in one phase ρ_f^a is that phase's density, also when the face is cut, and on every face it lies between the two densities; without solid $\rho_f^a = \rho^{\mathrm{l}}a_f^{\mathrm{l}} + \rho^{\mathrm{g}}a_f^{\mathrm{g}}$ and on a conforming patch (8.8) gives $(\mathsf{R}\mathcal{J}^{\boldsymbol{g}})_f = F_f/\widetilde{\rho}_f$ with the thesis' force $F = (\rho^{\mathrm{l}}a^{\mathrm{l}} + \rho^{\mathrm{g}}a^{\mathrm{g}})\mathfrak{S}(\boldsymbol{g}\cdot\boldsymbol{x}^I)$ [19, eq. (7.64)]; in a pure phase it is $\boldsymbol{g}\cdot\boldsymbol{e}_{k(f)}$ on uniform meshes. Until issue 74 the code used $\rho^{\mathrm{l}}a_f^{\mathrm{l}} + \rho^{\mathrm{g}}(1 - a_f^{\mathrm{l}})$, which put liquid next to a sloped solid out of hydrostatic equilibrium wherever a cut face in the liquid with $a_f^{\mathrm{o}} < 1$ carries a jump (in the fixtures below at one cut hanging face of the composite mesh; derivation audit, `proof_review/eq-forces-hydrostatic/`).

Hydrostatic balance. By (8.9) gravity is balanced if and only if there is a cell function φ with

$$\rho_f^a\,(\mathsf{J}\Phi)_f = (\mathsf{J}\varphi)_f \quad \text{for every } f\in\mathcal{F}_\Omega \text{ with } \gamma_f > 0; \tag{8.18}$$

since this condition is face by face, it does not depend on the metric, and holds at level transitions whenever it holds. For a planar interface normal to $\boldsymbol{g}$ through $\boldsymbol{x}^I$ it holds, with a solid of any shape in either phase, as soon as the PLIC line of every mixed cell is the plane, with $\varphi_c = \rho^\pi\boldsymbol{g}\cdot(\boldsymbol{x}_c^I - \boldsymbol{x}^I)$ in a pure cell of phase π and $\varphi_c = 0$ in a mixed cell: the state (8.3)(b) sampled at $\boldsymbol{x}_c^I$, the thesis' $p^\dagger$ of Lemma 7.1 [19, eq. (7.65), App. 7.A]. Indeed, across a face between two mixed cells neither Φ nor φ jumps, since both points lie on the plane; a face with a pure neighbour of phase π lies in that phase, because the plane does not cross the open part of a pure cell, so that $\rho_f^a = \rho^\pi$ also when the face is cut; and $\rho^\pi(\mathsf{J}\Phi)_f = (\mathsf{J}\varphi)_f$ there, since $\varphi_c = \rho^\pi(\Phi_c - \boldsymbol{g}\cdot\boldsymbol{x}^I)$ in the pure cells of phase π and $\Phi_c = \boldsymbol{g}\cdot\boldsymbol{x}^I$, $\varphi_c = 0$ in the mixed ones. The argument uses neither the metric nor the solid nor the potential point of a pure cell. Conversely, let every face with a pure neighbour of phase π lie in that phase. Then the jumps of (8.18) divided by $|f|$ add up, along a path of faces through pure cells of phase π from a mixed cell c to a mixed cell c', to $\rho^\pi(\Phi_{c'} - \Phi_c)$;

along the closed path that leaves c through liquid cells and returns through gas cells to $(\rho^{\mathrm{l}} - \rho^{\mathrm{g}})(\Phi_{c'} - \Phi_c)$; and along the closed path that leaves c through cells of π and returns through the face f between two adjacent mixed cells to $(\rho^{\pi} - \rho^a_f)(\Phi_{c'} - \Phi_c)$. All these sums vanish for the jumps of a cell function, so the mixed cells joined by such paths lie at one potential level (the plane through $\boldsymbol{x}^I$ is one choice; the argument above works for any common level Φ^* in its place). In general the sum of $\rho^a_f (\mathsf{J}\Phi)_f / |f|$ along every closed path of open faces must vanish. Where the PLIC is not exact this fails: in the triple cells of a plane that meets a solid the closed path through a mixed–mixed face has a nonzero sum (0.05 of $\rho^{\mathrm{l}} |\boldsymbol{g}| \, h$), and for an oblique plane within the reach of the height functions of a solid or a corner a face next to a pure cell carries an aperture of the other phase. The jumps then lie at a relative distance of 4×10^{-4} to 1.2×10^{-2} from those of any cell function, in triple cells 1.2×10^{-2} independently of the embedded contact angle (derivation audit, `proof_review/eq-forces-hydrostatic/`). An interface on a grid line, a case outside the thesis' proof, has no mixed cells: a face between a liquid and a gas cell has the phase apertures $\frac{1}{2}, \frac{1}{2}$ (checked without solid) and $\rho^a_f = \frac{1}{2}(\rho^{\mathrm{l}} + \rho^{\mathrm{g}})$, and the state is balanced, with $\varphi = p^\dagger$ if the two cells have the same width normal to the face; at a level transition along the interface the balancing pressure differs across the interface by 0.750 times the difference of $p^\dagger$ between the two cell centres (widths $\frac{1}{2}$ below and $\frac{1}{4}$ above, densities $(1000, 1)$). The energy and momentum identities of the thesis [19, eqs. (7.59), (7.67)] are derived for the diagonal metric and are not claimed here; the diagnostic `PotentialEnergy` integrates the exact first moments of the phase polygons, not Φ.

Checks (`gravity_check`). For a planar interface through $(4.3, 4.6)$ on the uniform and composite meshes of the drop fixtures (densities $(1000, 1)$), normal to $\boldsymbol{g} = (0, -1)$ without solid and with a sloped solid in the gas or in the liquid (23 cut faces in the liquid on the composite mesh), an independent evaluation of (8.15) and (8.17) (own segment midpoints, the phase apertures of Chapter 4) equals the code's jumps to 0; for these planes and for the plane normal to $\boldsymbol{g} = (0.6, -0.8)$ without solid it equals the code's force to 3.8×10^{-15} (relative), and one projection of $\delta t \mathsf{R}\mathcal{J}^{\boldsymbol{g}}$ leaves at most 1.6×10^{-9} of $\delta t \, |\boldsymbol{g}|$. The oblique plane is not checked with a solid: there it meets the solid or passes within the reach of its height functions, where the PLIC is not exact and the balance fails (above).

Deviation from the thesis 8.4 (Gravity potential point in cut cells). The thesis evaluates the potential at the interface centroid of a mixed control volume and at the centroid of a pure one [19, eq. (7.61)]. The code uses the cell centre for a pure cut cell and the midpoint of the unclipped PLIC segment in a triple cell (8.15). The balance condition (8.18) does not depend on this choice: the planar state is balanced for any point of a pure cell and any point of the PLIC segment of a mixed cell (derivation audit, random points on six fixtures); the thesis' energy and momentum identities, not claimed here, rely on its centroids.

Implementation. `gravity!(acceleration, interface, pressure_operator, workspace; density, gravity)` overwrites `acceleration` with $\mathsf{R}\mathcal{J}^{\boldsymbol{g}}$; potential point `_interface_centroid` (`src/interface/height_functions.jl`) (8.15), liquid apertures `_current_phase_face_apertures!` (`src/two_phase/two_velocity/transport_geometry.jl`) (cached per reconstruction; the call also prepares them for the two-velocity projection). Diagnostic `_gravitational_potential_energy` (`src/two_phase/forces.jl`). Tests: `test/two_phase/forces.jl` ("Hydrostatic balance") (uniform, composite and adapted

meshes, vertical and oblique gravity, density ratio 1000, velocity below $2048\epsilon_{\mathrm{mach}}$ after the projection), `test/indicator/geometry.jl` ("Cut-cell AMR forces retain balanced pressure cancellation") (solid in the gas), `test/two_phase/forces.jl` ("Gravity uses the open-face phase mixture") and `test/two_phase/forces.jl` ("Hydrostatic liquid at a sloped solid on a composite mesh") ((8.16), issue 74), `test/navier_stokes/two_phase_navier_stokes.jl` ("Flat heavy interface remains hydrostatic") (ten steps of `simulate!`, velocity below 10^{-9}).

8.7 Prescribed body acceleration

A body acceleration, the keyword `force` of `step!` and `simulate!`, is a function $(x, y, t, k) \mapsto \boldsymbol{a}^{\mathrm{b}} \cdot \boldsymbol{e}_k$ sampled at the face centroids at the midpoint of the step. A `FlowSimulation` stores the constructor's `force` as `flow.force`, the default for its steps and simulations:

$$u_f^* \leftarrow u_f^* + \delta t\, \boldsymbol{a}^{\mathrm{b}} \left(\boldsymbol{x}_f, t^{(n)} + \frac{1}{2}\delta t \right) \cdot \boldsymbol{e}_{k(f)}, \qquad f \in \mathcal{F}_\Omega, \tag{8.19}$$

added to the transported velocity, before its boundary values are set at $t^{(n)} + \frac{1}{2}\delta t$ and before the viscous step. In the two-velocity model it is added to both phase velocities, except where a phase is absent (NaN). There is no force map and no balance statement: a body acceleration is an increment of the predictor like any other. Without `force` the stage does nothing.

Implementation. `_body_force_substep!` (`src/navier_stokes/stages.jl`) and `_add_body_force!` (`src/navier_stokes/stages.jl`) (8.19). Tests: `test/navier_stokes/body_force.jl` ("Typed body force preserves phase data and callback semantics") (sampling points, midpoint time, absent phases), `test/navier_stokes/body_force.jl` ("Body-force midpoint update reaches shared and intrinsic stepping") (a time-dependent acceleration integrated exactly by the midpoint rule for one and two velocities).

8.8 Two velocities

The two-velocity step adds the same two increments to both intrinsic phase velocities,

$$u_f^{\pi,***} = u_f^{\pi,**} + \delta t(\mathsf{R}^{\mathrm{tv}} \mathcal{J}^{\sigma})_f + \delta t(\mathsf{R}^{\mathrm{tv}} \mathcal{J}^{\boldsymbol{g}})_f, \qquad \pi \in \{\mathrm{l}, \mathrm{g}\}, \tag{8.20}$$

except where a phase is absent (NaN), with the force map R^{tv} of the projection that follows (Chapter 10, CMDGFM): the map (8.6) with the common coefficient γ^{tv} (8.5) of the two-velocity projections in place of γ and the same K, evaluated with the pressure operator that the selected projection caches, its coefficient rebuilt from the transported fractions. Without a solid $\gamma^{\mathrm{tv}} = \gamma$ and $\mathsf{R}^{\mathrm{tv}} = \mathsf{R}$, and the step uses the one-velocity operator, refreshed only when $\sigma > 0$ or $\boldsymbol{g} \neq \boldsymbol{0}$. With a solid the two maps are congruent, $\mathsf{R} = \mathsf{A}^{1/2}\mathsf{R}^{\mathrm{tv}}\mathsf{A}^{1/2}$ with $\mathsf{A} = \mathrm{diag}\,(a^{\mathrm{o}})$, so that $\mathsf{R}\mathcal{J} = \mathsf{A}^{1/2}\mathsf{R}^{\mathrm{tv}}(\mathsf{A}^{1/2}\,[\![])$: at a face whose row of K is diagonal both maps are diagonal there, $(\mathsf{R}^{\mathrm{tv}}\mathcal{J})_f = \mathcal{J}_f/(\widetilde{\rho} f\,|\omega_f^{\mathrm{o}}|)$, so that $(\mathsf{R}\mathcal{J})_f = a_f^{\mathrm{o}}\,\mathcal{J}_f/(\widetilde{\rho} f\,|\omega_f^{\mathrm{o}}|)$, the face acceleration of (8.8); while at a level transition a row that K couples mixes faces whose apertures differ, so that $(\mathsf{R}\mathcal{J})_f \neq a_f^{\mathrm{o}}(\mathsf{R}^{\mathrm{tv}}\mathcal{J})_f$ there. The projection assembles its own system with the coefficient γ^{tv} and the gradient-jump unknowns ξ with phase coefficients Ξ^{π}; the forces enter it through the predictor only, and its common pressure gradient $\mathsf{R}^{\mathrm{tv}}\mathsf{J}p$

uses the map of the increment. The phase gradients of CMDGFM differ by ξ alone, and for equal phase increments the jump unknowns stayed zero in every case of the derivation audit (`proof_review/eq-forces-two-velocity/`), so that CMDGFM acts as the one-velocity projection with the coefficient γ^{tv}: by (8.9) for R^{tv}, it returns the rest state exactly if the jumps are those of a cell function on the faces with $a_f^{\mathrm{o}} > 0$, with or without a solid (for gravity the condition (8.18)). Measured, one inviscid step of `simulate!` from rest of the planar heavy liquid of Section 8.6 ($\boldsymbol{g} = (0, -1)$, densities $(1000, 1)$, $\delta t = 0.01$, tolerance 10^{-10}) leaves, relative to $\delta t\,|\boldsymbol{g}|$: at most 2.2×10^{-7} with one velocity (uniform and composite, without solid and with a solid in the gas or in the liquid); with CMDGFM 9.0×10^{-11} without solid, and 1.1×10^{-10} (uniform) and 2.5×10^{-10} (composite) with a solid in either phase. A drop with prescribed constant curvature, its CSF force added to both phases and projected by CMDGFM, keeps at most 4.5×10^{-13} of the increment without solid and 1.1×10^{-12} on the cut mesh (1.3×10^{-6} with one velocity). Until issue 76 (PR #81) the increments used the one-velocity map R of the cut-cell operator, i.e. the same jumps through $\mathsf{A}^{1/2}\mathsf{R}^{\mathrm{tv}}\mathsf{A}^{1/2}$, whereas the projection removes gradients of R^{tv} alone: it balances that increment if and only if $\mathsf{A}^{1/2}\mathsf{R}^{\mathrm{tv}}(\mathsf{A}^{1/2}\,[\![\,]\!])$ lies in the range of $\mathsf{R}^{\mathrm{tv}}\mathsf{J}$, which at a face whose row of K is diagonal reduces to $a_f^{\mathrm{o}}\,[\![\,]\!]_f$ being the jump of a cell function. Without a solid $\mathsf{R}^{\mathrm{tv}} = \mathsf{R}$ bitwise; on a conforming mesh with a solid every row is diagonal, and the one-velocity increment is balanced exactly when the aperture-weighted jumps are those of a cell function, which holds for the planar state of Section 8.6 because its $a_f^{\mathrm{o}} = 1$ on every face that carries a jump. At a level transition the row of K mixes those faces with the cut face at $a_f^{\mathrm{o}} = 0.8$, so $\mathsf{A}^{1/2}\mathsf{R}^{\mathrm{tv}}(\mathsf{A}^{1/2}\,[\![\,]\!])$ leaves the range even though $[\![\,]\!]$ is the jump of a cell function, and the one-velocity increment of the gravity or CSF jumps leaves 3.0×10^{-2} of $\delta t\,|\boldsymbol{g}|$ (projection only, the largest of the composite cases with a solid; the full step left 0.029 with the solid in the gas and 0.030 with it in the liquid). The cut-mesh drop behaves alike: the solid does not touch it, but one cut face with $a_f^{\mathrm{o}} = 0.8$ at the level transition is coupled through K to the drop's CSF jumps, and the one-velocity increment leaves 0.013 of it, while CGFM, whose jump never reaches that row, is balanced (1.5×10^{-13}). With the increment evaluated through R^{tv} the two maps agree, the balance reduces to the hydrostatic condition (8.18), and the same projections leave 1.1×10^{-10}, 4.3×10^{-11} and 2.2×10^{-12} (derivation audit, `proof_review/eq-forces-two-velocity/`).

Deviation from the thesis 8.5 (Two-velocity forces). The thesis puts surface tension into the jump conditions of the two-velocity gradient $\mathfrak{g}^{\pi}(p, \xi)$ through the value jump $\zeta = -\sigma\widetilde{\kappa}$ [19, eqs. (4.50)–(4.53)] and adds gravity per phase in step T4.II [19, §7.2]. The code adds both forces as one common explicit increment (8.20), computed with the pressure map of the projection, to both phases; its projections apply Ξ^{π} to ξ only, so no physical value jump ζ is imposed. The balance statements of Section 8.3 carry over to CMDGFM with γ^{tv} in place of γ.

Implementation.
`_phase_force_substep!` (`src/two_phase/two_velocity/stepping.jl`) and `_add_common_phase_acceleration!` (`src/two_phase/two_velocity/stepping.jl`) (8.20); the map of the projection `_phase_force_pressure_operator!` (`src/two_phase/two_velocity/stepping.jl`) (the cached operator of `_projection_pressure_operator` (`src/two_phase/projection/common.jl`)), the refresh of the one-velocity operator without a solid in

`step!` (`src/two_phase/two_velocity/stepping.jl`); the phase gradients `_projection_gradient_core!` (`src/two_phase/projection/common.jl`). Tests: `test/two_phase/forces.jl` ("CMDGFM cut composite constant-curvature drop force map (#76)") (the force stage applies the projection's map, and CMDGFM removes the constant-curvature CSF and CGFM force on the cut composite mesh to 10^{-9} of the increment, while the one-velocity increment of the same jumps is not removed), `test/two_phase/two_velocity/projection/cmdgfm_projection.jl` ("CMDGFM hydrostatic rest with embedded solids (issue #76)") (one step of the composite planar fixture with a solid in either phase stays at rest to $10^{-9}\delta t\,|\boldsymbol{g}|$), `test/two_phase/two_velocity/projection/cmdgfm_projection.jl` ("CMDGFM balance needs the projection's force map (#76)") (the coefficients of the two operators, $\gamma = a_f^{\mathrm{o}}\,\gamma^{\mathrm{tv}}$, and the two increments through the projection). Checks: `hydrostatic_step`, `two_velocity_tension` and `hydrostatic_former_step` (the one-velocity counterfactual); residual of the former map in the derivation audit `proof_review/eq-forces-two-velocity/`.

8.9 Fully discrete update and time-step limits

The forces are explicit increments at two places of the split step of `step!`, for one and two velocities:

$$
\begin{aligned}
&u^{(n)} \xrightarrow{\text{boundary values at } t^{(n)}} \xrightarrow{\text{transport}} u^{*},\ C^{\mathrm{l},(n+1)},\\
&u^{*} \leftarrow u^{*} + \delta t\,\boldsymbol{a}^{\mathrm{b}}\big(t^{(n)} + \tfrac{1}{2}\delta t\big) \xrightarrow{\text{boundary values at } t^{(n)}+\frac{1}{2}\delta t} \xrightarrow{\text{viscous step}} u^{**},\\
&u^{**} \xrightarrow{\text{two velocities: boundary values at } t^{(n+1)}}, \qquad \gamma \leftarrow \gamma\big(C^{\mathrm{l},(n+1)}\big),\\
&u^{***} = u^{**} + \delta t\,\mathsf{R}\mathcal{J}^{\sigma}\big(C^{\mathrm{l},(n+1)}, \widetilde{\kappa}^{(n+1)}\big) + \delta t\,\mathsf{R}\mathcal{J}^{\boldsymbol{g}}\big(C^{\mathrm{l},(n+1)}\big) \xrightarrow{\text{one velocity: boundary values at } t^{(n+1)}},\\
&u^{\star} = u^{***} - \delta t\,\mathsf{R}\mathsf{J}p^{(n)} \xrightarrow{\text{projection}} u^{(n+1)}, \quad p^{(n+1)} = p^{(n)} + \varphi,
\end{aligned}
\tag{8.21}
$$

with the transport of Chapters 5 and 6, the viscous step of Chapter 9 and the projection of Chapter 10. The last line is the old-pressure predictor and the pressure increment of the incremental one-velocity and CMDGFM projections ((10.26) and (10.27)); the boundary values at $t^{(n+1)}$ are imposed once, after the forces with one velocity and before them, for both phases, with two velocities. In the two-velocity step with a solid the increments and the old-pressure gradient use the map R^{tv} of the projection, and the one-velocity coefficient γ is not refreshed ((8.20)). The phase forces are evaluated once per step on the transported interface, with the curvature and phase apertures of its reconstruction, and added one after the other (surface tension, then gravity); both are first order (forward Euler) in time, and no global order of the split step is claimed here. Applying them after the viscous step keeps the hydrostatic predictor of the thesis' Remark 7.1: with $u^{(n)} = \mathbf{0}$ the viscous step returns zero and $u^{***} = \delta t\mathsf{R}\mathcal{J}^{\boldsymbol{g}}$, and once $p^{(n)}$ balances gravity, $\mathsf{R}\mathsf{J}p^{(n)} = \mathsf{R}\mathcal{J}^{\boldsymbol{g}}$ ((8.9)), the predictor $u^{\star}$ vanishes and so does the increment. A force that is switched off contributes nothing: for $\sigma = 0$ no curvature is interpolated and nothing is counted, for $\boldsymbol{g} = \mathbf{0}$ no potential is formed, and in the two-velocity step no pressure map is then refreshed for the forces (the CMDGFM step still prepares it for the old-pressure predictor).

The step size is limited by the capillary and gravity bounds (5.18), evaluated with the smallest cell width h of the mesh: δt_{σ} is the thesis' bound [19, eq. (4.1)], δt_{grav} the thesis'

gravity-wave bound [19, eq. (7.4)] multiplied by $1/\sqrt{2}$ (Chapter 13). `step!` rejects a step above $(1+10^{-6}+32\epsilon_{\mathrm{mach}})\min\{\delta t_{\mathrm{WY}}, \delta t_{\sigma}, \delta t_{\mathrm{grav}}\}$ ((13.3), issue 98), and the error message names the bound that is active; only a two-velocity step with the frozen interface fluxes whose step size `simulate!` has just chosen from these bounds skips the check.

> *Implementation.* Order in `step!` (`src/single_phase/stepping.jl`) (shared velocity, one and two phases) and `step!` (`src/two_phase/two_velocity/stepping.jl`); phase forces `_phase_force_substep!` (`src/two_phase/stepping.jl`) and `_apply_phase_forces!` (`src/two_phase/stepping.jl`), with `_add_common_phase_acceleration!` (`src/two_phase/stepping.jl`); old-pressure predictor `_pressure_predictor!` (`src/single_phase/stepping.jl`) and, for the CMDGFM, `_two_velocity_pressure_operator!` (`src/two_phase/two_velocity/stepping.jl`); bounds `_two_phase_timestep_bound` (`src/two_phase/stepping.jl`). Tests: `test/navier_stokes/two_phase_navier_stokes.jl` ("Two-phase gravity-wave timestep safety factor"), `test/navier_stokes/two_phase_navier_stokes.jl` ("Two-phase capillary and gravity bounds agree across velocity models") and `test/two_phase/forces.jl` ("Surface tension precedes gravity for each velocity model").

— 9 —

Viscous stress and implicit solve

Equation reference, audited against commit `bc0da10`.

Viscosity is the one implicit substep of the split time step: after the transport and the body acceleration, before the phase forces and the projection ((8.21)), the velocity is advanced by the implicit midpoint rule. Two operators implement it. A single phase has the constant kinematic viscosity μ/ρ and a componentwise staggered Laplacian on the dual graph of Chapter 6, with geometric dual-face contacts and a symmetric dissipative stiffness at level transitions; a cut-cell variant adds the embedded walls. Two phases have the coupled variable stress $\nabla \cdot (2\mu\mathbf{D})$, assembled as $\mathsf{E}^\intercal \mathsf{W}_\mu \mathsf{E}$ from normal-stress rows in the cells, shear rows at the cell corners and wall rows, with a mixture of the phase viscosities that is harmonic (series) by default and geometric as an option; the two-velocity model diffuses the mass-weighted mixture velocity with the same operator and gives both phases the same increment, so the rule applies to both velocity models. This chapter states the continuous model, the semi-discrete operators with their walls, the fully discrete midpoint solves with their stopping criteria, and the exact identities that hold for them: dissipation of the mass-weighted kinetic energy, conservation of momentum on periodic meshes and the unchanged velocity jump of two velocities. Derivations are in the thesis [19, §7.5]; the linear solvers and their multigrid preconditioners are in Chapter 11.

9.1 Continuous model

The viscous part of the phase momentum balance (8.1) is

$$\rho^\pi \partial_t \boldsymbol{u}^\pi = \nabla \cdot (2\mu^\pi \mathbf{D}(\boldsymbol{u}^\pi)) + \dots, \qquad \mathbf{D}(\boldsymbol{u}) = \frac{1}{2}\left(\nabla \boldsymbol{u} + (\nabla \boldsymbol{u})^\intercal\right), \tag{9.1}$$

which for a constant viscosity and $\nabla \cdot \boldsymbol{u} = 0$ is $\mu \Delta \boldsymbol{u}$. The one-velocity model has a single velocity with the piecewise constant ρ and μ [19, eq. (2.9)]; the thesis replaces the sharp viscosity by the volume-fraction-weighted geometric mean of the phase viscosities, a diffuse model chosen so that the one- and two-velocity formulations share it [19, §7.5.1]. The code implements that rule and, as the default, the volume-fraction-weighted harmonic mean, the series conductance across the interface that a centred shear difference needs; Section 9.3 states both and the deviation. With stationary no-slip or free-slip walls, or periodic bound-

aries, the viscous term dissipates kinetic energy,

$$\frac{\mathrm{d}}{\mathrm{d}t}\int_{\Omega^{\circ}} \tfrac{1}{2}\rho\,|\boldsymbol{u}|^{2}\,\mathrm{d}V = -\int_{\Omega^{\circ}} 2\mu\,\mathbf{D}(\boldsymbol{u}) : \mathbf{D}(\boldsymbol{u})\,\mathrm{d}V \le 0 \qquad \text{(viscous part only)}, \tag{9.2}$$

and conserves momentum $\int_{\Omega}\rho\boldsymbol{u}\,\mathrm{d}V$ on a periodic domain. The wall conditions are the stationary or moving no-slip condition $\boldsymbol{u} = \boldsymbol{u}^{\Gamma}$, the free-slip condition $\boldsymbol{u}\cdot\boldsymbol{n} = 0$ with zero tangential stress, a prescribed inflow velocity and, at outflow sides, a zero normal derivative.

9.2 Single-phase operator

The unknowns of component k are the face velocities u_f, $f \in \mathcal{F}^{[k]} \cap \mathcal{F}_{\Omega}$, with the transport volumes $\widetilde{V}_f$ of (6.3) as the diagonal of $\widetilde{\mathsf{V}}$. The operator is built from the dual faces of the momentum schedule of Section 6.2. For connections g along the component axis $t = k$, the net measure and normal distance are

$$|g| = \Big|\sum_{g'}\sum_{f'\in\mathcal{F}^{[t]}} \widetilde{w}_{g',f'}\,|f'|\Big|, \qquad d_g = \big|(\boldsymbol{x}_{f^{+}(g)} - \boldsymbol{x}_{f^{-}(g)})_t\big|\,, \tag{9.3}$$

summed over the dual faces g' of the pair; the nearest periodic image is used and negligible net measures are dropped. Transverse contacts $t \neq k$ are rebuilt from overlaps of bisected cell traces, including the contact between segments of a split side, as described below for open traces. Without a solid these are the full traces. This avoids using advection bookkeeping weights as geometric diffusion measures (issue 78, issue 80).

Across a connection the directional derivative is a row

$$\partial^{\mathrm{h}}_{g}\widetilde{\varphi} = \sum_{f} \mathfrak{c}_{g,f}\,\widetilde{\varphi}_f. \tag{9.4}$$

Aligned contacts use a two-point difference. Fine–coarse normal contacts use an interpolation of the fine siblings when available; transverse contacts with cancelling weighted offsets share a centred derivative. An offset contact that is left without such a group is closed against the wall where the wall value is known and otherwise takes a linear-exact two-point difference on the tangential level of one of its ends, as described below. With the wall terms included, the integrated stiffness and Laplacian are

$$\mathsf{S}^{\nu}_{k} = \mathrm{sym}\left(\sum_{g\in\mathcal{K}_k} |g| d_g\,\mathfrak{c}^{\mathsf{T}}_{g,}\mathfrak{c}_{g,} + \sum_{w\in\mathcal{W}_k} \varkappa_w\,\boldsymbol{e}_{f(w)}\boldsymbol{e}^{\mathsf{T}}_{f(w)}\right), \tag{9.5}$$

$$\widetilde{\mathfrak{L}}_k = -\widetilde{\mathsf{V}}^{-1}\mathsf{S}^{\nu}_{k}. \tag{9.6}$$

The symmetrisation removes only round-off. The interior terms are non-negative weighted squares; the signed wall-contact corrections complete non-negative wall-aware squares. Hence S^{ν}_{k} is symmetric positive semidefinite. On a periodic mesh constants are in its kernel and the rate conserves momentum. On a uniform mesh this is the five-point Laplacian.

The previous quadratically exact replacement of individual transition rows did not preserve the transpose rows, making $\widetilde{\mathsf{V}}\widetilde{\mathfrak{L}}_k$ nonsymmetric and its symmetric part indefinite.

The old midpoint step could increase energy by factors up to 1.1 (walled) and 2.7 (periodic) on the audit fixtures (`proof_review/eq-viscosity-laplacian/`). The shared geometric stiffness removes this growth for every step size (issue 80). Transition rows may have $O(1)$ pointwise truncation error, but the composite manufactured Helmholtz solution approaches second order, and the observed slopes depend on where the transition crosses the solution: they are no longer uniformly above 1.8. On the audit's walled half-refined mesh, with 16, 32 and 64 base cells per direction and a sine–sine manufactured Helmholtz solution, the previous maximum-norm orders were 2.36, 2.62 and the dual-volume L^2 orders were 2.61, 2.67. The geometric stiffness gives maximum-norm orders of 1.84, 1.92 on a half-refined strip and 1.78, 1.88 on a refined corner, and dual-volume L^2 orders of 1.99, 2.00 and 2.01, 2.01, respectively (base cells $16 \to 32 \to 64$; `test/operators/staggered_diffusion.jl`). Thus the solution remains approximately second order but the old fixture's superconvergent orders are not retained. For stationary walls or periodic boundaries, put $\mathsf{B} = \widetilde{\mathsf{V}}^{-1/2}\mathsf{S}_k^{\nu}\widetilde{\mathsf{V}}^{-1/2}$. Its eigenvalues are nonnegative, so for every $a = \delta t\mu/\rho \geq 0$,

$$\|\widetilde{\mathsf{V}}^{1/2}(\mathsf{I} - a\widetilde{\mathfrak{L}}_k)^{-1}\widetilde{\mathsf{V}}^{-1/2}\|_2 = \|(\mathsf{I} + a\mathsf{B})^{-1}\|_2 \leq 1.$$

The midpoint amplification is likewise bounded by one when its explicit and implicit halves use the same velocity. The split step (9.17) solves for u^{**} from the transported velocity u^* and evaluates its explicit half at the old velocity $u^{(n)}$: its right-hand side is $(\mathsf{I} + a\widetilde{\mathfrak{L}}_k)u^{(n)} + (u^* - u^{(n)})$. With $\mathsf{M} = \widetilde{\mathsf{V}}^{1/2}(\mathsf{I} - a\widetilde{\mathfrak{L}}_k)^{-1}(\mathsf{I} + a\widetilde{\mathfrak{L}}_k)\widetilde{\mathsf{V}}^{-1/2}$, $\|\mathsf{M}\|_2 \leq 1$, and, before the wall source,

$$\left((u^{**})^{\mathsf{T}}\widetilde{\mathsf{V}}\,u^{**}\right)^{1/2} \leq \left((u^{(n)})^{\mathsf{T}}\widetilde{\mathsf{V}}\,u^{(n)}\right)^{1/2} + \left((u^* - u^{(n)})^{\mathsf{T}}\widetilde{\mathsf{V}}\,(u^* - u^{(n)})\right)^{1/2}, \tag{9.7}$$

because $\widetilde{\mathsf{V}}^{1/2}(\mathsf{I} - a\widetilde{\mathfrak{L}}_k)^{-1}\widetilde{\mathsf{V}}^{-1/2}$ has norm at most one as well. The step therefore grows the energy by at most the transported-to-old difference; moving walls may add their work through u_w^{Γ}, and the split-step caveat in Section 9.4 still applies.

Walls. For a boundary face b of the cell $c = c(b)$ and the unknowns f of component k on the sides of c (side fractions $\varpi_{c,f}$ of (6.5)), the wall connections are

$$\varkappa_w = \begin{cases} \dfrac{|b|/2}{\left|(\boldsymbol{x}_f - \boldsymbol{x}_b)_{k(b)}\right|}, & k(b) \neq k \text{ and the dual strip touches } b, \\ \dfrac{\varpi_{c,f}|b|}{h_{c,k}}, & k(b) = k \text{ (normal, the } f \text{ opposite } b), \\ 0, & \text{otherwise.} \end{cases} \tag{9.8}$$

with the wall value u_w^{Γ} at the point where the wall meets the line of f: `NoSlip(u)` includes both kinds, with the value of `u` (default zero) on tangential and zero on normal connections; `FreeSlip` keeps the normal ones only (zero tangential stress); with `VelocityBoundaryConditions` each side chooses, `Inflow` with its function on both kinds, `Outflow` and `ZeroGradientVelocity` with none, a homogeneous Neumann condition. Connections of zero distance are skipped. Affine profiles, also between two moving walls, are reproduced exactly, including on composite meshes: the tendency of Couette flow between a stationary floor and a moving

lid stays at or below 2×10^{-12} of its velocity maximum across a level transition as well. A row next to a wall is not exact for quadratics: for $u_1 = y(X_2 - y)$ between no-slip walls it misses $\Delta u_1 = -2$ by 0.5, independently of the mesh width (derivation audit, `proof_review/eq-viscosity-walls/`). The operator returned by `diffusion!` is

$$\frac{\mu}{\rho}\Big((\widetilde{\mathfrak{L}}_k u)_f + \widetilde{V}_f^{-1} \sum_{w \in \mathcal{W}_k,\ f(w)=f} \varkappa_w\, u_w^{\Gamma}\Big), \qquad f \in \mathcal{F}^{[k]} \cap \mathcal{F}_\Omega, \tag{9.9}$$

and zero on the boundary faces, whose normal values are prescribed.

Cut cells. The unembedded and embedded operators share the geometric contact construction. With a solid, the constructor of `StaggeredDiffusionOperator` that takes a `GeometryField` uses the cut-cell schedule of Section 6.2 (weights times apertures, open volumes $|\omega_f^{\circ}|$, one where they vanish) and rebuilds the transverse connections from the open traces of the cells. A normal connection between the two unknowns of one cell takes the open area of that cell's mid-plane over the tangential overlap of the two faces: the schedule average halves whenever one of the two primal faces flanking the connection is closed, which is exactly the wall-adjacent row (issue 110). The open trace of a cut cell is bisected at the cell centre, as on an uncut cell, so a clipped trace does not move the boundary of a staggered dual strip. Across an open face h of axis $t \neq k$ between two cells, the extent of each cell along k is clipped to the open part of its side on h and bisected, each half belonging to the unknown of that cell on its side; every unknown of one cell that touches h is connected to every such unknown of the other cell, with the overlap of their halves within h as the measure and their distance along t (on every mesh, also without level transitions). The two segments of a split side of a cell, next to finer cells, are connected along t with half the open length of the line through the split point. The derivatives across the connections are exact for linear velocities: a fine–coarse connection along k differences the coarse unknown and the interpolation of the two fine siblings to its line; where transverse connections are offset along k (at a level transition), the connections of an unknown to one side whose measure-weighted offsets along k cancel share one centred derivative, with the coefficient $-1/d$ for the unknown and $|g|/(d\sum_{g'}|g'|)$ for each neighbour, d the measure-weighted offset along t; and an offset connection left without such a group, where a level transition meets a wall normal to k, takes the zero normal velocity of a `NoSlip` or `FreeSlip` wall as a Dirichlet partner, interpolating the far unknown to the wall distance of the near one (exact for linear profiles that vanish at the wall). An `Inflow` side prescribes that velocity too, so it closes the connection the same way with the prescribed value; because the wall values of the two signed corrections sit at the two faces' own tangential positions, that form is exact for inflow data constant along the wall and first order for data that varies along it. At `Outflow` and `ZeroGradientVelocity` walls no value is prescribed, so the connection is differenced on the tangential level of one of its ends instead: the endpoint that has a neighbour on its own level spanning the contact, and that neighbour. Differencing the offset pair itself would carry the tangential derivative of the offset into the t derivative, with weight $\Delta k/\Delta t$ and hence without vanishing under refinement; the level-matched pair is exact for linear velocities (issue 99). The one-ring face graph supplies the level-matched face; where none lies within two contact distances, the weighted minimum-norm derivative of (9.4) over the two-ring face graph is used, as for the unembedded transition. Both keep the coefficients summing to zero and the row energy a non-negative square. The domain walls are (9.8)

with each measure multiplied by the open fraction of the wall segment that the connection owns (for a tangential connection the half of the cell's wall segment on the side of f, owned whole by the segment of a split side that touches the wall, for a normal one the part of the wall segment over f; connections whose segment is covered are dropped). Embedded segments come from cut cells and from uncut open cells sharing a face with a covered cell (adjacent faces on one side are merged). For a Cartesian wall segment recovered from the face aperture, a tangential face owns the part of the segment within its dual strip; its conductance is that length divided by the wall-normal distance to the face, bounded below by one quarter of the smallest cell width. In particular, an open tangential node a distance $h/2$ from a grid-aligned wall uses $|f|/(h/2)$, not a distance to the primal-cell segment midpoint. A reconstructed PLIC segment of a cut cell, Cartesian or tilted, is split among the dual faces of that cell by the tangential strip of a component face and the half of the cell on that face's side, so the pieces partition the segment and each piece is owned once. The wall distance of an unknown f is the *perpendicular* distance d_f to the reconstructed plane of its cell, not the distance to the segment midpoint, and the conductance is

$$\varkappa_w = \frac{|I^{\mathrm{s}}_{c,f}|}{\max\left(|d_f|\,,\ \frac{1}{64}\min_t h_{c,t}\right)} \quad \text{(omitted for } \texttt{FreeSlip()}\text{)}, \tag{9.10}$$

with $|I^{\mathrm{s}}_{c,f}|$ the length of the piece. The floor is one sixty-fourth of the smallest cell width and it is active: a cut plane passes through or next to the centre of many wall-piece nodes, so a fifth to a third of the pieces on the manufactured fixtures sit closer than $h/64$ (the closest is a node exactly on the reconstructed plane). It bounds the largest wall conductance at $64|I^{\mathrm{s}}_{c,f}|/\min_t h_{c,t}$, which keeps the assembled stiffness scaling as $1/h^2$; the pinned errors and orders are measured with this floor in place. The factor $\frac{1}{2}\varpi_{c,f}$ of the previous closure is gone because the pieces already partition the segment. A fluid-side unknown satisfies $0 = u_w = u_f - d_f\partial_n u$ for the wall-normal derivative $\partial_n u$, so $\varkappa_w u_f$ equals the wall flux $|I^{\mathrm{s}}_{c,f}|\partial_n u$ of a linear profile exactly. A node whose centre lies inside the solid gets the same positive conductance, hence the opposite row contribution to the one its open dual volume requires: it has no fluid on the wall side, and prescribing it would remove a face that the cut-cell projection updates. Its rows therefore stay first order, which is the remaining defect of issue 110. Where a grid-aligned wall coincides with a normal component face, that face is prescribed zero and removed from the diffusion unknowns. The mid-plane contact of the adjacent cell is fully open there and carries the whole Dirichlet flux to the eliminated face, for both wall types; no separate wall connection is added, because an aperture-weighted half contact plus a compensating half connection is the same matrix split differently. The retained principal stiffness block also carries flux to the eliminated face. Its integrated reaction, the negative column sum of the eliminated-to-retained stiffness block applied to the retained velocities, is stored separately from wall connections and included when reporting embedded viscous forces; it is not added to diffusion twice.

Every embedded wall value is zero, also for a moving `NoSlip(u)` (since issue 75); with `VelocityBoundaryConditions` the cut kind is kept whatever the sides choose, so the embedded walls are no-slip. Away from eliminated wall-normal faces, a completely covered background face retains the unit-volume algebraic penalty $\varkappa_w = 1$ for both wall types. With $u_f^{(n)} = 0$, the midpoint step (9.17) scales such a covered value by $1/(1 + \frac{1}{2}\delta t\mu/\rho)$; the flow solver also sets closed-face values to zero at the start of every step and after projection

(`enforce_geometry_no_slip!`). Solid surfaces on cell faces are included as well as cut walls (issue 79). The aligned-wall manufactured Helmholtz solution now converges at second order on uniform and composite meshes in the maximum, L^2 and L^1 norms (issue 106; `test/operators/embedded_wall_rows.jl`). A cut-cell PLIC segment now uses its own reconstructed plane, and the wall flux of a fluid-side unknown is exact for a linear velocity, but the manufactured Helmholtz solution on a tilted wall still converges with observed L^2 orders of 1.1 to 1.4, and 0.74 on the composite grid, rather than two (issue 110; `test/operators/embedded_wall_rows.jl`). The remaining first-order error sits in the cut cells' interior contact measures, which are rebuilt from bisected one-dimensional traces: the normal component's wall rows keep an $h/(2\,|\nabla d|)$ residual, 0.091 rms at 32 cells on the two slopes; a node whose centre lies inside the solid also carries its wall piece with the flux sign of a fluid-side node, which a symmetric non-negative stiffness cannot avoid. The constructor builds when a solid cuts cells at a level transition (issue 78). Rows at a level transition can have a truncation error of order one; the derivation audit measured 0.09 for $|\Delta u| \le 0.31$ on its composite mesh, unchanged from 16 to 64 cells across while the other rows converged with order two. The solution of a Helmholtz problem with level transitions meeting no-slip or free-slip walls nevertheless converges with order at least 1.8 in the maximum norm (issue 87).

Checks (`figures/viscosity.jl`). On a uniform periodic mesh an independent five-point stencil equals `diffusion!` to 2.2×10^{-15} (relative; `uniform_laplacian_check`). On the walled composite mesh of Chapter 8, the weighted asymmetry is 0 and the smallest stiffness eigenvalue relative to the largest entry is 4.6×10^{-3} (`symmetric_laplacian_check`). On periodic, channel and walled composite fixtures and $\delta t\mu/\rho \in \{10^{-3}, 10^{-1}, 10, 10^{3}\}$, the largest backward-Euler and midpoint energy factors were 1.663 and 4.408 before the change, and $1 + 5.7 \times 10^{-13}$ and $1 + 4.5 \times 10^{-13}$ after it.

Deviation from the thesis 9.1 (Single-phase viscous operator). The thesis discretises the stress $\nabla \cdot (2\mu\mathbf{D})$ with staggered strain operators on conforming meshes [19, eq. (7.52)] and defers refinement interfaces to Van der Plas [28]. For a single phase the code uses the componentwise Laplacian $(\mu/\rho)\Delta$, equal to the stress form for constant viscosity and $\nabla \cdot \boldsymbol{u} = 0$, with shared geometric contact energies at level transitions. These are the package's own stencils.

Implementation. `StaggeredDiffusionOperator(mesh; viscosity=1, boundary=NoSlip())` (`StaggeredDiffusionOperator` (`src/operators/staggered_diffusion.jl`)) and `(geometry; viscosity, boundary)` (`StaggeredDiffusionOperator` (`src/embedded/operators.jl`)); the flow solver passes the kinematic viscosity μ/ρ (`NavierStokesSystem` (`src/navier_stokes/system.jl`)). Connections `_geometric_diffusion_connections` (`src/embedded/operators.jl`) (9.3), derivatives `_cut_cell_gradient_stencil` (`src/embedded/operators.jl`) (9.4), stiffness `_geometric_diffusion_stiffness` (`src/embedded/operators.jl`) (9.5), and $\widetilde{\mathfrak{L}}_k = -\widetilde{\mathsf{V}}^{-1}\mathsf{S}_k^{\nu}$ (9.6), walls `_wall_diffusion_connections` (`src/operators/staggered_diffusion.jl`), `_include_wall_connection` (`src/operators/staggered_diffusion.jl`) and `_wall_value` (`src/operators/staggered_diffusion.jl`) (9.8), cut cells

`_cut_cell_diffusion_component` (`src/embedded/operators.jl`) and `_embedded_wall_connections` (`src/embedded/operators.jl`) (9.10) with the segments on cell faces `_embedded_aligned_wall_segments` (`src/embedded/operators.jl`), transverse connections `_geometric_diffusion_connections` (`src/embedded/operators.jl`) with `_open_dual_trace` (`src/embedded/operators.jl`) and `_add_split_side_diffusion_connections!` (`src/embedded/operators.jl`), derivatives `_cut_cell_gradient_stencil` (`src/embedded/operators.jl`), `_cut_cell_transverse_stencils` (`src/embedded/operators.jl`), `_wall_contact_stencil!` (`src/embedded/operators.jl`) and `_level_matched_gradient` (`src/operators/staggered_diffusion.jl`) with `_minimum_norm_gradient` (`src/operators/staggered_diffusion.jl`), the clipped domain-wall measure `_wall_connection_measure` (`src/embedded/operators.jl`). `stable_timestep(operator; cfl=0.9)` returns the explicit bound $\mathtt{cfl}\,\rho/(\mu \max_f \sum_{f'} \left|(\widetilde{\mathfrak{L}}_k)_{f,f'}\right|)$, which the implicit flow step does not use. Tests: `test/operators/staggered_diffusion.jl` ("staggered diffusion stable core on uniform and AMR meshes"), `test/operators/staggered_diffusion.jl` ("composite diffusion solution convergence (#80)"), `test/operators/staggered_diffusion.jl` ("composite implicit diffusion is energy contractive (#80)"), `test/operators/staggered_diffusion.jl` ("uniform periodic staggered Laplacian convergence"), `test/operators/staggered_diffusion.jl` ("wall slip and no-slip closures"), `test/indicator/geometry.jl` ("Embedded free slip omits only physical wall-shear stresses"), `test/operators/staggered_diffusion.jl` ("moving domain walls do not move an embedded disc") and `test/operators/staggered_diffusion.jl` ("domain-wall diffusion uses only uncovered wall segments") (issue 75); cut cells `test/operators/staggered_diffusion.jl` ("cut-cell diffusion connections at an AMR transition (#78)"), `test/operators/staggered_diffusion.jl` ("cut transverse measures use overlapping traces (#87)"), `test/operators/staggered_diffusion.jl` ("wall conductance follows split dual strips (#87)"), `test/operators/staggered_diffusion.jl` ("wall-contact flux respects a linear normal profile (#87)"), `test/operators/staggered_diffusion.jl` ("cut-cell Helmholtz convergence at walls and level transitions (#87)") (order at least 1.8 in the maximum norm), `test/operators/staggered_diffusion.jl` ("orphan transverse rows are linear-exact at open walls (#99)"), `test/operators/staggered_diffusion.jl` ("open sides share the orphan closure (#99)"), `test/operators/staggered_diffusion.jl` ("inflow wall-contact closure is exact for constant wall data (#99)") and `test/operators/staggered_diffusion.jl` ("inflow wall-contact closure is first order for varying wall data (#99)") and `test/operators/staggered_diffusion.jl` ("orphan transverse rows converge on a manufactured field (#99)"), `test/two_phase/variable_staggered_diffusion.jl` ("Grid-aligned embedded walls have the cut-wall limit") (issue 79, also for the operator of Section 9.3).

9.3 Two-phase stress operator

The two-phase operator discretises $\nabla \cdot (2\mu \mathbf{D})$ as the negative adjoint of a strain-rate map. Its viscosity is, in every cell, a mixture of the phase viscosities with the conditional fraction C_c^{l} [19, §7.5.1]. The default is the harmonic (series) mixture and the thesis' geometric mixture is the `GeometricViscosityAveraging` option,

$$\mu_c = \begin{cases} \left(\dfrac{C_c^{\mathrm{l}}}{\mu^{\mathrm{l}}} + \dfrac{1 - C_c^{\mathrm{l}}}{\mu^{\mathrm{g}}} \right)^{-1} & \text{(harmonic, default)}, \\ (\mu^{\mathrm{l}})^{C_c^{\mathrm{l}}} (\mu^{\mathrm{g}})^{1 - C_c^{\mathrm{l}}} = \exp\left(C_c^{\mathrm{l}} \log \mu^{\mathrm{l}} + (1 - C_c^{\mathrm{l}}) \log \mu^{\mathrm{g}} \right) & \text{(geometric, GeometricViscosityAve} \end{cases} \tag{9.11}$$

The rows of the strain map $\mathsf{E} : \vec{\mathcal{F}} \to \mathbb{R}^{\text{rows}}$ are of three kinds:

$$\begin{aligned} &\text{(normal)} && (\mathsf{E}u)_{c,k} = \frac{1}{|c^{\mathrm{o}}|} \sum_{f \in \mathcal{F}(c) \cap \mathcal{F}^{[k]} \cap \mathcal{F}_\Omega} o_{c,f}\, a_f^{\mathrm{o}} |f|\, u_f, && \alpha_c^{\mathrm{o}} > 0,\ k = 1, 2, \\ &\text{(shear)} && (\mathsf{E}u)_v = (\mathfrak{d}_2 u_1)_v + (\mathfrak{d}_1 u_2)_v, && v \in \mathcal{V}^{\mathrm{s}},\ |\omega_v^{\mathrm{v}}| > 0, \\ &\text{(wall)} && (\mathsf{E}u)_w = u_{f(w)}, && w \text{ wall or covered unknown}, \end{aligned} \tag{9.12}$$

where u_1, u_2 are the two components. A normal row is the directional open divergence $\partial_k u_k$ of the cell; prescribed normal wall faces have zero velocity. At a Cartesian embedded wall their columns, and the adjacent covered-face columns, are eliminated from the physical strain map. Their global face slots remain isolated unit algebraic rows, with zero right-hand side and solution, so the coupled operator keeps the face-field ordering. The eliminated strain columns are retained separately for wall-reaction evaluation as viscosity changes. Wall rows exist for cut cells and uncut open cells sharing a face with a covered cell (issue 79), and for covered unknowns outside the constrained neighbourhood. Cartesian embedded-wall shear is represented by tangential wall rows; the corresponding cell contributions are omitted from vertex quadrature to avoid counting shear across the solid twice. The normal cell stress already supplies the normal flux. For free slip the tangential wall rows are omitted. The shear vertices $\mathcal{V}^{\mathrm{s}}$ are the corners of the leaf cells on the finest lattice (periodic images identified). With $\bar{h}_v$ the largest width of the cells at v, the derivative of component k along $t \neq k$ is

$$(\mathfrak{d}_t u_k)_v = \begin{cases} 0, & v \text{ on a wall normal to } \boldsymbol{e}_t,\ \texttt{FreeSlip}, \\ \dfrac{u_{f_v^+} - u_{f_v^-}}{(\boldsymbol{x}_{f_v^+} - \boldsymbol{x}_{f_v^-})_t}, & \text{a pair } f_v^{\pm} \text{ exists (see below)}, \\ \sum_j \mathfrak{s}_j u_{f_j}, & \text{otherwise (least squares)}. \end{cases} \tag{9.13}$$

The pair consists of the nearest unknowns of component k on the line through v along $\boldsymbol{e}_t$, one on each side, within the radius $3\bar{h}_v$; boundary faces count with the value zero, and at a no-slip wall normal to $\boldsymbol{e}_t$ the wall point v itself, with value zero, replaces the missing side (a one-sided difference over half a cell). Where one side has no such unknown, as at a hanging vertex for the component normal to the transition, the derivative is the weighted least-squares stencil on the at most 16 nearest unknowns of component k within

$3\bar{h}_v$ (boundary faces enter the fit with value zero and drop out of the row): the coefficients are exact for quadratics, $\sum_j \mathfrak{s}_j\, p(\boldsymbol{x}_{f_j}) = \partial_t p(\boldsymbol{x}_v)$ for every quadratic p, and minimise $\sum_j \mathfrak{s}_j^2 \left(1 + \left|\boldsymbol{x}_{f_j} - \boldsymbol{x}_v\right|_2^2 / \bar{h}_v^2\right)^2$, or are exact for affine p only if the quadratic system is rank deficient (smallest singular value below 10^{-10} of the largest). Every row that contains no wall value is exact for affine velocities (the one-sided wall differences and the rows that contain wall values only for fields that vanish at the wall); the two-point rows of a symmetric pair and the least-squares rows are also exact for quadratics, while the two-point rows of an asymmetric pair (one side coarse, one fine: stretching at a level transition) are first-order accurate. The affine fallback was not reached on the meshes of the derivation audit (`proof_review/eq-viscosity-shear/`).

Let $\mathcal{W}_v$ contain the cells whose Cartesian wall segment owns the vertex-side shear contribution. The stress weights are the measures of the remaining rows times the viscosity there,

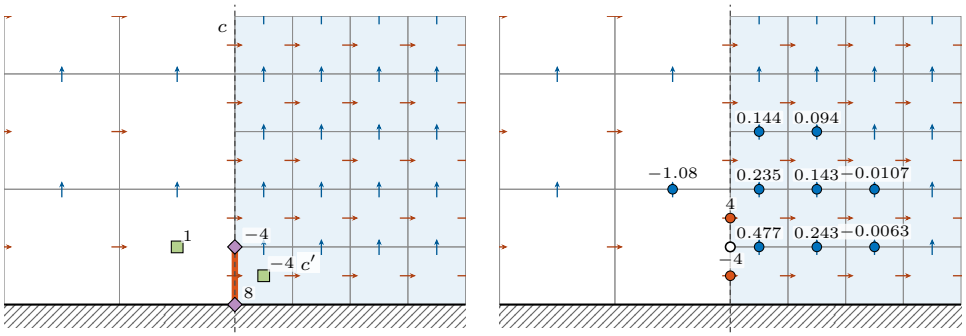
$$\mathsf{W}_\mu = \operatorname{diag}\left(2|c^{\mathrm{o}}|\mu_c,\ \ |\omega_v^{\mathrm{v}}|\mu_v,\ \ \varkappa_w \mu_{c(w)},\ \ 1_{\text{prescribed}}\right),$$
$$|\omega_v^{\mathrm{v}}| = \sum_{c \ni v,\ c \notin \mathcal{W}_v} \tfrac{1}{4}\alpha_c^{\mathrm{o}}|c|, \qquad \frac{1}{\mu_v} = \frac{1}{|\omega_v^{\mathrm{v}}|} \sum_{c \ni v,\ c \notin \mathcal{W}_v,\ \alpha_c^{\mathrm{o}} > 0} \tfrac{1}{4}\alpha_c^{\mathrm{o}}|c| \frac{1}{\mu_c}, \tag{9.14}$$

so that μ_v is the open-volume-weighted harmonic mean of the viscosities of the cell quarters the vertex spans. This is the series conductance across those quarters: the centred difference (9.13) differences two velocities through cells that lie in series, so their traction $\mu\partial_t u_k$ is transmitted exactly only if the coefficient is the harmonic mean. For a layer aligned with the shear and $\mu = (2, 1)$ the vertex coefficient is $4/3$; the geometric mean $\sqrt{2}$ overestimates the flux by $2\sqrt{\mu_1\mu_2}/(\mu_1+\mu_2)$, $6.07\,\%$ there, and the 1-D shear across such a layer converges with order 0.8–1.0 instead of 2 (issue 113). With `GeometricViscosityAveraging` the operator uses the thesis' $\prod_c \mu_c^{w_c}$, the geometric mean at the open-volume fraction, and reproduces that behaviour; both rules keep $\mu_v > 0$, so the stiffness remains symmetric positive semidefinite and the energy balance of Section 9.4 holds for either. Cartesian embedded-wall rows use the owned segment length divided by the wall-normal distance as in the componentwise operator; a non-Cartesian PLIC wall retains the earlier midpoint-distance conductance, half the open fraction of the face times the segment length divided by the distance from the face centre to the segment midpoint, bounded below by a quarter of the smallest cell width. That closure, and the cut closure of (9.10), are not continuous in the tilted-wall limit (issue 110). Wall rows use the viscosity of the cell of their solid segment; covered unknowns outside the constrained wall neighbourhood retain their algebraic penalty and the viscosity of $c^-(f)$. The integrated stiffness and the operator returned by `diffusion!` are

$$\mathsf{E}^{\mathsf{T}}\mathsf{W}_\mu\mathsf{E}, \qquad -\widetilde{\mathsf{V}}^{-1}\mathsf{E}^{\mathsf{T}}\mathsf{W}_\mu\mathsf{E}\,u, \tag{9.15}$$

with $\widetilde{\mathsf{V}} = \operatorname{diag}(|\omega_f|)$, or $\operatorname{diag}\left(|\omega_f^{\mathrm{o}}|\right)$ (one where it vanishes) with a solid; `diffusion!` returns zero at prescribed faces. The stiffness is symmetric positive semidefinite, and its quadratic form is the discrete dissipation

$$u^{\mathsf{T}}\mathsf{E}^{\mathsf{T}}\mathsf{W}_\mu\mathsf{E}u = \sum_{c,k} 2\mu_c|c^{\mathrm{o}}|(\partial_k u_k)_c^2 + \sum_v \mu_v|\omega_v^{\mathrm{v}}|\left((\mathfrak{d}_2 u_1)_v + (\mathfrak{d}_1 u_2)_v\right)^2 + \sum_w \varkappa_w \mu_{c(w)} u_{f(w)}^2 + \sum_{f \in \text{prescribe}} \tag{9.16}$$



A. Rows of E that contain u_f

B. Shear row of the hanging vertex

FIGURE 9.1. The strain-rate rows (9.12) next to a level transition (dashed) and a no-slip wall (hatched), read from the code's operator on the walled composite fixture (coarse cells of width $\frac{1}{2}$ on the left). (a) The 2 normal rows (green squares, at the centres of the coarse cell c and the fine cell c') and the 2 shear rows (purple diamonds, at the vertices) whose entry for the fine segment f (thick red) is not zero, with that entry: $\pm a_f^{\circ}|f|/|c^{\circ}|$ in the normal rows, the two-point difference to the wall and across the hanging vertex in the shear rows. (b) The complete shear row of the hanging vertex (open circle): 11 unknowns, u_1 (red) by the two-point difference along the fine segments, u_2 (blue) by the least-squares stencil of (9.13), since the coarse cell has no u_2 unknown on the line through the vertex.

the counterpart of $\int 2\mu\mathbf{D} : \mathbf{D}\,\mathrm{d}V$ in (9.2), which it approximates to second order for a constant viscosity, also on composite meshes (derivation audit, `proof_review/eq-viscosity-quadratic/`); for constant viscosity and $\nabla \cdot \boldsymbol{u} = 0$ the operator approximates $\mu\Delta\boldsymbol{u}$ to second order on a uniform mesh (`proof_review/eq-viscosity-operator/`). The kernel of the stiffness consists of the constant components on a periodic mesh and of the constant tangential component in a channel that is periodic along free-slip walls; a no-slip wall or a solid makes it definite. The walls are part of the rows: no-slip domain walls through the one-sided shear differences, free slip through the vanishing ones, the embedded solid through the wall rows (no-slip only). Only the stationary `NoSlip()` and `FreeSlip()` are accepted; moving walls and `VelocityBoundaryConditions` are rejected. When the interface moves only W_μ changes: the code refreshes μ_c from the transported fractions before every viscous step and keeps E until the next regrid, after which unchanged shear stencils are restored from a cache keyed by their local face geometry.

Checks (`stress_rows_check`, `affine_strain_check`). The 1-D shear $\mu(y)u_1'(y) = -\cos(2\pi y)$ across an interface aligned with $y = 1/2$, whose tangential traction is continuous and whose velocity derivative jumps by the viscosity ratio, converges at order 2.03, 2.01, 2.00 and 2.00 ($\mu = (2, 1)$, $n = 16 \to 128$; the equal-viscosity solve gives the same orders), while `GeometricViscosityAveraging` reproduces the reported first-order rates below 1.2 (`test/two_phase/variable_staggered_diffusion.jl` ("Aligned viscosity jump transmits the tangential traction (#113)"), `test/two_phase/variable_staggered_diffusion.jl` ("Geometric mixture keeps the first-order aligned shear (#113)")). Both rules keep the stiffness symmetric and the implicit midpoint step dissipative on a composite mesh with an aligned jump

(`test/two_phase/variable_staggered_diffusion.jl` ("Harmonic vertex viscosity stays dissipative (#113)")). An independent assembly of the normal rows (9.12) equals the code's to 0 on the composite, cut and periodic composite drop fixtures ($\mu = (0.1, 0.002)$); the independent weights (9.14) of all normal and 697 shear rows (composite fixture) agree to 1.9×10^{-15} (relative); the cut fixture has 272 wall rows. For the six affine velocities and both constants, the 461 rows more than four coarse cells from the domain edges return the exact strain rates to 4.6×10^{-15}, on the walled and on the periodic composite mesh.

Deviation from the thesis 9.2 (Two-phase stress rows). The thesis locates the off-diagonal strain at the edges of the cells and needs no interpolation on a rectilinear mesh, with the viscosity from the volume fractions interpolated to the edges [19, eq. (7.52), §7.5.1]. The code uses the conditional fractions, places the shear rows at the cell corners with the open quarter-cell volumes, and replaces the two-point difference by the least-squares stencil (9.13) where a side of the line through a corner has no unknown, as at hanging vertices; it closes no-slip walls by one-sided differences and adds wall rows for the embedded solid. The cell and vertex mixtures are harmonic (series) by default, (9.11) and (9.14); the thesis' volume-fraction-weighted geometric mean remains available as `GeometricViscosityAveraging` through the `viscosity_averaging` option of `NavierStokesOptions`, because it is a diffuse model whose vertex coefficient does not transmit the tangential traction across an aligned interface at second order (issue 113).

Implementation. `VariableStaggeredDiffusionOperator`, built by `StaggeredDiffusionOperator(interface; viscosity=(mu_l, mu_g), boundary=NoSlip())` in `_variable_staggered_diffusion` (`src/two_phase/diffusion/variable_diffusion_operator.jl`) (9.12) and (9.14); `dynamic_viscosity` and `AbstractViscosityAveraging` (`HarmonicViscosityAveraging` (`src/operators/viscosity_averaging.jl`), `GeometricViscosityAveraging` (`src/operators/viscosity_averaging.jl`)) (9.11); stress-row mixtures `_fill_stress_weights!` (`src/two_phase/diffusion/variable_diffusion_operator.jl`) (9.14); vertices `_stress_vertices!` (`src/two_phase/diffusion/variable_diffusion_geometry.jl`), shear stencils `_vertex_derivative` (`src/two_phase/diffusion/variable_diffusion_geometry.jl`) (cached by `_cached_vertex_derivative!` (`src/two_phase/diffusion/variable_diffusion_geometry.jl`)) (9.13); embedded wall rows `_embedded_variable_wall_stresses` (`src/embedded/operators.jl`); stiffness `_symmetric_stiffness!` (`src/two_phase/diffusion/variable_diffusion_geometry.jl`); refresh `_update_dynamic_viscosity!` (`src/two_phase/diffusion/variable_diffusion_operator.jl`); action `_factorized_variable_stiffness_mul!` (`src/two_phase/diffusion/variable_diffusion_operator.jl`). Tests: `test/two_phase/variable_staggered_diffusion.jl` ("Factorized variable-stress action matches assembled operator"), `test/two_phase/variable_staggered_diffusion.jl` ("Phase-viscosity averaging rules (#113)"), `test/two_phase/variable_staggered_diffusion.jl` ("Variable coupled stress diffusion"), `test/two_phase/variable_staggered_diffusion.jl` ("Variable

stress manufactured bulk convergence"),
`test/two_phase/variable_staggered_diffusion.jl` ("variable diffusion is independent of base-root partitioning"),
`test/operators/embedded_variable_wall_mms.jl` ("two-phase embedded-wall manufactured Helmholtz (#106)"),
`test/operators/embedded_variable_wall_mms.jl` ("two-phase aligned-wall constraints and reactions (#106)").

9.4 Fully discrete update: implicit midpoint solves

Single phase. With $u^{(n)}$ the old velocity with its boundary values at $t^{(n)}$ and u^* the transported velocity after the body acceleration and the boundary values at $t^{(n)} + \frac{1}{2}\delta t$ ((8.21)), each component solves

$$\left(\mathsf{I} - \tfrac{1}{2}\delta t \tfrac{\mu}{\rho}\widetilde{\mathfrak{L}}_k\right) u^{**} = u^* + \tfrac{1}{2}\delta t \tfrac{\mu}{\rho}\widetilde{\mathfrak{L}}_k u^{(n)} + \delta t \tfrac{\mu}{\rho}\widetilde{\mathsf{V}}^{-1}\sum_w \varkappa_w u_w^{\Gamma}\big(t^{(n)} + \tfrac{1}{2}\delta t\big), \tag{9.17}$$

a Crank–Nicolson step whose explicit half is evaluated at the old velocity $u^{(n)}$, not at the transported one, and whose wall data are taken at the midpoint. The system is multiplied by $\widetilde{\mathsf{V}}$ and solved by CG if $\widetilde{\mathsf{V}}\widetilde{\mathfrak{L}}_k$ is symmetric for every shift, including the embedded conservative operator, otherwise by BiCGSTAB, both preconditioned by default with the rediscretised tree multigrid of Chapter 11; a different step size only refreshes the shift.

Two phases, one velocity. With the mass matrix $\mathsf{M}_\rho = \operatorname{diag}\left(\widetilde{\rho}_f \widetilde{V}_f\right)$ of the transported fractions ((8.4); open volumes with a solid) and the viscosity refreshed from the same fractions,

$$\left(\mathsf{M}_\rho + \tfrac{1}{2}\delta t\, \mathsf{E}^{\mathsf{T}}\mathsf{W}_\mu \mathsf{E}\right) u^{**} = \mathsf{M}_\rho u^* - \tfrac{1}{2}\delta t\, \mathsf{E}^{\mathsf{T}}\mathsf{W}_\mu \mathsf{E}\, u^*, \tag{9.18}$$

the implicit midpoint rule about the transported velocity, as in the thesis' step T3.I with the stress [19, eq. (7.52)]; the matrix is symmetric positive definite, solved by CG with the factorised action $\mathsf{M}_\rho u + \frac{1}{2}\delta t \mathsf{E}^{\mathsf{T}}(\mathsf{W}_\mu(\mathsf{E}u))$ and the rediscretised multigrid of Chapter 11. If both viscosities are zero no operator is built and $u^{**} = u^*$; one zero viscosity is rejected, as is a non-positive single-phase viscosity.

Stopping criterion. With r the residual and b the right-hand side of the integrated system and m_f the diagonal of $\widetilde{\mathsf{V}}$ (single phase) or M_ρ (two phases), both solves stop when $\max_f |\mathsf{r}_f| / \mathsf{m}_f \le \max(\epsilon_{\mathrm{tol}} \max_f |\mathsf{b}_f| / \mathsf{m}_f,$ round-off floor): a velocity defect relative to the velocity scale of the right-hand side, with the step tolerance ϵ_{tol} (default $\epsilon_{\mathrm{mach}}^{1/2}$) and at most 100 (one phase) or 500 (two phases) iterations; otherwise the step raises an error.

Identities. Let $\mathsf{r} = \mathsf{b} - \left(\mathsf{M}_\rho + \frac{1}{2}\delta t\, \mathsf{E}^{\mathsf{T}}\mathsf{W}_\mu \mathsf{E}\right) u^{**}$ be the residual of (9.18), with the right-hand side $\mathsf{b} = \left(\mathsf{M}_\rho - \frac{1}{2}\delta t\, \mathsf{E}^{\mathsf{T}}\mathsf{W}_\mu \mathsf{E}\right) u^*$, as in the stopping criterion. Then $\mathsf{M}_\rho(u^{**} - u^*) = -\frac{1}{2}\delta t\, \mathsf{E}^{\mathsf{T}}\mathsf{W}_\mu \mathsf{E}\,(u^* + u^{**}) - \mathsf{r}$, and multiplying by $u^* + u^{**}$ gives the energy balance

$$\tfrac{1}{2}(u^{**})^{\mathsf{T}}\mathsf{M}_\rho u^{**} - \tfrac{1}{2}(u^*)^{\mathsf{T}}\mathsf{M}_\rho u^* = -\tfrac{1}{4}\delta t\,(u^* + u^{**})^{\mathsf{T}}\mathsf{E}^{\mathsf{T}}\mathsf{W}_\mu \mathsf{E}\,(u^* + u^{**}) - \tfrac{1}{2}(u^* + u^{**})^{\mathsf{T}}\mathsf{r}\,: \tag{9.19}$$

the viscous step does not increase the mass-weighted kinetic energy $\sum_f \frac{1}{2}\widetilde{\rho}_f \widetilde{V}_f u_f^2$, for every δt, up to the residual, the discrete counterpart of (9.2); the residual term is at most $\frac{1}{2}\max_f(|\mathsf{r}_f| / \mathsf{m}_f)\sum_f \mathsf{m}_f \left|u_f^* + u_f^{**}\right|$, with the maximum bounded by the stopping criterion. The balance uses only that M_ρ is diagonal and positive and that the stiffness is

symmetric positive semidefinite, which $\mathsf{E}^{\mathsf{T}}\mathsf{W}_{\mu}\mathsf{E}$ is by construction. It is an equality: on an eigenvector of $\mathsf{M}_{\rho}^{-1}\mathsf{E}^{\mathsf{T}}\mathsf{W}_{\mu}\mathsf{E}$ with eigenvalue λ an exact solve multiplies the energy by $\big((1-\frac{1}{2}\delta t\lambda)/(1+\frac{1}{2}\delta t\lambda)\big)^2$, so the kernel keeps its energy and stiff modes are hardly damped for large δt (the midpoint rule is not L-stable). On a periodic mesh without solid a constant component has no strain, so $\mathsf{E}\mathbf{1}^{[k]} = 0$ for the indicator $\mathbf{1}^{[k]}$ of $\mathcal{F}^{[k]}$ and, by symmetry,

$$\sum_{f\in\mathcal{F}^{[k]}} \widetilde{\rho}_f \widetilde{V}_f u_f^{**} - \sum_{f\in\mathcal{F}^{[k]}} \widetilde{\rho}_f \widetilde{V}_f u_f^{*} = -(\mathbf{1}^{[k]})^{\mathsf{T}}\mathsf{r}\,: \tag{9.20}$$

the momentum is conserved up to the residual. A solid exchanges momentum through its wall rows; a wall exchanges the momentum of the component normal to it through the normal rows, which omit the boundary faces (zero wall-normal velocity), and a no-slip wall that of the tangential component through the wall-adjacent shear differences, while a free-slip wall conserves the tangential momentum on a mesh that is periodic along it. The single-phase step (9.17) conserves momentum on any periodic composite mesh for any explicit argument, since $(\mathbf{1}^{[k]})^{\mathsf{T}}\widetilde{\mathsf{V}}\widetilde{\mathfrak{L}}_k = 0$. Its backward-Euler resolvent is contractive for every step size, and its midpoint update obeys the same energy balance when $u^{(n)} = u^*$. The actual split step evaluates the explicit half at $u^{(n)}$ and therefore has no such balance when these velocities differ.

Two velocities. The two-velocity step diffuses the mass-weighted mixture of the transported intrinsic velocities with the operator of (9.15), including its viscosity mixture, and gives both phases the same increment [19, §7.5.2, step T3.II]:

$$\begin{aligned} \bar{u}_f^{*} &= \frac{\rho^{\mathrm{l}}\hat{C}_f\, u_f^{\mathrm{l},*} + \rho^{\mathrm{g}}(1-\hat{C}_f)\, u_f^{\mathrm{g},*}}{\widetilde{\rho}_f}, \qquad \bar{u}^{**} \text{ from (9.18) with } \bar{u}^{*}, \\ u_f^{\pi,**} &= u_f^{\pi,*} + \left(\bar{u}_f^{**} - \bar{u}_f^{*}\right), \qquad \pi \in \{\mathrm{l},\mathrm{g}\}, \end{aligned} \tag{9.21}$$

with the clamped staggered fraction (8.4), a product with an absent (NaN) phase taken as zero, $\bar{u}_f^{*} = 0$ on faces without open volume, and absent values left absent. The velocity jump $u_f^{\mathrm{g}} - u_f^{\mathrm{l}}$ is unchanged and, on the faces with open volume, the mixture of the new phase velocities is $\bar{u}^{**}$; on a covered face the mixture is zero by definition, while $\bar{u}^{**}$ is the value that the penalty rows of the covered unknowns leave there (at most 2.6×10^{-5} of the largest value in the cut fixture; derivation audit, `proof_review/eq-viscosity-two-velocity/`). The combined momentum $\sum_f |\omega_f^{\mathrm{o}}|\left(\rho^{\mathrm{l}}\hat{C}_f u_f^{\mathrm{l}} + \rho^{\mathrm{g}}(1-\hat{C}_f)u_f^{\mathrm{g}}\right)$, with absent values counted as zero and $|\omega_f^{\mathrm{o}}| = |\omega_f|$ without a solid, therefore changes exactly as the mixture momentum $\sum_f \widetilde{\rho}_f|\omega_f^{\mathrm{o}}|\bar{u}_f$, on a periodic mesh without solid as in (9.20), and a continuous velocity, $u^{\mathrm{l}} = u^{\mathrm{g}}$, is diffused exactly as in the one-velocity model.

Checks (`midpoint_check`, `two_velocity_viscosity_check`, `single_phase_midpoint_check`). On the composite, cut and periodic composite drop fixtures (densities $(1000, 1)$, $\mu = (0.1, 0.002)$, $\delta t = 0.05$, tolerance 10^{-13}), (9.18) holds to 3.7×10^{-16} (strong residual relative to the velocity), (9.19) to 1.5×10^{-11} of the dissipation, and on the periodic mesh the momentum changes by 2.1×10^{-16} (relative) while $\mathsf{E}^{\mathsf{T}}\mathsf{W}_{\mu}\mathsf{E}\mathbf{1}^{[k]}$ is 2.6×10^{-16} of the largest stiffness entry. For two phase velocities that differ, the mixture (9.21) equals an independent evaluation to 0, the jump changes by

2.0×10^{-16} and the mixture of the updated phases equals $\bar{u}^{**}$ to 1.7×10^{-16} (relative). The single-phase solve satisfies (9.17) to 3.4×10^{-16} on the walled composite mesh.

Deviation from the thesis 9.3 (Single-phase diffusion time level). The thesis' implicit midpoint step T3 acts on the transported velocity u^* in both halves [19, §7.2, eq. (7.52)], which the two-phase step (9.18) follows. The single-phase step (9.17) evaluates the explicit half at the pre-transport velocity $u^{(n)}$, which loses the energy balance (9.19) also where $\widetilde{\mathrm{V}}\widetilde{\mathfrak{L}}_k$ is symmetric (it misses by 3 % in a uniform periodic test of the derivation audit, `proof_review/eq-viscosity-single-midpoint/`).

Implementation. `implicit_midpoint_diffusion!(destination, convected, old_velocity, operator, dt, solver, work; time, tolerance, max_cycles)` (9.17) in `implicit_midpoint_diffusion!` (`src/navier_stokes/implicit_diffusion.jl`) with `_add_wall_source!` (`src/navier_stokes/implicit_diffusion.jl`), the shift `_update_staggered_elliptic!` (`src/navier_stokes/implicit_diffusion.jl`) of `StaggeredEllipticOperator(operator; reaction=1, diffusion=dt*nu/2)` and `StaggeredEllipticSolver`; the two-phase form (9.18) in `implicit_midpoint_diffusion!` (`src/two_phase/diffusion/variable_diffusion_solver.jl`) with `VariableStaggeredDiffusionSolver` (`src/two_phase/diffusion/variable_diffusion_solver.jl`); stopping criterion `_solve_krylov_with_stats!` (`src/solvers/multigrid/elliptic_solver.jl`); stages `_diffusion_substep!` (`src/single_phase/stepping.jl`), `_diffusion_substep!` (`src/two_phase/stepping.jl`) and `_diffusion_substep!` (`src/two_phase/two_velocity/stepping.jl`) with `_mass_weighted_one_velocity!` (`src/two_phase/two_velocity/stepping.jl`) and `_apply_two_velocity_diffusion_update!` (`src/two_phase/two_velocity/stepping.jl`) (9.21). Tests: `test/operators/staggered_diffusion.jl` ("implicit midpoint staggered elliptic solve"), `test/operators/staggered_diffusion.jl` ("dimensionless diffusion tolerance and failure report"), `test/navier_stokes/two_phase_navier_stokes.jl` ("Two-phase diffusion uses the thesis split state") (the step equals the direct solve of (9.18) about the transported state), `test/two_phase/variable_staggered_diffusion.jl` ("Variable stress shifted solve on fixed and adapted trees"), `test/two_phase/two_velocity/diffusion.jl` ("Two-velocity implicit diffusion identities"), `test/two_phase/two_velocity/diffusion.jl` ("Two-velocity diffusion recovers the continuous-velocity limit"), `test/navier_stokes/navier_stokes.jl` ("single-phase material scaling").

— 10 —

Pressure and velocity constraints

Equation reference, audited against commit `bc0da10`.

Every time step ends with one pressure projection ((8.21)): the predictor u^{***}, which carries the transport, the viscous step, the forces and the boundary values at $t^{(n+1)}$, is corrected by a pressure gradient so that the end-of-step velocity satisfies the discrete incompressibility constraint and, with two velocities, a discrete condition on the normal velocity jump at the interface; this is step T5 of the thesis [19, §7.2]. Two projections are implemented: the Poisson projection of a shared velocity (one phase, or two phases with one velocity), and the projection of the intrinsic phase velocities by the conservative multidimensional ghost-fluid method (CMDGFM) [19, §7.3]. Both are incremental: the step first subtracts the gradient of the previous pressure from the predictor, and the projection, written in Sections 10.2 to 10.4 for a predictor u^{***} and a pressure p, then solves for the pressure increment (Section 10.5.2). This chapter states the continuous constraints, the discrete constraints and pressure maps, the matrices and right-hand sides of the two systems with their gauges and the rules that select the jump unknowns and rows, the properties that hold exactly (divergence and energy identities, momentum identity, diagonal dominance), and then the fully discrete projection stage: boundary values, the pressure update, the defects compared with the tolerance and the projection after a regrid. The operators are those of Chapter 3, the pressure coefficients and force maps those of Chapter 8; the linear solvers are named here and described in Chapter 11.

10.1 Continuous model

Continuous in space, the projection step of the thesis updates each phase velocity by its pressure gradient [19, eq. (7.8)],

$$\boldsymbol{u}^{\pi,(n+1)} = \boldsymbol{u}^{\pi,***} - \frac{\delta t}{\rho^{\pi}} \nabla p^{\pi} \quad \text{in } \Omega^{\circ} \cap \Omega^{\pi}, \qquad \nabla \cdot \boldsymbol{u}^{\pi,(n+1)} = 0, \tag{10.1}$$

and in the one-velocity model the same update holds with the one velocity $\boldsymbol{u}$ and the piecewise constant density ρ in place of $\boldsymbol{u}^{\pi}$ and ρ^{π}. The phase velocities are continuous in the interface normal direction, $[\![\boldsymbol{u} \cdot \boldsymbol{\eta}]\!] = 0$ [19, eq. (6.1)], but not tangentially; imposed on (10.1) this gives the jump condition [19, eq. (7.9)]

$$\left[\!\!\left[\frac{1}{\rho} \boldsymbol{\eta} \cdot \nabla p \right]\!\!\right] = \frac{[\![\boldsymbol{u}^{***} \cdot \boldsymbol{\eta}]\!]}{\delta t}, \tag{10.2}$$

which the two-velocity projections discretise. Integrated over a control volume the two velocities satisfy $\int_{\partial c} \boldsymbol{u}\cdot\boldsymbol{n}\,\mathrm{d}S = 0$, each phase velocity on its own part of ∂c [19, eq. (6.15)]. The pressure carries no value jump in the projection: surface tension enters through the predictor (deviation 8.2), so the Young–Laplace jump is a property of the pressure that balances it, not a condition of the solve. At walls and inflows the one-velocity projection preserves the prescribed predictor flux and uses a homogeneous Neumann pressure increment. At `Outflow` and `ZeroGradientVelocity` faces it instead uses a zero pressure increment at the boundary and corrects the normal velocity there. The two-velocity projections retain their prescribed phase boundary fluxes; each connected component must then have zero total mixture flux. For any velocity with $\boldsymbol{u}\cdot\boldsymbol{n} = 0$ on the boundary and a pressure without value jump, $[\![p]\!] = 0$, the pressure work splits as (cf. [19, eq. (7.24)])

$$-\int_\Omega \boldsymbol{u}\cdot\nabla p\,\mathrm{d}V = \int_\Omega p\,\nabla\cdot\boldsymbol{u}\,\mathrm{d}V + \int_I p\,[\![\boldsymbol{u}\cdot\boldsymbol{\eta}]\!]\,\mathrm{d}S, \tag{10.3}$$

by the divergence theorem in each phase, with the outward normal $\boldsymbol{\eta}$ of the liquid and $-\boldsymbol{\eta}$ of the gas at the interface (deviation 10.1). The pressure therefore does no work on a solenoidal field whose normal velocity is continuous at the interface. CMDGFM imposes normal continuity through its jump rows and a mixture-divergence target (Section 10.4).

Deviation from the thesis 10.1 (Sign of the interface pressure work). The thesis prints the interface term of (10.3) with a minus sign [19, eq. (7.24)]. With its own conventions, $[\![\cdot]\!]$ the gas minus the liquid value and $\boldsymbol{\eta}$ pointing into the gas [19, §2.1], the liquid contributes $-\int_I p\,\boldsymbol{u}^{\mathrm{l}}\cdot\boldsymbol{\eta}\,\mathrm{d}S$ and the gas $+\int_I p\,\boldsymbol{u}^{\mathrm{g}}\cdot\boldsymbol{\eta}\,\mathrm{d}S$, so the sign is a plus: the printed minus is a typographical error, confirmed by the derivation audit with exact rational integration on a polynomial example (`proof_review/eq-projection-continuous/`). The thesis uses the identity only to motivate its mimetic gradient; its discrete statements [19, eqs. (7.38)–(7.44)] do not depend on the printed sign. The code does not implement that gradient operator.

10.2 One velocity: the Poisson projection

The end-of-step velocity is made discretely solenoidal in the open divergence (3.13) ($\mathfrak{D}^{\mathrm{o}} = \mathfrak{D}$ without solid) and is corrected by the pressure map of Chapter 8: the weighted gradient $\mathfrak{S}_\gamma$, or $\mathfrak{S}^{\mathrm{o}}$ with a solid, with the coefficient $\gamma_f = 1/\widetilde{\rho}_f$, or $(a_f^{\mathrm{o}})^2/\widetilde{\rho}_f$ with a solid (8.5) ($1/\rho$ for one phase). Written with the force map, $\mathsf{R}\mathsf{J}p$ is this gradient ((8.7)), and with the face metric $\mathsf{K}^{\mathrm{o}} = \mathsf{W}\mathsf{K}\mathsf{W}$ ($\mathsf{K}^{\mathrm{o}} = \mathsf{K}$ without solid) the interior integrated stiffness is $\mathsf{J}^{\mathsf{T}}\mathsf{G}^{1/2}\mathsf{K}^{\mathrm{o}}\mathsf{G}^{1/2}\mathsf{J}$ ((3.26)). For open boundaries the stiffness S includes the positive diagonal flux contributions of the zero-pressure-increment boundary faces, and $\mathcal{G}$ denotes the corresponding weighted gradient on both interior and boundary faces. The projection solves

$$\mathsf{S}p = \mathsf{b}, \qquad \mathsf{b}_c = -\frac{1}{\delta t}|c^{\mathrm{o}}|\left(\mathfrak{D}^{\mathrm{o}}u^{***}\right)_c, \qquad u_f^{(n+1)} = u_f^{***} - \delta t\,(\mathcal{G}p)_f. \tag{10.4}$$

The gradient $\mathcal{G}$ is the force map on interior faces, vanishes at walls and inflows, and uses the distance from the adjacent cell centre to the zero-increment value at the open face. Its divergence includes exactly the same open-face flux as the stiffness. This is the strong-form problem $-\mathfrak{D}^{\mathrm{o}}\mathfrak{S}^{\mathrm{o}}p = -\mathfrak{D}^{\mathrm{o}}u^{***}/\delta t$ multiplied by the open cell volumes $\mathsf{V} = \mathrm{diag}\,(|c^{\mathrm{o}}|)$;

covered cells carry the identity row and get $p_c = 0$ (Section 3.7). For each connected fluid component $\mathcal{K}$ without an open pressure face or inactive Dirichlet anchor, its indicator is a null vector of S. The right-hand side is made compatible and the gauge fixed separately on each such component:

$$\mathsf{b}_c \leftarrow \mathsf{b}_c - \bar{\mathsf{b}}_{\mathcal{K}} |c^{\mathrm{o}}|, \qquad \bar{\mathsf{b}}_{\mathcal{K}} = \frac{\sum_{c\in\mathcal{K}} \mathsf{b}_c}{\sum_{c\in\mathcal{K}} |c^{\mathrm{o}}|}, \qquad \sum_{c\in\mathcal{K}} |c^{\mathrm{o}}|\, p_c = 0. \tag{10.5}$$

The removed component means and their smallest cell indices are reported in `removed_rhs_comp` and `removed_rhs_component_anchors`; `removed_rhs_mean` is their volume-weighted mean over unanchored components. With `compatibility=:error`, the solve throws if any unanchored component has $\left|\sum_{c\in\mathcal{K}} \mathsf{b}_c\right| > 256\epsilon_{\mathrm{mach}} \max(1, \sum_{c\in\mathcal{K}} |\mathsf{b}_c|)$. By the discrete divergence theorem (3.12), each such sum is $-1/\delta t$ times its boundary flux. A nonzero removed mean therefore represents a real flux imbalance, contributing a uniform divergence on that component in addition to the solve residual; the pressure correction cannot change its prescribed flux. Components touching a positive open-pressure conductance have no constant nullspace, so their means are not removed and the outlet flux can adjust during projection. The projection reports `compatible=false` when $\delta t^2 \max_{\mathcal{K}} \left|\bar{\mathsf{b}}_{\mathcal{K}}\right| > \varepsilon^p$ and warns once per projection workspace, independently of whether the projected linear system converged. The one-velocity solve and its multigrid hierarchy handle component nullspaces individually (issue 95, issue 97). The two-velocity projections instead fix one anchor per component ((10.16)).

Exact identities. Let $\mathsf{r} = \mathsf{b} - \mathsf{S}p$ be the residual of the compatible right-hand side. For a full reference step, before any subsequent boundary-value reset, every open cell satisfies

$$|c^{\mathrm{o}}| \left(\mathfrak{D}^{\mathrm{o}} u^{(n+1)}\right)_c = -\delta t \left(\mathsf{r}_c + \bar{\mathsf{b}}_{\mathcal{K}} |c^{\mathrm{o}}|\right), \qquad c \in \mathcal{K}, \tag{10.6}$$

since $-\mathsf{V}\mathfrak{D}^{\mathrm{o}}\mathcal{G} = \mathsf{S}$ (extending Section 3.7 to the open faces), with $\bar{\mathsf{b}} = 0$ when the pressure boundary fixes the gauge: the residual of the solve is, up to the factor $-\delta t$, the volume flux left in the cell. The solve stops when the strong residual satisfies

$$\delta t^2 \max_c \frac{|\mathsf{r}_c|}{|c^{\mathrm{o}}|} \le \max\left\{\varepsilon^p,\ \delta t^2 \epsilon_{\mathrm{fl}}\right\}, \tag{10.7}$$

over the cells with $|c^{\mathrm{o}}| > 0$, with the dimensionless tolerance ε^p (`tolerance`, $0 < \varepsilon^p < 1$, default $\sqrt{\epsilon_{\mathrm{mach}}}$) and the round-off floor ϵ_{fl} of the linear solver, $128\epsilon_{\mathrm{mach}}$ times the largest strong magnitude of the terms of $\mathsf{S}p$ and of b (at least $128\epsilon_{\mathrm{mach}}$). The initial guess defines the initial floor. The current-iterate floor may increase the stop level only when the true residual has decreased and the new floor is less than one hundredth of the initial residual; an inflated floor from a diverging iterate is rejected (issue 97). For $\varepsilon^p < \delta t^2 \epsilon_{\mathrm{fl}}$ the floor is the stop level. A solve that misses the rule is reported (`converged=false`), not thrown (Section 10.5.3). By (10.6), $\delta t \max_c \left|(\mathfrak{D}^{\mathrm{o}} u^{(n+1)})_c\right|$ equals the left-hand side when every $\bar{\mathsf{b}}_{\mathcal{K}} = 0$: the tolerance bounds the fraction of its volume that any cell gains or loses in one step through the remaining divergence, a quantity that does not depend on the initial residual or on the units. With prescribed boundary flux the projection is moreover an orthogonal projection. On the faces $\{\, f \mid \gamma_f > 0 \,\}$ let $\mathsf{K}^{\mathrm{o}}_{++}$ be the principal submatrix of K^{o} and

$$\mathsf{M}_\gamma = \mathsf{A}\mathsf{G}^{-1/2} \left(\mathsf{K}^{\mathrm{o}}_{++}\right)^{-1} \mathsf{G}^{-1/2}\mathsf{A}, \qquad u^{\mathsf{T}}\mathsf{M}_\gamma\, \mathsf{R}\mathsf{J}p = u^{\mathsf{T}}\mathsf{A}\mathsf{J}p, \tag{10.8}$$

($\mathsf{A} = \mathsf{I}$ without solid). Hence, if $\mathfrak{D}^{\mathrm{o}} u^{(n+1)} = 0$ and the boundary values vanish, $\|u^{***}\|^2_{\mathsf{M}_\gamma} = \|u^{(n+1)}\|^2_{\mathsf{M}_\gamma} + \delta t^2 \|\mathsf{R}\mathsf{J}p\|^2_{\mathsf{M}_\gamma}$, by the discrete adjoint relation (3.18): the projection removes exactly the kinetic energy of its correction. Where K^{o} is diagonal, M_γ is the diagonal of the staggered masses $\widetilde{\rho}_f |\omega_f|$ (no solid) and $\widetilde{\rho}_f |\omega_f^{\mathrm{o}}|$ (one velocity with a solid); for the two-velocity coefficient $\gamma^{\mathrm{tv}} = a_f^{\mathrm{o}} / \widetilde{\rho}_f$ it is $a_f^{\mathrm{o}} \widetilde{\rho}_f |\omega_f^{\mathrm{o}}|$, the kinetic norm minimised by the two-velocity projections on cut faces. With nonzero prescribed boundary values and no open pressure boundary, pressure does work at the boundary,

$$\|u^{***}\|^2_{\mathsf{M}_\gamma} = \|u^{(n+1)}\|^2_{\mathsf{M}_\gamma} + \delta t^2 \|\mathsf{R}\mathsf{J}p\|^2_{\mathsf{M}_\gamma} + 2\delta t \sum_{f \in \mathcal{F}_\Gamma} s_f |f| a_f^{\mathrm{o}} u_f p_{c(f)},$$

where $u^{(n+1)}$ is the corrected velocity and u_f the prescribed normal boundary velocity. An open pressure boundary adds boundary face corrections to the kinetic norm, so this interior-face identity is not used there.

Checks (`figures/projection.jl`, `one_velocity_check`). On the drop fixture of Table 10.1 (densities $(1000, 1)$, so γ varies by three decades) on the uniform and composite meshes, without solid and with the sloped solid, one projection of a smooth divergent predictor at $\varepsilon^p = 10^{-12}$ satisfies (10.6) cell by cell to 4.0×10^{-16} of the predictor's divergence, leaves $\delta t \max_c |(\mathfrak{D}^{\mathrm{o}} u)_c| \le 4.4 \times 10^{-13}$, keeps the volume mean of the pressure at 3.7×10^{-17} of $\sum_c |c^{\mathrm{o}}|\,|p_c|$, and the energy identity, evaluated outside the projection with the code's metric and coefficient, holds to 3.6×10^{-16} (relative), while the projection removes 17–60 % of the energy.

Implementation. `project!(velocity, pressure, solver, dt, workspace[, geometry]; velocity_boundary=nothing, boundary_time=0, tolerance=sqrt(eps(T)), max_cycles=100, compatibility=:project)` with `ProjectionWorkspace` and an `EllipticSolver` whose operator has zero reaction and unit diffusion; returns the `LinearSolverStats` of the solve (`residual_scale`$= 1/\delta t^2$) and adds the solution to `pressure` (Section 10.5.2). For open boundaries the standalone caller constructs the operator with the same `velocity_boundary` and supplies the prepared predictor boundary values; inconsistent boundary configurations are rejected. Implemented in `project!` (`src/navier_stokes/projection.jl`) and, with a solid, `project!` (`src/embedded/operators.jl`) with `_cut_cell_weighted_gradient!` (`src/embedded/operators.jl`); compatibility and gauge in `solve!` (`src/solvers/multigrid/elliptic_solver.jl`), `_project_component_rhs!` (`src/solvers/multigrid/elliptic_solver.jl`) and `_remove_component_volume_means!` (`src/solvers/multigrid/elliptic_solver.jl`); tolerance `_dimensionless_tolerance` (`src/solvers/multigrid/stats.jl`). The operator is `PoissonOperator(interface[, geometry]; density)` (Section 8.2). Tests: `test/operators/poisson.jl` ("pressure projection") ($\delta t \max |\mathfrak{D} u| = \delta t^2 \|\mathsf{r}\|_\infty$ to $8\epsilon_{\mathrm{mach}}$, zero pressure mean, composite mesh and variable coefficient), `test/operators/poisson.jl` ("mixed-level no-permeation projection"), `test/operators/poisson.jl` ("periodic nullspace convention") (removed mean, `compatibility=:error`), `test/operators/poisson.jl` ("dimensionless pressure tolerance validation").

10.3 Two velocities: constraints and phase gradients

The two-velocity projections act on the intrinsic phase velocities $u^{\mathrm{l}}, u^{\mathrm{g}}$. A phase is present on an interior face if its staggered volume is positive, $\hat{C}_f > 0$ for the liquid and $\hat{C}_f < 1$ for the gas (and $|\omega_f^{\mathrm{o}}| > 0$ with a solid), with the clamped staggered liquid fraction $\hat{C}_f$ of (8.4); an absent phase velocity is NaN and is never evaluated: every product with a zero weight below is zero. The face data of a projection, computed from the current reconstruction, are the staggered phase masses per volume and the coefficients of the thesis' gradient with the conservative distance $\widetilde{\phi}^{\pi} = \widetilde{\alpha}^{\pi}$ [19, eqs. (4.51), §7.3.7],

$$\begin{gathered}\widetilde{m}_f^{\mathrm{l}} = \rho^{\mathrm{l}}\hat{C}_f, \quad \widetilde{m}_f^{\mathrm{g}} = \rho^{\mathrm{g}}\left(1-\hat{C}_f\right), \quad \widetilde{m}_f = \widetilde{m}_f^{\mathrm{l}} + \widetilde{m}_f^{\mathrm{g}} = \widetilde{\rho}_f, \quad \widetilde{m}_f^{\mathrm{r}} = \frac{\widetilde{m}_f^{\mathrm{l}}\widetilde{m}_f^{\mathrm{g}}}{\widetilde{m}_f}, \\ \Xi_f^{\mathrm{l}} = -\widetilde{m}_f^{\mathrm{g}}, \quad \Xi_f^{\mathrm{g}} = \widetilde{m}_f^{\mathrm{l}}, \quad \Xi_f^{a} = \frac{a_f^{\mathrm{l}}\Xi_f^{\mathrm{l}} + a_f^{\mathrm{g}}\Xi_f^{\mathrm{g}}}{\widetilde{m}_f},\end{gathered} \tag{10.9}$$

with the phase apertures $a_f^{\mathrm{l}} + a_f^{\mathrm{g}} = a_f^{\mathrm{o}}$ of (4.22) ($a_f^{\mathrm{o}} = 1$ without solid). A face is *mixed* if both phases have a staggered fraction beyond the fraction floor and the face is open,

$$\mathcal{F}^{\mathrm{mix}} = \{\, f \in \mathcal{F}_\Omega \mid \varepsilon^{\alpha} < \hat{C}_f < 1-\varepsilon^{\alpha},\ a_f^{\mathrm{l}} + a_f^{\mathrm{g}} > 0 \,\}; \tag{10.10}$$

the jump unknowns ξ_f live on a subset $\mathcal{F}^{\xi} \subseteq \mathcal{F}^{\mathrm{mix}}$ chosen by the projection, and $\xi_f = 0$ elsewhere. The constraint is the cut-cell mixture divergence of the thesis [19, eq. (6.19)],

$$|c^{\mathrm{o}}|\left(\mathfrak{D}^{\mathrm{mix}}(u^{\mathrm{l}}, u^{\mathrm{g}})\right)_c = \sum_{f\in\mathcal{F}(c)} o_{c,f}\,|f|\left(a_f^{\mathrm{l}}u_f^{\mathrm{l}} + a_f^{\mathrm{g}}u_f^{\mathrm{g}}\right), \qquad \alpha_c^{\mathrm{o}} > 0, \tag{10.11}$$

where boundary faces enter with s_f and the boundary phase apertures of the cell's reconstruction. The phase gradients are the common pressure gradient, with the force map R^{tv} of the two-velocity coefficient γ^{tv} (8.5), plus a phase share of the jump unknown [19, eqs. (4.50), (7.48)],

$$\mathfrak{g}^{\pi}(p,\xi)_f = (\mathsf{R}^{\mathrm{tv}}\mathsf{J}p)_f + \frac{\Xi_f^{\pi}}{\widetilde{m}_f}\,\xi_f \quad (f\in\mathcal{F}_\Omega), \qquad \mathfrak{g}^{\pi}(p,\xi)_f = 0 \quad (f\in\mathcal{F}_\Gamma), \tag{10.12}$$

and every present phase velocity is updated by $u_f^{\pi,(n+1)} = u_f^{\pi,***} - \delta t\,\mathfrak{g}^{\pi}(p,\xi)_f$. Without solid $\gamma^{\mathrm{tv}} = \gamma$ and the common part is the one-velocity gradient $\mathfrak{S}_\gamma p$; with a solid it is $\mathfrak{S}^{\mathrm{o}}p$ with the coefficient $a_f^{\mathrm{o}}/\widetilde{\rho}_f$ (Section 8.8), and the stiffness of this map is $\mathsf{S}^{\mathrm{tv}} = \mathsf{J}^{\intercal}\mathsf{G}_{\mathrm{tv}}^{1/2}\mathsf{K}^{\mathrm{o}}\mathsf{G}_{\mathrm{tv}}^{1/2}\mathsf{J}$, $\mathsf{G}_{\mathrm{tv}} = \mathrm{diag}\,(\gamma^{\mathrm{tv}})$. Two identities follow from $\Xi_f^{\mathrm{g}} - \Xi_f^{\mathrm{l}} = \widetilde{m}_f$ and $\widetilde{m}_f^{\mathrm{l}}\Xi_f^{\mathrm{l}} + \widetilde{m}_f^{\mathrm{g}}\Xi_f^{\mathrm{g}} = 0$:

$$\mathfrak{g}^{\mathrm{g}}(p,\xi)_f - \mathfrak{g}^{\mathrm{l}}(p,\xi)_f = \xi_f, \qquad \widetilde{m}_f^{\mathrm{l}}\mathfrak{g}^{\mathrm{l}}(p,\xi)_f + \widetilde{m}_f^{\mathrm{g}}\mathfrak{g}^{\mathrm{g}}(p,\xi)_f = \widetilde{m}_f\,(\mathsf{R}^{\mathrm{tv}}\mathsf{J}p)_f; \tag{10.13}$$

the unknown ξ is the jump of the phase gradients, and the phase momentum feels only the common gradient. Equivalently, on a face where both phases are present, with the mass-weighted velocity $\bar{u}_f = (\widetilde{m}_f^{\mathrm{l}}u_f^{\mathrm{l}} + \widetilde{m}_f^{\mathrm{g}}u_f^{\mathrm{g}})/\widetilde{m}_f$ [19, eq. (7.23)] and the jump $[\![u]\!]_f = u_f^{\mathrm{g}} - u_f^{\mathrm{l}}$,

$$\begin{gathered}u_f^{\pi} = \bar{u}_f + \frac{\Xi_f^{\pi}}{\widetilde{m}_f}\,[\![u]\!]_f\,, \qquad \sum_{\pi}\widetilde{m}_f^{\pi}\left(u_f^{\pi}\right)^2 = \widetilde{m}_f\,\bar{u}_f^2 + \widetilde{m}_f^{\mathrm{r}}\,[\![u]\!]_f^2\,, \\ a_f^{\mathrm{l}}u_f^{\mathrm{l}} + a_f^{\mathrm{g}}u_f^{\mathrm{g}} = a_f^{\mathrm{o}}\,\bar{u}_f + \Xi_f^{a}\,[\![u]\!]_f\,,\end{gathered} \tag{10.14}$$

so that the projection moves $\bar{u}$ by $-\delta t \mathsf{R}^{\mathrm{tv}} \mathsf{J} p$ and the jump by $-\delta t \xi$, and changes the mixture divergence by

$$|c^{\mathrm{o}}| \left(\mathfrak{D}^{\mathrm{mix}}(u^{(n+1)}) \right)_c = |c^{\mathrm{o}}| \left(\mathfrak{D}^{\mathrm{mix}}(u^{***}) \right)_c + \delta t \left(\mathsf{S}^{\mathrm{tv}} p + \mathsf{J}^{\mathsf{T}} \operatorname{diag}(\Xi^a)\, \xi \right)_c , \tag{10.15}$$

using $\mathsf{J}^{\mathsf{T}} \mathsf{A} \mathsf{R}^{\mathrm{tv}} \mathsf{J} = \mathsf{S}^{\mathrm{tv}}$ and $\sum_{f \in \mathcal{F}(c)} o_{c,f} |f| u_f = -(\mathsf{J}^{\mathsf{T}} u)_c$ for a face function that vanishes on $\mathcal{F}_\Gamma$. The coefficient Ξ^a_f couples the jump to the volume balance: without solid and with phase apertures equal to the staggered fractions it is $\hat{C}_f(1-\hat{C}_f)(\rho^{\mathrm{l}} - \rho^{\mathrm{g}})/\widetilde{m}_f$.

Gauge. Both projections remove one pressure value per connected component. The components are those of the graph whose vertices are the cells with $\alpha^{\mathrm{o}}_c > 0$ and whose edges are the interior faces with $a^{\mathrm{o}}_f > 0$ (all cells and faces without solid); in each component the cell of smallest index is the anchor, $p = 0$ there, and its row is dropped, leaving the cells $\mathcal{C}^{\mathrm{r}}$. Subscripts on a matrix select rows and columns, faces or cells; $\mathsf{S}^{\mathrm{r}} = \mathsf{S}^{\mathrm{tv}}_{\mathcal{C}^{\mathrm{r}}\mathcal{C}^{\mathrm{r}}}$ is symmetric positive definite, and for every component $\mathcal{K}$

$$\sum_{c \in \mathcal{K}} |c^{\mathrm{o}}| \left(\mathfrak{D}^{\mathrm{mix}}(u^{\mathrm{l}}, u^{\mathrm{g}}) \right)_c = \sum_{f \in \mathcal{F}_\Gamma,\ c(f) \in \mathcal{K}} s_f |f| \left(a^{\mathrm{l}}_f u^{\mathrm{l}}_f + a^{\mathrm{g}}_f u^{\mathrm{g}}_f \right), \tag{10.16}$$

because the faces between two components are closed and carry no phase aperture. Summing the pressure rows of (10.25) over a component therefore gives zero on the left (the constant of the component is a null vector of S^{tv}) and $-1/\delta t$ times the boundary flux of the component on the right: the dropped row holds as soon as the other rows of its component hold if that flux vanishes; otherwise the anchor cell keeps the whole defect.

Implementation. Face data `_projection_face_data` (`src/two_phase/projection/common.jl`) (10.9) and (10.10) (`zeta_liquid`, `zeta_gas` are Ξ^π, `coupling` is Ξ^a, `relative_mass` is $\widetilde{m}^{\mathrm{r}}$); phase gradients `_projection_gradient_core!` (`src/two_phase/projection/common.jl`) (10.12), public as `cmdgfm_gradient!`; correction `_projection_correct_phase_velocities!` (`src/two_phase/projection/common.jl`); mixture flux `_cmdgfm_cell_flux!` (`src/two_phase/projection/cmdgfm_cache.jl`) (10.11); gauge `_projection_gauge_partition!` (`src/two_phase/projection/common.jl`); the pressure operator `_projection_pressure_operator` (`src/two_phase/projection/common.jl`); absent phases `_mark_inactive_phase_velocities!` (`src/two_phase/two_velocity/velocity_fields.jl`). Test: `test/two_phase/two_velocity/projection/cmdgfm_pressure_reuse.jl` ("CMDGFM compact connectivity and disconnected pressure gauges") (one anchor per component).

10.4 CMDGFM PROJECTION

The CMDGFM imposes the discrete counterpart of (10.2) by the thesis' jump interpolant [19, §§7.3.3–7.3.5], with a new rule for its weight β and with explicit rules for the faces where no condition can be imposed. Its origin is the ghost-fluid method of Liu et al. [13].

Staggered normal and ratio. With the cell normals $\boldsymbol{\eta}_c = \mathsf{H}_c^{-1}\hat{\boldsymbol{\eta}}_c / \left|\mathsf{H}_c^{-1}\hat{\boldsymbol{\eta}}_c\right|$ of the PLIC lines (zero if $\left|\mathsf{H}_c^{-1}\hat{\boldsymbol{\eta}}_c\right| \le \sqrt{\epsilon_{\mathrm{mach}}}$) and the mixing weights $w^{\mathrm{m}}_c = C^{\mathrm{l}}_c(1-C^{\mathrm{l}}_c)$, the staggered normal of $f \in \mathcal{F}^{\mathrm{mix}}$ with axis $k = k(f)$ and tangential axis $t = 3-k$ is

$$\widetilde{\boldsymbol{\eta}}_f = \frac{\breve{\boldsymbol{\eta}}_f}{|\breve{\boldsymbol{\eta}}_f|}, \quad \breve{\boldsymbol{\eta}}_f = w^{\mathrm{m}}_{c^-(f)} \boldsymbol{\eta}_{c^-(f)} + w^{\mathrm{m}}_{c^+(f)} \boldsymbol{\eta}_{c^+(f)}, \qquad R_f = \frac{\widetilde{\boldsymbol{\eta}}_{f,t}}{\widetilde{\boldsymbol{\eta}}_{f,k}}, \tag{10.17}$$

and $\breve{\boldsymbol{\eta}}_f = (C^{\mathrm{l}}_{c^-(f)} - C^{\mathrm{l}}_{c^+(f)})\boldsymbol{n}_f$ if $w^{\mathrm{m}}_{c^-(f)} + w^{\mathrm{m}}_{c^+(f)} = 0$ or $|\breve{\boldsymbol{\eta}}_f| \le \sqrt{\epsilon_{\mathrm{mach}}}(w^{\mathrm{m}}_{c^-(f)} + w^{\mathrm{m}}_{c^+(f)})$ (an interface on a grid line, pure cells on both sides); there is no normal if $|\breve{\boldsymbol{\eta}}_f| \le \sqrt{\epsilon_{\mathrm{mach}}}$. The ratio R_f is the thesis' $\mathcal{R}_{f,h}$ [19, eq. (7.15)] for the faces h of axis t, signed and independent of the orientation of $\widetilde{\boldsymbol{\eta}}_f$.

Transverse and inference faces. The transverse faces $\mathcal{F}^{\perp}(f)$ are the faces of axis t in the vertex patches $\mathcal{F}(v)$ of the two end points of f (Section 3.6) that are mixed by fraction, $\varepsilon^{\alpha} < \hat{C}_h < 1 - \varepsilon^{\alpha}$, and open, $a^{\mathrm{l}}_h + a^{\mathrm{g}}_h > \varepsilon^{\alpha}$; on a uniform mesh the four faces that touch f at its ends. Their weights are [19, eq. (7.11)]

$$w^{\perp}_{f,h} = \frac{w^{\mathrm{m}}_h}{\sum_{h' \in \mathcal{F}^{\perp}(f)} w^{\mathrm{m}}_{h'}}, \qquad w^{\mathrm{m}}_h = \hat{C}_h \left(1 - \hat{C}_h\right), \qquad h \in \mathcal{F}^{\perp}(f). \tag{10.18}$$

If $|R_f| > 1$ the row also uses two inference faces. The candidates $\mathcal{F}^{\parallel}(f)$ are the faces of axis k in the same two vertex patches other than f (on a uniform mesh the two collinear neighbours of f), with a solid only those with $|\omega^{\mathrm{o}}_h| > 0$. Ordered by the liquid mass fraction $\widetilde{m}^{\mathrm{l}}_h/\widetilde{m}_h$ (ties by index), the first is h^{g} and the last h^{l}, and

$$\beta_f = \begin{cases} 1, & |R_f| \le 1, \\ \dfrac{1 - s_f}{|R_f| - s_f}, & |R_f| > 1, \end{cases} \qquad s_f = \frac{\widetilde{m}^{\mathrm{l}}_{h^{\mathrm{g}}}}{\widetilde{m}_{h^{\mathrm{g}}}} + \frac{\widetilde{m}^{\mathrm{g}}_{h^{\mathrm{l}}}}{\widetilde{m}_{h^{\mathrm{l}}}}. \tag{10.19}$$

The code clamps β_f to $[0, 1]$ and throws if $|R_f| \le s_f$; neither happens, because the inference faces must be dominated by their phase ($\widetilde{m}^{\mathrm{g}}_{h^{\mathrm{g}}} \ge \widetilde{m}^{\mathrm{l}}_{h^{\mathrm{g}}}$ and $\widetilde{m}^{\mathrm{l}}_{h^{\mathrm{l}}} \ge \widetilde{m}^{\mathrm{g}}_{h^{\mathrm{l}}}$, else the row is unresolved), so that both cross fractions are at most $\frac{1}{2}$, $0 \le s_f \le 1 < |R_f|$ and $0 \le \beta_f < 1$.

Rows. For a resolved row the discrete normal velocity jump is

$$\mathcal{N}_f(u^{\mathrm{l}}, u^{\mathrm{g}}) = [\![u]\!]_f + \beta_f R_f \sum_{h \in \mathcal{F}^{\perp}(f)} w^{\perp}_{f,h} [\![u]\!]_h + (\beta_f - 1)\left(u^{\mathrm{g}}_{h^{\mathrm{g}}} - u^{\mathrm{l}}_{h^{\mathrm{l}}}\right), \tag{10.20}$$

the last term only if $|R_f| > 1$, and the row imposes $\mathcal{N}_f(u^{\mathrm{l},(n+1)}, u^{\mathrm{g},(n+1)}) = 0$. For $\beta_f > 0$, $\mathcal{N}_f = (\beta_f/\widetilde{\boldsymbol{\eta}}_{f,k})\,\widetilde{\boldsymbol{\eta}}_f \cdot \mathfrak{J}^{\beta}(u^{\mathrm{g}}, u^{\mathrm{l}})_f$ with the thesis' jump interpolant [19, eq. (7.16)]

$$\mathfrak{J}^{\beta}(u^{\mathrm{g}}, u^{\mathrm{l}})_f = \left[\beta_f^{-1} [\![u]\!]_f + \left(1 - \beta_f^{-1}\right)\left(u^{\mathrm{g}}_{h^{\mathrm{g}}} - u^{\mathrm{l}}_{h^{\mathrm{l}}}\right)\right] \boldsymbol{n}_f + \sum_{h \in \mathcal{F}^{\perp}(f)} w^{\perp}_{f,h} [\![u]\!]_h \, \boldsymbol{e}_t, \tag{10.21}$$

so the row is the thesis' condition $\widetilde{\boldsymbol{\eta}} \cdot \mathfrak{J}^{\beta}(u^{\mathrm{g}}, u^{\mathrm{l}})^{(n+1)} = 0$ of step T5 [19, eq. (7.13)]. If the normal is aligned with the face, $|\widetilde{\boldsymbol{\eta}}_{f,k}| \ge 1 - 64\epsilon_{\mathrm{mach}}$, the row has no transverse or inference terms, $\mathcal{N}_f = [\![u]\!]_f$, and the face-normal jump is removed. Substituting the update (10.12) (with (10.13)) turns the condition into the linear row

$$\xi_f + \beta_f R_f \sum_{h \in \mathcal{F}^{\perp}(f)} w^{\perp}_{f,h} \xi_h + (\beta_f - 1)\left[\frac{\widetilde{m}^{\mathrm{l}}_{h^{\mathrm{g}}}}{\widetilde{m}_{h^{\mathrm{g}}}} \xi_{h^{\mathrm{g}}} + \frac{\widetilde{m}^{\mathrm{g}}_{h^{\mathrm{l}}}}{\widetilde{m}_{h^{\mathrm{l}}}} \xi_{h^{\mathrm{l}}} + (\mathsf{R}^{\mathrm{tv}}\mathsf{J}p)_{h^{\mathrm{g}}} - (\mathsf{R}^{\mathrm{tv}}\mathsf{J}p)_{h^{\mathrm{l}}}\right]$$
$$= \frac{1}{\delta t} \mathcal{N}_f\left(u^{\mathrm{l},***}, u^{\mathrm{g},***}\right), \tag{10.22}$$

in which $\xi_h = 0$ for $h \notin \mathcal{F}^{\xi}$; the right-hand side contains the jumps of all transverse faces, also those without an unknown.

Unresolved rows. A face $f \in \mathcal{F}^{\mathrm{mix}}$ gets no row and no unknown if, in this order of the tests,

1. it is closed: $a_f^{\mathrm{l}} + a_f^{\mathrm{g}} \le \varepsilon^{\alpha}$;
2. it lies in one phase by aperture ($a_f^{\mathrm{l}} \le \varepsilon^{\alpha}$ or $a_f^{\mathrm{g}} \le \varepsilon^{\alpha}$) while a face of the same axis on the far side of one of its two cells has both phase apertures above ε^{α};
3. it has no staggered normal, or $|\widetilde{\boldsymbol{\eta}}_{f,k}| \le \sqrt{\epsilon_{\mathrm{mach}}}$;
4. its normal is not aligned with the face (an aligned normal, $|\widetilde{\boldsymbol{\eta}}_{f,k}| \ge 1 - 64\epsilon_{\mathrm{mach}}$, keeps the row $\mathcal{N}_f = [\![u]\!]_f$ without transverse or inference faces) and it has fewer than two transverse faces, or fewer than one if an end point lies on a physical boundary or a transverse face has another length than f (a level transition), or their mixing weights sum to zero;
5. its normal is not aligned, $|R_f| > 1$, and $\mathcal{F}^{\parallel}(f)$ has fewer than two faces, or an inference face is not dominated by its phase;

then

$$\mathcal{F}^{\xi} = \{\, f \in \mathcal{F}^{\mathrm{mix}} \mid \text{none of 1–5} \,\}, \qquad \xi_f = 0 \quad (f \in \mathcal{F}^{\mathrm{mix}} \setminus \mathcal{F}^{\xi}), \tag{10.23}$$

so the face-normal jump of an unresolved face is left as it is, $[\![u^{(n+1)}]\!]_f = [\![u^{***}]\!]_f$, rather than replaced by an invented condition. On the four drop fixtures of Table 10.1 between 31 and 41 % of the mixed faces are unresolved, between 84 and 96 % of them by rule 2 (a fraction-mixed face whose phase apertures put it in one phase next to a face that both phases cross) and the others by rules 4 and 5; an interface on a grid line consists of aligned faces, which are all resolved; Figure 10.1a shows the classification.

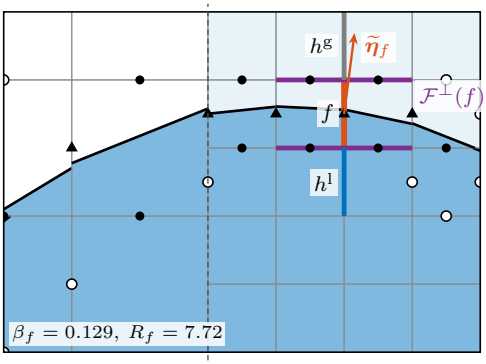
Diagonal dominance. The rows (10.22) form the jump block B (the coefficients of ξ) and the pressure block B^p (those of p). The diagonal of B is one, because no transverse or inference face is f, and the absolute values of the off-diagonal entries of row f satisfy

$$\sum_{h \ne f} |\mathsf{B}_{fh}| \le \beta_f\, |R_f| + (1 - \beta_f)\, s_f \le 1 = \mathsf{B}_{ff}, \tag{10.24}$$

since $\sum_h w^{\perp}_{f,h} = 1$. The left inequality is an equality if all transverse and inference faces carry unknowns; for a row with a transverse term the right one is an equality exactly when $|R_f| \ge 1$, and for $|R_f| < 1$ ($\beta_f = 1$) the margin is $1 - |R_f|$ when all transverse faces carry unknowns (an aligned row has no off-diagonal entries). Since $\beta \mapsto \beta\,|R_f| + (1-\beta)s_f$ increases for $|R_f| > s_f$, the rule (10.19) is the largest $\beta_f \in [0,1]$ for which the bound does not exceed one: B is weakly row diagonally dominant, strictly in the rows with $|R_f| < 1$, with as much of the interpolated tangential jump as this allows and the inference estimate making up the rest. The statistic `minimum_jump_diagonal_margin` is the smallest $\mathsf{B}_{ff} - \sum_{h \ne f} |\mathsf{B}_{fh}|$.

System. With the gauge of (10.16), the unknowns $p_{\mathcal{C}^{\mathrm{r}}}$ and $\xi_{\mathcal{F}^{\xi}}$ solve the nonsymmetric two-block system

$$\begin{bmatrix} \mathsf{S}^{\mathrm{r}} & \mathsf{J}^{\mathsf{T}}_{\mathcal{F}^{\xi}\mathcal{C}^{\mathrm{r}}}\, \mathrm{diag}\,(\Xi^{a})_{\mathcal{F}^{\xi}} \\ \mathsf{B}^{p} & \mathsf{B} \end{bmatrix} \begin{bmatrix} p_{\mathcal{C}^{\mathrm{r}}} \\ \xi_{\mathcal{F}^{\xi}} \end{bmatrix} = \begin{bmatrix} \left(-\delta t^{-1} |c^{\mathrm{o}}|\, \mathfrak{D}^{\mathrm{mix}}(u^{\mathrm{l},***}, u^{\mathrm{g},***})_c\right)_{c \in \mathcal{C}^{\mathrm{r}}} \\ \left(\delta t^{-1} \mathcal{N}_f(u^{\mathrm{l},***}, u^{\mathrm{g},***})\right)_{f \in \mathcal{F}^{\xi}} \end{bmatrix}, \tag{10.25}$$



A. CMDGFM rows

FIGURE 10.1. Constraint rows of the CMDGFM projection on the composite cut fixture of Table 10.1 (a drop on the sloped solid; level transition dashed; liquid shaded, solid hatched), near the top of the drop. Rows of the mixed faces: disc $\beta_f = 1$, triangle $\beta_f < 1$ (with inference faces), open circle unresolved (10.23). The stencil of one row with $|R_f| > 1$ (10.20): the face f (red) with its staggered normal $\widetilde{\boldsymbol{\eta}}_f$, the transverse faces $\mathcal{F}^{\perp}(f)$ (purple) and the inference faces h^{g} (grey) and h^{l} (blue).

where B^p holds, in row f, $(\beta_f - 1)$ times the difference of the rows h^{g} and h^{l} of $\mathsf{R}^{\mathrm{tv}}\mathsf{J}$ restricted to the columns $\mathcal{C}^{\mathrm{r}}$ (the same products with K^{o} as the gradient, evaluated only at the two inference faces). By (10.15) and (10.22), a solution makes $\mathfrak{D}^{\mathrm{mix}}(u^{(n+1)}) = 0$ in every cell of $\mathcal{C}^{\mathrm{r}}$, and by (10.16) in every open cell if each component has zero boundary flux, and makes $\mathcal{N}_f(u^{(n+1)}) = 0$ for every $f \in \mathcal{F}^{\xi}$. The momentum identity (10.13) holds for any solution. As in the thesis [19, §7.4], the CMDGFM conserves momentum but not kinetic energy: its jump rows are not the adjoint of its coupling block, and no energy identity is claimed.

Checks (`cmdgfm_check`). On the four fixtures of Table 10.1, an independent classification following (10.17) to (10.19) and (10.23) (own staggered normals, the code's vertex patches) reproduces the code's unknown set, transverse and inference faces exactly, and R_f, $w^{\perp}_{f,h}$ and β_f to 3.9×10^{-16}; the smallest diagonal margin of B is -1.1×10^{-16}, with the rows of zero margin counted in the table. After one projection at $\varepsilon^p = 10^{-10}$, the fixture independently evaluates $\max_f \delta t \left|\mathcal{N}_f(u^{(n+1)})\right| / h_f$ and $\delta t \max_c \left|\mathfrak{D}^{\mathrm{mix}}(u^{(n+1)})_c\right|$, checks that they equal the code's dimensionless defects and fall below the tolerance, and checks the identities (10.13). Here h_f is the smallest edge of either adjacent cell (the fine-cell minimum at a coarse–fine face), as in (10.30).

Deviation from the thesis 10.2 (CMDGFM jump weight). The thesis takes $\beta_f = 3x^2 - 2x^3$, $x = |\widetilde{\boldsymbol{\eta}}_f \cdot \boldsymbol{n}_f|$ [19, eqs. (7.18), (7.19)], which bounds $\beta_f |\mathcal{R}_{f,h}| \le 0.873$ but does not control the inference coefficients (with $s_f = 1$ the row sum reaches 1.47, and weak dominance fails for $s_f \ge 0.27$). The code uses (10.19), the largest weight for which the jump block is weakly diagonally dominant (10.24). For $|R_f| \le 1$ the code takes $\beta_f = 1$, the thesis less than one unless the normal is aligned with the face.

Deviation from the thesis 10.3 (CMDGFM inference faces and unresolved rows). The thesis lets one inference face be f itself, whichever phase has the larger mass in ω_f [19, §7.3.5]. The code takes both from the same-axis faces that share an end point with f, the extremes of the liquid mass fraction, and requires each to be dominated by its phase. Faces that fail the tests (10.23) get no unknown and keep their jump; the thesis has no such classification.

Implementation. `CMDGFMProjection(; preconditioner=:block, preconditioner_options=(;))` selects the method in `TwoVelocityModel`; the solver objects are `CMDGFMProjectionSolver(interface; density, preconditioner)` and `CMDGFMProjectionWorkspace`, and `project!(velocity, pressure, solver, dt, workspace; tolerance=sqrt(eps(T)), max_iterations=500, warm_start=true)` returns `CMDGFMProjectionStats` and adds the pressure part of the solution to `pressure`, keeping its anchor values (Section 10.5.2). Stencils `_cmdgfm_jump_stencils` (`src/two_phase/projection/cmdgfm_projection.jl`) with `_cmdgfm_face_normal` (`src/two_phase/projection/cmdgfm_projection.jl`) (10.17) to (10.19) and (10.23) (type `CMDGFMJumpStencil` (`src/two_phase/projection/cmdgfm_cache.jl`)); rows `_cmdgfm_jump_blocks` (`src/two_phase/projection/cmdgfm_projection.jl`) (10.22) with the gradient rows `_cmdgfm_append_gradient_row!` (`src/two_phase/projection/cmdgfm_cache.jl`); system `_assemble_cmdgfm_cached!` (`src/two_phase/projection/cmdgfm_projection.jl`) and `_cmdgfm_pressure_jump` (`src/two_phase/projection/cmdgfm_cache.jl`) (10.25); $\mathcal{N}_f$ `_cmdgfm_normal_velocity_defect` (`src/two_phase/projection/cmdgfm_projection.jl`); interpolant `cmdgfm_jump_interpolant` (10.21) (for $\beta_f > \sqrt{\epsilon_{\mathrm{mach}}}$). The system is solved by BiCGSTAB [29] (Krylov.jl), right-preconditioned by the block lower-triangular preconditioner `CMDGFMBlockPreconditioner` (`src/two_phase/projection/cmdgfm_projection.jl`): one V-cycle of the tree-geometric multigrid hierarchy of S^{tv} on the full cell space for the pressure, mapped to the gauge by adding the residual of a retained cell with opposite sign to its anchor and returning differences to the anchor value, then a Crout ILU (drop tolerance 10^{-3}) of B applied to the jump residual minus B^p times the pressure part; `:ilu` is a whole-system ILU. Without solid the hierarchy and operator of the one-velocity pressure solver are shared. Tests: `test/two_phase/two_velocity/projection/cmdgfm_projection.jl` ("CMDGFM revised beta and diagonal dominance"), `test/two_phase/two_velocity/projection/cmdgfm_projection.jl` ("CMDGFM jump interpolant consistency") (constant jumps reproduced exactly, first order for affine fields), `test/two_phase/two_velocity/projection/cmdgfm_projection.jl` ("CMDGFM reference impose/infer eligibility"), `test/two_phase/two_velocity/projection/cmdgfm_projection.jl` ("Equal-density CMDGFM phase-exchange symmetry"), `test/two_phase/two_velocity/projection/cmdgfm_projection.jl` ("CMDGFM projection invariants") (defects, margin and momentum identity on periodic and cut composite meshes), `test/two_phase/two_velocity/projection/cmdgfm_projection.jl` ("CMDGFM block-preconditioned BiCGSTAB").

Table 10.1. Unknowns and rows of the CMDGFM projection on the drop fixtures (a drop of radius 1.6 centred at $(4.3, 2.3)$ in the walled box of Chapter 8, densities $(1000, 1)$; composite: the right half refined once; with solid: resting on the sloped solid $y < 0.9 + 0.1x$), for one projection of a divergent predictor with a phase jump, $\delta t = 0.01$, $\varepsilon^p = 10^{-10}$ (`figures/projection.jl`).

	uniform		composite	
	no solid	solid	no solid	solid
mixed faces $\#\mathcal{F}^{\mathrm{mix}}$	72	51	119	83
resolved rows $\#\mathcal{F}^{\xi}$	47	35	70	51
of which $\beta_f < 1$	16	12	23	16
of which zero diagonal margin	8	6	8	7
unresolved by rule 2	24	14	43	27
unresolved by rules 4, 5	1	2	6	5
BiCGSTAB iterations	6	7	8	16

10.5 Fully discrete projection stage

The projection is an algebraic problem for the given predictor and δt; there is no time discretisation inside it. The time step enters as the scale of the correction and as the gauge of the defects, and in the incremental projections through the share of an inherited divergence that a shortened step removes (Section 10.5.2).

10.5.1 Boundary values

The one-velocity step applies the velocity boundary conditions of the solver at $t^{(n+1)}$, and with a solid sets the faces with $a_f^o = 0$ to zero, after the forces and before it subtracts the old pressure gradient (10.26). The old-pressure gradient and the increment use the same open-face pressure condition. The projection forms the divergence of the resulting predictor; the standalone `project!` prepares prescribed faces and retains the supplied open-face predictor values before forming the divergence. `Inflow` samples its function, while `NoSlip` and `FreeSlip` sides and the default walls give zero. Open-face initial values are input data: when extrapolation is wanted, the caller must first invoke `enforce_velocity_boundary!`, which copies the area- and aperture-weighted mean of parallel interior faces and, for `Outflow`, balances the total flux. Later the tentative open velocity is the previous projected value plus the same weighted extrapolation of the *change* in those interior velocities. Thus it approaches the projected value as $\delta t \to 0$, without a boundary impulse. The projection corrects the open normal faces and retains the corrected values instead of re-extrapolating them. Walls and inflows remain prescribed.

The old physical pressure in the predictor uses zero pressure at open faces as well: incompatible initial pressure data, including hydrostatic data with nonzero outlet pressure, can generate an initial outlet impulse. The gravity and surface-tension force maps supply no balancing open-face force, so a vertical open boundary under gravity need not preserve hydrostatic equilibrium. The two-velocity projection retains its phase boundary values. Its boundary preparation rejects a nonzero net phase flux when no `Outflow` supplies a balancing correction; standalone projections expect already compatible boundary data (`test/two_phase/two_velocity/projection/cmdgfm_projection.`

`jl` ("Two-velocity open boundary compatibility")). Two-velocity transport supports impermeable and periodic boundaries only (Chapter 5). The explicit initial-data contract is tested by `test/navier_stokes/high_level_api.jl` ("initial open-face velocities are explicit input data").

Check (`outflow_check`). In a 2×1 channel with parabolic inflow, no-slip walls and an `Outflow` side, one projection of a predictor with $\delta t \max_c |(\mathfrak{D} u^{***})_c| = 2.8 \times 10^{-2}$ at $\varepsilon^p = 10^{-12}$ gives a volume defect of 2.3×10^{-13} in the 16 outflow-adjacent cells and 4.4×10^{-13} elsewhere. The former must be at most ten times the latter. Four further projections give maximum volume defects $4.4 \times 10^{-13}, 4.4 \times 10^{-13}, 4.4 \times 10^{-13}, 4.4 \times 10^{-13}$. The same property, including zero-gradient outlets, is tested by `test/navier_stokes/navier_stokes.jl` ("open channel pressure flux and divergence").

Deviation from the thesis 10.4 (Boundary reset after the projection (fixed)). Previously the projection re-extrapolated `Outflow` and `ZeroGradientVelocity` values after solving, changing the adjacent cell fluxes and leaving a volume defect outside the tolerance. On the channel fixture the outflow defect was 5.7×10^{-3} versus 1.2×10^{-13} elsewhere; repeating the projection reduced it by only about $2/3$ per application (`proof_review/dev-projection-outflow/`). This defect is fixed: the open boundary face is now part of the projection, so the corrected boundary flux obeys the same residual–divergence identity as the interior (issue 93).

10.5.2 Pressure update

Incremental projections. The one-velocity and CMDGFM projections are incremental pressure corrections in the sense of Brown, Cortez and Minion [5] (issue 90). With $p^{(n)}$ the physical pressure at the start of the step, the step subtracts its gradient from the predictor of (8.21) once the boundary values at $t^{(n+1)}$ are set (Section 10.5.1),

$$u^\star = u^{***} - \delta t\, \mathcal{G} p^{(n)}, \qquad u^{\pi,\star} = u^{\pi,***} - \delta t\, \mathsf{R}^{\mathrm{tv}} \mathsf{J} p^{(n)} \quad \text{(CMDGFM, present phases)}, \tag{10.26}$$

with the pressure map of the projection that follows: the one-velocity map with the refreshed coefficient, and for the CMDGFM the two-velocity map of (10.12) at the transported interface. The one-velocity gradient vanishes on prescribed-flux boundaries and corrects open faces; the CMDGFM common gradient vanishes on boundary faces and carries no phase jump. The projection then solves (10.4), respectively (10.25), with $u^\star$ in place of u^{***} for the increment φ in place of p (and the jump ξ), corrects the velocities with φ and ξ, and accumulates

$$p^{(n+1)} = p^{(n)} + \varphi \quad \text{up to the component gauges.} \tag{10.27}$$

For a step with $\delta t^{\mathrm{ref}} = \delta t$ (below) this is the full-pressure projection of u^{***}, the non-incremental projection of the thesis' step T5 and of the code before issue 90, up to the tolerance of the solve. With one velocity, $-\mathsf{V}\mathfrak{D}^{\circ}\mathcal{G} = \mathsf{S}$ gives $\mathsf{b}(u^\star) = \mathsf{b}(u^{***}) - \mathsf{S} p^{(n)}$ with the same removed means on unanchored components, since each of their indicators lies in the left nullspace of S. Thus $p^{(n)} + \varphi$ solves the system for u^{***} and $u^{(n+1)} = u^{***} - \delta t \mathcal{G}(p^{(n)} + \varphi)$. With the CMDGFM, (10.15) with $\xi = 0$ shifts the pressure rows of (10.25) by $-(\mathsf{S}^{\mathrm{tv}} p^{(n)})_{\mathcal{C}^\Gamma}$, and a common increment of both phases changes the row (10.20) only through its inference term, which shifts the jump rows by $-\mathsf{B}^p p^{(n)}$ (with $p^{(n)}$ relative

to the anchors); hence $(p^{(n)}+\varphi,\xi)$ solves the system for u^{***}, and ξ is the whole jump of the phase gradients, not an increment. In the adaptive cylinder of issue 90, three steps of both forms solved to $\varepsilon^p=10^{-15}$ give the same pressure forces to 10^{-10}.

The one-velocity increment has zero volume mean on each unanchored component (10.5); at an open pressure face its value is fixed there instead. On a connected unanchored domain the incoming physical-pressure mean is preserved; on a split domain unanchored component means of the accumulated pressure are normalised. The CMDGFM increment is zero at the anchor cells and keeps the incoming anchor values. The physical pressure of a `FlowSimulation` starts at zero unless an initial pressure is given, so that the first projection builds it, and a regrid transfers it with the velocity. The lower-level `project!` methods of the two incremental projections add their increment to the pressure they are given and expect a velocity predicted with it: projecting an unpredicted velocity repeatedly with the same pressure field accumulates impulses, and an absolute projection takes a zero pressure. `project!(flow)`, meant for preparing an initial velocity, and the post-regrid projection of Section 10.5.4 use such a zero scratch pressure and leave the physical pressure unchanged.

Shortened steps. The velocity that enters a step is solenoidal only up to the tolerance of the previous projection, and an AMR transfer can leave further divergence. A projection of a step much shorter than the one that left this inherited divergence divides it by the short δt and turns it into pressure: in the adaptive cylinder at Reynolds number 200 of issue 90, this full-pressure treatment ($\delta t^{\mathrm{ref}}=\delta t$) of a last step of a tenth (a half) of the proposal, taken to reach an output time ten steps before the end of the run, moves drag and lift by 196 and 537 (26 and 73) times their change over a full step. The incremental projections therefore remove, in the same solve, only the share $\delta t/\delta t^{\mathrm{ref}}$ of the inherited divergence, with a reference step $\delta t^{\mathrm{ref}}\geq\delta t$; the right-hand side of (10.4) becomes

$$\mathsf{b}_c=-\frac{1}{\delta t}|c^{\mathrm{o}}|\left(\mathfrak{D}^{\mathrm{o}}u^{\star}\right)_c+\left(\frac{1}{\delta t}-\frac{1}{\delta t^{\mathrm{ref}}}\right)|c^{\mathrm{o}}|\left(\mathfrak{D}^{\mathrm{o}}u^{(n)}\right)_c, \tag{10.28}$$

with $u^{(n)}$ the velocity at the start of the step with its boundary values at $t^{(n)}$ and its covered faces set to zero. The CMDGFM pressure rows of (10.25) take the same form with $\mathfrak{D}^{\mathrm{mix}}$, the inherited term being the mixture divergence of $(u^{\mathrm{l},(n)},u^{\mathrm{g},(n)})$ on the phase apertures of the start-of-step reconstruction, saved before the transport; its jump rows keep their right-hand side and impose $\mathcal{N}_f(u^{(n+1)})=0$ on the current interface (keeping a share of the inherited jump instead left normal-velocity defects of 10^{-3} to 3×10^{-2} in the review of the change). The cell identity (10.6) becomes

$$|c^{\mathrm{o}}|\left(\mathfrak{D}^{\mathrm{o}}u^{(n+1)}\right)_c=\left(1-\frac{\delta t}{\delta t^{\mathrm{ref}}}\right)|c^{\mathrm{o}}|\left(\mathfrak{D}^{\mathrm{o}}u^{(n)}\right)_c-\delta t\left(\mathsf{r}_c+\bar{\mathsf{b}}_{\mathcal{K}}\,|c^{\mathrm{o}}|\right),\qquad c\in\mathcal{K}, \tag{10.29}$$

for the retained projected boundary values: the step keeps the share $1-\delta t/\delta t^{\mathrm{ref}}$ of the inherited divergence instead of turning it into pressure, and $\delta t^{\mathrm{ref}}=\delta t$ is the normal step. The CMDGFM measures its volume defect for the convergence test against the same target (Section 10.5.3).

In `simulate!` the reference step is $\delta t^{\mathrm{ref}}=\max(\delta t^{\mathrm{prop}},\delta t^{\mathrm{def}},\delta t)$ (13.2), with δt^{prop} the untruncated proposal of the step, δt^{def} the actual length of the previous physical step, which left the defect, and the step δt itself, which exceeds both only for a last step stretched

to reach the end time: a last step shortened to reach the end time and a step whose proposal dropped below the previous step both keep part of the defect. The first step of a run has $\delta t^{\mathrm{def}} = 0$, as has a call that does not continue the previous one with the same fields, solver, time and mesh topology; a field-level `step!` or `project!(flow)` ends that history. After a topology change the transfer leaves a new defect, and δt^{def} is then the proposal δt^{prop} of the step before the regrid, with which the two-velocity post-regrid projection gauges its defect (Section 10.5.4), not the actual step. A full `step!(flow, dt)` uses $\delta t^{\mathrm{ref}} = \max(\delta t, \delta t^{\mathrm{def}})$ and retains the new defect reference; a field-level step defaults to $\delta t^{\mathrm{ref}} = \delta t$. Both `step!` and the lower-level projections reject $\delta t^{\mathrm{ref}} < \delta t$. With this treatment the pressure of a shortened step no longer contains the inherited divergence divided by δt, the mechanism of these excursions: over five end times 8 to 12 steps before the end of the run, the last step of a tenth (a half) of the proposal moves the drag by 1.8–8.5 (0.67–4.9) and the lift by 2.6–4.0 (0.63–3.1) times their change over a full step, alternating with the parity of the step (issue 90; the test asserts at most 12).

First guess. The one-velocity Krylov iteration starts from a zero increment. The CMDGFM with its block preconditioner tries the zero increment and, if the mesh topology is unchanged, the increment of its previous projection (relative to the anchors), each with the jump that the ILU of B assigns to it, and starts from a candidate only if it lowers the residual 2-norm and lowers the largest residual defect by at least 5 % relative to the best candidate so far (initially the zero solution). The converged result depends on the first guess only within the tolerance.

Deviation from the thesis 10.5 (Incremental projection and reference step). The thesis' step T5 solves for the whole pressure in every step [19, §7.2]. The one-velocity and CMDGFM projections solve for the increment of the previous pressure (10.26) and (10.27), which for a normal step gives the same velocity and, up to a constant, the same pressure. On a step shorter than its reference step they keep the share $1 - \delta t/\delta t^{\mathrm{ref}}$ of the inherited divergence (10.29), so that the end-of-step velocity is solenoidal only up to that share.

Implementation. Predictor `_pressure_predictor!` (`src/single_phase/stepping.jl`), after `_enforce_current_velocity_boundary!` (`src/single_phase/stepping.jl`), and for the CMDGFM `_two_velocity_pressure_operator!` (`src/two_phase/two_velocity/stepping.jl`) with `_add_common_phase_acceleration!` (`src/two_phase/two_velocity/stepping.jl`) in `step!` (`src/two_phase/two_velocity/stepping.jl`) (10.26); solve and accumulation `_project_prepared!` (`src/navier_stokes/projection.jl`) (with a solid `_project_prepared!` (`src/embedded/operators.jl`)), `_projection_rhs!` (`src/navier_stokes/projection.jl`) (10.28) and `_accumulate_pressure_increment!` (`src/navier_stokes/projection.jl`) (10.27); for the CMDGFM `_assemble_cmdgfm_system!` (`src/two_phase/projection/cmdgfm_projection.jl`) (inherited weight) and `_project_cmdgfm_assembled!` (`src/two_phase/projection/cmdgfm_projection.jl`) (increment, accumulation, target defect), the inherited flux `_capture_two_velocity_inherited_flux!` (`src/two_phase/two_velocity/stepping.jl`); the reference step in `_simulate!` (`src/navier_stokes/simulate.jl`) (internal keyword `_reference_dt`, checked by `_validate_projection_reference_dt` (`src/navier_stokes/projection.jl`)); scratch pressures in

`project!` (`src/navier_stokes/flow_simulation.jl`) and `_project_regridded_two_velocity!` (`src/two_phase/two_velocity/flow_workspace.jl`); initial pressure `_initial_pressure` (`src/navier_stokes/flow_simulation.jl`). Tests: `test/operators/poisson.jl` ("incremental pressure projection"), `test/navier_stokes/navier_stokes.jl` ("increment predictor preserves open-boundary projection"), `test/navier_stokes/navier_stokes.jl` ("short step does not turn inherited divergence into pressure"), `test/navier_stokes/high_level_api.jl` ("adaptive cylinder short final step (issue #90)") (three steps of the incremental and the full-pressure form, and a shortened last step), `test/navier_stokes/high_level_api.jl` ("abrupt proposal drops retain the actual preceding step"), `test/navier_stokes/high_level_api.jl` ("standalone flow projection retains physical pressure"), `test/two_phase/two_velocity/projection/cmdgfm_pressure_reuse.jl` ("CMDGFM incremental projection satisfies the coupled system"), `test/two_phase/two_velocity/projection/cmdgfm_pressure_reuse.jl` ("CMDGFM shortened projection retains only the mixture target"), `test/two_phase/two_velocity/projection/cmdgfm_pressure_reuse.jl` ("CMDGFM moving interface enforces the actual short-step jump"), `test/two_phase/two_velocity/transport/two_velocity.jl` ("Two-velocity short-step reference flux").

10.5.3 Defects and tolerance

All projections compare dimensionless defects with the one tolerance ε^p of `step!` and `simulate!`. With one velocity the solve stops on (10.7), whose left-hand side is the volume defect e^V of every cell by (10.6), on a shortened step relative to the retained share of the inherited divergence (10.29). With two velocities the defects are measured on the end-of-step velocity,

$$e^V = \delta t \max_{c:\,|c^{\mathrm{o}}|>0} \left| \left(\mathfrak{D}^{\mathrm{mix}}(u^{(n+1)}) \right)_c \right|, \qquad e^\eta = \max_{f \in \mathcal{F}^\xi} \frac{\delta t \left| \mathcal{N}_f(u^{(n+1)}) \right|}{h_f}, \tag{10.30}$$

Here h_f is the smallest cell edge of either cell adjacent to face f, using the fine-cell minimum edge at a coarse–fine face. The CMDGFM converges if e^V, e^η and the residual defects $\delta t^2 \max_{c \in \mathcal{C}^{\mathrm{r}},\, |c^{\mathrm{o}}|>0} |\mathsf{r}_c| / |c^{\mathrm{o}}|$ and $\delta t^2 \max_{f \in \mathcal{F}^\xi} |\mathsf{r}_f| / h_f$ of (10.25), together with the same cell defect of the pressure rows omitted for the gauge, are at most ε^p, where on a shortened step the volume defect is taken relative to the retained share $(1 - \delta t / \delta t^{\mathrm{ref}})$ of the inherited mixture divergence while the reported e^V and e^η remain those of the end-of-step velocity; it restarts BiCGSTAB on the true residual, within its iteration budget, until the residual defects are met (Section 11.5). The normal and jump-row criteria previously compared dimensional velocities directly with the dimensionless tolerance; the length and time scaling above fixes that unit-dependent stopping rule (issue 94). Both Krylov systems are also nondimensionalised before iteration using the largest domain extent L and density ρ_0: the pressure and gradient-jump scales are $\rho_0 (L/\delta t)^2$ and $L/\delta t^2$; reported raw algebraic residual norms remain dimensional. Floating-point roundoff can still change the iteration count of an unpreconditioned solve near convergence. `project!` and the field-level `step!` report a projection that misses its rules (`converged=false`); `step!(flow, dt)` and the post-regrid projection throw instead (Section 10.5.4).

Implementation. Statistics `CMDGFMProjectionStats` (`mixture_volume_defect`, `normal_velocity_defect`, `minimum_jump_diagonal_margin`, `momentum_identity_error`, `underresolved_jump_rows`); residual defects `_cmdgfm_residual_defects` (`src/two_phase/projection/cmdgfm_projection.jl`); first guess `_cmdgfm_initial_solution!` (`src/two_phase/projection/cmdgfm_projection.jl`); boundary values `enforce_velocity_boundary!` (`src/operators/staggered_diffusion.jl`), `enforce_geometry_no_slip!` (`src/embedded/operators.jl`) and, for two velocities, `_enforce_phase_velocity_boundary!` (`src/two_phase/two_velocity/stepping.jl`); the stage in `step!` (`src/single_phase/stepping.jl`) and `step!` (`src/two_phase/two_velocity/stepping.jl`) (via `_project_two_velocity!` (`src/two_phase/two_velocity/flow_workspace.jl`)). Tests: `test/two_phase/two_velocity/projection/cmdgfm_pressure_reuse.jl` ("CMDGFM initial guesses preserve the converged projection"), `test/two_phase/two_velocity/projection/cmdgfm_projection.jl` ("Two-velocity projection convergence is independent of physical units"), `test/two_phase/two_velocity/projection/cmdgfm_projection.jl` ("CMDGFM dimensionless jump and residual defects"), `test/navier_stokes/navier_stokes.jl` ("pressure failure reports convergence diagnostics").

10.5.4 Projection after a regrid

The phase velocities are transferred to a new mesh independently (Chapter 12), which does not preserve the aperture-weighted mixture divergence (issue 84). Since PR #89, `step!(flow, dt)` therefore applies, after every topology change, the selected two-velocity projection to the transferred velocities, with the boundary values at the current time, the untruncated proposed step δt^{prop} (the supplied step for a direct call), and the tolerance, as an absolute projection (the CMDGFM after reconstructing the transferred interface),

$$u^{\pi} \leftarrow u^{\pi} - \delta t^{\mathrm{prop}}\, \mathfrak{g}^{\pi}(p,\xi)(\varphi,\xi), \tag{10.31}$$

where the potential φ is solved in a scratch field that starts at zero: the transferred physical pressure is kept. The matrix of (10.25) does not depend on the step and its right-hand side scales with $1/\delta t^{\mathrm{prop}}$, so (φ,ξ) scales with $1/\delta t^{\mathrm{prop}}$ and the corrected velocity does not depend on δt^{prop}; the step only sets the gauge of $e^{V} = \delta t^{\mathrm{prop}} \max\left|\mathfrak{D}^{\mathrm{mix}}\right|$, and the untruncated step keeps a step shortened to reach the end time from loosening the constraint for a resumed run. A failure to converge throws. The defect that the transfer leaves is associated with the same δt^{prop}, which enters the reference step of the next step (Section 10.5.2). The one-velocity model has no such projection: the transfer of a shared velocity corrects the refined faces with the Cartesian divergence (Chapter 12), the next step treats what remains as inherited divergence with the same reference, and standalone `adapt!` never projects.

Check (`regrid_scaling_check`). Projecting the same velocities on the composite cut fixture with δt and $3\delta t$ ($\varepsilon^{p} = 10^{-13}$) gives end-of-step velocities that differ by 6.8×10^{-14} of their maximum.

Deviation from the thesis 10.6 (Projection after a regrid). The thesis' time step has one projection [19, §7.2]. A two-velocity simulation with adaptivity adds one projection per regrid, outside the time integration, which changes the velocity by a pressure-gradient

correction without a time step of its own.

Implementation. `_project_regridded_two_velocity!` (`src/two_phase/two_velocity/flow_workspace.jl`) (10.31), called by `step!` (`src/navier_stokes/flow_simulation.jl`) after `adapt!` changes the topology. Tests:
`test/two_phase/two_velocity/transport/two_velocity.jl` ("Regrid on a shortened final step"),
`test/two_phase/two_velocity/transport/two_velocity.jl` ("Ordinary regrid tolerance is not tightened"),
`test/navier_stokes/high_level_api.jl` ("regrid defects keep the transfer reference"),
`test/two_phase/two_velocity/transport/two_velocity.jl` ("Fast phase velocities at regrid").

— 11 —

Operators and linear solvers

Equation reference, audited against commit `c46cf60`.

Every implicit part of a time step is a sparse linear system: the pressure projections (Chapter 10) and the implicit viscous step (Chapter 9). The thesis analyses the conditioning of its pressure matrices, whose largest-to-smallest nonzero eigenvalue ratio after diagonal (Jacobi) scaling grows as h^{-2}, the multidimensional ghost-fluid matrix being nonsymmetric with complex eigenvalues [19, §7.3.6], but it does not state how the systems are solved. This chapter states the matrices that the code assembles and the properties that the solvers rely on (Section 11.1), the Krylov driver shared by the elliptic solves with its stopping rule and round-off floor (Section 11.2), the tree geometric multigrid preconditioner (Section 11.3), the shifted viscous systems (Section 11.4) and the solvers of the projection systems (Section 11.5), and it reports measured iteration counts, convergence factors, spectra and allocations (Section 11.6). Table 11.1 lists the systems. There is no time discretisation in this chapter: a solver maps a matrix, a right-hand side and an initial guess to an approximate solution, and the time step enters only through the matrices and tolerances of the calling chapters.

11.1 Cell elliptic systems

Matrix. The cell operator $\mathfrak{L}$ of (3.25) is stored in integrated form. Let V be the diagonal matrix of the operator volumes: the cell volumes $|c|$, the open volumes $|c^{\mathrm{o}}|$ with a solid, or volumes supplied to the constructor (the liquid-carrier projection supplies the open volumes of the cells that contain liquid). The constructor requires that a cell is active exactly when

Table 11.1. The linear systems of a time step, their default Krylov method and preconditioner, and their iteration cap. "Tree MG" is one V-cycle of Section 11.3; the caps are the `max_cycles` or `max_iterations` defaults of `step!` (one phase / two phases).

System	Matrix	Property	Krylov	Preconditioner	Cap
one-velocity projection	(11.1), $r = 0$	SPSD	CG	tree MG (cells)	100 / 500
viscous, one phase	(11.21)	SPD	CG	tree MG (faces of an axis)	100
viscous, two phases	(9.18)	SPD	CG	tree MG (all faces)	500
CMDGFM projection	(10.25)	nonsymmetric	BiCGSTAB	V-cycle and ILU blocks	500

its volume is positive; the other cells form the inactive set $\mathcal{C}^{\times}$. The matrix of the solve is

$$\mathsf{L}_{cc'} = \begin{cases} r\,\mathsf{V}_{cc}\,\delta_{cc'} + \vartheta\left(\mathsf{S}+\mathsf{D}^{\partial}\right)_{cc'}, & c,c' \notin \mathcal{C}^{\times}, \\ \delta_{cc'}, & c \in \mathcal{C}^{\times} \text{ or } c' \in \mathcal{C}^{\times}, \end{cases} \qquad \mathsf{S} = \mathsf{J}^{\mathsf{T}}\mathsf{G}^{1/2}\mathsf{K}^{\mathrm{o}}\,\mathsf{G}^{1/2}\mathsf{J}, \tag{11.1}$$

with the face metric K^{o} (K without solid, WKW with a solid) and the face coefficients $\mathsf{G} = \operatorname{diag}(\gamma_f)$ of Section 3.7. The product S is assembled once per topology and replaced by $\frac{1}{2}(\mathsf{S}+\mathsf{S}^{\mathsf{T}})$, which is exactly symmetric in floating point; only its values are refreshed when γ changes. The diagonal D^{∂} holds the conductances of the open boundary faces $\mathcal{F}^{\mathrm{o}}_{\Gamma}$, the boundary faces on `Outflow` and `ZeroGradientVelocity` sides, where the pressure (increment) is zero:

$$\mathsf{D}^{\partial}_{cc} = \sum_{f\in\mathcal{F}^{\mathrm{o}}_{\Gamma},\ c(f)=c} |f|\,a^{\mathrm{o}}_{f}\,\frac{2\gamma^{\partial}_{f}}{h_{c,k(f)}}, \qquad c \notin \mathcal{C}^{\times}, \tag{11.2}$$

a two-point flux over the distance $\frac{1}{2}h_{c,k(f)}$ from the cell centre to the face, with the boundary aperture a^{o}_{f} (one without solid). The boundary coefficient is supplied per face, evaluated at the face for a constant or a function coefficient, and otherwise extended from the interior coefficients,

$$\gamma^{\partial}_{f} = \frac{\sum_g |g|\,\gamma_g}{\sum_g |g|}, \tag{11.3}$$

over the interior faces g of axis $k(f)$ on the side of $c(f)$ opposite to f; the constructor throws if there is no such face. The two-phase pressure operator supplies its own boundary coefficients. The weighted gradient of the projection uses the same conductances (Section 10.2).

Inactive cells. The rows and columns of $\mathcal{C}^{\times}$ are those of the identity, so inactive cells decouple whatever the coefficients of their faces. Two conditions are distinguished by the operator, with the same matrix treatment. With `inactive_condition=:decoupled` (cut-cell pressure) the faces next to a covered cell have zero aperture and therefore $\gamma_f = 0$ (3.6). With `:dirichlet` (the liquid-carrier projection, whose inactive cells contain no liquid) the faces between an active and an inactive cell keep their coefficient: their contribution stays on the diagonal of the active row, which imposes a zero value in the inactive cell. The active mask of a `:dirichlet` operator may change on a fixed mesh; that of a `:decoupled` operator may not.

Properties. For x with $\mathsf{x}_c = 0$ on $\mathcal{C}^{\times}$, replacing the rows and columns of the inactive cells by the identity does not change the quadratic form, so

$$\mathsf{x}^{\mathsf{T}}\mathsf{L}\,\mathsf{x} = r\sum_c \mathsf{V}_{cc}\mathsf{x}_c^2 + \vartheta\,\|\mathsf{K}^{\mathrm{o}\,1/2}\mathsf{G}^{1/2}\mathsf{J}\mathsf{x}\|_2^2 + \vartheta\sum_c \mathsf{D}^{\partial}_{cc}\mathsf{x}_c^2. \tag{11.4}$$

Since K^{o} is symmetric positive definite (Section 3.6) and $\gamma_f \geq 0$, every term is non-negative: L is symmetric positive semidefinite, and positive definite for $r > 0$. For $r = 0$ the null space is made of connected components. Consider the graph whose vertices are the active cells and whose edges are the interior faces with $\gamma_f \neq 0$. A component $\mathcal{K}$ of this graph is *anchored* if one of its cells has $\mathsf{D}^{\partial}_{cc} > 0$ or if one of its cells shares a face with $\gamma_f \neq 0$ with

an inactive cell. Then

$$\ker \mathsf{L} = \operatorname{span}\{\, \mathbf{1}_{\mathcal{K}} \mid \mathcal{K} \text{ an unanchored component} \,\}, \qquad \mathsf{L}x = \mathsf{b} \text{ is solvable} \iff \mathbf{1}_{\mathcal{K}}^{\mathsf{T}}\mathsf{b} = 0 \text{ for every } \mathcal{K} \tag{11.5}$$

where $\mathbf{1}_{\mathcal{K}}$ is the indicator of $\mathcal{K}$. Indeed, for the symmetric positive semidefinite L, $\mathsf{L}x = 0$ if and only if $x^{\mathsf{T}}\mathsf{L}x = 0$; the inactive entries vanish because their rows are those of the identity, and by (11.4) with K° definite the form vanishes exactly when x is equal across every face with $\gamma_f > 0$, including the faces to inactive cells, where the value is zero, and vanishes where $\mathsf{D}^{\partial}_{cc} > 0$. The solvability condition is $\mathsf{b} \perp \ker \mathsf{L}$, since L is symmetric. The code labels each component by its smallest cell index, flags the operator as singular if one component is unanchored, and caches the labels; they are rebuilt when the mesh topology (its identity or generation), the active mask or the set of faces with nonzero coefficient changes, and the anchors are re-evaluated on every solve. The pressure on an unanchored component is defined up to a constant; Section 11.2 fixes its volume mean to zero.

Checks (`solv_component_check`). On 32×32 cells of the unit square with the solid band $|y - 0.46| < 0.17$, which leaves two fluid components with cut and covered cells, the operator has two unanchored components; an `Outflow` side at the top anchors exactly one of them, and one on the right side, which both components touch, anchors both and makes the operator nonsingular. The indicators of the unanchored components are null vectors to 4.4×10^{-16}.

Factorised action. The Krylov iteration does not use the assembled matrix: it applies L as the product of its factors, the jumps $\mathsf{J}x$ with the inactive entries set to zero, $\mathsf{G}^{1/2}$, K°, $\mathsf{G}^{1/2}$ and J^{T}, plus the diagonal $r\mathsf{V} + \vartheta\mathsf{D}^{\partial}$, and the identity on the inactive rows. This action keeps the native, non-diagonal metric at level transitions and equals the assembled matrix up to round-off. An operator built with `assemble_matrix=false` stores no matrix; its multigrid hierarchy then reads the diagonal, the absolute row sums and the transfer smoothing of the finest level from an exact stencil of the factors (Section 11.3.2).

> *Implementation.* `EllipticOperator(mesh; reaction=0, diffusion=1, face_coefficient=1, velocity_boundary=nothing, boundary_face_coefficient=nothing, volumes, active_cells, inactive_condition=:decoupled, assemble_matrix=true)`, `PoissonOperator(mesh)`, `PoissonOperator(geometry)` and the two-phase `PoissonOperator(interface[, geometry]; density)`. Assembly `_integrated_stiffness` (`src/operators/elliptic.jl`), `_impose_inactive_constraints!` (`src/operators/elliptic.jl`); open faces `_elliptic_boundary_data!` (`src/operators/elliptic.jl`) (11.2) and `_boundary_material` (`src/operators/elliptic.jl`) (11.3), two-phase `_mixture_boundary_coefficient` (`src/two_phase/forces.jl`); active mask `_update_elliptic_activity!` (`src/operators/elliptic.jl`); components `_refresh_elliptic_components!` (`src/solvers/multigrid/elliptic_solver.jl`) with `_projection_component_labels!` (`src/two_phase/projection/common.jl`) (11.5); factorised action `FactorizedEllipticOperator` (`src/operators/elliptic.jl`). Tests: `test/operators/poisson.jl` (“Factorized pressure action matches assembled operator”), `test/operators/poisson.jl` (“Fixed-pattern elliptic refresh and matrix-free storage”), `test/operators/poisson.jl` (“Elliptic zero-Dirichlet inactive cells”), `test/operators/poisson.jl` (“open pressure boundary operators and solvers”), `test/operators/poisson.jl` (“opposite open faces require boundary material”),

`test/operators/poisson.jl` (“open-face Poisson component labels”),
`test/operators/poisson.jl` (“singleton open pressure components”),
`test/solvers.jl` (“elliptic component cache tracks topology identity and generation”),
`test/solvers.jl` (“mixed component anchors follow activity and coefficient updates”).

11.2 Krylov driver and stopping rule

The cell solves and both viscous solves run one driver around the Krylov methods of Krylov.jl [14]: CG [10], BiCGSTAB [29], GMRES [21] or FGMRES. The CMDGFM projection calls BiCGSTAB in a loop of its own (Section 11.5).

Norms and stop level. With V the diagonal weights of the solve (the operator volumes for the cell systems, the staggered volumes $\widetilde{\mathsf{V}}$ or the inertia M_ρ for the viscous ones), integrated residuals are measured in the strong norms

$$\|\mathsf{r}\|_{\mathsf{V},\infty} = \max_{c:\,\mathsf{V}_{cc}>0} \frac{|\mathsf{r}_c|}{\mathsf{V}_{cc}}, \qquad \|\mathsf{r}\|_{\mathsf{V},2} = \left(\frac{\sum_{c:\,\mathsf{V}_{cc}>0} \mathsf{r}_c^2/\mathsf{V}_{cc}}{\sum_{c:\,\mathsf{V}_{cc}>0} \mathsf{V}_{cc}} \right)^{1/2}, \tag{11.6}$$

the largest and the volume-weighted root-mean-square residual per unit volume. With $\mathsf{r}_0 = \mathsf{b} - \mathsf{L}\mathsf{x}_0$ the residual of the initial guess, a solve stops when the true residual satisfies $\|\mathsf{r}\|_{\mathsf{V},\infty} \le \varepsilon^\star$ with

$$\varepsilon^\star = \begin{cases} \max\left\{\varepsilon^{\mathrm{abs}},\ \varepsilon^{\mathrm{rel}} \max(\|\mathsf{r}_0\|_{\mathsf{V},\infty}, \epsilon_{\mathrm{mach}}),\ \epsilon_{\mathrm{fl}}(\mathsf{x}_0)\right\}, & \text{cell solves (}\texttt{solve!}\text{)}, \\ \max\left\{\epsilon_{\mathrm{tol}}\,\|\mathsf{b}\|_{\mathsf{V},\infty},\ \epsilon_{\mathrm{fl}}(\mathsf{x}_0)\right\}, & \text{viscous solves}, \end{cases} \tag{11.7}$$

where `solve!` defaults to $\varepsilon^{\mathrm{rel}} = 10^{-10}$, $\varepsilon^{\mathrm{abs}} = 0$ and 100 iterations. The one-velocity projection passes $\varepsilon^{\mathrm{rel}} = 0$ and $\varepsilon^{\mathrm{abs}} = \varepsilon^p/\delta t^2$, which is (10.7); the viscous solves stop relative to the right-hand side (Section 9.4), so that their stop level does not depend on the initial guess. The round-off floor is the scale of the terms that a residual entry sums,

$$\epsilon_{\mathrm{fl}}(\mathsf{x}) = 128\,\epsilon_{\mathrm{mach}} \max\left\{1,\ \max_{c:\,\mathsf{V}_{cc}>0} \frac{(|\mathsf{L}|\,|\mathsf{x}|)_c}{\mathsf{V}_{cc}},\ \|\mathsf{b}\|_{\mathsf{V},\infty}\right\}, \tag{11.8}$$

with entrywise absolute values; for the factorised action $|\mathsf{L}|$ is replaced by the entrywise larger $|r|\,\mathsf{V} + |\vartheta|\left(\mathsf{D}^\partial + |\mathsf{J}|^{\mathsf{T}}\,\mathsf{G}^{1/2}\,|\mathsf{K}^{\mathrm{o}}|\,\mathsf{G}^{1/2}\,|\mathsf{J}|\right)$. The floor of the initial guess always applies. A floor evaluated at a later iterate x with true residual r raises the stop level only under a guard (issue 97),

$$\varepsilon^\star(\mathsf{x}) = \begin{cases} \max\left\{\varepsilon^\star,\ \epsilon_{\mathrm{fl}}(\mathsf{x})\right\}, & \varepsilon^\star < \|\mathsf{r}\|_{\mathsf{V},\infty} < \|\mathsf{r}_0\|_{\mathsf{V},\infty} \text{ and } \epsilon_{\mathrm{fl}}(\mathsf{x}) < \tfrac{1}{100}\|\mathsf{r}_0\|_{\mathsf{V},\infty}, \\ \varepsilon^\star, & \text{otherwise :} \end{cases} \tag{11.9}$$

the iterate must have reduced the residual, and its floor must be small compared with the initial residual. An iterate that grows along a null direction makes $|\mathsf{L}|\,|\mathsf{x}|$ arbitrarily large while the residual barely changes; without the guard such an iterate certified convergence (issue 97: a pressure of about 10^{19} was reported as converged). The guard always compares with the original $\varepsilon^\star$, so raised levels do not accumulate.

Restarted iteration. A solve proceeds as follows.

1. Copy the initial guess and the integrated right-hand side. If the operator is singular, project the right-hand side onto the range of L and remove the component means of the guess (below).

2. Compute r_0, $\epsilon_{\mathrm{fl}}(\mathsf{x}_0)$ and $\varepsilon^\star$ (11.7); stop if $\|\mathsf{r}_0\|_{\mathsf{V},\infty} \le \varepsilon^\star$.

3. Run the Krylov method from the current iterate with the remaining iteration budget. CG and BiCGSTAB stop when a callback finds $\|\cdot\|_{\mathsf{V},\infty} \le \varepsilon^\star$ for their recursively updated residual; Krylov.jl's own tolerances are set to zero. GMRES and FGMRES expose the residual only at the end of an Arnoldi pass; they stop on the Euclidean norm,
$$\|\mathsf{r}\|_2 \le \varepsilon^\star \min\left\{1, \min_{c:\,\mathsf{V}_{cc}>0} \mathsf{V}_{cc}\right\} \quad\Longrightarrow\quad \|\mathsf{r}\|_{\mathsf{V},\infty} \le \varepsilon^\star, \tag{11.10}$$
because $|\mathsf{r}_c| \le \|\mathsf{r}\|_2$ for every c (the cap at one only tightens the test when every volume exceeds one). With right preconditioning their residual is that of the unpreconditioned system.

4. Remove the component means, recompute the true residual $\mathsf{b} - \mathsf{L}\mathsf{x}$ and stop if it meets the guarded level (11.9), if it is not finite, or if the pass made no iteration; otherwise continue with step 3 until the budget is spent.

5. Remove the component means, recompute the true residual and the guarded level, and report `converged` if the residual is finite and meets it.

A recursively updated residual drifts from $\mathsf{b} - \mathsf{L}\mathsf{x}$ by round-off, which the strong norm amplifies in cut cells with a small volume; the restart replaces it by the true residual without changing the discretisation. If BiCGSTAB returns a non-finite iterate with a preconditioner (a $0/0$ before an almost exact intermediate residual is detected), the pass is repeated without the preconditioner. CG receives the preconditioner as Krylov.jl's `M`, applied by `ldiv!` as $\mathsf{z} = \mathfrak{B}\mathsf{r}$; BiCGSTAB, GMRES and FGMRES apply it from the right (`N`). CG requires $\mathfrak{B}$ to be symmetric and positive definite on the space in which the iterates live (Section 11.3.3).

Singular systems. For each unanchored component $\mathcal{K}$ the right-hand side is made compatible and the solution gauged,
$$\mathsf{b} \leftarrow \mathsf{b} - \mathsf{V}\mathbf{1}_{\mathcal{K}} \frac{\mathbf{1}_{\mathcal{K}}^{\mathsf{T}}\mathsf{b}}{\mathbf{1}_{\mathcal{K}}^{\mathsf{T}}\mathsf{V}\mathbf{1}_{\mathcal{K}}}, \qquad \mathsf{x} \leftarrow \mathsf{x} - \mathbf{1}_{\mathcal{K}} \frac{\mathbf{1}_{\mathcal{K}}^{\mathsf{T}}\mathsf{V}\mathsf{x}}{\mathbf{1}_{\mathcal{K}}^{\mathsf{T}}\mathsf{V}\mathbf{1}_{\mathcal{K}}}, \tag{11.11}$$
which is (10.5); the removed means and their anchors are reported, and with `compatibility=:er` the solve throws if $|\mathbf{1}_{\mathcal{K}}^{\mathsf{T}}\mathsf{b}| > 256\epsilon_{\mathrm{mach}} \max(1, \sum_{c\in\mathcal{K}} |\mathsf{b}_c|)$. Covered cells get a zero right-hand side and a zero solution. For a consistent symmetric positive semidefinite system, CG converges to a solution [11]; the gauge removes the null-space part of the iterate after every pass. The projections are applied independently on every component, so that a component with a net boundary flux does not disturb the others (issue 95).

Statistics and failure. `LinearSolverStats` records the iterations summed over the passes (`cycles`), the strong norms of r_0 and of the final residual, the final $\varepsilon^\star$ (`target_linf`), `converged`, the removed means and anchors, the names of the method and the preconditioner, and a residual scale: $\max(\|\mathsf{r}_0\|_{\mathsf{V},\infty}, \epsilon_{\mathrm{mach}})$ for `solve!`, $\|\mathsf{b}\|_{\mathsf{V},\infty}$ for the viscous solves and $1/\delta t^2$ as set by the projections, which also overwrite

`compatible` (Section 10.2). Failure messages quote the final residual divided by this scale. `solve!` and the projections report a missed tolerance; the viscous steps and the liquid-carrier projection throw, and a full `step!(flow, dt)` throws for a projection that is not converged.

Initial guesses. `solve!` starts from the solution field it is given; the one-velocity and liquid-carrier projections start from a zero increment; the viscous solves start from the destination field, the velocity before the viscous step; the CMDGFM projection may start from the previous increment (Section 11.5).

Storage. A solver owns its Krylov workspace, its solution, right-hand-side, residual and product vectors and the scratch of the factorised action. After a regrid these are resized in place: Krylov.jl's vectors keep their capacity, Krylov.jl's own warm-start flag is cleared, and a GMRES basis longer than the new number of unknowns forces a new workspace. Section 11.6 reports the allocations of repeated solves.

Other preconditioners. The symbols `:amg` (AlgebraicMultigrid.jl, smoothed aggregation or Ruge–Stüben, with a pseudo-inverse coarse solve for a singular operator), `:ilu` (IncompleteLU.jl's Crout ILU [12], drop tolerance 10^{-3}), `:ichol` (the limited-memory incomplete $\mathsf{L}\mathsf{D}\mathsf{L}^{\mathsf{T}}$ of LimitedLDLFactorizations.jl, symmetric operators only), a factory $\mathsf{L} \mapsto \mathfrak{B}$ or a prebuilt object with `ldiv!` replace the tree multigrid; they act on the assembled matrix.

Implementation. `EllipticSolver(operator; krylov=Krylov.CgWorkspace, krylov_options=(;), preconditioner=:tree_geometric, preconditioner_options=(;), multigrid=MultigridOptions())` and `solve!(solution, solver, rhs; rtol=1e-10, atol=0, max_cycles=100, compatibility=:project)`; `LinearSolverOptions(; krylov=nothing, preconditioner=:auto)` of `NavierStokesOptions`, where `:auto` is `:tree_geometric` and `krylov=nothing` is CG for the pressure, CG for two-phase diffusion and the rule of Section 11.4 for one-phase diffusion; `residual_norms`, `LinearSolverStats`, `algebraic_multigrid_preconditioner`, `incomplete_lu_preconditioner`, `incomplete_cholesky_preconditioner`. Driver `_solve_krylov_with_stats!` (`src/solvers/multigrid/elliptic_solver.jl`), `_run_krylov_reliably!` (`src/solvers/multigrid/elliptic_solver.jl`), `_run_krylov!` (`src/solvers/multigrid/elliptic_solver.jl`) and `StrongResidualCallback` (`src/solvers/multigrid/elliptic_solver.jl`); level and floor `_roundoff_residual_floor!` (`src/solvers/multigrid/cycles.jl`) (11.8), `_guarded_roundoff_target!` (`src/solvers/multigrid/elliptic_solver.jl`) (11.9), `_gmres_absolute_tolerance` (`src/solvers/multigrid/elliptic_solver.jl`) (11.10); norms `_integrated_residual_norms` (`src/solvers/multigrid/cycles.jl`) (11.6); gauge `_project_component_rhs!` (`src/solvers/multigrid/elliptic_solver.jl`) and `_remove_component_volume_means!` (`src/solvers/multigrid/elliptic_solver.jl`) (11.11); methods `_krylov_traits` (`src/solvers/multigrid/cycles.jl`), resizing `_resize_krylov_workspace!` (`src/solvers/multigrid/cycles.jl`) and `_reconfigure_elliptic_solver!` (`src/solvers/multigrid/elliptic_solver.jl`); matrix preconditioners `_construct_matrix_preconditioner` (`src/solvers/multigrid/cycles.jl`), `build_algebraic_multigrid_preconditioner` (`ext/TwoPhaseAMRAlgebraicMultigridExt.jl`),

`build_incomplete_cholesky_preconditioner` (`ext/TwoPhaseAMRLimitedLDLFactorizationsExt.jl`). Tests:
`test/solvers.jl` ("final iterate cannot inflate the original stopping target"),
`test/solvers.jl` ("guarded iterate roundoff floor"),
`test/solvers.jl` ("roundoff-limited heterogeneous Poisson solve"),
`test/solvers.jl` ("shared Krylov residual scales and copy order"),
`test/solvers.jl` ("Krylov trait dispatch preserves direct solver results"),
`test/solvers.jl` ("Krylov workspace resize and re-solve"),
`test/solvers.jl` ("external preconditioners support in-place-only ldiv!"),
`test/operators/poisson.jl` ("external Krylov solver and independent preconditioner"),
`test/operators/poisson.jl` ("periodic nullspace convention"),
`test/operators/poisson.jl` ("split-domain Poisson compatibility and gauges").

11.3 Tree geometric multigrid

The default preconditioner of the cell systems is one V-cycle of a geometric multigrid method on a sequence of coarser block trees; the operator is rediscretised on every tree, the cell transfer is a Jacobi-smoothed piecewise-constant interpolation in the manner of smoothed aggregation [30], and the coarsest system is factorised. Geometric multigrid for quadtree and octree AMR is also used by, e.g., Afivo [26]. Here every coarse mesh is again a complete block tree, obtained by pruning, so that every level operator is a composite-mesh operator of Chapter 3. The viscous hierarchies of Section 11.4 reuse the coarsening, smoothing and coarse solve with a different transfer.

11.3.1 Hierarchy

The solver meshes $\mathcal{T}_0^{\mathrm{MG}}, \mathcal{T}_1^{\mathrm{MG}}, \ldots, \mathcal{T}_J^{\mathrm{MG}}$ start with the composite mesh $\mathcal{T}_0^{\mathrm{MG}}$. Above the root level, $\mathcal{T}_{j+1}^{\mathrm{MG}}$ replaces every complete family of four leaf blocks of $\mathcal{T}_j^{\mathrm{MG}}$ by its parent (pruning `:all`; with `:deepest` it caps the deepest level instead), and the tree constructor restores the 2:1 balance. Once all blocks are roots, $\mathcal{T}_{j+1}^{\mathrm{MG}}$ merges 2×2 root blocks, which halves the base mesh, while both root counts are even and every periodic axis keeps at least four cells (the gradient resolves a two-cell direction, whose half-period tie the face orientation settles, issue 36; the coarsest level keeps this resolution floor). Coarsening stops when the mesh has at most `coarse_size` cells (default 64), when $J+1$ reaches `max_levels` (default 32), when no coarser mesh exists, or, with pruning `:all`, when a pruned mesh is not smaller. If coarsening stops early the last mesh is factorised however large it is.

11.3.2 Coarse operators and transfer

Rediscretisation. The operator of $\mathcal{T}_{j+1}^{\mathrm{MG}}$ is the cell operator (11.1) on that mesh, with its own face metric, the same r, ϑ and open sides, and restricted coefficients,

$$\mathsf{L}_{j+1} = \mathsf{L}\left(\mathcal{T}_{j+1}^{\mathrm{MG}};\ \gamma^{j+1},\ \gamma^{\partial,j+1}\right), \qquad \gamma_F^{j+1} = \sum_{f \subset F} \frac{|f \cap F|}{|F|}\, \gamma_f^{j}, \tag{11.12}$$

where the sum runs over the faces f of $\mathcal{T}_j^{\mathrm{MG}}$ that lie on the line of the coarse face F: the length-weighted arithmetic mean along F (boundary coefficients likewise). No Galerkin

product $\mathsf{P}_j^{\mathrm{MG}\mathsf{T}}\mathsf{L}_j\mathsf{P}_j^{\mathrm{MG}}$ is formed. With a solid and the solver's geometry passed to the hierarchy, each coarse mesh carries its own geometry: the solid fraction of a coarse cell is the volume mean of the fractions of its fine cells, reconstructed by PLIC, which defines the coarse apertures, open volumes and active cells; the material part γ_F^{m} of the coefficient is the mean of $\gamma_f^{\mathrm{m}} = \gamma_f/(a_f^{\mathrm{o}})^2$ weighted by $a_f^{\mathrm{o}}\,|f \cap F|/|F|$ over the fine faces with $a_f^{\mathrm{o}} > 0$ (the aperture-weighted mean over all fine faces if there are none), and $\gamma_F = \gamma_F^{\mathrm{m}}(a_F^{\mathrm{o}})^2$. Without geometry, a coarse cell of a pressure operator with inactive cells is active if one of its fine cells is active, and then has its full volume.

Transfer. The tentative transfer assigns each cell of $\mathcal{T}_j^{\mathrm{MG}}$ to the coarse cell that contains its centre,

$$\left(\mathsf{P}_j^0\right)_{cC} = \begin{cases} 1, & \boldsymbol{x}_c \in C, \\ 0, & \text{otherwise}, \end{cases} \qquad c \in \mathcal{T}_j^{\mathrm{MG}},\ C \in \mathcal{T}_{j+1}^{\mathrm{MG}}, \tag{11.13}$$

which is well defined because the meshes are nested. One damped-Jacobi step smooths it,

$$\mathsf{P}_j^{\mathrm{MG}} = \left(\mathsf{I} - \omega_j^{\mathrm{P}}\mathsf{D}_j^{-1}\mathsf{A}_j\right)\mathsf{P}_j^0, \qquad \omega_j^{\mathrm{P}} = \frac{4}{3\Lambda_j^{\mathrm{G}}}, \tag{11.14}$$

with A_j the stiffness $\mathsf{S} + \mathsf{D}^{\partial}$ of L_j with its inactive rows (the full L_j on a matrix-free finest level; the two coincide for a pressure operator, $r = 0$, $\vartheta = 1$), D_j its diagonal, rows with a non-positive diagonal left unsmoothed, and the Gershgorin bound

$$\Lambda_j^{\mathrm{G}} = \max_{i:\,\mathsf{A}_{ii}>0} \sum_{i'} \frac{|\mathsf{A}_{ii'}|}{\mathsf{A}_{ii}} \;\geq\; \max\left|\mathrm{spec}\left(\mathsf{D}_j^{-1}\mathsf{A}_j\right)\right| \tag{11.15}$$

(with $\omega_j^{\mathrm{P}} = 0$ if $\Lambda_j^{\mathrm{G}} = 0$). The weight $4/(3\Lambda^{\mathrm{G}})$ uses the Gershgorin bound in place of the spectral radius of $\mathsf{D}_j^{-1}\mathsf{A}_j$ [20, Thm. 4.6]; for the five-point operator of a uniform mesh it gives $\omega^{\mathrm{P}} = 2/3$. Restriction is the transpose, $\mathsf{P}_j^{\mathrm{MG}\mathsf{T}}$, applied to integrated residuals. If the fine or the coarse level has inactive cells (cut-cell pressure, or $r = 0$ with inactive cells), the entries that couple an inactive fine or coarse cell are set to zero and every active fine row is normalised,

$$\left(\mathsf{P}_j^{\mathrm{MG}}\right)_{cC} \leftarrow \frac{\left(\mathsf{P}_j^{\mathrm{MG}}\right)_{cC}}{\sum_{C' \notin \mathcal{C}^{\times}} \left(\mathsf{P}_j^{\mathrm{MG}}\right)_{cC'}}, \qquad c, C \notin \mathcal{C}^{\times}, \qquad \text{so that} \quad \mathsf{P}_j^{\mathrm{MG}}\mathbf{1}_{j+1}^{\mathrm{act}} = \mathbf{1}_j^{\mathrm{act}}\,: \tag{11.16}$$

the transfer preserves the null vector of the active domain, which is one on the active and zero on the covered cells; a transfer that preserved the constant background vector would send coarse corrections into covered cells. An active fine row with a zero sum is an error. The sparsity patterns of the transfers and coarse matrices are fixed per topology; coefficient changes only refresh their values (Section 11.3.4).

11.3.3 V-cycle

Smoother. A sweep of damped Jacobi on level j is

$$\mathsf{x} \leftarrow \mathsf{x} + \omega_j^{\mathrm{J}}\,\mathsf{D}^{-1}\left(\mathsf{b} - \mathsf{L}_j\mathsf{x}\right), \qquad \omega_j^{\mathrm{J}} = \min\left\{1,\ \frac{4}{3\Lambda_j^{\mathrm{G}}}\right\}, \tag{11.17}$$

with D and Λ_j^{G} those of L_j; a non-positive diagonal is an error, except a zero diagonal (an isolated cell) on a singular level. Since the eigenvalues λ of $\mathsf{D}^{-1}\mathsf{L}_j$ lie in $[0, \Lambda_j^{\mathrm{G}}]$, $\omega_j^{\mathrm{J}}\lambda \leq \frac{4}{3} < 2$: the sweep reduces the error in the energy norm. On a uniform mesh the rows of the five-point operator give $\Lambda^{\mathrm{G}} = 2$ and $\omega^{\mathrm{J}} = 2/3$.

Component projections. On a level with unanchored components the cycle applies, with $\mathsf{v}_{\mathcal{K}} = \mathsf{V}_j \mathbf{1}_{\mathcal{K}}$,

$$\mathsf{Q}_j \mathsf{x} = \mathsf{x} - \sum_{\mathcal{K}} \mathbf{1}_{\mathcal{K}} \frac{\mathsf{v}_{\mathcal{K}}^{\mathsf{T}} \mathsf{x}}{\mathsf{v}_{\mathcal{K}}^{\mathsf{T}} \mathbf{1}_{\mathcal{K}}}, \qquad \mathsf{Q}_j^{\mathsf{T}} \mathsf{b} = \mathsf{b} - \sum_{\mathcal{K}} \mathsf{v}_{\mathcal{K}} \frac{\mathbf{1}_{\mathcal{K}}^{\mathsf{T}} \mathsf{b}}{\mathsf{v}_{\mathcal{K}}^{\mathsf{T}} \mathbf{1}_{\mathcal{K}}}, \tag{11.18}$$

the removal of the volume means of the solution and its transpose, the compatible projection of an integrated right-hand side, both over the unanchored components of $\mathcal{T}_j^{\mathrm{MG}}$. Coarsening may merge, split or anchor components; every level uses its own.

Cycle. Applied to an integrated residual r, the preconditioner returns $\mathfrak{B}\mathsf{r} = \mathsf{x}_0$ of the following recursion with $\mathsf{b}_0 = \mathsf{r}$, $\nu_1 = \nu_2 = 2$ sweeps by default:

1. on a singular level, $\mathsf{b}_j \leftarrow \mathsf{Q}_j^{\mathsf{T}} \mathsf{b}_j$; on $\mathcal{T}_J^{\mathrm{MG}}$ solve directly (below) and return;
2. $\mathsf{x}_j = 0$; ν_1 sweeps (11.17), each followed by $\mathsf{x}_j \leftarrow \mathsf{Q}_j \mathsf{x}_j$ on a singular level;
3. $\mathsf{b}_{j+1} = \mathsf{P}_j^{\mathrm{MG}\,\mathsf{T}} (\mathsf{b}_j - \mathsf{L}_j \mathsf{x}_j)$, and x_{j+1} by the recursion on level $j+1$;
4. $\mathsf{x}_j \leftarrow \mathsf{Q}_j \left(\mathsf{x}_j + \mathsf{P}_j^{\mathrm{MG}} \mathsf{x}_{j+1}\right)$ (Q_j on a singular level only), then ν_2 sweeps as in step 2.

Coarse solve. The last level is factorised by sparse LU (UMFPACK [6], through Julia's `lu`). If it has m unanchored components with indicator matrix $\mathsf{E} = [\mathbf{1}_{\mathcal{K}_1}, \ldots, \mathbf{1}_{\mathcal{K}_m}]$, the factorised matrix is the bordered system

$$\begin{bmatrix} \mathsf{L}_J & \mathsf{V}_J \mathsf{E} \\ \mathsf{E}^{\mathsf{T}} \mathsf{V}_J & 0 \end{bmatrix} \begin{bmatrix} \mathsf{x}_J \\ \boldsymbol{\mu} \end{bmatrix} = \begin{bmatrix} \mathsf{b}_J \\ 0 \end{bmatrix}, \tag{11.19}$$

which is nonsingular and returns the solution with zero volume mean on every component; for the compatible b_J of step 1 the multipliers $\boldsymbol{\mu}$ vanish. A factorisation is refactorised in place when the pattern of the new matrix, including stored zeros, equals the stored one: the values are copied and only the numerical factorisation is repeated, reusing the symbolic analysis; otherwise it is factorised anew.

Symmetry and definiteness. CG needs a preconditioner that is symmetric and positive definite on the space of its iterates. The V-cycle is arranged for this: restriction is the transpose of prolongation, the same number of sweeps of the same symmetric smoother precede and follow the coarse correction, the projections enter in transposed pairs, and the coarse solve is exact on the compatible space. Let $\mathfrak{B}_j$ be the cycle started on level j and $\mathcal{W}_j$ the volume-mean-free space of the unanchored components of $\mathcal{T}_j^{\mathrm{MG}}$, on which $\langle \mathsf{x}, \mathsf{y} \rangle_j = \mathsf{x}^{\mathsf{T}} \mathsf{L}_j \mathsf{y}$ is an inner product. With $\mathsf{S} = \mathsf{Q}_j (\mathsf{I} - \omega_j^{\mathrm{J}} \mathsf{D}^{-1} \mathsf{L}_j)$ and $\nu = \nu_1 = \nu_2$ the error of one cycle is

$$\mathsf{I} - \mathfrak{B}_j \mathsf{L}_j = \mathsf{S}^{\nu}\, \mathsf{T}\, \mathsf{S}^{\nu}, \qquad \mathsf{T} = \mathsf{I} - \mathsf{Q}_j \mathsf{P}_j^{\mathrm{MG}}\, \mathfrak{B}_{j+1}\, \mathsf{P}_j^{\mathrm{MG}\,\mathsf{T}} \mathsf{L}_j. \tag{11.20}$$

Since $\mathsf{L}_j \mathsf{Q}_j = \mathsf{L}_j$, both factors are selfadjoint for $\langle \cdot, \cdot \rangle_j$, and $\langle \mathsf{T}\mathsf{x}, \mathsf{x} \rangle_j = \|\mathsf{x}\|_j^2 - \mathsf{r}^{\mathsf{T}} \mathsf{P}_j^{\mathrm{MG}} \mathfrak{B}_{j+1} \mathsf{P}_j^{\mathrm{MG}\,\mathsf{T}} \mathsf{r} \leq \|\mathsf{x}\|_j^2$ with $\mathsf{r} = \mathsf{L}_j \mathsf{x}$ whenever $\mathfrak{B}_{j+1}$ is symmetric positive semidefinite, as the exact coarse

solve (11.19) is. The smoother contracts, $\|\mathsf{S}\mathsf{x}\|_j < \|\mathsf{x}\|_j$ on $\mathcal{W}_j \setminus \{0\}$, exactly when $0 < \omega_j^{\mathrm{J}}\lambda < 2$ on the nonzero spectrum of $\mathsf{D}^{-1}\mathsf{L}_j$, which (11.17) guarantees ($\omega_j^{\mathrm{J}}\lambda \le \frac{4}{3}$). Hence $\langle \mathfrak{B}_j \mathsf{L}_j \mathsf{e}, \mathsf{e}\rangle_j \ge \|\mathsf{e}\|_j^2 - \|\mathsf{S}^{\nu}\mathsf{e}\|_j^2 > 0$, and by induction from the coarsest level $\mathfrak{B}$ is symmetric, positive definite on the mean-free space and zero on the $\mathsf{v}_{\mathcal{K}}$. The argument uses no relation between $\mathsf{P}_j^{\mathrm{MG}}$ and L_j, hence no Galerkin product; for nonsingular levels it is Theorems 2 and 3 of [25]. It needs $\nu_1 = \nu_2$; CG with the tree multigrid preconditioner requires equal pre- and post-sweep counts, while nonsymmetric Krylov methods may use unequal counts (issue 128). The argument fails for $\omega_j^{\mathrm{J}}\lambda > 2$. It does not make the cycle a contraction: $\lambda_{\max}(\mathfrak{B}\mathsf{L})$ exceeds one on an operator with an open side (1.45) and on a Dirichlet-inactive operator without geometry (2.2), so the cycle is a preconditioner, not a stand-alone solver, and its convergence factor is not bounded a priori; it is measured (Section 11.6).

Checks (`solv_vcycle_check`). The dense matrix $\mathfrak{B}$, assembled from the $\mathfrak{B}$-images of the unit vectors, on the unit square with 16×16 and 32×32 uniform cells, on a composite mesh of 2560 cells, for the drop at density ratio 1000 and for the split domain of Section 11.1 with its geometry: $\mathfrak{B}$ is symmetric to 8.9×10^{-16} (largest entry of $\mathfrak{B} - \mathfrak{B}^{\mathsf{T}}$ relative to the largest entry of $\mathfrak{B}$); its columns have zero volume mean on every unanchored component to 1.4×10^{-16}; it couples active and covered cells by exactly zero; and the eigenvalues of $\mathfrak{B}\mathsf{L}$ on the volume-mean-free space lie in $[0.673, 1.035]$, with condition numbers $\kappa(\mathfrak{B}\mathsf{L}) = 1.27, 1.34$ (uniform), 1.38 (composite), 1.52 (drop) and 1.43 (split); operators with an open side or a Dirichlet-inactive region, outside these fixtures, give $\kappa = 2.0$ and 2.9 (derivation audit, `proof_review/ch-solvers/`). The CG bound $\|\mathsf{e}_m\|_{\mathsf{L}} \le 2q^m\|\mathsf{e}_0\|_{\mathsf{L}}$, $q = (\sqrt{\kappa}-1)/(\sqrt{\kappa}+1)$, for the energy norm of the error gives the factor $q = 0.07$ (uniform, 32×32) and 0.10 (drop) per iteration, and $\lceil \ln(2/\varepsilon^{\mathrm{rel}})/\ln(1/q) \rceil$ iterations, within one of the measured 10 and 12. The energy norm of the error reduces by this factor to within $8\,\%$ (never above 1.02 times it); the mean reduction of the strong residual (11.24), 0.08 and 0.12, is in another norm, for which the bound does not apply, and exceeds the factor by up to $15\,\%$. On every level of these and of the fixtures of Table 11.2 $\Lambda^{\mathrm{G}} = 2$, so $\omega^{\mathrm{J}} = 0.667$.

Threaded sums. The component sums $\mathsf{v}_{\mathcal{K}}^{\mathsf{T}}\mathsf{x}$ and $\mathbf{1}_{\mathcal{K}}^{\mathsf{T}}\mathsf{b}$ in (11.18) are evaluated in fixed chunks of 1024 cells: each chunk is summed in cell order and the chunk sums are added in chunk order. The chunk boundaries depend only on the number of cells, so the result, and every solve, is bitwise the same for any number of threads; a level of at most 1024 cells is summed left to right as before.

Threaded transfers. Prolongation and its transpose run the package's own products for every thread count: a fine cell gathers its coarse neighbours with `_prolongate!` (`src/solvers/multigrid/hierarchy.jl`) in increasing coarse index and a coarse cell sums its fine cells with `_restrict!` (`src/solvers/multigrid/hierarchy.jl`) in stored order, both starting from zero and fusing with `muladd`. The unthreaded path uses the same kernels, because SparseArrays' `mul!` fuses only from Julia 1.12; using it on one thread would make the result depend on the Julia version. The row schedule of the prolongation is cached per matrix object and non-zero count, so the values may be refreshed in place but the sparsity pattern may not change at constant non-zero count.

11.3.4 Refresh and reuse

Coefficient refresh. On a fixed mesh a new face coefficient, as for a two-phase pressure operator after the interface has moved, refreshes the hierarchy in place: the coarse activity (without geometry), the restricted coefficients and boundary coefficients of every level and the values of the coarse matrices, the smoothed transfers (11.14) with their restriction (11.16), the Jacobi weights and diagonals, the components, and the coarse factorisation, refactorised in place. The two-phase step refreshes the pressure hierarchy in every step when the densities differ, because a hierarchy built for an earlier interface position loses effectiveness at a large density ratio. A change between a pressure ($r = 0$) and a shifted operator, of the inactive-cell condition or of the open sides requires a new hierarchy.

Reuse after a regrid. A hierarchy for the new mesh may be built with the previous one: after each coarsening step the new coarse mesh is compared with the meshes of the old hierarchy (the same base mesh, blocks, faces, dual volumes and boundary faces), and at the first match the old levels from there on, with their transfers, restriction maps and coarse factorisation, are appended and refreshed as above. This requires the same pruning, r, ϑ, open sides and inactive condition and no more than `max_levels` levels; a hierarchy with geometry shares only the coarse mesh objects and rebuilds its levels. In a flow the pressure hierarchy is rebuilt this way after every regrid.

Checks (`solv_reuse_check`). After moving the refined band of the composite mesh of Table 11.2 (32 base cells per side), the new hierarchy of 5 levels reuses 3, and its V-cycle equals that of a hierarchy built from scratch to 4.8×10^{-16}. After moving the drop on a fixed 64×64 mesh, the refreshed hierarchy keeps its coarse factorisation object and its V-cycle equals that of a new hierarchy to 6.2×10^{-16} (both relative to the largest entry).

Implementation. `TreeGeometricMultigridPreconditioner(operator; multigrid=MultigridOptions(), pruning=:all, reuse=nothing, geometry=nothing)` and `MultigridOptions(; coarse_size=64, pre_sweeps=2, post_sweeps=2, max_levels=32)`. Coarsening `_tree_next_coarse_mesh!` (`src/solvers/multigrid/tree_coarsening.jl`), `_all_leaf_pruned_mesh` (`src/solvers/multigrid/tree_coarsening.jl`), `_capped_tree_mesh` (`src/solvers/multigrid/tree_coarsening.jl`), `_coarsened_base_mesh` (`src/solvers/multigrid/tree_coarsening.jl`); coefficients `_face_coefficient_restriction` (`src/solvers/multigrid/tree_coarsening.jl`) and `_restrict_face_coefficients!` (`src/solvers/multigrid/tree_coarsening.jl`) (11.12), `_restrict_geometry_material!` (`src/solvers/multigrid/tree_coarsening.jl`), `_restrict_geometry_to_mesh` (`src/embedded/geometry.jl`), `_coarse_cut_cell_activity` (`src/solvers/multigrid/geometric_preconditioner.jl`); transfer `_tree_cell_prolongation` (`src/solvers/multigrid/tree_coarsening.jl`) (11.13), `_refresh_smoothed_transfer!` (`src/solvers/multigrid/hierarchy.jl`) (11.14), `_restrict_transfer_to_active_domain!` (`src/solvers/multigrid/hierarchy.jl`) (11.16); weights `_jacobi_weight!` (`src/solvers/multigrid/hierarchy.jl`) (11.15); cycle `_vcycle!` (`src/solvers/multigrid/cycles.jl`), `_smooth!` (`src/solvers/multigrid/cycles.jl`) (11.17), `_multigrid_remove_component_means!` (`src/solvers/multigrid/hierarchy.jl`) and `_multigrid_project_component_rhs!` (`src/solvers/multigrid/`

hierarchy.jl) (11.18), _coarse_solve! (src/solvers/multigrid/cycles.jl) with _augmented_multigrid_coarse_matrix (src/solvers/multigrid/geometric_preconditioner.jl) (11.19) and _refactorize_coarse! (src/solvers/multigrid/geometric_preconditioner.jl); matrix-free finest level FactorizedEllipticStencil (src/solvers/multigrid/hierarchy.jl); refresh _refresh_geometric_multigrid! (src/solvers/multigrid/geometric_preconditioner.jl), called by _refresh_pressure_substep! (src/two_phase/stepping.jl); reuse _tree_reuse_suffix! (src/solvers/multigrid/tree_coarsening.jl) and _same_tree_topology (src/solvers/multigrid/tree_coarsening.jl), called by _rebuild_topology_dependent_state! (src/navier_stokes/workspace.jl). Tests: test/operators/poisson.jl ("Rediscretized pressure hierarchy on pruned trees"), test/operators/poisson.jl ("Root-block Poisson hierarchy and stopping"), test/operators/poisson.jl ("Periodic base meshes retain four cells per period"), test/operators/poisson.jl ("Root-block hierarchy reuse after AMR"), test/operators/poisson.jl ("Root-block V-cycle Poisson CG comparison"), test/operators/poisson.jl ("Jacobi policies retain smoothing semantics"), test/operators/poisson.jl ("CSC transfer schedules preserve stored entries and order"), test/operators/poisson.jl ("multigrid component projections"), test/operators/poisson.jl ("geometric multigrid Poisson convergence"), test/solvers.jl ("multigrid component merge preserves symmetric positive action"), test/solvers.jl ("multigrid coarse gauges follow coefficient connectivity"), test/solvers.jl ("multilevel mixed gauges follow activity changes"), test/solvers.jl ("coarse augmentation follows coarse connectivity on hierarchy reuse"), test/solvers.jl ("Coarse sparse LU symbolic reuse"), test/solvers.jl ("cut-cell root coarsening and AMR mesh reuse"), test/navier_stokes/two_phase_navier_stokes.jl ("Tree solvers retain matching hierarchy levels across AMR").

11.4 Shifted viscous systems

One phase. The implicit midpoint step (9.17) solves, for each velocity component k separately,

$$\widetilde{\mathsf{V}}\left(\mathsf{I} - s\,\widetilde{\mathfrak{L}}_k\right) u^{**} = \widetilde{\mathsf{V}}\,\mathsf{w}, \qquad s = \tfrac{1}{2}\delta t\,\mu/\rho, \tag{11.21}$$

with the explicit terms and wall sources of (9.17) in w. The matrix $\widetilde{\mathsf{V}}(\mathsf{I}-s\widetilde{\mathfrak{L}}_k)$ is symmetric for every s iff $\widetilde{\mathsf{V}}\widetilde{\mathfrak{L}}_k$ is. The automatic Krylov selection uses CG when this weighted symmetry is guaranteed for both components and BiCGSTAB otherwise; it certifies it by checking $\widetilde{\mathsf{V}}\widetilde{\mathfrak{L}}_k = -\mathsf{S}_k^{\nu}$ entrywise in the assembled arithmetic, and rejects an operator whose matrix was replaced or perturbed after construction. The unembedded and embedded geometric operators satisfy $\widetilde{\mathsf{V}}\widetilde{\mathfrak{L}}_k = -\mathsf{S}_k^{\nu}$ with symmetric positive semidefinite stiffness, so their shifted systems select CG on composite as well as uniform meshes (issue 80, issue 128). The default preconditioner is, per component, a tree multigrid with the coarsening of Section 11.3.1 (counted in faces of the axis), rediscretisation with the same viscosity, boundary conditions and shift on every coarse mesh, no Galerkin product, the smoother (11.17), and a dense LU coarse solve for at most 256 unknowns, a sparse LU otherwise. The coarse matrices always use the conservative symmetric stiffness of their rediscretized operator, so the V-

cycle remains symmetric for CG even on composite coarse meshes. Its transfer is the linear staggered interpolation from the coarser tree: a fine face on a coarse face line takes the length-weighted mean of the coarse faces that it overlaps, a new face interpolates linearly between the two neighbouring coarse face lines of its axis, a missing line at a physical wall contributes the homogeneous boundary value, and periodic distances wrap; restriction is the transpose, and the transfer is not smoothed. A new step size changes only s: the values of every level are rewritten on the fixed pattern (once extended by the diagonal if s was zero), the Jacobi weights recomputed and the coarse factorisation refactorised; an unchanged s returns at once. After a regrid the hierarchy is rebuilt with the reuse of Section 11.3.4, which here also requires the same boundary type and the same s: `step!(flow, dt)` builds the new operator with the step just taken, so runs with a fixed step reuse the coarse levels. Matrix preconditioners (`:ilu`, …) are built from the symmetric shifted stiffness $s\mathsf{S}_k^{\nu} + \widetilde{\mathsf{V}}$.

Two phases. The coupled system (9.18), $(\mathsf{M}_\rho + \frac{1}{2}\delta t\, \mathsf{E}^{\mathsf{T}}\mathsf{W}_\mu \mathsf{E})u^{**} = \mathsf{b}$, is symmetric positive definite and solved by CG with the factorised action of Section 9.4. Its preconditioner is a tree multigrid for the coupled system of all faces: the finest level is the assembled matrix; each coarser tree (coarsened to at most $2\,$`coarse_size` face unknowns above the root level and `coarse_size` cells below it) rediscretises the stress operator with the logarithm of the cell viscosity averaged over the children (a geometric mean of the viscosity), the staggered density averaged as in (11.12), and the same δt; the transfer is the linear staggered interpolation above on all faces; smoothing and the sparse LU coarse solve are those of Section 11.3.3. Because the density, the viscosity and δt change in every step, the hierarchy is refreshed on every solve. With two velocities there is one such solve for the mass-weighted mixture velocity (Section 9.4).

Implementation. `StaggeredEllipticOperator(diffusion_operator; reaction=1, diffusion=1)`, `StaggeredEllipticSolver(operator; krylov=nothing, preconditioner=:tree_geometric, multigrid)` and `solve!(solution, solver, rhs; tolerance=sqrt(eps(T)), max_cycles=100)`; `implicit_midpoint_diffusion!`. Method selection and shift `StaggeredEllipticSolver` (`src/navier_stokes/implicit_diffusion.jl`), `_staggered_shift_is_symmetric` (`src/navier_stokes/implicit_diffusion.jl`), `_update_staggered_elliptic!` (`src/navier_stokes/implicit_diffusion.jl`), `_refresh_staggered_component!` (`src/navier_stokes/implicit_diffusion.jl`); hierarchy `TreeStaggeredGeometricMultigridPreconditioner` (`src/navier_stokes/implicit_diffusion.jl`), `TreeStaggeredMultigridComponent` (`src/navier_stokes/implicit_diffusion.jl`), `DenseStaggeredCoarseLU` (`src/navier_stokes/implicit_diffusion.jl`), transfer `_tree_face_prolongation` (`src/solvers/multigrid/tree_coarsening.jl`); matrix preconditioners `_staggered_stable_matrix` (`src/operators/elliptic.jl`); two phases `VariableStaggeredDiffusionSolver` (`src/two_phase/diffusion/variable_diffusion_solver.jl`), `TreeVariableDiffusionPreconditioner` (`src/two_phase/diffusion/variable_diffusion_preconditioner.jl`), `_refresh_tree_variable_diffusion_preconditioner!` (`src/two_phase/`

`diffusion/variable_diffusion_preconditioner.jl`), `_cell_viscosity_from_log!` (`src/two_phase/diffusion/variable_diffusion_preconditioner.jl`); rebuild with the current step `step!` (`src/navier_stokes/flow_simulation.jl`). Tests: `test/operators/staggered_diffusion.jl` ("implicit midpoint staggered elliptic solve"), `test/operators/staggered_diffusion.jl` ("staggered base-mesh multigrid"), `test/operators/staggered_diffusion.jl` ("external BiCGSTAB and staggered preconditioners"), `test/navier_stokes/navier_stokes.jl` ("fixed-topology staggered diffusion refresh"), `test/navier_stokes/navier_stokes.jl` ("zero-shift expansion and non-tree staggered refresh"), `test/navier_stokes/navier_stokes.jl` ("large staggered coarse systems reuse sparse symbolic analysis"), `test/navier_stokes/navier_stokes.jl` ("staggered refresh after AMR hierarchy reuse"), `test/two_phase/variable_staggered_diffusion.jl` ("Rediscretized variable diffusion on pruned trees"), `test/two_phase/variable_staggered_diffusion.jl` ("Variable diffusion refactorization with Float32 coefficients"), `test/two_phase/variable_staggered_diffusion.jl` ("Variable stress shifted solve on fixed and adapted trees"), `test/navier_stokes/two_phase_navier_stokes.jl` ("Two-phase pressure and diffusion solver configuration").

11.5 Projection systems

One velocity. The projection (10.4) is a cell solve with $r = 0$, $\vartheta = 1$ and the stop level (10.7), from a zero increment, by CG with the tree multigrid; with a solid the hierarchy is built with the solver's geometry.

CMDGFM. The two-block system (10.25) is nondimensionalised before the iteration with the largest domain extent L and the largest density ρ_0: the unknowns are scaled by $\rho_0(L/\delta t)^2$ (pressure) and $L/\delta t^2$ (jump), the rows by $(\delta t/L)^2$ and $\delta t^2/L$,

$$\hat{\mathsf{L}} = \mathsf{D}_\mathrm{r}\mathsf{L}\mathsf{D}_\mathrm{c}, \qquad \hat{\mathsf{b}} = \mathsf{D}_\mathrm{r}\mathsf{b}, \qquad \mathsf{x} = \mathsf{D}_\mathrm{c}\hat{\mathsf{x}}, \qquad \hat{\mathfrak{B}} = \mathsf{D}_\mathrm{c}^{-1}\mathfrak{B}\,\mathsf{D}_\mathrm{r}^{-1}, \tag{11.22}$$

so that $\hat{\mathsf{L}}\hat{\mathfrak{B}} = \mathsf{D}_\mathrm{r}\mathsf{L}\mathfrak{B}\mathsf{D}_\mathrm{r}^{-1}$ is similar to $\mathsf{L}\mathfrak{B}$, so the preconditioned spectrum is unchanged. BiCGSTAB itself is not invariant under a nonuniform D_r: on the scaled system it is BiCGSTAB on the unscaled right-preconditioned system in the inner product $\mathsf{u}^\mathsf{T}\mathsf{D}_\mathrm{r}^2\mathsf{v}$ (shadow residual, residual-minimising step and stopping norm), so the scaling weighs the pressure and the jump rows in these. BiCGSTAB is right-preconditioned; each pass requests the relative reduction $\min(\varepsilon^p, \varepsilon^p/e)$ of the scaled residual, with e the largest current residual defect of Section 10.5.3, that of the pressure rows omitted for the gauge included, and the passes restart from the true residual until the defects are met, the iteration budget, against which every pass counts, is spent, or a pass leaves the solution unchanged. A pass whose right-hand side is so small that the requested reduction would fall below the machine precision is solved for the normalised right-hand side. Until PR #130 the loop stopped after eight passes, also with budget left (deviation 11.1). The default block preconditioner is lower block-triangular,

$$\mathfrak{B}\begin{bmatrix}\mathsf{r}_p\\ \mathsf{r}_\xi\end{bmatrix} = \begin{bmatrix}\mathsf{z}_p\\ \mathsf{z}_\xi\end{bmatrix}, \qquad \mathsf{z}_p = \mathsf{G}\,\mathfrak{B}_\mathrm{c}\,\mathsf{G}^\mathsf{T}\mathsf{r}_p, \qquad \mathsf{z}_\xi = \tilde{\mathsf{B}}^{-1}\left(\mathsf{r}_\xi - \mathsf{B}^p\,\mathsf{z}_p\right), \tag{11.23}$$

where $\mathfrak{B}_c$ is one V-cycle of the tree multigrid of the pressure operator on all cells (Section 11.3), $(\mathsf{G}\mathsf{x})_c = \mathsf{x}_c - \mathsf{x}_{a(c)}$ maps a cell function to the retained cells $\mathcal{C}^{\mathrm{r}}$ with $a(c)$ the anchor of the component of c (so G^{T} adds a residual to its cell and subtracts it from the anchor), and $\tilde{\mathsf{B}}$ is the Crout ILU with drop tolerance 10^{-3} of the jump block B. The ILU factor is reused while the jump block is unchanged, and without a solid the one-velocity operator and hierarchy are shared when their settings agree. The alternatives are `:ilu`, one ILU with drop tolerance 10^{-3} of the whole scaled matrix $\hat{\mathsf{L}}$, and no preconditioner. The first guess is zero or the previous increment with a jump regenerated from the current residual, accepted only if it reduces the largest defect below 0.95 times that of zero and lowers the Euclidean residual (Section 10.5.3).

Implementation. `project!` of `ProjectionWorkspace` (one velocity), `CMDGFMProjection(; preconditioner=:block, preconditioner_options=(;))` with `CMDGFMProjectionSolver`. One velocity `project!` (`src/navier_stokes/projection.jl`); CMDGFM `_project_cmdgfm_assembled!` (`src/two_phase/projection/cmdgfm_projection.jl`) with the defect of the gauge rows `_cmdgfm_anchor_residual_defect` (`src/two_phase/projection/cmdgfm_projection.jl`), scaling `_cmdgfm_krylov_scaling!` (`src/two_phase/projection/cmdgfm_projection.jl`) (11.22) with `_ScaledProjectionPreconditioner` (`src/two_phase/projection/cmdgfm_projection.jl`), preconditioner `CMDGFMBlockPreconditioner` (`src/two_phase/projection/cmdgfm_projection.jl`) and `CMDGFMReducedPressurePreconditioner` (`src/two_phase/projection/cmdgfm_projection.jl`) (11.23), `_cmdgfm_pressure_preconditioner` (`src/two_phase/projection/cmdgfm_projection.jl`), first guess `_cmdgfm_initial_solution!` (`src/two_phase/projection/cmdgfm_projection.jl`). Tests: `test/two_phase/two_velocity/projection/cmdgfm_projection.jl` ("CMDGFM block-preconditioned BiCGSTAB"), `test/two_phase/two_velocity/projection/cmdgfm_projection.jl` ("Two-velocity projection convergence is independent of physical units"), `test/two_phase/two_velocity/projection/cmdgfm_pressure_reuse.jl` ("CMDGFM shares only compatible scalar pressure data"), `test/two_phase/two_velocity/projection/cmdgfm_pressure_reuse.jl` ("CMDGFM initial-guess residual gate and zero fallback"), `test/two_phase/two_velocity/projection/cmdgfm_projection.jl` ("CMDGFM AMR whole-system ILU regression (issue #126)") (the composite fixture of Section 11.6 converges with both preconditioners), `test/two_phase/two_velocity/projection/cmdgfm_projection.jl` ("CMDGFM omitted gauge row and iteration budget"), `test/navier_stokes/navier_stokes.jl` ("pressure failure reports convergence diagnostics").

11.6 Measured convergence

Table 11.2 gives, for representative systems, the iterations m to the stop level and the mean reduction of the strong residual per iteration,

$$\bar{\varrho} = \left(\frac{\|\mathsf{r}_m\|_{\mathsf{V},2}}{\|\mathsf{r}_0\|_{\mathsf{V},2}} \right)^{1/m}, \tag{11.24}$$

with the default preconditioner and without one. The fixtures are the unit square with walls and 8×8-cell blocks; the composite meshes refine twice the blocks within 0.02 of the

Table 11.2. Iterations m and mean residual reduction $\bar{\varrho}$ per iteration (11.24) with the default tree multigrid (levels $J+1$) and without preconditioner, from a zero guess (`figures/solvers.jl`, `solv_pressure_cases` and `solv_viscous_cases`).

			multigrid		none	
System	unknowns	levels	m	$\bar{\varrho}$	m	$\bar{\varrho}$
Cell systems, CG						
Poisson, 64×64	4096	4	10	0.08	292	0.92
Poisson, 128×128	16384	5	10	0.09	576	0.959
Poisson, 256×256	65536	6	10	0.09	1148	0.979
Poisson, composite, base 32^2	7552	5	10	0.09	506	0.953
Poisson, composite, base 64^2	19648	6	11	0.09	780	0.969
drop, ratio 1000, 128×128	16384	5	15	0.18	6710	0.9964
drop, ratio 1000, composite	19648	6	16	0.19	10417	0.9976
split cut-cell domain	16384	5	12	0.11	690	0.964
One-phase viscous, both components						
$\sigma = 1$, 128×128, CG	32512	5	8	0.04	35	0.50
$\sigma = 100$, 128×128, CG	32512	5	12	0.12	310	0.93
$\sigma = 100$, composite, CG	39496	6	12	0.11	283	0.92
Two-phase viscous, drop, CG						
64×64	8064	4	25	0.32	733	0.964
composite, base 64^2	39496	6	34	0.43	1994	0.986

circle of radius 0.2371 centred at $(0.513, 0.468)$; the drop is the liquid inside that circle with densities $(1000, 1)$, so that the face coefficient $1/\widetilde{\rho}_f$ varies by three decades; the split domain is the solid band of Section 11.1 on 128×128 cells. The cell systems have a right-hand side of deterministic noise, $\mathsf{b}_c = \mathrm{frac}(43758.5453 \sin(12.9898\, c)) - \frac{1}{2}$, and stop at $\varepsilon^{\mathrm{rel}} = 10^{-10}$; the one-phase viscous systems are those of unit viscosity with the shift $s = \sigma h^2$, h the smallest cell width, for both components (the larger count), with free-slip walls; the two-phase viscous systems take the drop with viscosities $(0.1, 0.002)$ and $\delta t = 1$, for which $\frac{1}{2}\delta t\,(\mathsf{E}^{\mathsf{T}}\mathsf{W}_\mu\mathsf{E})_{ff}/(\mathsf{M}_\rho)_{ff}$ reaches 43 (uniform) and 754 (composite); both stop at $\epsilon_{\mathrm{tol}} = 10^{-10}$.

Cell systems. With the multigrid preconditioner CG needs 10–10 iterations on 64×64 to 256×256 uniform cells, independently of the mesh size, while without preconditioner the count doubles with every refinement, as the condition number $\mathcal{O}(h^{-2})$ of the thesis predicts for CG. The composite meshes, the density ratio 1000 and the two-component cut-cell domain keep the count within 10–16; the drop is the slowest, with $\bar{\varrho} = 0.19$, where CG without preconditioner needs 10417 iterations. Each multigrid iteration costs one V-cycle with four Jacobi sweeps per level.

Viscous systems. The shifted one-phase systems are well conditioned for small shifts ($\sigma = 1$: 8 against 35 iterations) and the multigrid keeps the count low at $\sigma = 100$ on the uniform mesh (12 against 310). On the composite mesh multigrid-preconditioned CG needs 12 iterations with $\bar{\varrho} = 0.11$, against 283 without preconditioner. The previous nonsymmetric operator needed 42 BiCGSTAB iterations and 199 without the preconditioner. For a separate half-refined 16×16 base mesh, $s = 0.1$, smooth right-hand side, coarse size 16 and tolerance 10^{-10}, the two component counts changed from $4/4$ BiCGSTAB iterations

(periodic) and $5/5$ (walled) to $8/8$ CG and $9/9$ CG. For the coupled two-phase stress the counts are 25 and 34 against 733 and 1994.

CMDGFM projection (`solv_cmdgfm_cases`). One projection of a divergent predictor with a phase jump, density ratio 1000, $\delta t = 0.01$, $\varepsilon^p = 10^{-10}$ and the drop above, from a zero guess: on 64×64 cells (4287 unknowns, of which 192 jumps) BiCGSTAB needs 15 iterations with the block preconditioner (11.23), 8 with the whole-system ILU and 26758 without preconditioner; on the composite mesh with base 32×32 (7933 unknowns, 382 jumps) the counts are 16, 12 and 54248. The block system has no single volume norm; its reduction is measured with the residual defect that the projection compares with ε^p (Section 10.5.3),

$$e(\mathsf{r}) = \delta t^2 \max\left\{ \max_{c \in \mathcal{C}^{\mathrm{r}},\ |c^{\mathrm{o}}|>0} \frac{|\mathsf{r}_c|}{|c^{\mathrm{o}}|},\ \max_{f \in \mathcal{F}^{\xi}} \frac{|\mathsf{r}_f|}{h_f} \right\}, \qquad \bar{\varrho} = \left(\frac{e(\mathsf{r}_m)}{e(\mathsf{b})} \right)^{1/m}, \tag{11.25}$$

with b the right-hand side, the residual of the zero guess ($e(\mathsf{b}) = 6.9 \times 10^{-1}$ on the uniform fixture), and r_m the final true residual. The block preconditioner reduces e by $\bar{\varrho} = 0.13$ (uniform) and 0.16 (composite) per iteration, the whole-system ILU by 0.02 and 0.08, and BiCGSTAB without preconditioner by 0.9988 and 0.9995.

Deviation from the thesis 11.1 (Whole-system ILU for the CMDGFM (fixed)). With `preconditioner=:ilu` the CMDGFM projection of the composite fixture above stopped at $\varepsilon^p = 10^{-10}$ without meeting the defects (`converged=false`) after the eight BiCGSTAB passes of the restart loop, although iteration budget remained (issue 126). Since PR #130 the restarts continue within the budget, and the fixture converges after 12 iterations with defects of at most 1.4×10^{-13}. The whole-system ILU needs fewer iterations than the default block preconditioner on both fixtures but allocates up to 92 MiB per projection on them; the counts of the default block preconditioner did not change with the fix.

Allocations. A repeated cell solve allocates nothing, with and without the multigrid, on every fixture of Table 11.2, as does one V-cycle. A repeated one-phase viscous solve allocates 0 bytes per call as well: until PR #130 it allocated 624 bytes per call, independently of the mesh, when `solve!` iterated over the two components of different types to zero their Dirichlet faces, and the contract test that asserts zero bytes failed (issue 127). The two-phase viscous solve allocates up to 155 KiB per call with the multigrid, for the refresh of its hierarchy and coarse factorisation, and nothing without it; a repeated CMDGFM projection with the block preconditioner allocates 215 and 398 KiB on the two fixtures.

> *Implementation.* `docs/techref/figures/solvers.jl`: fixtures `solv_uniform`, `solv_amr`, `solv_drop_field` and `solv_band_solid`; counts `solv_pressure_cases`, `solv_viscous_cases` and `solv_cmdgfm_cases`; spectra `solv_vcycle_check`; components `solv_component_check`; reuse `solv_reuse_check`. Allocation tests:
> `test/performance/performance.jl` ("elliptic solver allocation contracts"),
> `test/performance/performance.jl` ("staggered solver allocation contract"),
> `test/performance/two_phase_performance.jl` ("Variable-stress solver allocation contracts").

— 12 —

Refinement criteria and conservative transfer

Equation reference, audited against commit `2ca00c4`.

Between two time steps the forest of Chapter 2 may change. Refinement indicators give every leaf block one label, *refine*, *keep* or *coarsen*; `adapt!` turns the labels into a new graded forest, rebuilds the schedules (Section 2.5) and transfers every registered field to the new cells and faces. This chapter states the indicators with their thresholds and limits, the regrid, the transfer maps of each kind of field, the quantities that these maps conserve and those that they do not, how the time step deals with the divergence that a transfer leaves, and the changes measured per regrid. The thesis states three refinement criteria [19, §2.5] and a fourth one for wave impacts [19, §8.3.2], but not how fields are transferred; the transfer maps below are the package's. A regrid has no time discretisation: it maps the fields at the end of one step to the new mesh before the next (Section 12.3.2).

12.1 Labels, limits and their combination

Every indicator i assigns to every leaf block $\mathcal{B}$ a label $\lambda_i(\mathcal{B}) \in \{\textsc{refine}, \textsc{keep}, \textsc{coarsen}\}$. The labels of all indicators are combined block by block in the order of precedence of the thesis [19, §2.5],

$$\lambda(\mathcal{B}) = \max\nolimits_{\prec}\big(\{\textsc{coarsen}\} \cup \{\, \lambda_i(\mathcal{B}) \mid i \,\}\big), \qquad \textsc{coarsen} \prec \textsc{keep} \prec \textsc{refine}, \tag{12.1}$$

and every block is labelled `KEEP` if there is no indicator. The built-in indicators share three limits.

- *Activity window.* An indicator is active if $t_{\text{init}} \le t \le t_{\text{end}}$, with t the time passed to `adapt!`; an inactive indicator labels every block `COARSEN`, the neutral element of (12.1).
- *Region.* A block whose closed rectangle does not intersect the closed rectangle $\mathcal{R}$ is labelled `COARSEN`, after and regardless of the rule of the indicator (so it also escapes a level floor).

- *Level bounds.* The error-based indicators reduce a cell estimate e_c to the block value $E_{\mathcal{B}} = \max_c e_c$ over the $N_1 N_2$ owned cells and label, with bounds $0 \le L_{\min} \le L_{\max} \le L$ and thresholds $0 \le \varepsilon_{\mathrm{c}} < \varepsilon_{\mathrm{r}}$, the first case that applies of

$$\lambda_i(\mathcal{B}) = \begin{cases} \textsc{refine}, & \ell(\mathcal{B}) < L_{\min}, \\ \textsc{coarsen}, & \ell(\mathcal{B}) > L_{\max}, \\ \textsc{refine}, & E_{\mathcal{B}} > \varepsilon_{\mathrm{r}} \text{ and } \ell(\mathcal{B}) < L_{\max}, \\ \textsc{coarsen}, & E_{\mathcal{B}} < \varepsilon_{\mathrm{c}} \text{ and } \left(\ell(\mathcal{B}) = 0 \text{ or } \ell(\mathcal{B}) > L_{\min}\right), \\ \textsc{keep}, & \text{otherwise.} \end{cases} \tag{12.2}$$

  The band indicators have one target level instead (Section 12.2.1).

A label acts on a whole block: *refine* splits all its cells, *coarsen* is a vote for merging it with its three siblings (Section 12.3), and roots are never merged. The thresholds $\varepsilon_{\mathrm{c}} < \varepsilon_{\mathrm{r}}$ give value hysteresis; the mesh options add temporal hysteresis (Section 12.3).

> *Implementation.* `AbstractRefinementIndicator` and `refinement_labels!(labels, indicator; time)` or `(workspace, indicators; time)`; combination `_combine_refinement_actions` (`src/mesh/adaptivity/indicators.jl`) with `_refinement_priority` (`src/mesh/adaptivity/indicators.jl`), in `refinement_labels!` (`src/mesh/adaptivity/workspace.jl`); limits `_indicator_active` (`src/mesh/adaptivity/indicators.jl`), `_apply_refinement_region!` (`src/mesh/adaptivity/indicators.jl`), `_refinement_levels` (`src/mesh/adaptivity/indicators.jl`) and `_indicator_can_coarsen` (`src/mesh/adaptivity/indicators.jl`). Keywords of every built-in indicator: `t_init`, `t_end` (default $\mp\infty$), `xmin`, `xmax`, `ymin`, `ymax` (default unbounded); of the error-based ones also `minimum_level`, `maximum_level` (default 0 and L). Tests: `test/mesh/adaptivity.jl` ("interpolation-error refinement indicators") (combination), `test/mesh/adaptivity.jl` ("AMR criterion time windows and level bounds"), `test/mesh/adaptivity.jl` ("indicator spatial regions"), `test/examples/stokes3_example.jl` ("Stokes-wave refinement window").

12.2 Refinement indicators

12.2.1 Interface and solid bands

`InterfaceRefinementIndicator` keeps the blocks near the liquid–gas interface at a target level $L_* \le L$. With $\mathcal{B}^+$ the owned cells of $\mathcal{B}$ together with n_{b} rings of its halo slots ($0 \le n_{\mathrm{b}} \le H$, (2.9)), whose conditional fractions are those of (2.10) after the fractions have been snapped by (4.5) (zero gradient beyond a wall), the preliminary label $\widehat{\lambda_i(\mathcal{B})}$ is

$$\widehat{\lambda_i(\mathcal{B})} = \begin{cases} \textsc{refine}, & \mathcal{B} \text{ near and } \ell(\mathcal{B}) < L_*, \\ \textsc{keep}, & \mathcal{B} \text{ near and } \ell(\mathcal{B}) = L_*, \\ \textsc{coarsen}, & \text{otherwise,} \end{cases} \tag{12.3}$$

$$\mathcal{B} \text{ near} \iff \min_{\substack{c \in \mathcal{B}^+ \\ \alpha_c^{\mathrm{o}} > 0}} C_c^{\mathrm{l}} < 1 - \varepsilon^{\alpha} \text{ and } \max_{\substack{c \in \mathcal{B}^+ \\ \alpha_c^{\mathrm{o}} > 0}} C_c^{\mathrm{l}} > \varepsilon^{\alpha},$$

with the fraction floor ε^{α} of the interface. The extrema skip the covered cells of the ring ($\alpha_c^{\circ} = 0$): such a cell holds no fluid, so its conditional fraction is zero by construction rather than gas ((4.4)) and a wall it touches is wetted by one phase alone, not an interface (issue 119). The owned values are snapped, so a block that contains a mixed cell, or a liquid and a gas cell, is near; so is a block whose ring holds such values, which detects an interface on the faces between pure cells of neighbouring blocks. `GeometryRefinementIndicator` applies the same rule to the solid fraction of the solid's reconstruction (Chapter 7) with its floor: it keeps a band around the solid boundary at its target level and coarsens blocks inside the solid and away from it.

The ring in (12.3) is measured in cells of the block's own level. A parent therefore sees twice the physical buffer width of its children. To prevent the resulting coarsen–refine cycle (issue 117), a coarsening guard acts on each complete family of four leaf siblings at level $1 \le \ell \le L_*$ whose labels, after the spatial-region filter, are all COARSEN. Let P be their parent. The guard expands P by $n_{\mathrm{b}}\, h_{\ell-1,k}$ on each side in direction k and takes the extrema, over the same open cells, of the conditional fractions of all owned leaf cells that overlap the resulting rectangle in positive area, including periodic images. At a physical wall the rectangle is clipped to the domain. If these extrema satisfy the same strict test as the near condition in (12.3), all four siblings are instead labelled KEEP; otherwise their filtered labels are unchanged. The guard also applies when $n_{\mathrm{b}} = 0$, using the children's cells. Since extrema are taken before averaging to the hypothetical parent, it can conservatively keep a family even if that parent would not actually be near. The spatial-region filter is applied again after the guard, so siblings outside the region still request COARSEN.

Deviation from the thesis 12.1 (Interface refinement). The thesis' α-AMR (criterion 1) refines a block that contains both phases, $\max_{c\in\mathcal{B}} \alpha_c^{\pi} \ge \varepsilon^{\alpha}$ for $\pi = \mathrm{l}$ and $\pi = \mathrm{g}$ with, for example, $\varepsilon^{\alpha} = 10^{-8}$, up to the maximum level of the criterion, and suggests coarsening otherwise [19, §2.5]. The code uses the conditional fractions with the fraction floor ($\varepsilon^{\alpha} = \sqrt{\epsilon_{\mathrm{mach}}} \approx 1.5 \times 10^{-8}$ by default) and strict inequalities, adds the ring of n_{b} halo slots (default 1), labels near blocks above L_* COARSEN, and protects sibling families with the prospective-parent guard. The solid band has no counterpart in the thesis.

Implementation. `InterfaceRefinementIndicator(interface; level, buffer_cells=1)` and `GeometryRefinementIndicator(geometry; level, buffer_cells=1)` with the common keywords of Section 12.1; `_fixed_interface_refinement_labels!` (`src/mesh/adaptivity/phase_criteria.jl`) (12.3) and the guard `_protect_fixed_band_coarsening!` (`src/mesh/adaptivity/phase_criteria.jl`), called by `refinement_labels!` (`src/mesh/adaptivity/phase_criteria.jl`) and `refinement_labels!` (`src/mesh/adaptivity/geometry_criteria.jl`); the fractions are refreshed by `_refresh_conditional_fraction!` (`src/interface/interface_field.jl`). Tests: `test/indicator/geometry.jl` ("Geometry-presence refinement indicator"), `test/mesh/adaptivity.jl` ("indicator spatial regions"), `test/mesh/adaptivity.jl` ("Fixed bands refine the interface and release distant blocks (#117)"), `test/mesh/adaptivity.jl` ("Static interface and geometry bands reach a fixed point (#117)"), `test/mesh/adaptivity.jl` ("Stationary droplet transfer does not regrid indefinitely (#117)"), `test/mesh/adaptivity.jl` ("Interface band skips covered cells of a wetted wall (#119)"), and `test/mesh/adaptivity.jl` ("Interface band keeps refining a

| wetted contact line (#119)").

12.2.2 Curvature

`CurvatureRefinementIndicator` reconstructs the interface and evaluates, in every owned cell with $0 < C_c^{\mathrm{l}} < 1$ (the open-fluid conditional fraction after snapping), and in every cut cell ($0 < \alpha_c^{\mathrm{o}} < 1$) that is full of liquid in its open part within a block that holds a mixed cell in its cells or its one-cell halo, with N_κ cells per radius and the coarsening ratio q_κ ($0 < 2q_\kappa < 1$),

$$e_c = \begin{cases} \infty, & c \text{ unresolved}, \\ N_\kappa \, \bar{h}_c \, |\kappa_c| \,, & \text{otherwise}, \end{cases} \tag{12.4}$$

$E_{\mathcal{B}} = 0$ if the block has no such cell, and labels the block by (12.2) with $\varepsilon_{\mathrm{r}} = 1$ and $\varepsilon_{\mathrm{c}} = q_\kappa$: it is refined if a cell has $\bar{h}_c \, |\kappa_c| > 1/N_\kappa$ and coarsened if all have $\bar{h}_c \, |\kappa_c| < q_\kappa / N_\kappa$. The curvature κ_c is that of Section 4.6, clipped by (4.17), so that $e_c \leq N_\kappa/2$ for a resolved cell. A cell is unresolved if κ_c is undefined, if no signed principal direction has three valid height columns, those through c and its two transverse neighbours (Section 4.4), or if it lies in a filament: along an axis $\boldsymbol{e}_k$ the seven stencil values $C^{\mathrm{l}}_{c+n\boldsymbol{e}_k}$, $|n| \leq 3$, of (4.10) are available, both end values are pure in the same phase and an inner value is not (all within $64\epsilon_{\mathrm{mach}}$). Covered cells and cells whose open part is filled by a single phase are not mixed by this criterion: in particular, a liquid-filled cut cell does not trigger unresolved-interface refinement merely because its full-cell fraction is between zero and one (issue 118). Such a cell counts only in a block that holds a mixed cell in its own cells or its one-cell halo, which is the contact-line case: the interface meets the solid inside the wetted part of the cut cell, so its conditional fraction is one and its curvature undefined while the interface it straddles is not resolved (issue 148). The test is per block because refinement is: a cell-level test would read a finer neighbour's restriction average at a level step, which is mixed even where the fine interface is resolved, so the coarse block would refine while its own children coarsened again. Coarsening siblings of a family that holds such a wetted cut cell are kept when the family plus a two-cell ring spans both phases, the same conservative parent test the fixed-band criteria use (Section 12.2.1); covered cells are excluded from that test because their conditional fraction is zero by construction, not gas (issue 118). Level bounds, other indicators and balancing still apply.

Deviation from the thesis 12.2 (Curvature refinement). The thesis' κ-AMR (criterion 2) keeps $\varepsilon_{\mathrm{c}}^\kappa \leq \max_c \bar{h}_c \, |\kappa_c| \leq \varepsilon_{\mathrm{r}}^\kappa$ (square cells) with $2\varepsilon_{\mathrm{c}}^\kappa < \varepsilon_{\mathrm{r}}^\kappa$, usually $\varepsilon_{\mathrm{r}}^\kappa = 3\varepsilon_{\mathrm{c}}^\kappa$, and also refines where both oppositely oriented local height functions of one direction are valid in a cell (a filament) or where an interface cell has no valid one [19, §2.5]. The code's thresholds are $\varepsilon_{\mathrm{r}}^\kappa = 1/N_\kappa$ and $\varepsilon_{\mathrm{c}}^\kappa = q_\kappa/N_\kappa$, so $2q_\kappa < 1$ is the thesis' condition and the default $q_\kappa = \frac{1}{3}$ its usual ratio; it uses the largest width of the cell, the clipped curvature and the seven-value filament test above. The condition $2q_\kappa < 1$ assumes that merging doubles $\bar{h}_c \, |\kappa_c|$; with the numerical curvature the ratio of a merged block to its children was 1.98 to 2.39 in the derivation audit (`proof_review/dev-amr-curvature/`), so a q_κ close to $\frac{1}{2}$ does not prevent re-refinement, while the default $\frac{1}{3}$ keeps a margin.

| *Implementation.* `CurvatureRefinementIndicator(interface; cells_per_radius=8, coarsen_ratio=1/3)` with the common keywords; `refinement_labels!` (`src/mesh/adaptivity/phase_criteria.jl`) (12.4),

contact-line cells `_wetted_cut_cell` (`src/mesh/adaptivity/phase_criteria.jl`), validity and filament test `_lhf_refinement_status` (`src/interface/height_functions.jl`), stencil values `_fill_height_values!` (`src/interface/height_functions.jl`). Tests: `test/indicator/phase/height_functions.jl` ("Curvature AMR detects invalid heights and filaments"), `test/mesh/adaptivity.jl` ("AMR criterion time windows and level bounds"), `test/mesh/adaptivity.jl` ("Curvature ignores liquid-filled sloped solid cuts (#118)"), `test/mesh/adaptivity.jl` ("Curvature refines contact-line cells of a wall-cut liquid wedge (#148)"), and `test/mesh/adaptivity.jl` ("Curvature keeps contact-line families across the interface (#148)"), and `test/mesh/adaptivity.jl` ("Contact-line refinement reaches a fixed point at a level step (#148)").

12.2.3 Vorticity

`VorticityRefinementIndicator` estimates the vorticity of a face velocity in every cell. Its cell velocity is, per component k, the mean of the length-weighted averages over the faces of $\mathcal{F}^{[k]}$ on the sides $s = \pm 1$ of c that carry a value, $S_k(c)$ (both sides, or one),

$$(\hat{\boldsymbol{u}}_c)_k = \frac{1}{\#S_k(c)} \sum_{s \in S_k(c)} \frac{\sum_{f \in \mathcal{F}_s(c)} |f| u_f}{\sum_{f \in \mathcal{F}_s(c)} |f|}, \tag{12.5}$$

with the side sets $\mathcal{F}_s(c)$ of (6.5) including boundary faces. The gradient is the least-squares fit over the face neighbours, and the vorticity its antisymmetric part,

$$\begin{aligned} \mathsf{G}_c &= \arg\min_{\mathsf{G}} \sum_{f \in \mathcal{F}_\Omega:\ c \in \mathcal{C}(f)} \frac{|f|}{|\boldsymbol{d}_f|^2} \left| \hat{\boldsymbol{u}}_{c^+(f)} - \hat{\boldsymbol{u}}_{c^-(f)} - \mathsf{G}\boldsymbol{d}_f \right|^2, \\ \omega_c &= (\mathsf{G}_c)_{21} - (\mathsf{G}_c)_{12}, \qquad e_c = \bar{h}_c \, |\omega_c| \,, \end{aligned} \tag{12.6}$$

with $\boldsymbol{d}_f$ the shortest periodic centroid displacement; ω_c is undefined (*unresolved*) if the 2×2 normal matrix has a determinant below $128\epsilon_{\mathrm{mach}}$ times its scale or the cell has no velocity. The labels follow (12.2) with the tolerances ε_{r} and ε_{c} (default $\varepsilon_{\mathrm{r}}/3$, $0 \le 2\varepsilon_{\mathrm{c}} < \varepsilon_{\mathrm{r}}$), and an unresolved cell forbids the coarsening case. For a phase velocity of a two-velocity flow (given with its interface, or registered with it in the AMR workspace), only cells in which the phase is present (open and $C_c^{\mathrm{l}} > 0$ for the liquid, < 1 for the gas) are estimated, only faces with phase volume carry values, and a missing cell velocity next to a present one is supplied, for the fit only, by the weighted least-squares affine fit (weights $|\boldsymbol{x}_{c'} - \boldsymbol{x}_c|^{-2}$) over the present cell velocities within two rings, or their weighted mean if that fit is singular.

Deviation from the thesis 12.3 (Vorticity refinement). The thesis' $\bar{\omega}$-AMR (criterion 3) uses the magnitude of the discrete curl at the vertices, interpolated to the cells, times the cell width, with $2\varepsilon_{\mathrm{c}} < \varepsilon_{\mathrm{r}}$ [19, §2.5]. The code fits the gradient of centred velocities (12.6), uses the largest cell width, evaluates each phase velocity separately and does not coarsen a block with an unresolved cell.

Implementation. `VorticityRefinementIndicator(velocity; refine_tolerance, coarsen_tolerance, interface, phase)`; labels `_vorticity_refinement_labels!` (`src/mesh/adaptivity/phase_`

`criteria.jl`) (phase association `_adaptive_refinement_labels!` (`src/two_phase/two_velocity/velocity_fields.jl`)); estimate `_cell_centered_velocity!` (`src/visualization/sampling.jl`) (12.5), `_vorticity_from_centered!` (`src/visualization/sampling.jl`) (12.6) and the stencil extension `_extrapolate_curl_stencil!` (`src/visualization/sampling.jl`), the same as the `Vorticity` of the visualisation. Test: `test/mesh/adaptivity.jl` ("vorticity refinement indicator").

12.2.4 Interpolation error

`InterpolationErrorIndicator` fills the halo of a cell field with its boundary rule (Section 2.4) and compares every owned value with the mean of its four axial neighbours on the block's lattice, relative to the local magnitude,

$$e_c = \frac{\left|\varphi_c - \frac{1}{4}\sum_{k=1}^{2}\left(\varphi_{c-\boldsymbol{e}_k} + \varphi_{c+\boldsymbol{e}_k}\right)\right|}{\max\left\{\varepsilon_{\mathrm{f}}, \left|\varphi_c\right|, \max_{k,\pm}\left|\varphi_{c\pm\boldsymbol{e}_k}\right|\right\}}, \tag{12.7}$$

where $c \pm \boldsymbol{e}_k$ are cells of the block or its halo slots and $\varepsilon_{\mathrm{f}} = \sqrt{\epsilon_{\mathrm{mach}}}$ by default; the labels follow (12.2) with $0 \le \varepsilon_{\mathrm{c}} < \varepsilon_{\mathrm{r}}$. The thesis names the bilinear interpolation error of Popinet as an alternative criterion that it does not use [19, §2.5]; this indicator has no counterpart there. For a smooth field the numerator is $\frac{1}{4}\bar{h}_c^2\left|\Delta\varphi\right| + O(\bar{h}_c^4)$ on square cells. To leading order, where $\Delta\varphi$ does not vanish, coarsening therefore multiplies the numerator by about four (by $4\cos^2(kh/2) < 4$ for a Fourier mode of wavenumber k), and the indicator of the sinusoidal test field of Section 12.8 grew by 3.83; as a heuristic, a merged block whose children were below ε_{c} is not refined again if $4\varepsilon_{\mathrm{c}} \le \varepsilon_{\mathrm{r}}$. This is no guarantee: where $\Delta\varphi$ vanishes the leading term is $O(\bar{h}_c^4)$ and the numerator grows by about 16 (for the harmonic field $1 + \delta(x^4 - 6x^2y^2 + y^4)$ it is $\delta\bar{h}_c^4$, for point values and cell averages alike), the normalisation changes with the stencil as well, and for fields that the mesh does not resolve the growth has no bound. The code requires only $\varepsilon_{\mathrm{c}} < \varepsilon_{\mathrm{r}}$.

Implementation. `InterpolationErrorIndicator(field; E_refine, E_coarsen, relative_floor=sqrt(eps(T)), boundary=ZeroGradient())` with the common keywords; `refinement_labels!` (`src/mesh/adaptivity/indicators.jl`) (12.7). `adapt!` registers the field of an indicator for transfer with the indicator's boundary rule. Test: `test/mesh/adaptivity.jl` ("interpolation-error refinement indicators").

Deviation from the thesis 12.4 (Instability refinement). The thesis' λ^*-AMR (criterion 4) refines interface blocks so that the most unstable wavelength of the local Kelvin–Helmholtz and Rayleigh–Taylor instability is resolved [19, §8.3.2]. It is not implemented.

12.3 The regrid

12.3.1 Marking, balancing and rebuilding

A call `adapt!(workspace, indicators; step, time)` computes the labels (12.1) and then edits the forest as follows.

1. If fewer than n_{u} steps have passed since the previous attempt (as counted by the argument `step`), nothing changes unless `force=true`; otherwise this call is the new previous attempt.
2. Every leaf labelled REFINE with $\ell(\mathcal{B}) < L$ is split into its four children (Section 2.2).
3. Every parent of four leaves is merged if all four are labelled COARSEN, none of them was created fewer than n_{d} steps before, and the parent's level is at least L_0. Because the refinements are made first, a family with one child labelled REFINE or KEEP is never merged.
4. Refinement-only balancing restores the grading (2.5); a merged parent that violates it is split again, which restores its old children.
5. If the leaf keys are unchanged, nothing is transferred. Otherwise the schedules of Section 2.5 are rebuilt, every registered field is transferred (Sections 12.4 to 12.6), and every new block whose parent was an old leaf records `step` as its creation step.

The transfer requires that every new leaf be an old leaf, a child of an old leaf, or the parent of four old leaves, and throws otherwise, so a regrid that succeeds changes every block by at most one level. Figure 12.1 shows labels of the earlier band criteria, the blocks they split and those split by the balance.

The transfer runs in a fixed order. (i) The absent phase velocities of a two-velocity field are marked with the old interface and their working values extended (Section 12.6). (ii) Every cell field is staged: an interface is reconstructed on the old mesh and the planes of the cells to be refined are copied, together with the solid's planes; halos are filled with the field's boundary rule. (iii) The old faces are grouped into lines; the leaf array, the faces, the dual volumes and the halo schedule are rebuilt and the generation of the topology caches is advanced. (iv) The fields are transferred in their order of registration. (v) Every interface is reconstructed on the new mesh, the solid first: its geometry is refreshed (Section 7.6), then the phase interface is reconstructed with the snapping of (4.5), then the staggered fractions of its `PhaseFaceScalar`s are recomputed. (vi) The phase velocities are marked absent again with the new interface.

Implementation. `adapt!(workspace, indicators or labels; step, time=0, force=false)` returns `AMRStats(changed, refined_blocks, coarsened_families)`; `AMRWorkspace(mesh, fields...)` and `register_adaptive_field!`; `MeshAdaptationOptions(update_interval=1, coarsen_delay=4, minimum_level=0)` gives n_{u}, n_{d} and L_0. Order `_adapt_with_labels!` (`src/mesh/adaptivity/adapt.jl`); tree edits `_apply_refinement_actions!` (`src/mesh/adaptivity/workspace.jl`) and `_can_coarsen` (`src/mesh/adaptivity/workspace.jl`), balance `_balance!` (`src/mesh/topology/mesh.jl`); creation steps `_actual_adaptation_counts!` (`src/mesh/adaptivity/adapt.jl`); rebuild `_rebuild_mesh_schedules!` (`src/mesh/adaptivity/field_transfer.jl`) and `_invalidate_mesh_topology_cache!` (`src/mesh/topology/mesh.jl`). Tests: `test/mesh/adaptivity.jl` ("in-place conservative adaptive field transfer") (interval, delay, storage reuse), `test/mesh/adaptivity.jl` ("adaptive refinement remains 2:1 graded"), `test/mesh/adaptivity.jl` ("multi-root adaptation and conservative

transfer"), `test/mesh/adaptivity.jl` ("non-dyadic dynamic balancing and transfer"), `test/mesh/adaptivity.jl` ("mesh adaptation option validation"), `test/mesh/amr_connectivity.jl` ("Connectivity epochs include same-size AMR changes").

12.3.2 When the mesh adapts

`step!(flow, dt)` uses an AMR workspace when refinement indicators are configured. It registers the velocity (the two phase velocities together with the interface), the pressure, the interface and its phase scalars, the solid, the additional `adaptive_fields` and the fields of the indicators. After the physical step passes its projection test (otherwise `step!` throws), the mesh's count of completed steps is advanced and `adapt!` is called with that count and the time at the end of the step; the indicators therefore see the end-of-step state. If the leaf set changed, `step!` rebuilds the topology-dependent operators and workspaces (pressure operator and solver, viscous operators, transport workspaces, reusing multigrid levels whose meshes are unchanged), associates the divergence that the transfer leaves with the untruncated proposed step (the supplied step for a direct call; Section 10.5.2), and, with two velocities, projects the transferred velocities by (10.31). No physical step is taken in a regrid. Standalone `adapt!` only transfers; a caller that transports with the transferred velocity must impose its constraint itself.

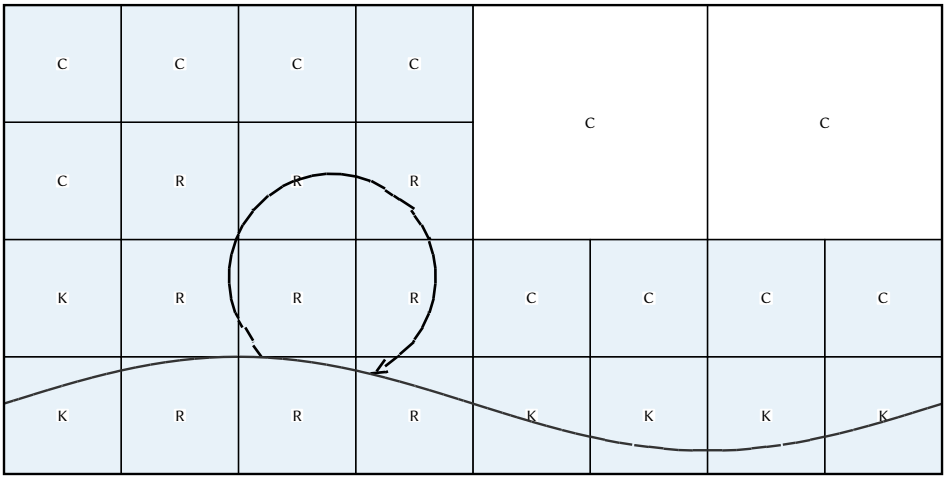
Implementation. `Adaptivity(; criteria, fields)` of `FlowSimulation` and the keywords `refinement_indicators`, `adaptive_fields` of `NavierStokesSystem`; registration `_navier_stokes_amr_workspace` (`src/navier_stokes/workspace.jl`), call in `step!` (`src/navier_stokes/flow_simulation.jl`), rebuild `_rebuild_topology_dependent_state!` (`src/navier_stokes/workspace.jl`), projection `_project_regridded_two_velocity!` (`src/two_phase/two_velocity/flow_workspace.jl`). Tests: `test/mesh/adaptivity.jl` ("Navier–Stokes optionally adapts and rebuilds its operators"), `test/navier_stokes/high_level_api.jl` ("FlowSimulation preserves Float32 through a regrid"), `test/navier_stokes/high_level_api.jl` ("regrid defects keep the transfer reference"), `test/navier_stokes/high_level_api.jl` ("full steps match simulate! on an adapting mesh").

12.4 Transfer of cell fields

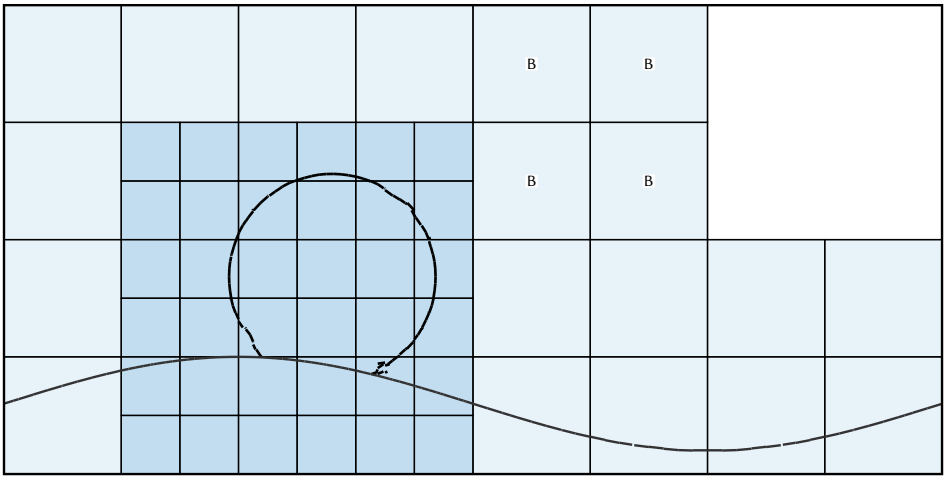
Cell values are averages (3.8). Let c be a cell of an old or new parent block and c_r, $r = 1, \dots, 4$, the cells of the children that it covers, with $|c_r| = \frac{1}{4}|c|$ and side $s_{r,k} = \pm 1$ of $\boldsymbol{x}_{c_r} - \boldsymbol{x}_c$ in direction k. Cells of blocks that the regrid keeps keep their values. A merged cell receives the mean, and a refined cell gives its children a limited linear reconstruction,

$$\mathcal{R}^{\mathrm{AMR}}: \quad \varphi_c = \frac{1}{4}\sum_{r=1}^{4} \varphi_{c_r}, \tag{12.8}$$

$$\mathcal{P}^{\mathrm{AMR}}: \quad \varphi_{c_r} = \varphi_c + \frac{1}{4}\sum_{k=1}^{2} s_{r,k}\,\sigma_k, \qquad \sigma_k = \mathrm{MC}\left(\varphi_c - \varphi_{c-\boldsymbol{e}_k},\, \varphi_{c+\boldsymbol{e}_k} - \varphi_c\right), \tag{12.9}$$



A. After the first regrid: the label of every block for the second one, R (REFINE), K (KEEP) and C (COARSEN).



B. After the second regrid; B: blocks split only by the 2:1 balance.

FIGURE 12.1. Two regrids of the walled fixture of Section 12.8, under `InterfaceRefinementIndicator` ($L_* = 2$) and `GeometryRefinementIndicator` ($L_* = 1$), both with $n_\mathrm{b} = 1$, starting from the base mesh ($h_0 = 1/16$, blocks of 8×8 cells, cells not drawn; shading by level as in Figure 2.1). Thick lines: PLIC segments of the drop; grey lines: the solid's segments. In (a) 9 blocks are labelled R, 6 K (the solid band at its target) and 11 C; the level-zero blocks labelled C cannot be merged, and in (b) 4 blocks were split because their neighbours reached level two. Drawn from `figures/generated/amr_regrid.tex`.

with the neighbours on the parent block's lattice (halo slots of (2.10) at its border, the field's boundary rule at a wall, by default zero gradient) and the monotonized-central slope

$$\mathrm{MC}(a,b) = \begin{cases} 0, & ab \le 0, \\ \operatorname{sgn}(a+b)\min\left\{\tfrac{1}{2}\,|a+b|\,,\ 2\,|a|\,,\ 2\,|b|\right\}, & ab > 0. \end{cases} \tag{12.10}$$

Both maps conserve the cell integral: the offsets of (12.9) sum to zero over the four children, so $\sum_r |c_r|\varphi_{c_r} = |c|\varphi_c$ for either map, and $\sum_{\mathcal{C}} \varphi$ (3.10) is unchanged up to rounding. Each child lies between the smallest and the largest of φ_c and its four neighbours, because a nonzero slope has the sign of both differences and $\frac{1}{4}\,|\sigma_k| \le \frac{1}{2}\min|\cdot|$ of them, and the two offsets add to at most the larger of the two neighbour differences; constants are reproduced exactly, and affine data whenever the two differences of each direction are equal, as on a uniform neighbourhood; they are not equal in the layer next to a coarser block (the halo slot copies the coarse cell, (2.10)) nor next to a wall with the zero-gradient rule, where the reconstruction is first order. At extrema the slope is zero and the reconstruction first order.

Check (`amr_transfer_check`). Over the 8 regrids of the two static fixtures of Section 12.8, an independent replica of (12.8) and (12.9) reproduces the transferred pressure to 0, no child leaves the bound, and the cell integral changes by at most 4.7×10^{-15} of $\sum_{\mathcal{C}} |p|$.

Implementation. Public `restrict!(parent, children, halo)` and `prolong!(children, parent, halo)` for whole blocks (children ordered low-low, high-low, low-high, high-high); the regrid uses `_prolong_child!` (`src/mesh/adaptivity/field_transfer.jl`) per child and `_transfer_cell_field!` (`src/mesh/adaptivity/field_transfer.jl`) (copies, storage pool, halo); slope `_mc_slope` (`src/mesh/transfer.jl`). Tests: `test/mesh/variables.jl` ("cell fields, halos, restriction, and prolongation"), `test/mesh/adaptivity.jl` ("in-place conservative adaptive field transfer"), `test/mesh/adaptivity.jl` ("multi-root adaptation and conservative transfer") (constants exactly).

12.5 Transfer of fractions, phase scalars and the solid

Volume fractions. The liquid fraction α_c^{l} of an `InterfaceField` (liquid volume per full cell volume) is restricted by the mean (12.8). A refined cell gives each child the liquid area of its own reconstruction over the child's quadrant $\hat{Q}_r \subset \hat{Q}$ of its local coordinates ((4.7); with a solid, of the open part cut by the parent's solid half-plane),

$$\alpha_{c_r}^{\mathrm{l}} = \operatorname{clamp}\left(4|\hat{Q}_r \cap \hat{Q}_c^{\mathrm{l}}|,\ 0,\ 4|\hat{Q}_r \cap \hat{Q}_c^{\mathrm{o}}|\right), \tag{12.11}$$

which is $\mathcal{A}(\hat{\boldsymbol{\eta}}_c, \hat{\phi}_c; \hat{Q}_r)$ without a solid; a pure cell gives pure children. Then the interface is reconstructed on the new mesh, which snaps every new cell by (4.5). Restriction conserves $\sum_c |c|\alpha_c^{\mathrm{l}}$ up to rounding; refinement conserves it up to the accuracy with which the plane constant reproduces the fraction (4.9), since the four quadrants partition the parent. The snap changes the liquid volume by at most $\varepsilon^{\alpha}|c^{\mathrm{o}}|$ per snapped cell, a child cut near a corner by its parent's line or a merged cell with an almost empty child.

Solid. The solid fraction is transferred in the same way with the solid's own planes ((7.7)), conserving the solid volume up to rounding, and the open fraction, apertures and

open dual volumes are recomputed from it (Section 7.6). The child's open part in (12.11) is cut by the same parent plane, so $\alpha^{\mathrm{l}}_{c_r} \le \alpha^{\mathrm{o}}_{c_r}$ up to rounding and the conditional fraction of every child stays in $[0, 1]$.

Phase scalars. The intrinsic values φ^{π}_{c} of a `PhaseScalar` are transferred with the weights $w^{\mathrm{l}}_{c} = \alpha^{\mathrm{l}}_{c}$ and $w^{\mathrm{g}}_{c} = \alpha^{\mathrm{o}}_{c} - \alpha^{\mathrm{l}}_{c}$, so that $|c| w^{\pi}_{c} \varphi^{\pi}_{c}$ is the amount $|c^{\mathrm{o}}| C^{\pi}_{c} \varphi^{\pi}_{c}$ of (5.28). A merged cell receives the weighted mean of its children (their plain mean if all weights vanish, and the common value if all four values are equal). A refined cell first forms the limited linear values $\varphi^{\mathrm{lin}}_{c_r}$ of (12.9) and then shifts them by one constant, using the children's weights $w^{\pi}_{c_r}$ of (12.11) before the snap (for the gas with the open area of the parent's solid half-plane),

$$\varphi^{\pi}_{c_r} = \varphi^{\mathrm{lin}}_{c_r} + \frac{4 w^{\pi}_{c} \varphi^{\pi}_{c} - \sum_s w^{\pi}_{c_s} \varphi^{\mathrm{lin}}_{c_s}}{\sum_s w^{\pi}_{c_s}} \quad \text{if } \sum_s w^{\pi}_{c_s} > \sqrt{\epsilon_{\mathrm{mach}}}, \tag{12.12}$$

and $\varphi^{\pi}_{c_r} = \varphi^{\mathrm{lin}}_{c_r}$ otherwise; if both slopes vanish the children take φ^{π}_{c} without shift. Then $\sum_r |c_r| w^{\pi}_{c_r} \varphi^{\pi}_{c_r} = |c| w^{\pi}_{c} \varphi^{\pi}_{c}$: both maps conserve the phase amounts up to rounding (without shift up to the volume defect of (12.11)) and up to the snap of the following reconstruction, which changes the amounts of a snapped cell. Unlike the limited values, the shifted ones need not lie between the parent's value and its neighbours' (up to $14\,\%$ of that range outside it on noise data in the derivation audit, `proof_review/eq-amr-vof/`).

Staggered phase scalars. The intrinsic face values of a `PhaseFaceScalar` are transferred by the face map of Section 12.6 without the integral correction, and their staggered fractions (5.23) are recomputed from the new reconstruction; their phase amounts are not conserved.

Check (`amr_transfer_check`). An independent replica of (12.11) by polygon clipping reproduces the fractions of all 12288 children of the static fixtures to 2.2×10^{-16} except 8 snapped children (a change of at most 3.1×10^{-19} in volume); per regrid the liquid volume changes by at most 3.8×10^{-16}, the solid volume by 7.4×10^{-16} and the phase amounts by 5.3×10^{-16}, relative.

Implementation.
`InterfaceProlongation` (`src/interface/interface_field.jl`) (12.11) (planes copied by `_prepare_prolongation!` (`src/interface/interface_field.jl`), mean `_restrict_parent!` (`src/interface/interface_field.jl`), reconstruction `_finish_prolongation!` (`src/interface/interface_field.jl`)); solid `GeometryProlongation` (`src/embedded/geometry.jl`); phase scalars `PhaseProlongation` (`src/vof/scalar_transfer.jl`) (12.12) and `_restrict_parent!` (`src/vof/scalar_transfer.jl`); staggered scalars `_finish_phase_face_transfer!` (`src/two_phase/phase_face_transfer.jl`). Tests: `test/indicator/geometry.jl` ("Geometry and phase PLIC transfer conserve physical volumes on AMR"), `test/two_phase/transport.jl` ("Geometric phase transfer through dynamic refinement and coarsening"), `test/two_phase/transport.jl` ("Intrinsic phase scalar AMR transfer convergence") (second order, amounts to $128\epsilon_{\mathrm{mach}}$), `test/two_phase/phase_face_lifecycle.jl` ("phase-face scalar survives uniform and mixed AMR schedules"), `test/indicator/geometry.jl` ("Geometry hierarchy is rebuilt consistently after AMR").

12.6 Transfer of face fields

Lines. A face field stores the normal component u_f on every owned face (Section 2.3). The old faces are grouped into lines: the faces of one axis k on one grid line $x_k = \text{const}$, in tangential order, positions measured on the lattice of the finest possible level L. For an old face g of a line, the tangential slope $\sigma_g^{\parallel}$ is the slope (12.10) of the two difference quotients with its neighbours on the line (the neighbours in tangential order, which across a gap of the line may be distant; across a periodic seam wrapped; one quotient at an end of the line). A new face f of axis k whose line existed and is covered by old faces over the extent of f receives

$$u_f = \begin{cases} u_g + \sigma_g^{\parallel}\,(\boldsymbol{x}_f - \boldsymbol{x}_g)\cdot\boldsymbol{e}_t, & f \subseteq g \text{ for an old face } g, \\ \dfrac{1}{|f|}\sum_{g\subset f} |g|\,u_g, & \text{otherwise,} \end{cases} \tag{12.13}$$

($t \neq k$; $u_f = u_g$ for $f = g$; no slope if the field was registered as piecewise constant). Any other new face, in practice an internal face of a refined cell, lies between two old lines: with $x^{\pm}$ the nearest line coordinates below and above $(\boldsymbol{x}_f)_k$ that have an old face $g^{\pm}$ containing the midpoint of f (wrapped over a periodic seam) and $u^{\pm} = u_{g^{\pm}} + \sigma_{g^{\pm}}^{\parallel}(\boldsymbol{x}_f - \boldsymbol{x}_{g^{\pm}})\cdot\boldsymbol{e}_t$,

$$u_f = u^- + \frac{(\boldsymbol{x}_f)_k - x^-}{x^+ - x^-}\left(u^+ - u^-\right), \tag{12.14}$$

or the one point value that exists. Boundary faces are transferred along their own lines. Two properties follow from (12.13): the two halves of a split face carry offsets of opposite sign and equal length, so $|f_1|u_{f_1} + |f_2|u_{f_2} = |g|u_g$, and a merged face carries the length-weighted mean; the integrated flux through every piece of an old line that survives is therefore unchanged. Every branch is exact for affine data, since the difference quotients of an affine function along a line are equal.

Divergence-preserving refinement. For a `FaceVelocity` (and for each phase velocity), let $F_c = |c|(\mathfrak{D}u)_c$ be the net integrated outflow of (3.11). For every cell c of a refined block, the defects of its children after the interpolation, $\varrho_r = \frac{1}{4}F_c - F'_{c_r}$, are removed by corrections $\delta_{12}, \delta_{34}, \delta_{13}, \delta_{24}$ of the integrated fluxes of the four internal faces, the faces between c_1 and c_2, c_3 and c_4 (normal $\boldsymbol{e}_1$) and c_1 and c_3, c_2 and c_4 (normal $\boldsymbol{e}_2$), children numbered low-low, high-low, low-high, high-high (Figure 12.2),

$$\delta_{12} = \tfrac{1}{4}\left(2\varrho_1 - \varrho_2 + \varrho_3\right), \quad \delta_{34} = \varrho_1 + \varrho_3 - \delta_{12}, \quad \delta_{13} = \varrho_1 - \delta_{12}, \quad \delta_{24} = \varrho_2 + \delta_{12},$$
$$u_f \leftarrow u_f + \delta_f/|f| \quad \text{on the four internal faces } f. \tag{12.15}$$

The children's outflows change by $\delta_{12} + \delta_{13}$, $-\delta_{12} + \delta_{24}$, $\delta_{34} - \delta_{13}$ and $-\delta_{34} - \delta_{24}$, which equal $\varrho_1, \varrho_2, \varrho_3$ and $-\varrho_1 - \varrho_2 - \varrho_3$. The last equals ϱ_4 because $\sum_r F'_{c_r}$ is the flux through the sides of c, which (12.13) preserves, so $\sum_r \varrho_r = 0$. Of the solutions, which differ by a circulation around the centre vertex, (12.15) is the one of least Euclidean norm in the integrated fluxes; for children whose sides differ in length, the minimiser of the velocity changes would differ. With the line property this gives, for the Cartesian divergence and

arbitrary face values,

$$\begin{aligned}(\mathfrak{D}u')_c &= (\mathfrak{D}u)_c && \text{(cells of kept blocks)},\\ (\mathfrak{D}u')_{c_r} &= (\mathfrak{D}u)_c && \text{(children of a refined cell)},\\ (\mathfrak{D}u')_c &= \tfrac{1}{4}\textstyle\sum_r (\mathfrak{D}u)_{c_r} && \text{(merged cells)},\end{aligned} \tag{12.16}$$

up to rounding: kept and merged cells only see sides whose integrated fluxes are preserved, and the corrections of different cells touch disjoint faces. Hence $\max_c |(\mathfrak{D}u)_c|$ does not grow, and a velocity that is solenoidal in $\mathfrak{D}$ stays so. Tóth and Roe [27] derive divergence-preserving prolongation formulas for staggered fields of the same kind; the correction (12.15) is the package's own.

Periodic integral. For a `FaceVelocity` or `FaceField` registered with the integral correction (the default), each periodic axis k then restores the staggered integral $M_k = \sum_{f\in\mathcal{F}_\Omega\cap\mathcal{F}^{[k]}} |\omega_f| u_f$ with one constant,

$$u_f \leftarrow u_f + \frac{M_k - M_k'}{\sum_{f'\in\mathcal{F}_\Omega\cap\mathcal{F}^{[k]}} |\omega_{f'}|}, \qquad f \in \mathcal{F}_\Omega \cap \mathcal{F}^{[k]}, \tag{12.17}$$

which leaves (12.16) intact: in a periodic direction a constant normal component has zero divergence. On an axis with walls no correction is made, which keeps affine exactness and the divergence of the wall cells; M_k then changes. The two phase velocities and the staggered phase scalars are registered without it.

Phase velocities. Before the transfer, the absent values (Section 6.8) of each phase velocity are replaced in a working copy, never in the field: two passes of the neighbour mean of (6.35) over the dual faces of the phase schedule, then, for every value so obtained, the weighted least-squares affine fit (weights $|\boldsymbol{x}_{f'} - \boldsymbol{x}_f|^{-2}$) over the originally present values within three rings of dual-face neighbours, if its normal matrix is regular; values still missing get zero. Each phase velocity is then transferred as above, its divergence corrections computed with these working values, and afterwards marked absent with the new interface. The identities (12.16) thus hold for each phase velocity with its working values, but the constraint of the two-velocity projection is the mixture divergence (10.11), whose phase apertures change with the reconstruction (Section 12.7).

Check (`amr_transfer_check`). On the static fixtures the three identities (12.16) hold to 4.6×10^{-15} relative to $\max|\mathfrak{D}u|$, a stream-function velocity keeps $h\max|\mathfrak{D}u'| \le 1.1\times10^{-15}\max|u|$ (h the smallest cell width), affine face fields are reproduced to 0 on the walled fixture, and (12.17) restores M_k on the periodic axes to 3.3×10^{-15} of $\sum|\omega_f|\,|u_f|$. On the walled axes no correction is made: for these fixtures, whose velocity does not vanish at the walls, the staggered integral over the interior faces changes by 8.3×10^{-3} to 1.2×10^{-2} and the kinetic energy by up to 8.6×10^{-3} per regrid, a first-order effect of the half dual volumes of the omitted wall faces (second order, about 10^{-4} at $h_0 = 1/16$, for a velocity with $u\cdot\boldsymbol{n} = 0$ at the walls, `proof_review/eq-amr-momentum/`); the kinetic-energy change of the transfer itself is second order in the cell width, and a refinement followed by the reverse coarsening restores it.

> *Implementation.* `register_adaptive_field!(workspace, field; piecewise_constant=false, conserve_integral=true)`; lines `_build_face_transfer_geometry!` (`src/mesh/adaptivity/field_transfer.jl`)

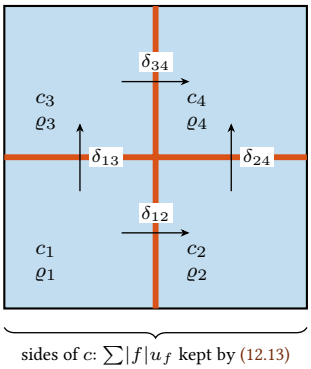


Figure 12.2. A cell c refined into $c_1, \dots, c_4$ (low-low, high-low, low-high, high-high). The fluxes through the sides of c are kept by the line map, so the outflow defects ϱ_r of the children after the interpolation sum to zero, and the four internal faces (thick) take the corrections $\delta_{12}, \delta_{34}, \delta_{13}, \delta_{24}$ of (12.15), positive in the direction of the arrows. The cell is one refined in the second regrid of Figure 12.1 (`figures/generated/amr_regrid.tex`).

and `_face_lattice_geometry` (`src/mesh/topology/mesh.jl`); values `_face_transfer_value` (`src/mesh/adaptivity/field_transfer.jl`) with `_line_value` (`src/mesh/adaptivity/field_transfer.jl`) (12.13), `_tangential_slope` (`src/mesh/adaptivity/field_transfer.jl`), `_line_point_value` (`src/mesh/adaptivity/field_transfer.jl`) and `_bracketing_line_segments` (`src/mesh/topology/mesh.jl`) (12.14); correction `_preserve_refined_divergence!` (`src/mesh/adaptivity/field_transfer.jl`) (12.15); integral `_conserve_face_momentum!` (`src/mesh/adaptivity/field_transfer.jl`) (12.17); phase velocities `_prepare_adaptive_phase_pair!` (`src/two_phase/two_velocity/velocity_fields.jl`), `_extend_phase_transfer_values!` (`src/two_phase/two_velocity/velocity_fields.jl`) and `_mark_inactive_phase_velocities!` (`src/two_phase/two_velocity/velocity_fields.jl`). Tests: `test/mesh/adaptivity.jl` (“face prolongation is affine exact”), `test/mesh/adaptivity.jl` (“in-place conservative adaptive field transfer”) (staggered integrals and divergence on a periodic mesh), `test/mesh/adaptivity.jl` (“adaptive refinement remains 2:1 graded”), `test/mesh/adaptivity.jl` (“non-dyadic dynamic balancing and transfer”), `test/mesh/adaptivity.jl` (“mesh face dual volumes follow AMR topology”), `test/two_phase/two_velocity/transport/two_velocity.jl` (“Intrinsic two-phase velocity transfer on AMR with geometry”).

12.7 Constraints after a regrid

The projection of every step makes the end-of-step velocity solenoidal up to its tolerance (Chapter 10); the transfer then changes the constraint as follows.

- *One velocity, no solid.* The constraint is $\mathfrak{D}u = 0$, which the transfer keeps cell by cell (12.16).

- *One velocity with a solid.* The constraint is the open divergence (3.13). The face map and the corrections act on Cartesian fluxes, and the apertures change with the solid's reconstruction, so a cell next to a refined or merged cut face may violate it. There is no extra projection: the next step treats the remainder as inherited divergence with the reference step δt^{prop} ((10.28) and (10.29)).

- *Two velocities.* The constraint is the mixture divergence (10.11) on the phase apertures of the new reconstruction, which the independent phase transfers do not keep where the regrid touches mixed cells (issue 84); the projection after a regrid (10.31) restores it with the scratch pressure, keeping the transferred physical pressure (deviation 10.6).

The pressure itself is a cell field (Section 12.4). In the incremental projections the transferred pressure enters the predictor of the next step (Section 10.5.2); its gradient on the new faces is not the transfer of the old gradient.

12.8 Conservation summary and measurements

Table 12.1 collects the statements of Sections 12.4 to 12.7 with the largest changes per regrid measured by `figures/amr.jl`. The static fixtures are the walled box of Figure 12.1, a drop resting on a wavy solid, and a drop in a periodic unit square, both with base cells $h_0 = 1/16$ and $1/32$ in blocks of 8×8 cells and $L = 2$, each refined twice by its bands and coarsened twice (48 blocks split and 48 families merged in all). The flow runs replicate the per-step actions of `step!(flow, dt)` around `adapt!` with the fixed step $\frac{1}{2}$`stable_timestep(flow; cfl=0.5)`; over 12 steps of the periodic case this loop reproduces `simulate!` to 3.6×10^{-13}, including the topology changes. All flows have densities $(1, 0.1)$, viscosities $(10^{-3}, 10^{-4})$ and surface tension 0.01, start from a projected velocity and use $n_{\mathrm{d}} = 2$. The periodic case is a drop of radius 0.1 in the velocity $(1 + \frac{1}{4}\sin 2\pi y,\ \frac{1}{2} + \frac{1}{4}\sin 2\pi x)$ under the interface band at level 1 and the curvature criterion, run for 60 steps with one and with two velocities; the cut case is the walled fixture with the stream function $\sin^2(\pi x/2)\sin^2(\pi y)$ and no-slip walls, run for 40 steps with one velocity. Defects are the dimensionless volume defects $\delta t \max_c |(\cdot)_c|$ of (10.30), here measured independently of the solvers, and compared with the run's tolerance $\varepsilon^p = 1.0 \times 10^{-10}$.

In the periodic case the mesh changed in 4 of the 60 steps with one velocity and in 4 with two; momentum and kinetic energy change by the amounts of Table 12.1 per regrid (momentum relative to $\sum \widetilde{\rho}|\omega_f|\,|u_f|$). With two velocities the transfer leaves a mixture defect of up to 2.1×10^{-4}, far above ε^p, and the projection (10.31) reduces it to 1.5×10^{-14}, changing the kinetic energy by up to 4.4×10^{-8}. With one velocity the defect after the transfer never exceeds that before it, as (12.16) imply. In the cut case the mesh changed in 1 steps, and the open-divergence defect after the transfer reached 7.6×10^{-3} in a cut cell; it is left to the next step.

Checks of the indicators (`amr_indicator_check`). An independent replica of the band and parent guard, and replicas of the error-based criteria, agree on 316 block labels. On a pool over a sloped solid, the interface band labels 0 of 4 blocks for refinement without a liquid–gas mixed cell: the wetted wall alone is not near (issue 119). The curvature indicator labels 0 such blocks (issue 118). In the static-drop fixture with a one-cell buffer, the mesh

Table 12.1. Quantities under a regrid: the statement of this chapter and the largest changes per regrid measured on the static fixtures (integrals over the interior faces relative to $\sum|\omega_f|\,|u_f|$, other rows relative to the stated scale) and in the flow runs (one velocity, two velocities, one velocity with a solid). Volume defects are $\delta t \max|\cdot|$ after the transfer and, for two velocities, after the projection of (10.31).

Quantity	Statement	Measured
$\sum_{\mathcal{C}} \varphi$ (cell fields)	conserved, (12.8) and (12.9)	4.7×10^{-15}
Liquid volume	conserved up to snapping, (12.11)	3.8×10^{-16}; flows 2.2×10^{-16}, 2.2×10^{-16}, 0
Solid volume	conserved, (7.7)	7.4×10^{-16}
Phase amounts	conserved up to snapping, (12.12)	5.3×10^{-16}
$\mathfrak{D}u$	identities (12.16)	4.6×10^{-15}
M_k, periodic axis	restored, (12.17)	3.3×10^{-15}
M_k, walled axis	not conserved; first order from the omitted wall half volumes	1.2×10^{-2}
Momentum, relative to $\sum \widetilde{\rho}\|\omega_f\|\,\|u_f\|$	not conserved	flows 4.2×10^{-6}, 3.8×10^{-6}, 5.2×10^{-5}
Kinetic energy	not conserved; second order, restored by the reverse regrid	8.6×10^{-3}; flows 1.4×10^{-5}, 9.2×10^{-6}, 3.0×10^{-5}
Defect, one velocity	kept, (12.16)	3.7×10^{-11} (before: 3.7×10^{-11})
Defect, cut cells	not kept	7.6×10^{-3} (before: 8.5×10^{-11})
Defect, two velocities	not kept; projected	2.1×10^{-4}, then 1.5×10^{-14}
Staggered phase amounts	not conserved	—

changes 0 times after the initial refinements over 14 attempts and has period 1 (a fixed point), addressing issue 117.

Implementation. The figure data and all numbers of this section are produced by `figures/amr.jl` (`generate.jl amr`): `amr_transfer_check`, `amr_indicator_check` and `amr_flow_check`; the latter replicates the per-step actions in `step!` (`src/navier_stokes/flow_simulation.jl`). Tests of the constraints after a regrid:
`test/two_phase/two_velocity/transport/two_velocity.jl` ("Moving two-velocity WY carrier after regridding (#84)"),
`test/two_phase/two_velocity/transport/two_velocity.jl` ("Two-velocity interface advection after dynamic cut-cell AMR"),
`test/navier_stokes/high_level_api.jl` ("regrid defects keep the transfer reference").

— 13 —

Stability and accuracy

Equation reference, audited against commit `2ca00c4`.

The explicit parts of a time step limit its size: the geometric transport of the fractions and of the momentum (Chapters 5 and 6) and the surface-tension and gravity increments (Chapter 8). The viscous step is implicit (Chapter 9) and the projection has no time step of its own (Chapter 10). This chapter states the bounds that the code evaluates and how a flow simulation combines them (Section 13.1), how `simulate!` turns them into a sequence of steps and how `step!` checks a given step (Section 13.2), the bounds, step sequences and empirical stability margins of representative flows (Section 13.3), the configurations in which a step within the bounds still fails (Section 13.4), and the order of accuracy of the building blocks together with the orders that the tests assert (Section 13.5). The thesis states the transport bound [19, eqs. (3.50)–(3.51)], the capillary bound [19, eq. (4.1)] and the gravity bound [19, eq. (7.4)], and its refinement studies refine the step through these bounds [19, §7.7]; how the bounds become a sequence of steps is not stated there. The step selection has no discretisation of its own: it maps the state at the start of a step to the size of that step.

13.1 Time-step bounds

Every bound is the largest step for which a set of rates satisfies an inequality (Table 13.1). The advective bounds limit the volume that a carrier moves through the faces of a control volume relative to its open volume; the capillary and gravity bounds limit the step relative to the period of the shortest interface wave that the mesh represents. A bound without rates (a vanishing velocity, $\sigma = 0$, $\boldsymbol{g} = \mathbf{0}$ or $\rho^{\mathrm{l}} = \rho^{\mathrm{g}}$) is ∞.

Flow bound. `stable_timestep(velocity, solver, workspace; cfl=0.45)` and `stable_timestep(flow; cfl=0.45)` return

$$\delta t_{\mathrm{flow}}(\nu^{\mathrm{f}}) = \min\left\{\delta t_{\mathrm{adv}}(\nu^{\mathrm{f}}),\ \delta t_{\sigma},\ \delta t_{\mathrm{grav}}\right\},$$
$$\delta t_{\mathrm{adv}}(\nu^{\mathrm{f}}) = \begin{cases} \delta t_{\mathrm{stag}} \text{ at } \nu^{\mathrm{s}} = \nu^{\mathrm{f}}, & \text{one phase,} \\ \delta t_{\mathrm{WY}} \text{ of the velocity at } \nu^{\dagger} = \nu^{\mathrm{f}}, & \text{two phases, one velocity,} \\ \delta t_{\mathrm{WY}} \text{ of (5.26)}, & \text{two velocities,} \end{cases} \tag{13.1}$$

for the velocity passed to them, with δt_{σ} and δt_{grav} only in two phases; the last row is the transport of Section 5.8.1. Both force bounds use $h = \min_{c,k} h_{c,k}$, the smallest width

Table 13.1. Time-step bounds of the code. The parameter is the `cfl` keyword of `stable_timestep` for the transport or operator alone; the last column marks the bounds that a flow combines in (13.1).

Bound	Definition	Parameter (default)	In a flow
δt_{stag}, staggered convection	(6.23)	$0 < \nu^{\mathrm{s}} \leq 1$ (0.9)	one phase
δt_{WY}, WY sweeps of one carrier	(5.15) and (5.17)	$0 < \nu^{\dagger} \leq \frac{1}{2}$ ($\frac{1}{2}$)	two phases
δt_{WY}, frozen phase subfaces	(5.26)	as above	two velocities
δt_{cell}, cell-centred scalar	(6.39)	$0 < \nu^{\mathrm{s}} \leq 1$ (0.9)	no
δt_{σ}, capillary	(5.18)	none	two phases
δt_{grav}, gravity	(5.18)	none	two phases
δt_{μ}, explicit viscous	Section 9.2	$0 <$ `cfl` ≤ 1 (0.9)	no

of all leaf cells in either direction, not the width at the interface; on a mesh refined elsewhere, at a solid for instance, they are stricter than the interface needs. In two phases ν^{f} must lie in $(0, \frac{1}{2}]$ and in one phase in $(0, 1]$; outside that range `stable_timestep` and `simulate!` throw an error that names the range, also when a prescribed `dt` makes it unused. The ranges are those of the bounds the code evaluates, not measured limits of the transports (Section 13.3 finds the WY bound sufficient rather than necessary): the WY sweeps of one carrier limit the face extrusion to half the donor width, and at $\nu^{\mathrm{s}} \leq 1$ every dual-face Courant number of the staggered bound stays within one, so the Lax–Wendroff face state is a convex combination of its upwind neighbours (Section 6.6; the first-order upwind update stays convex up to $\nu^{\mathrm{s}} \leq 2$). `step!` checks every step against δt_{flow} at the top of the range of the model ((13.3)), so every step that `stable_timestep` returns is accepted and the keyword alone sets the margin. Before issue 121 was fixed, the one-phase check used $\nu^{\mathrm{s}} = \frac{1}{2}$, and `simulate!` with $\nu^{\mathrm{f}} > \frac{1}{2}$ stopped at its first advection-limited step. Only δt_{adv} scales with ν^{f}. Since $\delta t_{\sigma} \propto h^{3/2}$ and $\delta t_{\mathrm{grav}} \propto h^{1/2}$ while the advective bounds are proportional to h for a given velocity, the capillary bound becomes the active one and the gravity bound an inactive one under mesh refinement; the thesis notes the latter for its gravity bound [19, eq. (7.4)]. For a translation with speed U on a uniform mesh, where $\delta t_{\mathrm{adv}} = \nu^{\mathrm{f}} h / U$, the capillary bound is the smaller one below the width $4\pi {\nu^{\mathrm{f}}}^2 \sigma / ((\rho^{\mathrm{l}} + \rho^{\mathrm{g}}) U^2)$ and the gravity bound only above $(\rho^{\mathrm{l}} + \rho^{\mathrm{g}}) U^2 / (2{\nu^{\mathrm{f}}}^2 \left|\rho^{\mathrm{l}} - \rho^{\mathrm{g}}\right| |\boldsymbol{g}|)$.

Transport bounds on composite meshes and in cut cells. The advective bounds are those of Sections 5.6 and 6.6. The WY bound limits the inflow and the outflow of every open cell, the extrusion of every face relative to its donor width, and the available open portions of cut donor strips and opposing strip pairs (5.15) and (5.16). The staggered bound sums the absolute rates of every dual control volume, including the boundary dual faces where a level transition meets a wall (6.23). Both divide by open volumes, and no cells are merged, so small cut cells reduce the step (Section 13.3). The two-phase momentum transport has no bound of its own (Section 6.6).

Capillary bound. δt_{σ} is the bound of Brackbill, Kothe and Zemach [4] in the form of the thesis [19, eq. (4.1)]. With the phase speed $c_{\sigma} = (\pi\sigma / ((\rho^{\mathrm{l}} + \rho^{\mathrm{g}}) h))^{1/2}$ of the capillary wave of wavelength $2h$, $\delta t_{\sigma} = h / (2c_{\sigma})$, which is $2^{-1/2}$ times the period of the capillary wave of wavelength h. Denner and van Wachem attribute the bound to the temporal sampling of capillary waves, irrespective of an explicit or implicit surface tension, propose the static

bound $h/(\sqrt{2}\,c_\sigma)$, a factor $\sqrt{2}$ less restrictive, and add the Doppler shift of a flow with speed U along the interface, $h/(\sqrt{2}\,c_\sigma + U)$ [7, eqs. (43)–(44)]. The code adds no speed to c_σ: (13.1) takes the minimum of δt_σ and the advective bound. For a flat interface translated along itself with speed U on a uniform mesh without solid, where $\delta t_{\mathrm{WY}} = \nu^\dagger h/U$, the selected step $\min\{\nu^\dagger h/U,\ h/(2c_\sigma)\}$ exceeds the dynamic bound $h/(\sqrt{2}\,c_\sigma + U)$ by at most the factor $\nu^\dagger + 2^{-1/2}$ (1.16 at $\nu^\dagger = 0.45$), attained where the two bounds of the code are equal ($U = 2\nu^\dagger c_\sigma$).

Gravity bound. The thesis bounds the step by the period of a gravity wave of wavelength $h/(2\pi)$, a wavelength found necessary in numerical experiments, and remarks that the bound is not restrictive [19, eq. (7.4)]; the code multiplies it by $2^{-1/2}$ (deviation 13.1).

No viscous bound. The viscous step is implicit (Section 9.4), and the flow bound is the convective one also for a large viscosity. In two phases the implicit midpoint step does not increase the mass-weighted kinetic energy for any δt, up to the residual (9.19). The single-phase geometric stiffness is also positive semidefinite: backward Euler is contractive for every δt, as is midpoint when $u^{(n)} = u^*$. The actual single-phase split step evaluates its explicit half at $u^{(n)}$ (deviation 9.3), so, by (9.7), its energy grows by at most $\big((u^* - u^{(n)})^{\mathsf{T}}\widetilde{\mathsf{V}}(u^* - u^{(n)})\big)^{1/2}$, the dual-volume distance between the transported and the old velocity, plus the work of moving walls; the old transition-band energy growth is removed (issue 80). δt_μ, the explicit estimate of `stable_timestep(operator; cfl=0.9)` for a `StaggeredDiffusionOperator`, is used by no flow step. The body force is integrated with the midpoint rule and has no bound (8.19).

Deviation from the thesis 13.1 (Gravity time-step factor). The thesis' gravity bound is the period of a gravity wave of wavelength $h/(2\pi)$, $\big((\rho^{\mathrm{l}} + \rho^{\mathrm{g}})h/(\big|\rho^{\mathrm{l}} - \rho^{\mathrm{g}}\big|\,|\boldsymbol{g}|)\big)^{1/2}$ [19, eq. (7.4)]. The code multiplies it by the safety factor $2^{-1/2}$ that the thesis applies to the capillary bound [19, eq. (4.1)], so that δt_{grav} (5.18) is stricter by $\sqrt{2}$.

Implementation. Flow bound (13.1): `stable_timestep(velocity, solver, workspace; cfl=0.45)` in `stable_timestep` (`src/single_phase/stepping.jl`) (shared velocity) and `stable_timestep` (`src/two_phase/two_velocity/stepping.jl`) (two velocities), `stable_timestep(flow; cfl=0.45)` in `stable_timestep` (`src/navier_stokes/flow_simulation.jl`), the diagnostic `CFLTimeStep(; cfl=0.45)`; the force bounds and the minimum in `_two_phase_timestep_bound` (`src/two_phase/stepping.jl`), with h from `_minimum_mesh_spacing` (`src/two_phase/stepping.jl`) and the name of the active bound from `_two_phase_limit_name` (`src/two_phase/stepping.jl`); the two-velocity transports in `stable_timestep` (`src/two_phase/two_velocity/interface_transport.jl`) and `_frozen_interface_stable_timestep` (`src/two_phase/two_velocity/transport_geometry.jl`); the explicit viscous estimate in `stable_timestep` (`src/operators/staggered_diffusion.jl`). The step check of (13.3) uses the top of the accepted `cfl` range of the model, `_admissible_transport_cfl` (`src/single_phase/stepping.jl`): δt_{stag} at $\nu^{\mathrm{s}} = 1$ in one phase and δt_{WY} at $\nu^\dagger = \frac{1}{2}$ in two phases, so that every step `stable_timestep` returns is accepted (issue 121). Tests: `test/navier_stokes/high_level_api.jl` ("time-step safety") (a step at 1.01 times the bound at $\nu^{\mathrm{s}} = 1$ is refused, a step of 0.75 times it and `simulate!` with $\nu^{\mathrm{f}} = 0.8$ are

accepted, and $\nu^{\mathrm{f}} = 1.01$ is refused),
`test/navier_stokes/navier_stokes.jl` ("Navier–Stokes timestep is convective only") (the flow bound is δt_{stag} also at viscosity 100),
`test/navier_stokes/two_phase_navier_stokes.jl` ("Two-phase gravity-wave timestep safety factor") (δt_{grav} with $2^{-1/2}$, for one and two velocities),
`test/navier_stokes/high_level_api.jl` ("tiny final timestep is absorbed") (the capillary bound does not scale with `cfl`).

13.2 Step selection and the step check

Without a prescribed step (`dt=nothing`, the default of `simulate!` and of a `FlowSimulation` built without `dt`), `simulate!` advances from $t^{(0)}$ to t_{end} as follows. At the start of step n it imposes the boundary values at $t^{(n)}$, evaluates $\delta t^{(n)}_{\mathrm{flow}} = \delta t_{\mathrm{flow}}(\nu^{\mathrm{f}})$ of (13.1) with $\nu^{\mathrm{f}} = 0.45$ (keyword `cfl`) and sets

$$\delta t^{\mathrm{prop}(n)} = \min\left\{\delta t^{\mathrm{prop}(n-1)},\ \delta t^{(n)}_{\mathrm{flow}}\right\}, \qquad \delta t^{\mathrm{ref}(n)} = \max\left\{\delta t^{\mathrm{prop}(n)},\ \delta t^{\mathrm{def}(n)},\ \delta t^{(n)}\right\},$$
$$\delta t^{(n)} = \begin{cases} t_{\mathrm{end}} - t^{(n)}, & t_{\mathrm{end}} - t^{(n)} - \delta t^{\mathrm{prop}(n)} < 10^{-6}\,\delta t^{\mathrm{prop}(n)}, \\ \delta t^{\mathrm{prop}(n)}, & \text{otherwise}, \end{cases}$$
$$t^{(n+1)} = \begin{cases} t_{\mathrm{end}}, & \delta t^{(n)} = t_{\mathrm{end}} - t^{(n)}, \\ \min\left\{t_{\mathrm{end}},\ t^{(n)} + \delta t^{(n)}\right\}, & \text{otherwise}, \end{cases} \tag{13.2}$$

with $\delta t^{\mathrm{prop}(-1)}$ the proposal retained from a previous call (below) or ∞. If $\delta t^{\mathrm{prop}(0)}$ is still infinite, the step is capped at $(t_{\mathrm{end}} - t^{(0)})/128$, so that a flow at rest without forces takes at least 128 steps. Steps are taken while $t_{\mathrm{end}} - t^{(n)} > 16\epsilon_{\mathrm{mach}} \max\{1, |t_{\mathrm{end}}|, |t^{(0)}|\}$ (a threshold fixed at the start of the call), and the call ends at t_{end}. The reference step δt^{ref} of the incremental projections (Section 10.5.2) uses δt^{def}, the length of the previous physical step, after a regrid the proposal of the step before it, and 0 at the first step of a run; its third argument matters only for a stretched last step, which must not exceed its reference ((10.28)). With a prescribed step (keyword `dt`) the proposal is that step in every step, no bound is evaluated, and the rest of (13.2) applies.

The following hold by construction. The proposal never increases: a transient velocity peak, a solenoidal correction of an impulsive start or a regrid to finer cells lowers the step for the rest of the run, and a decaying flow or a coarsening never raises it again. Every step of a call but the last equals its proposal. The last step of a call that follows a step of the same call lies between 10^{-6} and $1 + 10^{-6}$ times its proposal: a remainder below 10^{-6} of the proposal is absorbed into the step before it (issue 98: before, a step of 5×10^{-14} remained), and a remainder above it is a step of its own. Such a shortened step keeps, through $\delta t^{\mathrm{ref}} \geq \delta t^{\mathrm{prop}}$, the share $1 - \delta t/\delta t^{\mathrm{ref}}$ of the divergence that the previous projection left instead of dividing it by the short step ((10.29), issue 90).

Retained proposals. A call continues the previous one if its velocity, pressure and solver are the same objects, its start time equals the end time of the previous call, and the mesh topology has not changed since; it then starts from the retained proposal (not from a shortened or stretched last step) and with δt^{def} of the previous step (issue 91). After a call with a prescribed step no proposal is retained, but δt^{def} is; a call that takes no step keeps a finite

proposal. A full `step!(flow, dt)` resets the proposal and retains the defect reference of the completed step or transfer. A field-level `step!`, `project!(flow)` or a replaced workspace ends both histories. A run split into calls whose ends lie on its step sequence therefore takes the same steps as the uninterrupted run, whereas an end between two steps shortens one step and changes the trajectory. The cap of an infinite first proposal depends on the requested interval, so the first step of a flow at rest also depends on how the run is split.

The step check. `step!` accepts a step if, for the velocity at the start of the step with its boundary values at $t^{(n)}$,

$$\delta t \le \left(1 + 10^{-6} + 32\epsilon_{\mathrm{mach}}\right) \delta t_{\mathrm{flow}}(\nu^{\max}), \qquad \nu^{\max} = \begin{cases} 1, & \text{one phase,} \\ \frac{1}{2}, & \text{two phases,} \end{cases} \tag{13.3}$$

and otherwise stops with an error that names the active bound (advective, capillary or gravity). The check uses the top $\nu^{\max}$ of the accepted `cfl` range, so that every step `stable_timestep` returns is accepted and the keyword alone sets the margin: a step that `simulate!` selects with $\nu^{\mathrm{f}} = 0.45$ has the margin $\nu^{\max}/0.45$ against the check if it is advection-limited (2.22 in one phase, 1.11 in two), and only the tolerance if it is capillary- or gravity-limited; the relative 10^{-6} admits the stretched last step. With frozen interface fluxes, `step!` skips the check for a step that `simulate!` has just selected for the same state (that evaluation also freezes the apertures of the time-step start, Section 5.8.1). The WY `advect!` makes the same check with δt_{WY} at $\nu^{\dagger} = \frac{1}{2}$ unless `check_timestep=false`; the staggered and cell-centred `advect!` make none. The relative 10^{-6} came with the stretched last step (issue 98); a step at $(1+10^{-6})\delta t_{\mathrm{WY}}$ is therefore accepted, and the rejection probe of the check in Section 5.6 lies at $(1+2\times10^{-6})\delta t_{\mathrm{WY}}$ (issue 115).

Checks (`figures/stability.jl`, `step_selection_check`). On a decaying Taylor–Green vortex of 8×8 periodic cells (one phase, viscosity 0.05), `simulate!` keeps the first proposal for all 10 steps of a run of ten proposals, while the bound grows by the factor 3.24; a remainder of 0.5×10^{-6} of the proposal stretches the third step to 1.0000005 proposals, whereas one of 2×10^{-6} is a fourth step of 2.0×10^{-6} proposals; a call to 2.5 proposals ends with half a proposal and the next call resumes with a whole one, and two calls that end on the step sequence reproduce the velocity and pressure of the uninterrupted run bitwise; the fluid at rest takes 128 steps of $1/128$ to $t_{\mathrm{end}} = 1$; and `step!` accepts $(1+0.9\times10^{-6})\delta t_{\mathrm{flow}}(\nu^{\max})$ and rejects $(1+2\times10^{-6})\delta t_{\mathrm{flow}}(\nu^{\max})$, with $\nu^{\max} = 1$ here. The selection rule (13.2) also reproduces every recorded step of the flows of Section 13.3.

> *Implementation.* Selection (13.2) in `_simulate!` (`src/navier_stokes/simulate.jl`) (`simulate!(velocity, pressure, solver, final_time, workspace; time=0, dt=nothing, cfl=0.45, ...)`); for a `FlowSimulation` the default is `dt=flow.default_dt`. The retained state is in `SimulationStepSelection` (`src/navier_stokes/workspace.jl`), updated by `step!` (`src/navier_stokes/flow_simulation.jl`) and invalidated by `_invalidate_simulation_step!` (`src/navier_stokes/workspace.jl`); the check (13.3) in `_transport_substep!` (`src/single_phase/stepping.jl`), `_transport_substep!` (`src/two_phase/stepping.jl`) and `_transport_substep!` (`src/two_phase/two_velocity/stepping.jl`), and

Table 13.2. Bounds at $\nu^{\mathrm{f}} = 0.45$ at the start of four flows, the active one in bold, and the advective bound at the end of the runs of `simulate!` described in the text.

Flow	h	δt_{adv}	δt_σ	δt_{grav}	δt_{adv} after the steps
Capillary wave	$\frac{1}{64}$	∞	$\mathbf{5.5 \times 10^{-4}}$	∞	0.251
Cylinder	$\frac{1}{8}$	$\mathbf{2.6 \times 10^{-3}}$	–	–	7.4×10^{-3}
Kelvin–Helmholtz	$\frac{1}{8}$	$\mathbf{0.150}$	0.196	∞	0.117
KH–RT combined	$\frac{1}{16}$	0.137	$\mathbf{0.069}$	0.512	–

for the WY transport alone in `_advect_wy!` (`src/vof/fluxes.jl`). Tests: `test/navier_stokes/high_level_api.jl` ("tiny final timestep is absorbed"), `test/navier_stokes/high_level_api.jl` ("infinite initial bound retains the chosen proposal"), `test/navier_stokes/high_level_api.jl` ("automatic timestep resumes on adaptive cylinder"), `test/navier_stokes/high_level_api.jl` ("abrupt proposal drops retain the actual preceding step"), `test/navier_stokes/high_level_api.jl` ("standalone projection invalidates the resumed automatic selection"), `test/navier_stokes/two_phase_navier_stokes.jl` ("two-velocity automatic timestep survives resumption and regrid"), `test/navier_stokes/high_level_api.jl` ("adaptive cylinder short final step (issue #90)").

13.3 Measured bounds and stability margins

Table 13.2 gives the bounds (13.1) of four flows at $\nu^{\mathrm{f}} = 0.45$, computed by `case_bounds_check`: the standing capillary wave of `examples/06_capillary_wave` (density ratio 1000, Laplace number 3000, one velocity) on 32×96 cells with its interface band refined once; the flow past the cylinder of `examples/04_cylinder_flow` at Reynolds number 100 with unit inflow on 56×32 cells whose cut cells are refined once; the Kelvin–Helmholtz wave of the two-velocity tests (equal densities, Weber number 5, frozen interface fluxes and the CMDGFM) on their coarsest mesh of 8 cells per wavelength; and the combined Kelvin–Helmholtz–Rayleigh–Taylor wave of `examples/07_interfacial_instability` (density ratio 3, Weber number 0.5, Bond number 0.75) on 16 cells per wavelength. For the first three, `simulate!` then advanced over 12 times the first step: in 12 steps for the wave and the cylinder, whose proposals keep the first step, and in 13 steps for the Kelvin–Helmholtz wave, whose falling bound shortens its steps.

The capillary wave starts at rest, so its first step is capillary-limited with an infinite advective bound; after the steps the advective bound is 455 times larger than δt_σ. In the cylinder flow the cut cells reduce the bound of the unit inflow from 0.028 on the same mesh without the solid to 2.6×10^{-3}, a factor 10.7 (the smallest open fraction of a cut cell is 0.042); the first step projects the impulsive start and raises the bound to 7.0×10^{-3}, and it reaches 7.4×10^{-3}, but the proposal keeps the first step, 2.8 times below the bound for the rest of the run. The Kelvin–Helmholtz wave is advection-limited and grows; its steps follow the falling bound. In the combined wave all three bounds are finite and the capillary one is active.

Empirical stability margin (`stability_margin_check`). From the initial state, a run takes a fixed number of steps of size $q\,\delta t^{(0)}$, with $\delta t^{(0)}$ the step selected at the start, through the prepared two-velocity path of `simulate!`, in which `step!` does not check the step; the largest stable multiple q^* is the largest tested multiple below the first unstable one. The capillary wave, run with two velocities, takes 40 steps and is stable if its largest velocity stays below the velocity amplitude of the linear standing wave (initial amplitude times angular frequency): it is stable up to $q^* = 2.5$ (largest velocity 0.77 times that amplitude), while at $q = 2.75$ the velocity exceeds it after 14 steps and reaches 37 times it during the 40-step run. The criterion is a blow-up test over 40 steps: bisection with it places the limit in $[2.594, 2.602]$, and over 160 steps the stable runs also reach the amplitude, which the linear wave itself attains after about 70 steps, so that the criterion no longer discriminates there (derivation audit, `proof_review/ch-stability-accuracy/`). The capillary bound, with its factor $2^{-1/2}$, is thus about 2.6 times below the limit observed within 40 steps, which exceeds the static bound $h/(\sqrt{2}\,c_\sigma)$ of Denner and van Wachem. The Kelvin–Helmholtz wave takes 6 steps and is stable if they complete: it does up to $q^* = 1.35$ ($\nu^\dagger = 0.61$ relative to the initial bound), while at $q = 1.5$ ($\nu^\dagger = 0.68$) the WY transport stops in step 5 with a conditional fraction above one; bisection places the limit in $[1.425, 1.434]$. Since the bound falls during the run, the last step of the run at q^* is 1.29 times $\delta t_{\mathrm{flow}}(\frac{1}{2})$ of its own state, an effective $\nu^\dagger \approx 0.65$ (derivation audit). The bound $\nu^\dagger \le \frac{1}{2}$ is sufficient for the boundedness of the transport on this mesh, not necessary. The single-phase step check admits no step above $\delta t_{\mathrm{flow}}(1)$, so the margin of the cylinder flow is not measured here beyond $1/0.45 \approx 2.22$.

> *Implementation.* `docs/techref/figures/stability.jl`: fixtures `stab_capillary_wave`, `stab_cylinder`, `stab_kelvin_helmholtz` and `stab_taylor_green`; bounds `case_bounds_check`; selection `step_selection_check`; margins `stability_margin_check`. The margin runs pass the prepared-geometry flag through `workspace.transport_geometry_prepared` after evaluating the bound, as `simulate!` does, before calling `step!`.

13.4 Known gaps between the bounds and stability

The bounds are sufficient for the properties that the thesis proves on meshes without solid. The cut-donor geometric constraints extend the WY donating-region argument to the cut cells and frozen subfaces described in Section 5.5. The run-time checks remain active if a caller supplies a larger step or disables the time-step check; `simulate!` does not retry a refused step.

Deviation from the thesis 13.2 (Open strip of a cut coarse donor). At a level transition a segment's open strip can be smaller than suggested by the whole donor volume or the shared aperture. The former bound could request more than that strip held at its returned step (issue 52). The present bound includes the strip's open area ((5.16)), while retaining the whole-cell incoming and outgoing constraints.

Deviation from the thesis 13.3 (Boundedness of cut donors). The thesis proves $0 \le C_c^{\mathrm{l}} \le 1$ using disjoint donating regions [19, Lemma 3.3, eqs. (3.50)–(3.51)]. In a cut donor a rectangle can extend past the cell midpoint, so the previous bound did not ensure disjoint-

ness for hanging sides or frozen phase subfaces (issue 56). The current bound separates the open portions of opposing outgoing strips within their common support and thereby restores the geometric argument (Section 5.5).

Deviation from the thesis 13.4 (Frozen subface flux in cut donors). The bound uses each frozen phase subface's own tangential interval and donor open volume, rather than the whole-cell volume. Its ordering and aperture are frozen together for both sweeps. A nonzero shared-aperture flux with no open volume behind its subface admits no usable step, which the bound reports as an error naming the face and donor cell; otherwise the strip condition accounts for the volume the sweep cuts (issue 56, Section 5.8.1).

Momentum Courant numbers next to a solid. The two-phase momentum transport is bounded only through the WY bound, which bounds its dual Courant numbers by $\nu^\dagger$ on 2:1-balanced meshes without solid and, through the cut-donor strip bound, also with a solid (Section 6.6); the measured ratio to $\nu^\dagger$ over random cut meshes with a level transition is one to round-off, so the selected Lax–Wendroff or limited face state there is a convex combination of its two neighbours as well (issue 72).

Energy of the split step. The full split step is not claimed to be energy-stable: the centred and Lax–Wendroff convection add energy in time ((6.22)), the CMDGFM has no energy identity (Section 10.4), and the single-phase viscous step has different explicit and implicit arguments (Section 13.1). The bounds therefore limit the step by the time scales of transport and interface waves, not by an energy estimate.

> *Implementation.* Run-time checks: `_swept_fraction` (`src/vof/fluxes.jl`) (open strip), `_advect_wy!` (`src/vof/fluxes.jl`) (fraction bounds and the snapping tolerance) and `_compute_directional_wy_fluxes!` (`src/two_phase/two_velocity/interface_transport.jl`) (frozen subfaces). No test exercises the three failures; the reproducers are those of Section 5.4 and of the issues.

13.5 Accuracy

Table 13.3 collects the formal orders that the chapters state for the building blocks and the orders that the tests assert. An order is the exponent of the mesh width in the error under refinement; the tests measure it from the errors on two or three meshes, as the logarithm of the error ratio divided by that of the width ratio, and assert a lower bound. Where the step is refined with the mesh (all flow tests), the measured order is that of the space–time error.

Space. The orders of Table 13.3 are those of operator applications or single solves on manufactured fields, at fixed geometry. The known first-order spots are local: the closures of inclined embedded walls (issue 110; the test pins the observed 0.6 and 0.8 and a non-growing error, (9.10)), the variable-viscosity stress at level transitions without solids (issue 112: 0.985 and 1.002 in L^2 on fixed composite trees, second order on uniform meshes), the geometric-mean vertex viscosity of the thesis [19, §7.5.1] at a material jump parallel to an interface-aligned shear (issue 113: L^2 orders 1.48, 0.78, 0.82, 0.90, 0.95 for $n = 16$ to 512 against 1.96 to 1.98 for equal viscosities; no test). An orphaned transverse edge next to a level transition is linear-exact at every wall type (issue 99): a `NoSlip`, `FreeSlip` or `Inflow` side closes it against the prescribed wall velocity – exact for wall data constant along the wall, first order for data that varies along it – and an `Outflow` or `ZeroGradientVelocity`

Table 13.3. Formal orders of the building blocks and the orders that the tests assert (lower bounds); "first order" marks the known first-order spots.

Building block	Stated order	Asserted
Interface (Chapter 4)	WY normal exact for lines (4.12); LHF curvature; contact-line curvature first order (deviation 4.7)	LHF curvature 1.7; PLIC position 1.49 (L^2, 1.499 measured); triple-point curvature 0.7 (1.8 at 90°); embedded contact curvature 0.8
VOF transport (Chapter 5)	exact space–time fluxes of the reconstruction, no method-of-lines form (Section 5.2)	symmetric difference 1.8 (translation, cut walls); vortex reversal 1.5
Staggered convection (Chapter 6)	forward Euler sweeps with Lax–Wendroff or limited states (6.19)	Lax–Wendroff translation 1.9 (uniform); convection on composite meshes 1.3
Pressure gradient and projection (Chapters 3 and 10)	CMDGFM jump interpolant first order for affine fields (Section 10.4)	gradient on composite meshes 1.4, stretched 1.0; Poisson 1.7; jump interpolant 0.8
Viscous operator (Chapter 9)	two-point difference second order; transition rows can be $O(1)$ (9.6); PLIC wall rows first order (issue 110); variable stress at transitions first order (issue 112); geometric-mean viscosity at a jump first order (issue 113)	uniform 1.9; composite Helmholtz 1.8 (max, L^2); cut walls with transitions 1.8 (max); aligned embedded walls 1.5 (max), 1.8 (L^1, L^2); inclined walls 0.6 (max), 0.8 pinned; variable stress bulk 1.8, at transitions 0.8 pinned
AMR transfer (Chapter 12)	conservative; prolongation first order next to coarser blocks	phase scalar transfer 1.8
Forces in time (Chapter 8)	forward Euler (8.21)	–

side differences a level-matched pair of unknowns instead of the offset pair. Next to triple points and at the contact line the reconstruction is first order (Chapter 4).

Time. The step is a sequence of stages ((8.21)): the transport, the body force at the midpoint, the implicit viscous step, the forward-Euler forces and the incremental projection. No order in time is claimed for the complete step, and no test asserts one: the temporal series at a fixed mesh assert the temporal self-convergence, that the difference between a run and the finest-step run of the same series, sampled at the coarse run's step times, does not increase when the step is halved. The error against the exact solution is not that gate. At a fixed mesh it converges to the spatial error, and it increases once a temporal error of the opposite sign stops cancelling it, which the correct one-velocity capillary wave at density ratio one shows beyond a quarter step: 0.01257, 0.00455, 0.00302, 0.00418 and 0.00501 for the steps 1 to $\frac{1}{16}$, while its self-convergence is non-increasing (issue 178). The frozen two-velocity wave at density ratio one is also not consistent with the one-velocity wave: the CMDGFM inference term of a cut face carries a tangential velocity jump of 0.098 (issue 171), so at the step $\frac{1}{8}$ its error is 0.01334 against 0.00418, and the test records that comparison as a known failure (issue 178). In a refinement study the step follows the active bound: $\delta t \propto h$ when advection-limited, $\delta t \propto h^{3/2}$ when capillary-limited and $\delta t \propto h^{1/2}$ when gravity-limited, so that an error of first order in δt limits the order in h to 1, $3/2$ or $1/2$ when it dominates: a ceiling, not a prediction (at density ratio one the capillary wave converges with 1.8). The Kelvin–Helmholtz test runs on meshes where the advective bound is active and expects the first-order split time error.

Flows. The flow tests assert the space–time orders: the Taylor–Green vortex 1.8 (step

proportional to h); a manufactured wave equation on composite meshes 1.5 in L^2 (1.0 for the velocity in the maximum norm); Prosperetti's standing capillary waves 1.8 at density ratio one and 1.5 at 1000, the gravity wave 0.4; the two-velocity Kelvin–Helmholtz dispersion 0.8, the Rayleigh–Taylor dispersion 1.7 (1.3 at density ratio 3) and the combined wave 0.79; the vortex reversal 1.5; and the sessile droplet 1.2.

Implementation. Tests (the asserted lower bounds of Table 13.3): `test/indicator/phase/height_functions.jl` ("Circular LHF curvature converges"), `test/indicator/geometry.jl` ("PLIC geometry converges and produces filled solid visualization"), `test/indicator/phase/height_functions.jl` ("Contact-angle curvature converges at triple points"), `test/indicator/geometry.jl` ("Embedded GHF contact curvature converges on sloped walls"), `test/indicator/geometry.jl` ("Cut-cell WY reverses wall-tangent contact-line translation"), `test/integration/anisotropic_cells.jl` ("anisotropic geometric VOF advection"), `test/integration/two_phase_vortex.jl` ("Classical T=1 vortex refinement study"), `test/operators/advection/staggered_advection.jl` ("Lax-Wendroff convergence"), `test/operators/advection/staggered_advection.jl` ("mixed-level staggered convection convergence"), `test/operators/gradient.jl` ("mixed-level gradient convergence"), `test/operators/gradient.jl` ("compatible pressure gradient"), `test/operators/poisson.jl` ("geometric multigrid Poisson convergence"), `test/two_phase/two_velocity/projection/cmdgfm_projection.jl` ("CMDGFM jump interpolant consistency"), `test/operators/staggered_diffusion.jl` ("uniform periodic staggered Laplacian convergence"), `test/operators/staggered_diffusion.jl` ("composite diffusion solution convergence (#80)"), `test/operators/staggered_diffusion.jl` ("composite implicit diffusion is energy contractive (#80)"), `test/operators/staggered_diffusion.jl` ("cut-cell Helmholtz convergence at walls and level transitions (#87)"), `test/operators/embedded_wall_rows.jl` ("grid-aligned embedded-wall Helmholtz convergence (#106)"), `test/operators/embedded_wall_rows.jl` ("oblique embedded-wall Helmholtz convergence (#110)"), `test/two_phase/variable_staggered_diffusion.jl` ("Variable stress manufactured bulk convergence"), `test/two_phase/variable_staggered_diffusion.jl` ("Variable stress shifted solve on fixed and adapted trees"), `test/two_phase/transport.jl` ("Intrinsic phase scalar AMR transfer convergence"); flows: `test/navier_stokes/navier_stokes.jl` ("Taylor–Green Navier–Stokes integration"), `test/integration/wave_equation.jl` ("mixed-level manufactured wave equation"), `test/integration/two_phase_waves.jl` ("Prosperetti standing waves on uniform and AMR meshes"), `test/integration/two_velocity_instabilities.jl` ("Two-velocity Kelvin–Helmholtz dispersion convergence"), `test/integration/two_velocity_instabilities.jl` ("Two-velocity Rayleigh–Taylor dispersion convergence"), `test/integration/two_velocity_instabilities.jl` ("Combined KH/RT dispersion convergence"), `test/integration/two_velocity_vortex.jl` ("Continuous two-velocity vortex

> reversal"), `test/integration/two_phase_sessile_droplet.jl` ("Sessile droplet equilibrium convergence").

— 14 —

Verification

Equation reference, audited against commit `c46cf60`.

The preceding chapters state what the code computes. This chapter states how that is checked. It defines the diagnostics through which tests, examples and users observe a flow (Section 14.1), describes the test suite and the tiers in which continuous integration runs it (Section 14.2), the regression oracle that detects any change of the results of the examples (Section 14.3) and the derivation audits that check the statements of this reference against the code (Section 14.4). It then lists the verification cases with what each asserts: manufactured solutions and observed orders (Section 14.5), conservation and other invariants (Section 14.6), benchmark flows with exact, theoretical or published references (Section 14.7) and the examples gallery (Section 14.8), and it closes with the known gaps (Section 14.9). The thesis validates the VOF transport [19, §3.5], the surface-tension models [19, §4.8], the two-fluid transport [19, §6.5] and the two-fluid flow solver [19, §7.7]; several cases below repeat those studies. Validation against experiments is not part of the suite.

14.1 Diagnostics

A diagnostic maps the state at the end of a step to a number or a pair of numbers. The kinetic energy and the linear momentum are sums over the interior faces $\mathcal{F}_\Omega$, weighted with the open staggered volumes $|\omega_f^{\mathrm{o}}|$ ($= |\omega_f|$ without a solid), the staggered mixture density $\widetilde{\rho}_f$ and the clamped staggered liquid fraction $\hat{C}_f$ of (8.4): with one velocity

$$E_k = \tfrac{1}{2} \sum_{f \in \mathcal{F}_\Omega} |\omega_f^{\mathrm{o}}|\, \widetilde{\rho}_f\, u_f^2, \qquad P_k = \sum_{f \in \mathcal{F}_\Omega \cap \mathcal{F}^{[k]}} |\omega_f^{\mathrm{o}}|\, \widetilde{\rho}_f\, u_f, \tag{14.1}$$

with $\widetilde{\rho}_f$ replaced by the density ρ in one phase, and with two velocities

$$\begin{aligned} E_k &= \tfrac{1}{2} \sum_{f \in \mathcal{F}_\Omega} |\omega_f^{\mathrm{o}}| \left(\rho^{\mathrm{l}} \hat{C}_f (u_f^{\mathrm{l}})^2 + \rho^{\mathrm{g}} (1 - \hat{C}_f)(u_f^{\mathrm{g}})^2 \right), \\ P_k &= \sum_{f \in \mathcal{F}_\Omega \cap \mathcal{F}^{[k]}} |\omega_f^{\mathrm{o}}| \left(\rho^{\mathrm{l}} \hat{C}_f u_f^{\mathrm{l}} + \rho^{\mathrm{g}} (1 - \hat{C}_f) u_f^{\mathrm{g}} \right), \end{aligned} \tag{14.2}$$

where a product with a vanishing weight ($|\omega_f^{\mathrm{o}}|$, $\hat{C}_f$ or $1 - \hat{C}_f$) is zero, also where the phase velocity is undefined. Boundary faces carry prescribed values, not unknowns, and do not

enter. The phase volumes $V^{\mathrm{l}} = \sum_c |c| \alpha_c^{\mathrm{l}}$ and $V^{\mathrm{g}} = |\Omega^{\mathrm{o}}| - V^{\mathrm{l}}$, with the open volume $|\Omega^{\mathrm{o}}| = \sum_c |c| \alpha_c^{\mathrm{o}}$ ($|\Omega|$ without a solid), give the phase masses $\rho^\pi V^\pi$. The potential and surface energies are

$$E_{\mathrm{p}} = -\boldsymbol{g} \cdot \left(\rho^{\mathrm{l}} \boldsymbol{M}^{\mathrm{l}} + \rho^{\mathrm{g}} \left(\boldsymbol{M}^{\mathrm{o}} - \boldsymbol{M}^{\mathrm{l}} \right) \right), \qquad E_\sigma = \sigma |I|,$$
$$|I| = \sum_{c:\ 0 < C_c^{\mathrm{l}} < 1} |I_c| + \sum_{f \in \mathcal{F}_\Omega \text{ separating}} |f| \, a_f^{\mathrm{o}}, \tag{14.3}$$

where $\boldsymbol{M}^{\mathrm{o}}$ and $\boldsymbol{M}^{\mathrm{l}}$ are the first moments $\int \boldsymbol{x} \, \mathrm{d}\boldsymbol{x}$ of the open region and of the reconstructed liquid region: in a cell with $0 < C_c^{\mathrm{l}} < 1$ the part of c^{o} on the liquid side of its PLIC line (Chapter 4), and c^{o} itself if $C_c^{\mathrm{l}} \geq 1$. A mixed cell thus contributes the moment of its phase polygons, not its mass at its centre; the potential is zero at the origin. I_c is the PLIC segment of a mixed cell, clipped to the open part, and a face is separating if one neighbour has $C_c^{\mathrm{l}} \leq \varepsilon^\alpha$ and the other $C_c^{\mathrm{l}} \geq 1 - \varepsilon^\alpha$: an interface on a grid line has no PLIC segment and is measured by the face. The measure $|I|$ is the length of the reconstruction; on a circle it converges with an observed order of about 0.7, so E_σ is a diagnostic of changes, not a second-order approximation of the exact surface energy. The total energy is $E_k + E_{\mathrm{p}} + E_\sigma$; in one phase $E_{\mathrm{p}} = E_\sigma = 0$. The divergence norm is

$$\|\mathfrak{D}\|_2 = \Big(\sum_{c:\ \alpha_c^{\mathrm{o}} > 0} |c^{\mathrm{o}}| \, (\mathfrak{D}^\star u)_c^2 \Big)^{1/2}, \qquad \mathfrak{D}^\star u = \begin{cases} \mathfrak{D} u, & \text{one velocity, no solid,} \\ \mathfrak{D}^{\mathrm{o}} u, & \text{one velocity, solid,} \\ \mathfrak{D}^{\mathrm{mix}}(u^{\mathrm{l}}, u^{\mathrm{g}}), & \text{two velocities,} \end{cases} \tag{14.4}$$

with the divergences of (3.11), (3.13) and (10.11): the constraint of the projection (Chapter 10), not a density-weighted divergence. The boundary faces enter the divergence with their prescribed values (and, with two velocities, the boundary phase apertures), unlike the energies. The largest velocity is $\max |u_f|$ over the interior and boundary faces, with two velocities over the faces on which a phase is active; the remaining diagnostics are the time-step bound $\delta t_{\mathrm{flow}}(\nu^{\mathrm{f}})$ of (13.1) and the numbers of leaf blocks and cells.

Implementation. Diagnostics `KineticEnergy`, `PotentialEnergy`, `SurfaceEnergy`, `TotalEnergy`, `LinearMomentum`, `PhaseVolume`, `PhaseMass`, `InterfaceMeasure`, `MaximumVelocity`, `CFLTimeStep`, `DivergenceNorm`, `LeafBlockCount`, `CellCount`, evaluated by `diagnostic_value` (`src/simulation/diagnostics/interface.jl`) with `_one_velocity_kinetic_energy` (`src/simulation/diagnostics/interface.jl`), `_two_velocity_kinetic_energy` (`src/simulation/diagnostics/interface.jl`), `_one_velocity_linear_momentum` (`src/simulation/diagnostics/interface.jl`), `_two_velocity_linear_momentum` (`src/simulation/diagnostics/interface.jl`), `_diagnostic_volume_flux!` (`src/simulation/diagnostics/interface.jl`); clamped fraction `_staggered_liquid_fraction` (`src/two_phase/forces.jl`); first moments `_phase_first_moments` (`src/two_phase/forces.jl`) and `_gravitational_potential_energy` (`src/two_phase/forces.jl`); segments `plic_segments` (`plic_segments` (`src/visualization/plic_geometry.jl`)). A `FlowSimulation` records each diagnostic when it is added, after the steps that its schedule selects (`EveryStep` by default) and at the end of a call of `simulate!`, never twice for the same state (`_diagnostic_due!` (`src/simulation/diagnostics/history.jl`));

`record_diagnostics!` (`src/navier_stokes/flow_diagnostics.jl`) records in a hand-written loop and `diagnostic_history` returns the record. Tests: `test/diagnostics.jl` ("one- and two-velocity physical diagnostics") (exact energies, momenta, volumes, masses and interface measure on a 4×4 mesh, including an interface on a grid line and a mixed one), `test/diagnostics.jl` ("embedded and adaptive diagnostic measures") (cut and adapted meshes), `test/diagnostics.jl` ("two-velocity divergence uses the projection volume flux") (equal to an independent face sum to 2×10^{-13}, independent of the densities), `test/diagnostics.jl` ("two-velocity projected-step divergence with slip").

14.2 Test suite and continuous integration

Every Julia file under `test/` is registered in `test/groups.jl`: in one or more test groups (an integration group lists its prerequisites), among the helpers, or among the exclusions (the entry points, the separately provisioned Makie suite and the MimeticFV comparison). The group `registry` fails when a file is unregistered or a registered file is missing, and when a default group is not in exactly one unit shard. The default suite (`Pkg.test()`) is `registry` and the eleven groups that are neither integration groups nor one of the separate groups `performance` and `threading_equivalence`, which have their own jobs. Arguments select groups by name, `integration` selects the thirteen integration groups and `shard:A`, `shard:B`, `shard:C` the unit shards. The unit groups check contracts, exact identities and short manufactured-solution studies; the integration groups run refinement studies and benchmark flows; `performance` holds allocation contracts; `threading_equivalence` repeats five short flows (Taylor–Green vortices on an adaptively refined mesh, two-phase CMDGFM on an embedded sloped wall, a capillary wave with interface refinement, the two-velocity Stokes-3 wave, which must regrid during the run, and the cylinder) in fresh processes with one, two and four Julia threads and the threading grain at one, so that every loop takes its threaded branch. It requires bitwise equal SHA-256 hashes of every state field (velocity components per phase, pressure, liquid fraction) and of every recorded diagnostic history, equal step, mesh-update and leaf-block counts, and warm steps of the Taylor–Green and capillary cases within a recorded allocation budget, so the oracle of Section 14.3 holds for every thread count. Table 14.1 lists when continuous integration runs what.

A commit on `main` is thus checked by the unit and main tiers; the integration groups run on it only at the next weekly run and the heavy groups at the next monthly run, unless a run is dispatched or the pull request is labelled. Three checks are manual: the regression oracle (Section 14.3); the developed-wake cylinder study `benchmark/validation/cylinder_study.jl` (Section 14.7); and the comparison of one first-order upwind step with the package MimeticFV.jl, `test/integration/mimeticfv_reference.jl`, excluded from the registry so that the reference package is not a test dependency, which passed at relative and absolute tolerances of 3×10^{-14} on 2026-09-13 (`test/reference-validation.md`). At the audited commit the `contracts` job fails on `main`: one solve of the staggered elliptic solver allocates 624 bytes (issue 127, since fixed by PR #130).

Implementation. Registry `TEST_GROUPS` (`test/groups.jl`), `UNIT_SHARDS` (`test/groups.jl`), `TEST_HELPERS` (`test/groups.jl`), `TEST_EXCLUSIONS` (`test/groups.jl`); selection and registry check `selected_groups` (`test/runtests.jl`),

Table 14.1. Tiers of continuous integration (workflow `CI`, and the technical-reference job of workflow `Documentation`). Julia "1" is the current release; the `unit` shards run on Julia 1.11 (the lower compat bound) and 1.12, the unit and integration jobs run on two threads and `threading-equivalence` on four. The `ci-ok` check of a pull request requires every job of the first row; the `scaling` tier is a measurement and is not part of it.

Tier	Trigger	Jobs and contents
Pull request	every pull request and push to `main`	`format` (Runic 1.10.0); `unit` shards A (`two_phase`), B (`registry`, `mesh`, `threading`, `indicators`, `operators`, `solvers`, `aqua`), C (`navier_stokes`, `diagnostics`), D (`examples`) and E (`recording`) on Julia 1.11 (the lower compat bound) and 1.12, with coverage from the 1.12 shards; `threading-equivalence` (group `threading_equivalence`: complete flows in fresh 1-, 2- and 4-thread processes, Julia 1.12); `makie` (visualisation extension); `studio` (Studio server, Julia 1.11 and 1.12); `frontend` (browser client tests and bundle check)
Main	push to `main`	`contracts` (group `performance`, one thread); `benchmark` (one-sample smoke run of the benchmarks and the latency probe)
Integration	weekly (Monday 03:00 UTC), dispatch, or a pull request labelled `run-integration`	eleven integration groups in three shards: A `two_velocity_vortex`, `two_velocity_transport`, `staggered_wave_equation`; B `two_phase_waves`, `two_phase_droplet`, `wave_equation`, `amr_manufactured_flow`; C `two_phase_vortex`, `two_velocity_rotation`, `anisotropic_cells`, `two_phase_sessile_droplet`
Heavy	monthly (1st, 04:00 UTC) or dispatch	`two_velocity_instabilities` and `single_phase_cylinder`; a failure does not fail the run (`continue-on-error`)
Scaling	dispatch, tier `scaling`	`thread-scaling`: the ten target cases of `benchmark/threading/scaling.jl` (Taylor–Green, cylinder, capillary wave and the one- and two-velocity Stokes-3 wave, each at its default resolution and one level finer) at one, two and four threads, one job per thread count, with the logs uploaded as an artifact
Browser	weekly and monthly schedule, dispatch	`studio-browser`: Studio smoke, live AMR and examples tests in Chromium
Reference	every pull request and push to `main`	`check_links.py --skip-api` (blocking), PDF build, rejection of undefined references (Section 14.4)

Table 14.2. Cases of the regression oracle: the examples of the gallery (Section 14.8) at a reduced maximum level and final time. All flows except the two marked use one velocity.

Case	Example	Run	Steps
`taylor_green`	01 Taylor–Green vortex	level 2, $t_{\mathrm{end}} = 0.02$, $\delta t = 0.002$	10
`cavity`	02 lid-driven cavity	level 1, $t_{\mathrm{end}} = 0.02$, $\delta t = 0.002$	10
`cylinder`	04 flow past a cylinder	level 1, $t_{\mathrm{end}} = 0.01$, $\delta t = 0.001$	10
`vortex_reverse`	05 vortex reversal (transport only)	level 2, $t_{\mathrm{end}} = 0.05$, automatic step	6
`capillary_one`	06 capillary wave	level 1, $t_{\mathrm{end}} = 0.02$, $\delta t = 0.001$	20
`capillary_cmdgfm`	06, two velocities, CMDGFM	as above	20
`sessile`	08 sessile droplet	level 1, $t_{\mathrm{end}} = 0.01$, $\delta t = 0.001$	10
`sloped`	09 sloped sessile droplet	level 1, $t_{\mathrm{end}} = 0.01$, $\delta t = 0.001$	10
`stokes3`	10 third-order Stokes wave	level 1, $t_{\mathrm{end}} = 0.01$, $\delta t = 0.001$	10

`SEPARATE_GROUPS` (`test/runtests.jl`), `default_groups` (`test/runtests.jl`), `check_registry` (`test/runtests.jl`); the equivalence cases `test/threading_equivalence_cases.jl` and their comparison `test/threading_equivalence.jl`. Workflows `.github/workflows/CI.yml` and `.github/workflows/Documentation.yml`; the entry points and their use are documented in `docs/src/testing.md`.

14.3 Regression oracle

The regression oracle detects every change of the results of ten short runs of the examples (Table 14.2); it does not judge whether a change is an improvement. For each case it records the number of steps, the final time, the last value of every diagnostic history of the flow (the volume error for the transport-only vortex), and for each velocity component (per phase with two velocities, on the owned interior faces of its direction), the pressure and the liquid fraction (on the cell interiors) four statistics: the SHA-256 hash of the raw values, the Euclidean norm and the largest magnitude of the finite values, and the numbers of values and of finite values. Inactive two-velocity faces hold NaN: they enter the hash and the counts, not the norms. `benchmark/oracle/check.jl` compares the steps and the counts exactly and the final time and the diagnostics with relative and absolute tolerance 10^{-10}; it compares the fields by their hashes, that is bitwise, when the Julia version and the thread count equal those of the record (Julia 1.12.7, one thread), and otherwise by their norms and largest magnitudes with the same tolerance. The mode and the tolerance can be set, and ORACLE_PERTURB=1 changes one velocity of the Taylor–Green case by one unit in the last place to exercise the failure path. `benchmark/oracle/perf.jl` records first-run latency and the time and allocations of ten warm steps (`perf_baseline.toml`); these are machine-local reference points, not contracts.

No workflow runs the oracle. A pull request that restructures code without intending a numerical change runs it by hand and reports the result ("oracle bitwise (10 cases)" for pull request 124, merged as the audited commit). `benchmark/oracle/record.jl` rewrites the record; this is done only when the numerical reference or the measurement environment changes intentionally, together with the tests that verify the change. Since its introduction on 2026-11-07 the record was rewritten for the deterministic face order needed on Julia 1.11 and 1.13 (every face list reordered; tolerance mode passes at 10^{-10}), and after

the fixes of the triple-cell reconstruction (issue 48), of the staggered momentum transport at walls, level transitions and cut walls (issue 60, issue 61, issue 62), of the cut-face gravity (issue 74), of the cut-cell diffusion (issue 79, issue 87), of the incremental projection (all cases within 10^{-10} relative) and of the projection (issue 93, issue 94: outflow reset, dimensionless two-velocity defects, mixed pressure gauges).

Implementation. Cases CASES (`benchmark/oracle/cases.jl`); statistics `stats` (`benchmark/oracle/cases.jl`) of the fields gathered by `capture` (`benchmark/oracle/cases.jl`); record `record_cases` (`benchmark/oracle/cases.jl`); comparison `check_all` (`benchmark/oracle/check.jl`) and `close_value` (`benchmark/oracle/check.jl`). Usage in `benchmark/README.md`; the record is `benchmark/oracle/baseline.toml`.

14.4 Derivation audits

A derivation audit checks one claim of this reference (an equation, an identity, a bound or a measured statement) independently of the chapter that states it: symbolically, and numerically against the package, mostly through an independent re-implementation of the discrete formula evaluated on random 2:1-balanced, periodic, walled and cut meshes. Each audit is archived in a folder of `proof_review/`, mostly named after the label of the claim, with `audit.md` (the steps, the errors found, the sharpness of the claim and its history) and `scripts/`, whose `run_all.jl` reruns every check and exits with an error if one fails; `proof_review/README.md` has one row per audited claim with the audited lines, the date, the status and the outcome. In this version of the reference 67 audit folders cover Chapters 3 to 14. A finding is either applied to the chapter or, if the code is at fault, filed as an issue and stated in a deviation box or in the text, as issue 63 and issue 70 from the audits of Chapter 6 and issue 78 and issue 79 from those of Chapter 9.

Each reference chapter states the commit against which it was audited (the optional argument of `\chapterstatus`, printed below its title); `coverage.toml` repeats the commit, and `check_links.py` fails if the two differ. The same script checks that every `\srcsym` names a symbol defined in its file, every `\testsym` an existing `@testset` title, and every claim of `coverage.toml` its source symbols and test titles. Its re-audit report, which does not fail, lists for each reference chapter the files it refers to (through `\srcsym`, the sources of its claims and the source files classified for it in `coverage.toml`) that changed between the audited commit and HEAD. A listed chapter is re-read against the changes and re-pinned, with a new audit where a stated equation changed. Numbers quoted in the chapters are computed by the scripts of `docs/techref/figures/`, whose assertions fail when the implementation no longer matches the statements; they need Julia and are run by hand (`generate.jl`, or `generate.jl amr` and `generate.jl stability` for one chapter), while the PDF build reads only their committed output.

Implementation. `proof_review/README.md` (index), `docs/techref/coverage.toml` (claims, chapter status and audited commits, classification of every file of `src/` and `ext/`), `docs/techref/check_links.py` (checks and re-audit report), `docs/techref/figures/generate.jl` (figure data and the number macros of `figures/generated/checks.tex`).

14.5 Manufactured solutions and observed orders

Table 14.3 lists the refinement studies with an exact or manufactured reference. A study computes the errors on two to four meshes refined by a common ratio and asserts lower bounds on the rates between consecutive meshes (*all*), on the rate between the two finest (*fin*) or on the endpoint rate between the coarsest and the finest (*end*); several also assert that the errors decrease. The formal orders of the building blocks and the time-step policy of the flow studies are those of Section 13.5. The measured values are those that the tests print in continuous integration (two threads): the unit groups in the run of the audited commit, the integration and heavy groups in the runs of commit `e736e6b` (pull request 116, merged as `800e4ae`), after which the sources changed only by recording code and by the refactor of pull request 124, which reproduces the oracle bitwise; "–" marks a test that prints nothing. The rows that pin a known first-order defect are those of issue 110 (inclined walls) and issue 112 (variable stress at level transitions).

Table 14.3. Refinement studies with an exact or manufactured reference (mm: manufactured). Meshes as cells per unit length or per diameter; rates as defined in the text.

Case	Quantity	Reference	Measured	Asserted
Operators				
Gradient, composite	face-normal gradient, displacement-weighted L^2	mm pressure	–	both > 1.4
Gradient, stretched composite	face gradient, area-weighted L^2	mm pressure	–	all > 1.0, decreasing
Poisson, composite, multigrid	cell L^2	mm, Dirichlet	–	all > 1.7, decreasing
Staggered Laplacian, uniform periodic	dual-volume L^2 of the tendency	mm velocity	–	both > 1.9
Composite Helmholtz (issue 80)	maximum and dual-volume L^2 solution error	mm velocity	1.78–1.84, 1.88–1.92 (max.); 1.99–2.01 (L^2)	first pair ≥ 1.7, second ≥ 1.85
Cut-cell Helmholtz, walls and transitions (issue 87)	maximum norm	mm velocity	–	all ≥ 1.8
Aligned embedded wall (issue 106)	maximum, L^2, L^1	mm velocity	–	all ≥ 1.5 (max.), ≥ 1.8 (L^2, L^1)
Inclined embedded wall (issue 106, issue 110)	maximum, L^2, L^1	mm velocity	–	all ≥ 0.6 (max.), ≥ 0.8 (L^2, L^1); finest error $\leq 1.02\times$ pinned
Variable stress, bulk	L^2 of the tendency	mm, $\mu(x)$	–	all ≥ 1.8
Variable stress, shifted solve on fixed and adapted trees	solution L^2	mm, $\mu(x)$	–	all ≥ 0.8, decreasing
Two-phase embedded-wall Helmholtz (issue 106)	dual-volume L^2 per direction	mm, piecewise stream function	1.86–2.06	all ≥ 1.9 (equal viscosities, uniform); else within $\log_2 1.1$ of the open domain

Case	Quantity	Reference	Measured	Asserted
Stress with uniform viscosity versus scalar diffusion	L^2 difference	scalar Laplacian	2.11–2.83 (uniform); 0.48–0.50 (composite)	all ≥ 2.0 (uniform); composite: below $1.1\times$ the same-domain open difference
Lax–Wendroff translation	L^2	exact translation	–	both > 1.9
Staggered convection, composite	dual-volume L^2 of the tendency	mm velocity	–	all > 1.3; > 1.35 (Lax–Wendroff, limited at transitions)
Interface				
LHF curvature, circle	maximum over LHF cells	exact circle	–	all ≥ 1.7
PLIC position, circle	RMS distance of segment midpoints	exact circle	–	all > 1.49
Contact-angle curvature at triple points, five angles	maximum	exact circle	–	all ≥ 0.7; ≥ 1.8 at 90°
Triple-cell normals, three methods, 60°–120°	median normal error	exact circle	0.039, 0.010, 7.4×10^{-4}	decreasing; finest below half the coarsest
Embedded contact curvature, sloped walls	median	exact circle	–	end > 0.8
Embedded height reconstruction	symmetric difference	clipped circle	3.70, 2.13	all ≥ 1.8
Geometry-aware level-set initialisation	open-volume symmetric difference	clipped circle	–	all ≥ 1.8
Anisotropic (2:1) cells	mean normal and curvature errors	exact circle	–	end > 0.9 (normal), > 1.5 (curvature)
Transport				
Anisotropic VOF translation	L^1 fraction	exact translation	–	all > 1.8
Anisotropic centred scalar advection	volume-weighted L^2	exact translation	–	all > 0.9
Cut-cell WY contact-line translation and reversal	symmetric difference	exact positions	2.77, 3.70; reversal end 2.90	all ≥ 1.8; reversal end ≥ 1.8
Vortex reversal $T = 1$, one velocity, uniform (16–64)	symmetric difference; L^1 fraction; L^1 cell and staggered phase scalars	initial state	2.16, 2.24; 2.21, 2.16; 1.89, 1.63; 1.59, 2.12	end and fin ≥ 1.5
composite and adaptive (8–32)	as above	initial state	composite 1.87, 2.29; adaptive 3.20, 2.55 (symmetric difference)	end and fin ≥ 1.5
Vortex reversal, two velocities	symmetric difference; L^1 fraction; staggered liquid and gas scalars	initial state	–	end ≥ 1.5; fin ≥ 1.5
Split-WY reversal, two velocities (16, 24, 32)	L^1 fraction; phase scalars	initial state	–	fin ≥ 1.8 (uniform), ≥ 1.5 (scalars)
Rotating disc, tangential jump	L^1 fraction after the rotation	initial state	end 1.31	end ≥ 0.8

Case	Quantity	Reference	Measured	Asserted
Rotating disc, CMDGFM projection (8–32)	L^1 fraction	initial state	end 1.40 (CMDGFM)	end ≥ 0.9
Phase-scalar AMR transfer	phase-weighted L^1 after one refinement	mm scalars	–	all ≥ 1.8
CMDGFM jump interpolant	mean jump error over resolved stencils	affine velocity with a constant jump	–	end ≥ 0.8, decreasing
Flows				
Taylor–Green vortex (12, 24)	velocity L^2	exact decay	–	> 1.8; divergence $\leq 2 \times 10^{-8}$
Wave equation, composite	pressure and velocity L^2 and maximum	exact standing wave	–	all > 1.5 (L^2); maximum > 1.5 (pressure), > 1.0 (velocity)
Staggered wave equation	L^2, maximum	exact standing wave	–	uniform all > 1.9; composite > 1.5 (L^2), > 0.9 (maximum)
Localised manufactured vortex, AMR, 1 to 3 levels	velocity L^2	mm vortex	–	all > 0.8; smaller error than a uniform mesh with as many cells, fewer than 0.6 of the cells of the uniform mesh at the finest width
Anisotropic Navier–Stokes (8, 16)	phase velocity L^2	exact decaying shear	–	> 1.8 (one phase), > 1.35 (two phases)

Where the tests print their errors, every measured order exceeds its bound. On the adaptive meshes, whose refined band follows the interface, the vortex reversal converges with 3.2 and 2.5 in the symmetric difference. The maximum norm of the fraction is not asserted and does not converge (uniform 0.60, 0.37; composite -0.16, 0.28). With a level transition the two-phase embedded-wall rates stay within $\log_2 1.1$ of the open-domain rates. The scalar Laplacian now uses the same geometric transition contacts as the fully open cut-cell operator (issue 80). Pointwise transition-row truncation errors need not predict solution-level convergence: the composite Helmholtz solution above approaches second order in both norms (≥ 1.7 between the first two base widths on the tested layouts) despite transition rows with $O(1)$ error. The separate variable-viscosity stress transition remains first order (issue 112; the shifted-solve test pins 0.8).

Implementation. Tests in the order of Table 14.3:
`test/operators/gradient.jl` (“mixed-level gradient convergence”),
`test/operators/gradient.jl` (“compatible pressure gradient”),
`test/operators/poisson.jl` (“geometric multigrid Poisson convergence”),
`test/operators/staggered_diffusion.jl` (“uniform periodic staggered Laplacian convergence”), `test/operators/staggered_diffusion.jl` (“composite diffusion solution convergence (#80)”),
`test/operators/staggered_diffusion.jl` (“composite implicit diffusion is energy contractive (#80)”), `test/operators/staggered_diffusion.jl` (“cut-cell Helmholtz convergence at walls and level transitions (#87)”),

`test/operators/embedded_wall_rows.jl` (“grid-aligned embedded-wall Helmholtz convergence (#106)”), `test/operators/embedded_wall_rows.jl` (“oblique embedded-wall Helmholtz convergence (#110)”), `test/two_phase/variable_staggered_diffusion.jl` (“Variable stress manufactured bulk convergence”), `test/two_phase/variable_staggered_diffusion.jl` (“Variable stress shifted solve on fixed and adapted trees”), `test/operators/embedded_variable_wall_mms.jl` (“two-phase embedded-wall manufactured Helmholtz (#106)”), `test/operators/embedded_variable_wall_mms.jl` (“uniform-viscosity stress versus scalar diffusion (#106)”), `test/operators/advection/staggered_advection.jl` (“Lax-Wendroff convergence”), `test/operators/advection/staggered_advection.jl` (“mixed-level staggered convection convergence”); `test/indicator/phase/height_functions.jl` (“Circular LHF curvature converges”), `test/indicator/geometry.jl` (“PLIC geometry converges and produces filled solid visualization”), `test/indicator/phase/height_functions.jl` (“Contact-angle curvature converges at triple points”), `test/indicator/geometry.jl` (“Triple-cell normals converge with matching and mismatched angles”), `test/indicator/geometry.jl` (“Embedded GHF contact curvature converges on sloped walls”), `test/indicator/geometry.jl` (“Embedded height reconstruction converges in symmetric difference”), `test/indicator/geometry.jl` (“Geometry-aware level-set initialization converges in open volume”), `test/indicator/phase/anisotropic_cells.jl` (“anisotropic reconstruction refinement”); `test/integration/anisotropic_cells.jl` (“anisotropic geometric VOF advection”), `test/indicator/phase/anisotropic_cells.jl` (“anisotropic centred advection refinement”), `test/indicator/geometry.jl` (“Cut-cell WY reverses wall-tangent contact-line translation”), `test/integration/two_phase_vortex.jl` (“Classical T=1 vortex refinement study”), `test/integration/two_velocity_vortex.jl` (“Continuous two-velocity vortex reversal”), `test/integration/two_velocity_transport.jl` (“Two-velocity split-WY reversal convergence”), `test/integration/two_velocity_rotation.jl` (“Tangentially discontinuous circular-disc rotation”), `test/integration/two_velocity_rotation.jl` (“Rotating disc with the CMDGFM projection”), `test/two_phase/transport.jl` (“Intrinsic phase scalar AMR transfer convergence”), `test/two_phase/two_velocity/projection/cmdgfm_projection.jl` (“CMDGFM jump interpolant consistency”); `test/navier_stokes/navier_stokes.jl` (“Taylor–Green Navier–Stokes integration”), `test/integration/wave_equation.jl` (“mixed-level manufactured wave equation”), `test/integration/staggered_wave_equation.jl` (“manufactured staggered wave equation”), `test/integration/amr_manufactured_flow.jl` (“localized manufactured vortex demonstrates AMR benefit”), `test/integration/anisotropic_cells.jl` (“anisotropic Navier–Stokes model refinement”). The vortex studies share `phase_vortex` (`test/integration/vortex_problem.jl`).

14.6 Conservation and invariants

Table 14.4 lists the tests of the properties that the discretisation has by construction: conservation of the phase volumes, masses and momenta, boundedness of the fractions, the energy identities, the divergence after a projection or a regrid, the balance of forces at rest and the exactness for constant and affine data. The tolerances are multiples of the unit round-off ϵ_{mach}, scaled with the number of faces and steps where errors accumulate, or of the solver tolerance where a linear solve enters.

Table 14.4. Conservation and invariant tests; n is the number of steps and $\#\mathcal{F}_\Omega$ that of interior faces.

Property	Setting	Asserted
Volume, phase constants, bounds	WY sweeps on uniform, composite, adapted and multi-root meshes, density ratio 1000, 20 steps	volume and constants to 2×10^{-14}; fractions in $[0, 1]$
Phase constants with dilation	exact face fluxes with a time-dependent dilation	3×10^{-14}
Root partition	one versus four root blocks	transported values to 4×10^{-14}
Volume and amounts in regrids	refine and coarsen with advection	volume 3×10^{-14}, liquid amount 3×10^{-11}
Volume and amounts in flows	vortex reversals; rotating discs; standing waves; two-velocity reversal	$16n\,\#\mathcal{F}_\Omega\,\epsilon_{\mathrm{mach}}$, plus $2n\varepsilon^\alpha\lvert\Omega\rvert$ for the discs and waves; reversal $16n\max(10^{-12}, \epsilon_{\mathrm{mach}})$
Volume, sessile and contact-line runs	sessile droplet; contact-line translation	$4096\epsilon_{\mathrm{mach}}$
Momentum, two-phase transport	one-velocity momentum with every flux interpolant	$512\epsilon_{\mathrm{mach}}$
Momentum, cut coarse cells (issue 63)	mass–momentum consistency of a cut coarse–fine cell	operator identity to 2×10^{-13}, drift $128\epsilon_{\mathrm{mach}}$
Momentum, viscous step	two-velocity implicit diffusion on uniform, composite and cut meshes	2×10^{-10} relative; jump unchanged to $16\epsilon_{\mathrm{mach}}$
Momentum, AMR transfer	refine and coarsen of a face velocity	2×10^{-13}; divergence does not grow
Momentum, wall force (issue 106)	aligned embedded wall, three wall types, single and double precision	momentum loss equal to the viscosity times the wall length, to $512\epsilon_{\mathrm{mach}}$ times that length
Skew-symmetry	centred staggered convection, uniform and composite	2×10^{-14}; constant field $64\epsilon_{\mathrm{mach}}$
Energy, viscous operator	uniform and composite	energy rate $\leq 256\,\#\mathcal{F}_\Omega\,\epsilon_{\mathrm{mach}}$; stiffness eigenvalues non-negative to round-off; implicit energy factor $\leq 1 + 10^{-12}$
Divergence, projections	CMDGFM projection, after a regrid, with slip, open boundaries	defects $\leq$ solver tolerance; flows $\leq 2 \times 10^{-8}$
Gradient–divergence duality	elliptic operator	8×10^{-13}; symmetric to 4×10^{-14}
Hydrostatic rest	flat heavy interface, vertical and oblique gravity, density ratio 1000; sloped solid on a composite mesh; CMDGFM with solids (issue 76)	velocity $< 10^{-9}$; $\leq 2048\epsilon_{\mathrm{mach}}$; $\leq 10^{-7}\delta t\,\lvert\boldsymbol{g}\rvert$
Static drop	variable-density CSF and CGFM with constant curvature	velocity after the projection $< 10^{-9}$

Property	Setting	Asserted
Exactness	affine face prolongation; limited cell prolongation; constant fields in transfer and advection; translation by the frozen subfaces; prescribed through-flow	5×10^{-14}; $4\epsilon_{\mathrm{mach}}$; exact; $512\epsilon_{\mathrm{mach}}$; $64\epsilon_{\mathrm{mach}}$

No test asserts the balance of the total energy over a complete step: the energy enters through the identities of the projection and of the viscous operator and the decay of the parasitic currents of the floating droplet (Section 14.7).

Implementation. Tests in the order of Table 14.4: `test/two_phase/transport.jl` ("WY volume and phase constants on uniform, mixed and regridded meshes"), `test/two_phase/transport.jl` ("WY dimensional dilation preserves phase constants"), `test/two_phase/transport.jl` ("WY transport is independent of base-root partitioning"), `test/two_phase/transport.jl` ("Geometric phase transfer through dynamic refinement and coarsening"), `test/integration/two_phase_vortex.jl` ("Classical T=1 vortex refinement study"), `test/integration/two_velocity_rotation.jl` ("Tangentially discontinuous circular-disc rotation"), `test/integration/two_phase_waves.jl` ("Prosperetti standing waves on uniform and AMR meshes"), `test/integration/two_velocity_transport.jl` ("Two-velocity split-WY reversal convergence"), `test/integration/two_phase_sessile_droplet.jl` ("Sessile droplet equilibrium convergence"), `test/indicator/geometry.jl` ("Cut-cell WY reverses wall-tangent contact-line translation"), `test/two_phase/transport.jl` ("All staggered flux interpolants transport one-velocity momentum"), `test/two_phase/transport.jl` ("Cut coarse-fine mass-momentum consistency"), `test/two_phase/two_velocity/diffusion.jl` ("Two-velocity implicit diffusion identities"), `test/mesh/adaptivity.jl` ("in-place conservative adaptive field transfer"), `test/operators/embedded_wall_force.jl` ("aligned embedded-wall force balances viscous momentum loss (#106)"), `test/operators/advection/staggered_advection.jl` ("centred staggered convection is skew-symmetric"), `test/operators/advection/staggered_advection.jl` ("AMR staggered flux compatibility and stability"), `test/operators/staggered_diffusion.jl` ("staggered diffusion stable core on uniform and AMR meshes"), `test/two_phase/two_velocity/projection/cmdgfm_projection.jl` ("CMDGFM projection invariants"), `test/two_phase/two_velocity/projection/cmdgfm_pressure_reuse.jl` ("CMDGFM incremental projection satisfies the coupled system"), `test/navier_stokes/navier_stokes.jl` ("open channel pressure flux and divergence"), `test/operators/poisson.jl` ("elliptic operator preserves gradient-divergence duality"), `test/navier_stokes/two_phase_navier_stokes.jl` ("Flat heavy interface remains hydrostatic"), `test/two_phase/forces.jl` ("Hydrostatic balance"), `test/two_phase/forces.jl` ("Hydrostatic liquid at a sloped solid on a composite mesh"), `test/two_phase/two_velocity/projection/cmdgfm_projection.jl` ("CMDGFM hydrostatic rest with embedded solids (issue #76)"), `test/two_phase/two_velocity/projection/cmdgfm_projection.jl` ("CMDGFM balance needs the projection's force map (#76)"), `test/two_phase/forces.jl` ("Variable-density balanced CSF and CGFM projection"), `test/mesh/adaptivity.jl` ("face prolongation is affine

exact”), `test/mesh/variables.jl` (“cell fields, halos, restriction, and prolongation”), `test/operators/advection/cell_advection.jl` (“conservative centred advection”), `test/two_phase/two_velocity/transport/two_velocity.jl` (“Frozen-subface WY common-normal translation”).

14.7 Benchmark flows

Table 14.5 lists the flows compared with an exact solution of the flow problem, with linear theory or with published data. The amplitude errors of the standing and unstable waves are the time-integrated L^2 errors of the amplitude history (the cosine moment of the liquid fraction) normalised by the norm of the reference history. The Prosperetti and instability studies refine the step with the mesh through the bounds of (13.1).

Table 14.5. Benchmark flows. Density ratio $\rho^{\mathrm{l}}/\rho^{\mathrm{g}}$ (DR); Weber (We), Bond (Bo) and Laplace (La) numbers as defined in the tests; cells per wavelength or diameter.

Flow	Quantity	Reference	Measured	Asserted
Lid-driven cavity, $\mathrm{Re} = 100$, 32×32, $t = 15$	centre-line velocities at 15 points each	Ghia, Ghia and Shin [9], Tables I–II	–	RMS < 0.004 (u), < 0.006 (v); maximum < 0.006, < 0.012; change to $t = 16 < 10^{-4}$
Cylinder, $\mathrm{Re} = 100$, symmetric start, $t = 1$ (4, 8)	total drag; largest lift	pinned transient values 0.89447, 1.0506	0.8982, 1.0607; 4×10^{-14}, 1×10^{-13}	drag within 0.02; lift $< 10^{-9}$
Capillary wave, La 3000, DR 1, one velocity (8, 16, 32)	amplitude error	Prosperetti [17]	0.0646, 0.0126, 0.0043; end 1.96	decreasing; end ≥ 1.8
DR 1000	as above	as above	end 1.63	end ≥ 1.5
gravity wave, DR 1000	as above	as above	end 0.66	end ≥ 0.4
capillary–gravity, DR 1000, uniform, composite, adaptive	as above	as above	end 1.63	end ≥ 1.5
two velocities	as above	as above	DR 1: end 1.98; DR 1000: 1.67	as one velocity
flat interface at rest	amplitude and speed	rest	1.1×10^{-14}	$\leq 10^{-9}$
Floating droplet, mode 4, DR 1000 (16, 32)	period of the kinetic energy; largest kinetic energy	Lamb period	–	period error and energy fall by more than 4; viscous peaks decay
Sessile droplet, 90°, Bo 1.21, La 550 (12, 24, 48)	L^1 fraction at a quarter capillary time	Laplace–Young profile	2.23, 2.72	all ≥ 1.2, decreasing
Kelvin–Helmholtz, two velocities (8, 16, 32)	amplitude error	dispersion relation [19, App. 7.B]	DR 1, We 0.5: end 1.56; DR 1000: 0.91; DR 1, We 5: 1.02	end ≥ 0.8; decreasing
Rayleigh–Taylor, frozen (8, 16, 32)	amplitude error	as above	DR 3, Bo 0.5: end 3.05; DR 1000: 1.78; DR 3, Bo 2: 1.36	end ≥ 1.7; ≥ 1.3 (Bo 2); decreasing

Flow	Quantity	Reference	Measured	Asserted
Combined KH–RT, DR 3, We 0.5, Bo 0.75	as above	as above	end 0.95	end ≥ 0.79; decreasing

The capillary-wave and floating-droplet cases repeat studies of the thesis [19, §§4.8.1, 4.8.2, 7.7.2]; the dispersion relation of the Kelvin–Helmholtz and Rayleigh–Taylor waves is the inviscid relation of the thesis [19, App. 7.B]. Several bounds lie below the order of the scheme for a reason recorded in the test: the gravity wave (the gravity bound makes $\delta t \propto h^{1/2}$, so that the first-order splitting limits the order to $\frac{1}{2}$ until the advective bound is active, Section 13.5), the Rayleigh–Taylor wave at Bond number 2 (1.3) and the combined wave (0.79). The sessile droplet converges faster than the order of about 1.5 that the thesis observes for the symmetric difference of its sessile droplets [19, §4.8.3].

Cylinder. The regression test does not measure the developed wake: a symmetric start keeps the lift at round-off, and it pins the total drag at $t = 1$ on two uniform meshes. The developed wake at $\mathrm{Re} = 100$ is the manual study `benchmark/validation/cylinder_study.jl`: perturbed start, $t = 100$, 4, 8 and 16 cells per diameter, compared with the Strouhal number 0.165, the drags 0.93 (pressure) and 1.24 (total) and the largest lift 0.3 of Verstappen and Dröge [31]. It asserts that the finest mesh is closer to each reference than the coarsest and a pressure-drag order of at least 1.75; the earlier unperturbed study to $t = 40$, recorded in the test, measured -0.884. The gallery flow at $\mathrm{Re} = 200$ (Section 14.8) is compared with other reference data and asserts nothing.

Implementation. Tests in the order of Table 14.5:
`test/navier_stokes/navier_stokes.jl` ("Ghia Re=100 lid-driven cavity"),
`test/integration/single_phase_cylinder.jl` ("Re=100 cylinder short symmetric-start regression"), `test/integration/two_phase_waves.jl` ("Prosperetti standing waves on uniform and AMR meshes")
(`wave_case` (`test/integration/two_phase_waves.jl`), reference `prosperetti_amplitude` (`examples/06_capillary_wave/prosperetti_amplitude.jl`)), `test/integration/two_phase_droplet.jl` ("Floating droplet: equilibrium and Lamb frequency")
(`floating_droplet_case` (`test/integration/two_phase_droplet.jl`)),
`test/integration/two_phase_sessile_droplet.jl` ("Sessile droplet equilibrium convergence"),
`test/integration/two_velocity_instabilities.jl` ("Two-velocity Kelvin–Helmholtz dispersion convergence"),
`test/integration/two_velocity_instabilities.jl` ("Two-velocity Rayleigh–Taylor dispersion convergence"),
`test/integration/two_velocity_instabilities.jl` ("Combined KH/RT dispersion convergence"); dispersion relation `instability_dispersion` (`test/integration/interfacial_instability_problem.jl`), checked analytically by `test/integration/two_velocity_instabilities.jl` ("General two-velocity dispersion relation"). Manual study
`benchmark/validation/cylinder_study.jl` ("Re=100 cylinder paper validation") with the reference values `CYLINDER_REFERENCE` (`test/integration/cylinder_reference_problem.jl`).

Table 14.6. Numbers stated in the gallery READMEs at documentation size; none is asserted by a test.

Example	Quantity	Reference	README value
01 Taylor–Green	kinetic energy at $t = 5$	exact decay, 6.61579	6.28788 (error 0.327914)
02 lid-driven cavity	divergence norm; kinetic energy at $t = 15$	none (Ghia et al. not compared)	2.39154×10^{-7}; 0.0327184
03 decaying turbulence	divergence norm; projected kinetic energy	none	1.03542×10^{-6}; $715.044 \to 344.946$
04 cylinder, $\mathrm{Re} = 200$	mean pressure drag ($t = 30$–50); Strouhal number	Qu et al. [18]: 1.068, 0.1958	1.02563 ($-4.0\,\%$); 0.2303 ($+17.6\,\%$)
05 vortex reversal	volume change at $T = 8$	zero	4.75607×10^{-11}
06 capillary wave	amplitude at $t = 0.399142$; largest sampled error	Prosperetti [17], 0.00614806	0.00625680; 1.32120×10^{-4} (two velocities 1.31329×10^{-4})
07 interfacial instability	amplitude at $t = 2\,\mathrm{arcosh}\,2$; largest sampled error	linear theory, 0.02	0.0196321; 3.67921×10^{-4}
08 sessile droplet	area change; final kinetic energy	zero; none	-3.49632×10^{-10}; 3.39393×10^{-5}
09 sloped sessile droplet	volume change; divergence norm	zero	-1.46794×10^{-11}; 7.9249×10^{-8}
10 Stokes wave	interface after two periods versus initial profile (RMS; maximum)	third-order Stokes profile (not an exact solution)	0.0449043; 0.103414
11 LGPI	liquid volume change	zero	-3.61×10^{-9} of 0.182426
12 manufactured Poisson	pressure RMS/maximum error, 32 cells	manufactured	CMDGFM 1.04378/8.98783

14.8 The examples gallery

Each example of `examples/` states in its `README.md` numbers of a run at documentation size (Table 14.6), the size at which `examples/make_all_figures.jl` generates the gallery. They are snapshots: no test asserts them and no job regenerates them. The tests of the group `examples` run every example at reduced size and assert its structure, the convergence of the pressure solves, finite fields, bounded fractions and the conservation of the phase volumes; the group `navier_stokes` runs the quick start of the package `README.md`. The oracle (Section 14.3) shows whether the code has moved since a number was written: between the commits that introduced the numbers and the audited commit, its records of the Taylor–Green, cavity, vortex, one-velocity capillary and sessile runs are unchanged to round-off, those of the two-velocity capillary and Stokes runs changed by at most 5×10^{-9} relative and that of the sloped droplet by 6×10^{-7}, while the cylinder velocities changed by 5×10^{-3} relative when the record was rewritten for issue 93 and issue 94; the README numbers of the cylinder flow predate that change. The oracle runs only a few steps at reduced size: an unchanged record does not prove an unchanged documentation run, while a changed one shows that the numbers predate the code.

Implementation. Gallery driver `gallery_options` (`examples/make_all_figures.jl`); tests `test/examples/basic_examples.jl` ("Adaptive vortex-reversal example"), `test/examples/basic_examples.jl` ("Manufactured Poisson example"),

`test/examples/basic_examples.jl` (“Adaptive decaying turbulence example”), `test/examples/stokes3_example.jl` (“Stokes-wave refinement window”), `test/examples/lgpi_example.jl` (“Adaptive LGPI example”), `test/examples/two_phase_examples.jl` (“Sloped droplet transport through sliver cells”), `test/readme.jl` (“README quick start”).

14.9 Known gaps

Open defects. Of the issues labelled *bug* and open at the audited commit, these concern accuracy or the completion of a step: first-order velocity next to inclined embedded no-slip walls (issue 110) and at level transitions with variable viscosity (issue 112), both pinned by tests at the observed orders; first-order shear across an aligned material viscosity jump with the geometric-mean vertex viscosity (issue 113), not tested; a CMDGFM projection with the ILU preconditioner that stops unconverged with iterations left (issue 126); conjugate gradients with non-symmetric multigrid cycles (issue 128). The open bugs issue 117 and issue 118 (refinement criteria) and issue 127 (an allocation contract) do not affect accuracy. Since the audited commit, PR #125 has fixed issue 117 and issue 118 (Sections 12.2.1 and 12.2.2), PR #130 issue 126 and issue 127, issue 121, the one-phase `cfl` range, is fixed by the step check of (13.3), which now uses the top of the accepted range (Section 13.1); issue 49, a level-set initialisation that could fail for valid input, is fixed by the certified acceptance of Section 4.9; and issue 36 is fixed: a periodic direction with two cells is resolved by the face orientation, whose normal component fixes the sign of the seam displacement, while the multigrid coarsest level still keeps four cells per periodic axis (Section 11.3.1); issue 70 is fixed by the outflow limiter of deviation 6.4, issue 71 by the open-strip phase schedule of Section 6.8, and issue 72 no longer reproduces: the cut-donor bound keeps the dual Courant number at or below its limit next to a solid (Section 6.6), and issue 99 is fixed: an orphaned transverse edge next to a level transition takes the prescribed wall velocity at a `NoSlip`, `FreeSlip` or `Inflow` side and a linear-exact level-matched difference at an `Outflow` or `ZeroGradientVelocity` side instead of the offset pair, which carried the tangential derivative of the offset into the normal gradient (Section 9.2). The others are open.

Order in time. No test asserts an order in time (Section 13.5). The temporal series at a fixed mesh assert the temporal self-convergence for the one- and two-velocity capillary waves, with the step halved three times, and for the two-velocity Kelvin–Helmholtz waves, halved twice. The error against the exact solution is printed, not asserted: the one-velocity capillary wave at density ratio one gave 0.0126, 0.0046, 0.0030, 0.0042 and 0.0050, which grows again below a quarter step while the self-convergence stays non-increasing (issue 178). At density ratio one the test also requires the two-velocity capillary wave, whose step is then converged, to reproduce the one-velocity wave; it does not (0.0133 against 0.0042 at the step $\frac{1}{8}$), and that check is a known failure of the CMDGFM inference term of a cut face (issue 171, issue 178).

References. The examples’ numbers are not asserted and can lag the code (Section 14.8). The cylinder regression pins a transient, the developed-wake comparison is manual, and the gallery’s cylinder flow is at another Reynolds number than the test and the manual study. No test compares the sloped sessile droplet, the decaying turbulence, the Stokes wave or the LGPI case with a reference, and none asserts a contact angle or the size of the parasitic

currents of a sessile drop. The Ghia comparison runs on one mesh.

Tiers. The integration groups run weekly and the heavy groups monthly, the latter without failing the run, so that a regression only they detect reaches `main` first; the oracle, the cylinder study and the MimeticFV comparison run by hand (Section 14.2).

— A —

Deviations from the thesis

This list is collected automatically from the deviation boxes in the chapters; each entry links back to the corresponding implementation difference.

1.1 Dimension and compressibility . . . 2
3.1 Staggered volume at a level transition . . . 24
3.2 Pressure gradient at level transitions . . . 28
4.1 Fraction floor . . . 32
4.2 Indicator at equal fractions . . . 32
4.3 Reconstruction family . . . 37
4.4 Height-function quality . . . 38
4.5 GHF centroid fallback . . . 39
4.6 Signs of the height-function formulas . . . 41
4.7 Acute and obtuse contact curvature . . . 42
5.1 Directional splitting . . . 49
5.2 First moments . . . 49
5.3 Donating regions in cut cells and at level transitions . . . 52
5.4 Time-step bound . . . 55
5.5 Phase-scalar face states . . . 56
5.6 Frozen interface fluxes . . . 59
6.1 Primal–dual fluxes on composite meshes and in cut cells . . . 67
6.2 Directional face states . . . 69
6.3 Modified flux interpolant in directional sweeps . . . 78
6.4 Outflow limiter at hanging segments . . . 78
6.5 Two-velocity momentum . . . 81
7.1 Open apertures . . . 89
7.2 ComFLOW boundary features . . . 91
8.1 Face curvature . . . 97
8.2 Surface tension as a predictor force . . . 99
8.3 Surface-tension models . . . 99
8.4 Gravity potential point in cut cells . . . 101
8.5 Two-velocity forces . . . 103
9.1 Single-phase viscous operator . . . 112
9.2 Two-phase stress rows . . . 117
9.3 Single-phase diffusion time level . . . 120
10.1 Sign of the interface pressure work . . . 122

10.2 CMDGFM jump weight . . . 129
10.3 CMDGFM inference faces and unresolved rows . . . 129
10.4 Boundary reset after the projection (fixed) . . . 132
10.5 Incremental projection and reference step . . . 134
10.6 Projection after a regrid . . . 136
11.1 Whole-system ILU for the CMDGFM (fixed) . . . 155
12.1 Interface refinement . . . 159
12.2 Curvature refinement . . . 160
12.3 Vorticity refinement . . . 161
12.4 Instability refinement . . . 162
13.1 Gravity time-step factor . . . 175
13.2 Open strip of a cut coarse donor . . . 179
13.3 Boundedness of cut donors . . . 179
13.4 Frozen subface flux in cut donors . . . 180

— B —

Notation

The notation is that of the thesis [19], extended for the block forest, owned face segments and halo slots. Every symbol is a macro of `techref-notation.sty`, and this table is generated from those definitions, so it lists exactly the notation of this reference. The column *Thesis* gives the section or equation of the thesis that introduces a symbol (*nomencl.*: the thesis' nomenclature; *new*: introduced here). Groups marked with chapter numbers collect the notation introduced there. Bound index letters (c, f, k, t, m, n, …) are not listed. A square $\square$ stands for any quantity.

Symbol	Meaning	Code	Thesis
Domain, phases and fields			
Ω	Rectangular domain $\prod_k (o_k, o_k + X_k)$; periodic directions identified (Section 2.1)	`origin`, `extent`, `periodic`	§2.1
o_k	Lower domain coordinate in direction k	`origin`	new
X_k	Domain length in direction k	`extent`	new
Γ	Physical (non-periodic) part of $\partial\Omega$	`periodic`	(2.23)
$\boldsymbol{n}$	Outward unit normal of a surface		§2.2.1
$\boldsymbol{x}$	Position in Ω		§2.2.1
x_k	k-th coordinate of $\boldsymbol{x}$		§2.2.1
$\boldsymbol{u}$	Continuous velocity		§2.1
u_f	Face-normal (staggered) velocity on face f, (3.9)	`FaceVelocity`	(2.17)
p_c	Pressure; cell average (3.8)	`pressure`	(2.16)
φ_c	Generic cell-centred quantity	`CellField`	§6.4
π	Phase label, $\pi \in \{\mathrm{l}, \mathrm{g}\}$	`:liquid`, `:gas`	§2.1
$\square^{\mathrm{l}}$	Liquid phase		§2.1
$\square^{\mathrm{g}}$	Gas phase		§2.1
$\square^{\mathrm{o}}$	Part open for flow, $\square^{\mathrm{o}} = \square \setminus \square^{\mathrm{s}}$		§2.4
$\square^{\mathrm{s}}$	Fixed solid part		§2.4
Forest, blocks and levels			
$\mathcal{T}$	Forest of quadtrees of blocks, one tree per root (Section 2.2)	`mesh.root`	new
$\mathcal{L}$	Active (leaf) blocks of $\mathcal{T}$	`mesh.blocks`	new
$\mathcal{B}$	Block of $N_1 \times N_2$ cells (a tree node)	`LeafBlock`	§2.5

Symbol	Meaning	Code	Thesis
ℓ	Refinement level index	`BlockKey.level`	new
$\ell(\mathcal{B})$, $\ell(c)$	Level of a block, or of the block owning a cell	`block.key.level`	new
$\mathrm{key}(\mathcal{B})$	Dyadic address (ℓ, i, j), (2.3)	`BlockKey`	new
L	Refinement ceiling (maximum level)	`max_level`	§2.5
L_{act}	Finest level present in $\mathcal{L}$		new
M_k	Level-zero cells in direction k	`base_mesh`	new
N_k	Cells per block in direction k (even, ≥ 2)	`block_size`	new
R_k	Roots in direction k, M_k/N_k	`RootGrid.layout`	new
h_0	Base mesh width (isotropic case)		§2.5
$h_{0,k}$	Level-zero cell width in direction k, X_k/M_k		new
$h_{\ell,k}$	Level-ℓ cell width $2^{-\ell}h_{0,k}$	`LeafBlock.spacing`	new
h	Minimum mesh width $2^{-L}h_0$ (isotropic)		§2.5
H	Halo width in cells	`halo`	new
CELLS, FACES AND INCIDENCE			
$\mathcal{C}$	Owned leaf cells ((2.4))	`CellRef`	§2.2.1
$\mathcal{C}_\ell$	Cells owned by level-ℓ blocks		new
$\mathcal{C}(f)$	Cells adjacent to face f	`minus`, `plus`	§2.2.1
$\mathcal{F}$	Owned faces, $\mathcal{F}_\Omega \cup \mathcal{F}_\Gamma$ (Section 2.3)		§2.2.1
$\mathcal{F}_\Omega$	Interior faces, including periodic seams	`mesh.faces`	new
$\mathcal{F}_\Gamma$	Physical boundary faces	`mesh.boundary_faces`	(2.23)
$\mathcal{F}^{[k]}$	Faces with normal $\pm\boldsymbol{e}_k$	`FaceField`	§2.2.3
$\mathcal{F}(c)$	Owned faces of cell c		§2.2.1
$\bar{f}$	Geometric side of a coarse cell at a level transition		new
$\mathcal{H}(\bar{f})$	Owned segments partitioning $\bar{f}$ ((2.7))		new
$c^-(f)$	Cell on the low side of an interior face	`face.minus`	new
$c^+(f)$	Cell on the high side of an interior face	`face.plus`	new
$c(f)$	Cell of a boundary face	`face.cell`	new
o	Orientation symbol		§2.2.1
$o_{c,f}$	± 1 such that $o_{c,f}\boldsymbol{n}_f$ points out of c ((2.8))	`face_signs`	§2.2.1
s_f	Outward side (± 1) of a boundary face	`face.side`	new
$k(f)$	Coordinate axis normal to face f	`face.axis`	new
$\boldsymbol{n}_f$	Face normal $\boldsymbol{e}_{k(f)}$		§2.2.1
$\boldsymbol{e}_k$	k-th standard basis vector		§2.2.3
$\boldsymbol{x}_c$, $\boldsymbol{x}_f$, $\boldsymbol{x}_v$	Centroid of a cell or face; position of a vertex	`cellcenter`, `face.center`	§2.2.1
$\boldsymbol{d}_f$	Centroid displacement $\boldsymbol{x}_{c^+(f)} - \boldsymbol{x}_{c^-(f)}$, shortest periodic image	`_face_displacement`	new
$\mathcal{V}$	Vertices: endpoints of interior faces	`_vertex_key`	§2.2.1
$\mathcal{F}(v)$	Vertex patch: interior faces with endpoint v	`_vertex_patches`	new

Symbol	Meaning	Code	Thesis
Halo (ghost) slots			
$\mathcal{C}^{\mathrm{h}}(\mathcal{B})$	Halo slots of block $\mathcal{B}$ ((2.9))	`mesh.halo_entries`	new
$\hat{c}$	A halo slot	`HaloEntry.destination`	new
$\mathcal{D}(\hat{c})$	Donor cells of a halo slot	`mesh.halo_donors`	new
$w_{\hat{c},b}$	Weight of donor b, (2.10)	`mesh.halo_weights`	new
$\boldsymbol{y}_{\hat{c},r}$	Quarter-cell sample points, $r = 1, \dots, 4$	`_build_halo_schedule`	new
$\Pi_{\bar{\Omega}}(\boldsymbol{x})$	Closest point of the closed domain (after periodic wrapping)	`_wrap_or_outside`	new
Measures and open (cut-cell) geometry			
$\lvert c\rvert,\ \lvert f\rvert$	Area (2-D volume) or length of a set	`volume`, `area`	§2.2.1
$h_{c,k}$	Width of cell c in direction k, $h_{\ell(c),k}$	`LeafBlock.spacing`	new
ω_f	Staggered control volume of face f ((3.3))		§2.2.1
$\lvert\omega_f\rvert$	Measure of ω_f	`mesh.face_dual_volumes`	(2.15)
$\omega(\mathcal{F})$	Set of staggered control volumes		(2.21)
h_f	Face-projected centroid distance $\lvert\omega_f\rvert/\lvert f\rvert$		§2.2.1
α	Volume fraction		§2.3
$c^{\mathrm{o}},\ f^{\mathrm{o}}$	Part of a cell or face open for flow		§2.4
α_c^{o}	Open volume fraction $\lvert c^{\mathrm{o}}\rvert/\lvert c\rvert$	`open_fraction`	§2.4
α_c^{s}	Solid volume fraction	`solid.fraction`	§2.4
a_f^{o}	Shared open face aperture ((3.6))	`geometry.aperture`	§2.4
$a_{c,f}^{\mathrm{o}}$	Open fraction of f seen by the solid reconstruction in c	`_cell_face_open_fraction`	new
$\Lambda_{c,f}$	Strip of c over the tangential extent of f		new
$\lvert\omega_f^{\mathrm{o}}\rvert$	Open staggered volume ((3.7))	`geometry.face_dual_volumes`	new
Function spaces and discrete integration			
$\vec{\mathcal{C}}$	Cell functions $\mathcal{C} \to \mathbb{R}$	`CellField`	(2.16)
$\vec{\mathcal{F}}$	Face functions $\mathcal{F} \to \mathbb{R}$	`FaceField`, `FaceVelocity`	(2.17)
$\sum \varphi$	$\sum_c \lvert c\rvert\varphi_c$ ((3.10))	`integrate`	(2.20)
$\sum_{\omega(\mathcal{F})} u\boldsymbol{n}$	$\sum_f \lvert\omega_f\rvert u_f \boldsymbol{n}_f$		(2.21)
$\langle u, w\rangle_{\mathsf{K}^{-1}}$	Face inner product $u^{\mathsf{T}}\mathsf{K}^{-1}w$ ((3.18))		new
Discrete operators and matrices			
$\mathfrak{D}$	Primary divergence $\vec{\mathcal{F}} \to \vec{\mathcal{C}}$ ((3.11))	`divergence!`	(2.22)
$\mathfrak{D}^{\mathrm{o}}$	Divergence over open cell parts ((3.13))	`divergence!`	new
$\mathfrak{S}$	Derived gradient $\vec{\mathcal{C}} \to \vec{\mathcal{F}}$ ((3.17))	`gradient!`	(2.25)
$\mathfrak{S}_\gamma$	Coefficient-weighted gradient ((3.24))	`weighted_gradient!`	new
$\mathfrak{S}_\gamma^{\mathrm{o}}$	Cut-cell weighted gradient ((3.27))	`_cut_cell_weighted_gradient!`	new
$\mathfrak{L}$	Cell elliptic operator ((3.25))	`EllipticOperator`	new
$\mathfrak{I}$	Interpolation $\vec{\mathcal{C}} \to \vec{\mathcal{F}}$		(2.28)

Symbol	Meaning	Code	Thesis
$\mathfrak{J}^{\mathrm{h}}$	Distance-weighted harmonic mean ((3.23))	`harmonic_face_coefficients`	new
J	Integrated pressure jump $\lvert f\rvert(p_{c^+(f)} - p_{c^-(f)})$	`_pressure_jump!`	new
K	Inverse face metric (Riesz map) ((3.19))	`inverse_metric`	new
K_v	Vertex-patch contribution to K	`_local_inverse_metric`	new
Y_v	Patch moments, rows $\lvert f\rvert \boldsymbol{d}_f^{\mathsf{T}}$	`moments`	new
N_v	Patch normals, rows $\frac{1}{2}\boldsymbol{e}_{k(f)}^{\mathsf{T}}$	`normals`	new
P_v	Projector onto $\mathrm{range}(\mathsf{Y}_v)^{\perp}$	`projector`	new
τ_v	Patch stabilisation weight	`stabilization`	new
V	Diagonal matrix of cell (open) volumes	`volumes`	new
S	Integrated elliptic stiffness ((3.26))	`stiffness`	new
G	Diagonal matrix of face coefficients γ_f		new
W	Diagonal cut-cell metric scaling $(\lvert\omega_f\rvert/\lvert\omega_f^{\mathrm{o}}\rvert)^{1/2}$	`scale`	new
I	Identity matrix		nomencl.
$\mathbf{1}$	Constant cell function one		new
γ_f	Non-negative face coefficient	`face_coefficient`	new
γ_f^{m}	Material part of the cut-cell coefficient, $\gamma_f = \gamma_f^{\mathrm{m}}(a_f^{\mathrm{o}})^2$	`face_coefficient`	new
r	Reaction coefficient ≥ 0	`reaction`	new
ϑ	Diffusion coefficient >0	`diffusion`	new
TIME LEVELS AND SPLITTING			
δt	Time-step size	`dt`	§2.1
t	Time	`time`	§2.1
$u^{(n)}$, $\alpha^{\mathrm{l},(n)}$	Superscript for time level $t^{(n)}$		nomencl.
$\alpha^{[k]}$	Superscript after the k-th directional sweep		§3.2.3
u^{*}, u^{**}, u^{***}	Superscripts after the first, second and third split stage		§7.2
t_{end}	Final time of a `simulate!` call	`final_time`	new
ν^{f}	CFL parameter of the flow bound, ν^{s} in one phase and $\nu^{\dagger}$ in two phases ((13.1))	`cfl`	new
$\nu^{\max}$	Top of the accepted range of ν^{f}, 1 in one phase and $\frac{1}{2}$ in two phases; the step check (13.3) uses it	`cfl`	new
δt_{adv}	Advective bound of a flow ((13.1))	`stable_timestep`	new
$\delta t_{\mathrm{flow}}(\nu^{\mathrm{f}})$	Flow bound, the smallest of the advective, capillary and gravity bounds ((13.1))	`stable_timestep`	new
δt_{μ}	Explicit viscous estimate, not used by the flow solvers (Table 13.1)	`stable_timestep`	new
c_{σ}	Phase speed of the capillary wave of wavelength $2h$ (Section 13.1)		new
U	Speed of a uniform translation along a flat interface (Section 13.1)		new

Symbol	Meaning	Code	Thesis
$q,\ q^*$	Multiple of the selected step in a margin run; largest stable multiple (Section 13.3)		new
Interface, reconstruction and curvature (Chapter 4)			
$I,\ I_c$	Interface; its PLIC segment in cell c	`plic_segments`	§2.3
q	Initialisation level set, liquid where $q \le 0$ (not evolved)	`levelset`	§2.3.1
Ω_0^{l}	Prescribed initial liquid region	`PolygonIndicator`	new
α_c^{π}	Phase volume per full cell volume ((4.4))	`interface.fraction`	(2.35)
C_c^{π}	Phase volume per open cell volume, $\alpha_c^{\pi}/\alpha_c^{\mathrm{o}}$	`conditional_fraction`	new
ε^{α}	Fraction floor: snapping tolerance of C_c^{l}	`fraction_floor`	§3.3
χ_c^{l}	Discrete phase indicator, $\chi_c^{\mathrm{l}} = 1$ iff $C_c^{\mathrm{l}} > \frac{1}{2}$	`indicator`	(2.37)
$\boldsymbol{\eta},\ \boldsymbol{\eta}_c$	Unit interface normal, pointing into the gas		§2.3
$\boldsymbol{\tau}$	Unit tangent $(-(\boldsymbol{\eta})_2, (\boldsymbol{\eta})_1)$ of the interface		§4.7.1
H_c	diag $(h_{c,1}, h_{c,2})$	`LeafBlock.spacing`	new
$\hat{\boldsymbol{x}}$	Local coordinate $\mathsf{H}_c^{-1}(\boldsymbol{x} - \boldsymbol{x}_c)$ of cell c		new
$\hat{Q}$	Reference cell $[-\frac{1}{2}, \frac{1}{2}]^2$ of the local coordinates	`_unit_cell_polygon`	new
$\hat{Q}_c^{\mathrm{o}},\ \hat{Q}_c^{\mathrm{l}}$	Local image of the open or phase part of cell c	`_open_cell_polygon`, `_clip_polygon`	(3.58)
$\hat{f}_c$	Local image of face f in the coordinates of cell c	`_face_local_tangent_interval`	new
$\hat{\boldsymbol{\eta}}_c$	PLIC normal in local coordinates	`normal_x`, `normal_y`	new
$\hat{\phi}_c$	PLIC plane constant in local coordinates	`intercept`	new
ϕ_c	Signed distance of the PLIC line from $\boldsymbol{x}_c$ (thesis plane constant)		§3.3
$\mathcal{A}(\hat{\boldsymbol{\eta}}_c, \hat{\phi}_c; \square)$	Area fraction of a rectangle below a line ((4.8))	`_plane_fraction`	new
$\hat{r}_k^-,\ \hat{r}_k^+$	Bounds of a rectangle $\prod_k [\hat{r}_k^-, \hat{r}_k^+]$ in local coordinates	`xlo`, `xhi`, `ylo`, `yhi`	new
$\hat{r}_k^+$	Upper rectangle bound		new
W_k	Projected rectangle width $\lvert(\hat{\boldsymbol{\eta}})_k\rvert\,(\hat{r}_k^+ - \hat{r}_k^-)$	`ax`, `ay`	new
z	Plane constant measured from the lowest corner of the rectangle	`t`	new
$F(z)$	Normalised area below the line, measured from the lowest corner		new
$\mathrm{S}(z)$	Smoothstep $z^2(3 - 2z)$ of z clamped to $[0, 1]$	`_height_quality`	new
ι	Inward sign (± 1) of a domain wall	`inward`	new
$\boldsymbol{x}_{I_c}$	Midpoint of the PLIC segment I_c	`_interface_centroid`	§4.7.1

Symbol	Meaning	Code	Thesis
g_k	Central difference of C^1 in direction k (WY normal)	`_wy_normal`	new
S_c	Sum of three conditional fractions along the principal axis (WY normal)	`_wy_normal`	new
d	Side (± 1) of the one-sided WY difference	`_wy_normal`	new
$H^{\pm k}$	Local height function, principal direction $\pm \boldsymbol{e}_k$ (not the halo width)		§2.3.1
$\bar{H}_m$	Discrete column height of column m, in cells	`_local_height`	(2.40)
H_m	Column height in length units (contact formulas)	`H0`, `H1`	§4.7.2
λ_c	Liquid fraction of a column line in cell c	`_line_liquid_fraction`	new
Q_i	Quality of a height-function normal	`_height_quality`	(3.62)
ψ_i	Angle between a height-function normal and its principal direction		§3.3.1
ψ^*	Quality transition half-width, $\pi/20$	`_height_quality`	§3.3.1
w_j	Weight of a height-function candidate or of a GHF point		(3.61)
$\boldsymbol{p}_j$	Interface position used by the GHF fit	`points`	§4.7.1
x_τ	Tangential coordinate of the GHF frame	`tangent`	§4.7.1
x_η	Normal coordinate of the GHF frame		§4.7.1
A_k	Coefficients of a fitted parabola $A_2x^2 + A_1x + A_0$		(4.93)
κ, κ_c	Interface curvature $\nabla \cdot \boldsymbol{\eta}$ (convex liquid: positive)	`curvature`	(2.6)
θ	Static contact angle, measured in the liquid	`contact_angles`, `embedded_contact_angle`	(2.8)
$\boldsymbol{n}^\Gamma$	Unit wall normal pointing into the fluid		(2.8)
$\boldsymbol{x}_*$	Triple (contact) point		§4.7.2
a_f^π	Phase face aperture ((4.22))	`phase_liquid_aperture`	(2.41)
$a_{c,f}^{\mathrm{l}}$	Fraction of face f in the phase part of cell c (one-sided)	`_face_aperture`	(2.41)
ε^q	Absolute tolerance of a level-set cell fraction	`atol`	new
Λ_q	Optional global Lipschitz bound of q	`lipschitz`	new
$\mathcal{Q}$	Rectangle of the recursive level-set quadrature	`lo`, `hi`	new
VOF AND PHASE-SCALAR TRANSPORT (CHAPTER 5)			
c^π, f^π	Part of a cell or face in phase π; for faces also the frozen phase subface ((5.24))	`_frozen_subface_interval`	(6.14)
v_f^π, v_f°	Volume flux per unit face length; $\delta t \lvert f \rvert v_f$ is the signed volume through f in one sweep ((5.5) and (5.13))	`open_flux`, `liquid_flux`, `gas_flux` (time-integrated)	(3.45)
Δ_f	Donating region of face f ((5.11))	`_swept_fraction`	(3.45)

Symbol	Meaning	Code	Thesis
$c^{\mathrm{u}}(f)$	Donor (upwind) cell of f for the carrier velocity	`donor`	new
$\hat{R}_f(\hat{d})$	Donating rectangle of f in the local coordinates of $c^{\mathrm{u}}(f)$ ((5.9))	`swept_bounds`	new
$\hat{d}_f$	Depth of $\hat{R}_f$ in units of the donor width ((5.10))	`distance`	new
C_f^{Δ}	Liquid fraction of the volume swept through f ((5.12))	`_swept_fraction`	new
$k_1,\ k_2$	Axis of the first and second sweep of a time step	`sweep_order`	§3.2.3
$\check{C}_c^{\mathrm{l},[j]}$	Conditional fraction after a sweep, before clamping and snapping ((5.14))	`c` in `_advect_wy!`	new
V^{π}	Total volume $\sum_c \lvert c^{\mathrm{o}}\rvert C_c^{\pi}$ of a phase ((5.27))	`integrate`	new
$\delta C_c^{[j]}$	Change of C_c^{l} by clamping and snapping after sweep j ((5.14))		new
$\widetilde{C}_f^{\pi}$	Staggered conditional fraction, open-volume weighted ((5.23))	`_phase_face_average!`	(2.36)
ϵ_{fl}	Round-off floor of the solver's residual target, at least $128\,\epsilon_{\mathrm{mach}}$ ((11.8))	`_roundoff_residual_floor!`	new
$(\mathfrak{F}\varphi^{\pi})_f$	Face state (flux interpolant) of a cell quantity ((5.21))	`_phase_flux_state`	(6.18)
Θ_f	Smoothness ratio of a limited face state ((5.22))	`smoothness`	§6.4.2
$\Psi^{\mathrm{MC}}(\Theta_f)$	Flux limiter function (Table 5.1)	`_flux_limiter`	§6.4.2
$\nu^{\dagger}$	CFL limit, $0 < \nu^{\dagger} \le \frac{1}{2}$	`cfl`	(3.50)
ν_f	Courant number of a face ((5.21))	`courant`	(3.51)
ν_c^{in}	Incoming local CFL number ((5.15))	`incoming`	(3.51)
ν_c^{out}	Outgoing local CFL number ((5.15))	`outgoing`	(3.55)
ν_f^{ext}	Extrusion number $\delta t\,\lvert u_f\rvert / h_{c^{\mathrm{u}}(f),k(f)}$ ((5.15))	`maxrate`	new
δt_{WY}	Largest step allowed by (5.15)	`stable_timestep`	new
δt_{σ}	Capillary time-step bound ((5.18))	`_two_phase_timestep_bound`	(4.1)
δt_{grav}	Gravity time-step bound ((5.18))	`_two_phase_timestep_bound`	(7.4)
Staggered transport (Chapter 6)			
$\widetilde{\varphi}_f$	Transported staggered quantity: a velocity component, a phase velocity or a staggered phase scalar	`FaceField`, `PhaseFaceScalar`	(6.39)
$\widetilde{V}_f$	Transport volume of the staggered control volume of f ((6.3))	`schedule.volumes`	(2.15)
$\varpi_{c,f}$	Fraction $\lvert f\rvert / h_{c,t}$ of a side of c covered by its face f ((6.5))	`ComponentFaceIncidence.fraction`	new
$\mathcal{F}_s(c)$	Faces of the transported component on side $s = \pm 1$ of c, $o_{c,f} = s$ ((6.5))	`ComponentFaceIncidence.side`	new

Symbol	Meaning	Code	Thesis
$\mathcal{G}$	Dual faces: oriented connections of two staggered control volumes ((6.6))	`schedule.faces`	(6.5)
$\mathcal{G}(f)$	Dual faces of the staggered control volume of f	`dual_offsets`	(6.5)
$f^-(g)$	Tail of a dual face ($\widetilde{o}_{f,g} = +1$ for $f = f^-(g)$)	`face.minus`	(6.5)
$f^+(g)$	Head of a dual face ($\widetilde{o}_{f,g} = -1$ for $f = f^+(g)$)	`face.plus`	(6.5)
$\widetilde{o}_{f,g}$	$+1$ if $f = f^-(g)$, -1 if $f = f^+(g)$	`dual_signs`	(6.5)
$\widetilde{w}_{g,f}$	Weight of the primal flux through f in the dual face g ((6.6))	`flux_weights`	(6.10)
$\widetilde{\mathfrak{H}}$	Primal-to-dual flux interpolation, the weights $\widetilde{w}_{g,f}$	`AMRDualAdvectionSchedule`	(6.10)
$\widetilde{\Upsilon}_g$	Integrated volume flux through a dual face ((6.7))	`_amr_dual_flux`, `_phase_dual_flux`	(6.42)
$\widetilde{\mathfrak{D}}\widetilde{\Upsilon}$	Staggered divergence ((6.9))	`_directional_flux_divergence!`	(6.5)
$\widetilde{\mathfrak{D}}^\Gamma\widetilde{\Upsilon}$	Closed staggered divergence: $\widetilde{\mathfrak{D}}$ plus the boundary dual faces ((6.18))	`_add_boundary_flux_divergence!`	new
S_c	Covered fraction $\sum \varpi_{c,f}$ of the two sides of c normal to $\boldsymbol{e}_k$ ((6.10))		new
$\varpi^{\mathrm{o}}_{c,f}$	Open-strip fraction $\lvert\Lambda_{c,f} \cap c^{\mathrm{o}}\rvert/\lvert c^{\mathrm{o}}\rvert$ of the one-velocity momentum schedule ((6.24))	`_momentum_advection_workspace`	new
λ	Weight of the tail value in a two-point face state ((6.21))		new
$\mathfrak{H}\varphi$	Volume-weighted cell-to-face interpolation ((6.4))	`_phase_face_average!`	(2.30)
$\mathcal{G}^*$	Non-regular dual faces, which use the AMR-interface scheme ((6.11))	`refinement_interface`	new
$\mathcal{G}_\Gamma(f)$	Boundary dual faces of the staggered control volume of f ((6.17))	`BoundaryDualFace`	new
$(\widetilde{\mathfrak{F}}\widetilde{\varphi})_g$	Face state of a staggered quantity on a dual face ((6.12))	`_dual_state`, `_phase_bulk_state`	(6.40)
$\widetilde{\mathfrak{U}}_1\widetilde{\varphi}$	Upwind interpolants of the thesis' unsplit states (not implemented)		(6.47)
$f_g^{\mathrm{u}},\ f_g^{\mathrm{d}},\ f_g^{\mathrm{uu}}$	Upwind, downwind and second-upwind unknowns of a dual face ((6.15))	`minus_stencil`, `plus_stencil`	(6.48)
$h_g^{\mathrm{d}},\ h_g^{\mathrm{uu}}$	Axial distances of the unknowns of a three-point state ((6.14))	`ratio_scale`	(6.48)
$\widetilde{\mathfrak{A}}[u]\widetilde{\varphi}$	Staggered convection operator ((6.16))	`convection!`	(6.44)
$\mathfrak{A}[m]\varphi$	Cell-centred advection operator of the thesis, $\mathfrak{D}(m\mathfrak{F}\varphi)$		(6.18)
$\langle\widetilde{\varphi},\psi\rangle_{\widetilde{V}}$	Inner product $\sum_f \widetilde{V}_f\widetilde{\varphi}_f\psi_f$ of staggered functions	`staggered_inner_product`	(2.21)
δt_{stag}	Largest step of the staggered convection bound ((6.23))	`stable_timestep`	new

Symbol	Meaning	Code	Thesis
ν^{s}	CFL parameter of the staggered bound, $0 < \nu^{\mathrm{s}} \leq 1$ ((6.23))	`cfl`	new
$\mathcal{D}_c^{[j]}$	Carrier dilation of sweep j per open volume (dimensionless, (6.26))	`work.outgoing`	new
$\widetilde{\rho}_f$	Staggered mixture density ((6.25))	`mixture_density`	(6.53)
$\widetilde{C}_f^{\pi,[j]}$	Staggered phase fraction predicted for the end of sweep j ((6.27))	`predicted_fraction`	new
$\mathcal{M}_c^{[j]}$	Net phase-mass outflow of c in sweep j per open volume ((6.31))		new
P_g	Sum of the positive partial volumes of a dual face ((6.33))		new
N_g	Sum of the magnitudes of the negative partial volumes of a dual face ((6.33))		new
γ_f	Coefficient of the old value in the update of the unknown f ((6.34))		new
$\alpha^{\dagger}$	Threshold of the switch to the CTU state (default $\frac{1}{2}$, (6.32))	`ctu_threshold`	(6.51)
$\widetilde{\varphi}_f^{\mathrm{e},\pi}$	Working extension of an absent intrinsic staggered value ((6.35))	`_extend_phase_face_values!`	new
δt_{cell}	Largest step of the cell-scalar upwind bound ((6.39))	`stable_timestep`	new
Embedded solid (Chapter 7)			
$\hat{\boldsymbol{\eta}}_c^{\mathrm{s}}$	Solid PLIC normal in local coordinates (into the fluid)	`solid.normal_x`, `solid.normal_y`	new
$\hat{\phi}_c^{\mathrm{s}}$	Solid PLIC plane constant in local coordinates	`solid.intercept`	new
Materials, transport and forces (Chapters 5–9)			
ρ^{π}	Mass density	`density`	§2.1
μ^{π}	Dynamic viscosity	`dynamic_viscosity`	§2.1
σ	Surface-tension coefficient	`sigma`	§2.1
$\boldsymbol{g}$	Gravitational acceleration	`gravity`	§2.1
m	Mass flux		§6.3
G	Gravitational potential energy density		§7.6
E_k	Kinetic energy	`KineticEnergy`	§4.2
$\widetilde{\boldsymbol{\eta}}_f$	Staggered interface normal		§7.3
Forces (Chapter 8)			
$\mathbf{T}^{\pi}$	Newtonian stress tensor $-p\mathbf{I} + \mu(\nabla\boldsymbol{u} + (\nabla\boldsymbol{u})^{\intercal})$		(2.7)
δ^I	Interface delta function		(4.8)
S_f^{CSF}, S_f^{CGFM}	The thesis' surface-tension force on a face		(4.29), (4.55)
F_f	The thesis' gravity force on a face		(7.64)
$\widetilde{\phi}_f^{\pi}$	The thesis' GFM distance function; conservative choice $\widetilde{\alpha}^{\pi}$		(4.54)

Symbol	Meaning	Code	Thesis
$\hat{C}_f$	Staggered liquid fraction clamped to $[0,1]$ ((8.4))	`_staggered_liquid_fraction`	new
$\bar{h}_c$	Largest width $\max_k h_{c,k}$ of a cell	`maximum(spacing)`	new
$\tilde{w}_c$	Weight of a cell in the curvature extension ((8.11))	`geometric_weight`	new
$\hat{\boldsymbol{m}}_c$	Midpoint of the PLIC segment in the reference cell ((8.15))	`_interface_centroid`	new
γ_f^{tv}	Common pressure coefficient of the two-velocity projections ((8.5))	`_projection_pressure_operator`	new
$\mathsf{R}\mathcal{J}^{\sigma}$	Force map: the pressure's weighted Riesz map applied to integrated face jumps ((8.6))	`_weighted_jump_force!`	new
$\mathsf{R}^{\mathrm{tv}}\mathcal{J}$	Force map ((8.6)) with the two-velocity coefficient γ^{tv} in place of γ (Section 8.8)	`_projection_pressure_operator`	new
A	Diagonal matrix $\mathrm{diag}\left(a_f^{\mathrm{o}}\right)$ of the open face apertures	`geometry.aperture`	new
$\mathcal{J}_f^{\sigma}$, $\mathcal{J}_f^{\boldsymbol{g}}$	Integrated face jump of the tension and gravity forces ((8.13) and (8.17))	`pressure_jump`	new
$\widetilde{\kappa}_f$	Face curvature of the tension force ((8.10))	`_face_curvature`	§4.7.1
w_c^{κ}	Curvature weight $C_c^{\mathrm{l}}(1-C_c^{\mathrm{l}})$ of a cell	`_face_curvature`	§4.7.1
r_f	Radius of the curvature extension around face f ((8.11))	`radius`	new
$\mathcal{F}_{\emptyset}^{\kappa}$	Faces with a tension jump but no face curvature ((8.12))	`unresolved_curvature_faces`	new
χ_c^{σ}	Phase function of the tension jump: C_c^{l} (CSF) or χ_c^{l} (CGFM)	`_surface_tension_jump`	(4.29), (4.55)
Φ_c	Gravity potential $\boldsymbol{g}\cdot\boldsymbol{x}_c^I$ of a cell ((8.15))	`potential`	(7.64)
$\boldsymbol{x}_c^I$	Potential point: PLIC segment midpoint (mixed cell) or cell centre	`_interface_centroid`	(7.61)
ρ_f^a	Aperture density of the gravity force ((8.16))	`gravity!`	(7.64)
$\boldsymbol{a}^{\mathrm{b}}(\boldsymbol{x},t)$	Prescribed body acceleration ((8.19))	`force`	new
Viscous stress (Chapter 9)			
$\mathcal{K}_k$	Connections of the single-phase operator of component k ((9.3))	`DualDiffusionConnection`	new
d_g	Distance of the two unknowns of a connection along its axis	`distance`	new
$(\partial_g^{\mathrm{h}}\widetilde{\varphi})$	Derivative across a connection, two-point or widened ((9.4))	`_gradient_stencil`	new
$\mathcal{W}_k$	Wall connections of component k ((9.8))	`wall_connections`	new
$\varkappa_w$	Conductance (measure over distance) of a wall connection	`conductance`	new
S_k^{ν}	Integrated stiffness of component k ((9.5))	`component.stiffness`	new
$\widetilde{\mathfrak{L}}_k$	Staggered Laplacian of component k ((9.6))	`component.laplacian`	new

Symbol	Meaning	Code	Thesis
$\widetilde{\mathsf{V}}$	Diagonal matrix of the staggered (open) volumes	`volumes`	new
E	Strain-rate rows of the coupled stress operator ((9.12))	`strain`	(7.52)
W_μ	Diagonal stress weights, measure times viscosity ((9.14))	`stress_weights`	new
$\mathcal{V}^{\mathrm{s}}$	Shear-stress vertices: corners of leaf cells	`_stress_vertices!`	§7.5.1
$\lvert\omega_v^{\mathrm{v}}\rvert$	Quarter-cell (open) volume of a stress vertex ((9.14))	`measures`	new
$(\mathfrak{d}_t u)_v$	Shear derivative stencil at a vertex ((9.13))	`_vertex_derivative`	new
M_ρ	Staggered inertia diag $\left(\widetilde{\rho}_f \widetilde{V}_f\right)$ of the viscous solve	`inertia`	(T3)
$\mathbf{D}(\boldsymbol{u})$	Rate of strain $\frac{1}{2}(\nabla \boldsymbol{u} + (\nabla \boldsymbol{u})^{\mathsf{T}})$		(7.52)
$\mathfrak{c}_{g,f}$	Coefficient of u_f in the derivative across connection g ((9.4))	`_gradient_stencil`	new
$\mathfrak{s}_j$	Coefficient of a least-squares shear derivative ((9.13))	`_vertex_derivative`	new
$f_v^{\pm}$	Nearest unknowns on both sides of a vertex along a line ((9.13))	`low_index`, `high_index`	new
$\mathbf{1}^{[k]}$	Indicator of the faces of axis k (constant velocity component)		new
r	Residual vector of a linear solve	`residual`	new
b	Right-hand side of a linear solve	`right_hand_side`	new
ϵ_{tol}	Relative tolerance of a viscous solve	`tolerance`	new
AMR transfer and multigrid (Chapters 11–12)			
$\mathcal{R}^{\mathrm{AMR}}$	Regrid restriction (one map per field type)	`restrict!`	new
$\mathcal{P}^{\mathrm{AMR}}$	Regrid prolongation (one map per field type)	`prolong!`	new
REFINE	Block label; precedence COARSEN $\prec$ KEEP $\prec$ REFINE ((12.1))	`:refine`	§2.5
KEEP	Block label	`:keep`	§2.5
COARSEN	Block label; the neutral element of the combination	`:coarsen`	§2.5
$\lambda(\mathcal{B})$	Combined label of a leaf block ((12.1))	`workspace.labels`	new
$\lambda_i(\mathcal{B})$	Label of a block by indicator i	`refinement_labels!`	new
$E_{\mathcal{B}}$	Block value of an error-based indicator, the maximum of e_c over its cells	`maximum_error`, `error`	new
e_c	Cell estimate of an error-based indicator ((12.4), (12.6) and (12.7))		new
$\varepsilon_{\mathrm{r}},\ \varepsilon_{\mathrm{c}}$	Refinement and coarsening thresholds, $0 \le \varepsilon_{\mathrm{c}} < \varepsilon_{\mathrm{r}}$	`E_refine`, `refine_tolerance`	§2.5
ε_{c}	Coarsening threshold	`E_coarsen`, `coarsen_tolerance`	§2.5

Symbol	Meaning	Code	Thesis
$L_{\min},\ L_{\max}$	Level bounds of an error-based indicator, $0 \le L_{\min} \le L_{\max} \le L$	`minimum_level`, `maximum_level`	new
$L_{\max}$	Upper level bound of an indicator	`maximum_level`	new
L_*	Target level of a band indicator	`level`	§2.5
n_{b}	Buffer: rings of halo slots added to a block, $0 \le n_{\mathrm{b}} \le H$	`buffer_cells`	new
$\mathcal{B}^+$	Cells of $\mathcal{B}$ with n_{b} rings of its halo slots ((12.3))		new
N_κ	Cells per radius of the curvature criterion	`cells_per_radius`	new
q_κ	Coarsening ratio of the curvature criterion, $0 < 2q_\kappa < 1$	`coarsen_ratio`	new
ω_c	Vorticity estimate of a cell ((12.6))	`_vorticity_values!`	§2.5
$\hat{\boldsymbol{u}}_c$	Cell-centred velocity of the vorticity estimate ((12.5))	`_cell_centered_velocity!`	new
G_c	Least-squares velocity gradient of a cell ((12.6))	`a11`, …	new
ε_{f}	Floor of the relative interpolation defect	`relative_floor`	new
$t_{\mathrm{init}},\ t_{\mathrm{end}}$	Activity window of an indicator	`t_init`, `t_end`	new
t_{end}	End of the activity window	`t_end`	new
$\mathcal{R}$	Closed rectangle selecting an indicator's blocks	`xmin`, `xmax`, `ymin`, `ymax`	new
n_{u}	Minimum number of steps between regrid attempts	`update_interval`	new
n_{d}	Steps for which a new block cannot be merged	`coarsen_delay`	new
L_0	Lowest level of a parent created by coarsening	`minimum_level` (mesh)	new
$\mathrm{MC}(a, b)$	Monotonized-central slope of two differences ((12.10))	`_mc_slope`	new
σ_k	Limited slope of a parent cell in direction k ((12.9))	`slope_x`, `slope_y`	new
$s_{r,k}$	Side (± 1) of child c_r in direction k	`offset_x`, `offset_y`	new
$\varphi^{\mathrm{lin}}_{c_r}$	Limited linear child value before the phase correction ((12.12))	`candidate`	new
w^π_c	Phase volume per cell volume: α^{l}_c, or $\alpha^{\mathrm{o}}_c - \alpha^{\mathrm{l}}_c$ for the gas	`w`, `parent_weight`	new
$\sigma^{\parallel}_g$	Limited tangential slope of an old face segment ((12.13))	`_tangential_slope`	new
$x^{\pm}$	Normal coordinates of the nearest old face lines ((12.14))	`_bracketing_line_segments`	new
$u^{\pm}$	Point values on those lines	`lower_value`, `upper_value`	new
F_c	Net integrated outflow $\lvert c\rvert(\mathfrak{D}u)_c$ of a cell	`old_cell_flux`, `new_cell_flux`	new
ϱ_r	Outflow defect of child c_r ((12.15))	`residuals`	new
δ_{12}	Integrated correction of an internal face of a refined cell ((12.15))	`vertical_lower`, …	new

Symbol	Meaning	Code	Thesis
M_k	Staggered integral $\sum \lvert\omega_f\rvert u_f$ over the interior faces of axis k ((12.17))	`_old_face_momentum`	new
$\square'$	Quantity after a regrid (transferred to the new mesh)		new
j	Multigrid hierarchy index (not a level)		new
$\mathcal{T}_j^{\mathrm{MG}}$	Solver mesh j; $\mathcal{T}_0^{\mathrm{MG}}$ is the composite mesh		new
$\mathsf{P}_j^{\mathrm{MG}}$	Jacobi-smoothed cell transfer from $\mathcal{T}_{j+1}^{\mathrm{MG}}$ to $\mathcal{T}_j^{\mathrm{MG}}$ ((11.14))	`prolongation`	new
L	Matrix of a linear solve in integrated form; the cell operator of (11.1)	`matrix`	new
L_j	Operator rediscretised on $\mathcal{T}_j^{\mathrm{MG}}$; $\mathsf{L}_0 = \mathsf{L}$ ((11.12))	`operators`	new
$\mathcal{C}^{\times}$	Inactive cells: identity rows and columns of L ((11.1))	`constraints`	new
$\mathcal{F}_\Gamma^{\mathrm{o}}$	Boundary faces on `Outflow` and `ZeroGradientVelocity` sides	`boundary_sides`	new
D^{∂}	Diagonal of the open-boundary conductances ((11.2))	`boundary_diagonal`	new
γ_f^{∂}	Coefficient of an open boundary face ((11.3))	`boundary_face_coefficient`	new
$\mathfrak{B}$	Preconditioner, applied as $\mathsf{z} = \mathfrak{B}\mathsf{r}$; one V-cycle of the tree multigrid	`ldiv!`	new
P_j^0	Tentative (piecewise constant) cell transfer ((11.13))	`_tree_cell_prolongation`	new
Q_j	Removal of the volume-weighted means of the unanchored components ((11.18))	`_multigrid_remove_component_means!`	new
Λ_j^{G}	Gershgorin bound of the Jacobi-scaled operator ((11.15))	`spectral_bound`	new
ω_j^{J}	Weight of the damped-Jacobi smoother ((11.17))	`relaxation_weight`	new
ω_j^{P}	Weight of the Jacobi-smoothed transfer ((11.14))	`weight`	new
$\lVert\mathsf{r}\rVert_{\mathsf{V},\infty}$	Strong maximum norm of an integrated residual ((11.6))	`final_linf`	new
$\lVert\mathsf{r}\rVert_{\mathsf{V},2}$	Strong volume-weighted root-mean-square norm ((11.6))	`final_l2`	new
$\varepsilon^{\star}$	Stop level of the strong residual ((11.7))	`target_linf`	new
$\varepsilon^{\mathrm{rel}}$	Relative tolerance of `solve!`	`rtol`	new
$\varepsilon^{\mathrm{abs}}$	Absolute strong tolerance of `solve!`	`atol`	new
m	Number of Krylov iterations	`cycles`	new
$\bar{\varrho}$	Mean reduction of $\lVert\mathsf{r}\rVert_{\mathsf{V},2}$ per iteration ((11.24))		new
Pressure and velocity constraints (Chapter 10)			
ε^{p}	Dimensionless projection tolerance: a fraction of a cell volume per step ((10.7))	`tolerance`	new

Symbol	Meaning	Code	Thesis
$e^V,\ e^\eta$	Defects compared with ε^p ((10.30))	`mixture_volume_defect`, `normal_velocity_defect`	new
$\bar{\mathsf{b}}$	Volume mean removed from a one-velocity right-hand side component	`removed_rhs_component_means`	new
K^{o}	Face metric of the pressure map, WKW (K without solid)	`inverse_metric`	new
M_γ	Kinetic face metric of the pressure map with coefficient γ ((10.8))		new
δt^{prop}	Untruncated proposed step, the defect gauge of the post-regrid projection	`proposed`	new
δt^{ref}	Reference step of an inherited divergence, $\delta t^{\mathrm{ref}} \geq \delta t$ ((10.28))	`_reference_dt`	new
δt^{def}	Step that left the inherited divergence: the previous step, or δt^{prop} after a regrid	`defect_dt`	new
φ_c	Pressure increment of an incremental projection; scratch potential of a projection that keeps the physical pressure ((10.27))	`increment`, `pressure_increment`	new
$\widetilde{m}_f^{\pi},\ \widetilde{m}_f$	Staggered phase mass per volume, $\widetilde{m}_f = \widetilde{m}_f^{\mathrm{l}} + \widetilde{m}_f^{\mathrm{g}} = \widetilde{\rho}_f$ ((10.9))	`liquid_mass`, `gas_mass`, `total_mass`	(4.51)
$\widetilde{m}_f^{\mathrm{r}}$	Reduced mass $\widetilde{m}_f^{\mathrm{l}}\widetilde{m}_f^{\mathrm{g}}/\widetilde{m}_f$	`relative_mass`	new
Ξ^{π}	Phase coefficient multiplying ξ, $\Xi^{\mathrm{l}} = -\widetilde{m}^{\mathrm{g}}$, $\Xi^{\mathrm{g}} = \widetilde{m}^{\mathrm{l}}$	`zeta_liquid`, `zeta_gas`	(4.51)
Ξ_f^{a}	Coefficient of ξ_f in the mixture flux ((10.9))	`coupling`	(7.49c)
ζ	Physical pressure value jump		§4.5
ξ_f	Jump $\mathfrak{g}^{\mathrm{g}}(p,\xi)_f - \mathfrak{g}^{\mathrm{l}}(p,\xi)_f$ of the phase gradients; unknown on $\mathcal{F}^{\xi}$	`jump`	(4.48b)
$\mathfrak{g}$	Scaled gradient operator		§4.5
$\mathfrak{g}^{\pi}(p,\xi)$	Phase pressure gradient ((10.12))	`cmdgfm_gradient!`	(4.50)
$\mathcal{F}^{\mathrm{mix}}$	Mixed faces ((10.10))	`data.mixed`	§7.3.3
$\mathcal{F}^{\xi}$	Mixed faces with a resolved jump unknown	`mixed_faces`	new
$\mathcal{C}^{\mathrm{r}}$	Pressure cells kept after removing one anchor cell per component ((10.16))	`pressure_indices`	new
$\mathfrak{D}^{\mathrm{mix}}(u^{\mathrm{l}},u^{\mathrm{g}})$	Phase-aperture mixture divergence ((10.11))	`_cmdgfm_cell_flux!`	(6.19)
S^{tv}	Stiffness of the two-velocity pressure map (coefficient γ^{tv})	`operator.stiffness`	new
J^{r}	Columns $\mathcal{C}^{\mathrm{r}}$ of J	`incidence_pressure`	new
$w_c^{\mathrm{m}},\ w_f^{\mathrm{m}}$	Mixing weight $C(1-C)$ of a cell or staggered liquid fraction		(7.11)
β_f	Jump-interpolation weight ((10.19)); the thesis' rule is not implemented	`CMDGFMJumpStencil.beta`	(7.16)
$\mathfrak{J}^{\beta}$	Jump interpolant ((10.21))	`cmdgfm_jump_interpolant`	(7.16)

Symbol	Meaning	Code	Thesis
R_f	Signed ratio $\widetilde{\boldsymbol{\eta}}_{f,t}/\widetilde{\boldsymbol{\eta}}_{f,k}$ of the staggered normal, the thesis' $\mathcal{R}_{f,h}$ in 2-D	`ratio`	(7.15)
$\mathcal{F}^{\perp}(f)$	Transverse faces of the jump interpolant ((10.18))	`transverse_faces`	§7.3.3
$w^{\perp}_{f,h}$	Weight of a transverse face	`transverse_weights`	(7.11)
$\mathcal{F}^{\parallel}(f)$	Inference candidates: same-axis faces sharing an end point with f	`_cmdgfm_vertex_candidates`	new
h^{π}	Inference face dominated by phase π	`liquid_face`, `gas_face`	(7.16)
s_f	Sum of the cross-phase mass fractions on the inference faces	`inferred_sum`	new
$\mathcal{N}_f(u^{\mathrm{l}}, u^{\mathrm{g}})$	Discrete normal velocity jump of a CMDGFM row ((10.20))	`_cmdgfm_normal_velocity_defect`	(7.13)
B	CMDGFM jump–jump block	`jump_block`	new
B^{p}	CMDGFM jump–pressure block	`jump_pressure`	new
S^{r}	Gauge-reduced pressure block: rows and columns $\mathcal{C}^{\mathrm{r}}$ of S^{tv}	`reduced_pressure`	new
$\mathcal{K}$	Cells of one connected component of the open pressure graph ((10.16))	`_projection_gauge_partition!`	new
$\breve{\boldsymbol{\eta}}_f$	Unnormalised staggered normal ((10.17))	`_cmdgfm_face_normal`	new
Diagnostics (Chapter 14)			
E_{p}	Gravitational potential energy, zero at the origin ((14.3))	`PotentialEnergy`	new
E_{σ}	Surface energy $\sigma\lvert I\rvert$ ((14.3))	`SurfaceEnergy`	new
P_k	Linear momentum in direction k ((14.1) and (14.2))	`LinearMomentum`	new
$\boldsymbol{M}^{\mathrm{l}},\ \boldsymbol{M}^{\mathrm{o}}$	First moment of the reconstructed liquid region and of the open region ((14.3))	`_phase_first_moments`	new
$\lVert\mathfrak{D}\rVert_2$	Divergence norm ((14.4))	`DivergenceNorm`	new
Decorations and operations			
$\widetilde{\square}$	Staggered equivalent		§6.2
$\bar{u}$	One-velocity (mixture) face velocity		§2.1.2
$\square^{\mathsf{T}}$	Adjoint (transpose) in the stated inner products		(2.24)
$[\![\square]\!]$	Jump $\square^{\mathrm{g}} - \square^{\mathrm{l}}$		§2.1
$\{\!\{\square\}\!\}$	Sum $\square^{\mathrm{g}} + \square^{\mathrm{l}}$ (not an average)		§2.1
$\mathcal{R}_{\rho}$	Ratio $\square^{\mathrm{g}}/\square^{\mathrm{l}}$; also the thesis normal ratio $\mathcal{R}_{f,h}$		§7.3.5
ϵ_{mach}	Unit round-off of the floating-point type	`eps(T)`	new
$[\square]^{+}$	Non-negative part $\max(0, \square)$		nomencl.
$[\square]^{-}$	Non-positive part $\min(0, \square)$		nomencl.
∇	Continuous gradient		§2.1
$\nabla \cdot \boldsymbol{u}$	Continuous divergence		§2.1

— C —

API index

Public names link to the matching version of the Documenter API. The implementation boxes at the ends of sections give the relevant names in context; internal names are given with their source file, linked to the main branch; Table 1.3 lists the public types by the chapter that states them. An explicit stable anchor for each public function is planned for the API reference.

Simulation `FlowSimulation`, `step!`, `simulate!`.

Mesh and geometry `BlockTreeMesh`, `InterfaceField`, `GeometryField`.

Models and projections `TwoPhase`, `OneVelocityModel`, `TwoVelocityModel`, `CMDGFMProjection`.

Operators and solvers `GradientOperator`, `EllipticOperator`, `LinearSolverOptions`, `MultigridOptions`.

Bibliography

[1] S. Afkhami and M. Bussmann. Height functions for applying contact angles to 2D VOF simulations. *International Journal for Numerical Methods in Fluids*, 57:453–472, 2008. https://doi.org/10.1002/fld.1651.

[2] M. J. Berger and P. Colella. Local adaptive mesh refinement for shock hydrodynamics. *Journal of Computational Physics*, 82:64–84, 1989. https://doi.org/10.1016/0021-9991(89)90035-1.

[3] J. Bonelle and A. Ern. Analysis of compatible discrete operator schemes for elliptic problems on polyhedral meshes. *ESAIM: Mathematical Modelling and Numerical Analysis*, 48:553–581, 2014. https://doi.org/10.1051/m2an/2013104.

[4] J. U. Brackbill, D. B. Kothe, and C. Zemach. A continuum method for modeling surface tension. *Journal of Computational Physics*, 100:335–354, 1992. https://doi.org/10.1016/0021-9991(92)90240-Y.

[5] D. L. Brown, R. Cortez, and M. L. Minion. Accurate projection methods for the incompressible navier–stokes equations. *Journal of Computational Physics*, 168:464–499, 2001. https://doi.org/10.1006/jcph.2001.6715.

[6] T. A. Davis. Algorithm 832: UMFPACK V4.3—an unsymmetric-pattern multifrontal method. *ACM Transactions on Mathematical Software*, 30:196–199, 2004. https://doi.org/10.1145/992200.992206.

[7] F. Denner and B. G. M. van Wachem. Numerical time-step restrictions as a result of capillary waves. *Journal of Computational Physics*, 285:24–40, 2015. https://doi.org/10.1016/j.jcp.2015.01.021.

[8] M. Dröge and R. Verstappen. A new symmetry-preserving cartesian-grid method for computing flow past arbitrarily shaped objects. *International Journal for Numerical Methods in Fluids*, 47:979–985, 2005. https://doi.org/10.1002/fld.924.

[9] U. Ghia, K. N. Ghia, and C. T. Shin. High-Re solutions for incompressible flow using the navier–stokes equations and a multigrid method. *Journal of Computational Physics*, 48:387–411, 1982. https://doi.org/10.1016/0021-9991(82)90058-4.

[10] M. R. Hestenes and E. Stiefel. Methods of conjugate gradients for solving linear systems. *Journal of Research of the National Bureau of Standards*, 49:409–436, 1952. https://doi.org/10.6028/jres.049.044.

[11] E. F. Kaasschieter. Preconditioned conjugate gradients for solving singular systems. *Journal of Computational and Applied Mathematics*, 24:265–275, 1988. https://doi.org/10.1016/0377-0427(88)90358-5.

[12] N. Li, Y. Saad, and E. Chow. Crout versions of ILU for general sparse matrices. *SIAM Journal on Scientific Computing*, 25:716–728, 2003. https://doi.org/10.1137/S1064827502405094.

[13] X.-D. Liu, R. P. Fedkiw, and M. Kang. A boundary condition capturing method for Poisson's equation on irregular domains. *Journal of Computational Physics*, 160:151–178, 2000. https://doi.org/10.1006/jcph.2000.6444.

[14] A. Montoison and D. Orban. Krylov.jl: A Julia basket of hand-picked Krylov methods. *Journal of Open Source Software*, 8:5187, 2023. https://doi.org/10.21105/joss.05187.

[15] J. E. Pilliod and E. G. Puckett. Second-order accurate volume-of-fluid algorithms for tracking material interfaces. *Journal of Computational Physics*, 199:465–502, 2004. https://doi.org/10.1016/j.jcp.2003.12.023.

[16] S. Popinet. An accurate adaptive solver for surface-tension-driven interfacial flows. *Journal of Computational Physics*, 228:5838–5866, 2009. https://doi.org/10.1016/j.jcp.2009.04.042.

[17] A. Prosperetti. Motion of two superposed viscous fluids. *Physics of Fluids*, 24:1217–1223, 1981. https://doi.org/10.1063/1.863522.

[18] L. Qu, C. Norberg, L. Davidson, S.-H. Peng, and F. Wang. Quantitative numerical analysis of flow past a circular cylinder at Reynolds number between 50 and 200. *Journal of Fluids and Structures*, 39:347–370, 2013. https://doi.org/10.1016/j.jfluidstructs.2013.02.007.

[19] Ronald A. Remmerswaal. *Numerical Modelling of Variability in Liquid Impacts*. PhD thesis, University of Groningen, 2023. https://doi.org/10.33612/diss.562133319.

[20] Y. Saad. *Iterative Methods for Sparse Linear Systems*. SIAM, Philadelphia, 2nd edition, 2003. https://doi.org/10.1137/1.9780898718003.

[21] Y. Saad and M. H. Schultz. GMRES: A generalized minimal residual algorithm for solving nonsymmetric linear systems. *SIAM Journal on Scientific and Statistical Computing*, 7:856–869, 1986. https://doi.org/10.1137/0907058.

[22] R. Scardovelli and S. Zaleski. Analytical relations connecting linear interfaces and volume fractions in rectangular grids. *Journal of Computational Physics*, 164:228–237, 2000. https://doi.org/10.1006/jcph.2000.6567.

[23] I. E. Sutherland and G. W. Hodgman. Reentrant polygon clipping. *Communications of the ACM*, 17:32–42, 1974. https://doi.org/10.1145/360767.360802.

[24] P. K. Sweby. High resolution schemes using flux limiters for hyperbolic conservation laws. *SIAM Journal on Numerical Analysis*, 21:995–1011, 1984. https://doi.org/10.1137/0721062.

[25] O. Tatebe. The multigrid preconditioned conjugate gradient method. In *Sixth Copper Mountain Conference on Multigrid Methods*, pages 621–634, 1993. https://ntrs.nasa.gov/citations/19940017009.

[26] J. Teunissen and U. Ebert. Afivo: A framework for quadtree/octree AMR with shared-memory parallelization and geometric multigrid methods. *Computer Physics Communications*, 233:156–166, 2018. https://doi.org/10.1016/j.cpc.2018.06.018.

[27] G. Tóth and P. L. Roe. Divergence- and curl-preserving prolongation and restriction formulas. *Journal of Computational Physics*, 180:736–750, 2002. https://doi.org/10.1006/jcph.2002.7120.

[28] P. H. A. J. M. van der Plas. *Local grid refinement for free-surface flow simulations*. PhD thesis, University of Groningen, 2017. https://research.rug.nl/en/publications/local-grid-refinement-for-free-surface-flow-simulations/.

[29] H. A. van der Vorst. Bi-CGSTAB: A fast and smoothly converging variant of Bi-CG for the solution of nonsymmetric linear systems. *SIAM Journal on Scientific and Statistical Computing*, 13:631–644, 1992. https://doi.org/10.1137/0913035.

[30] P. Vaněk, J. Mandel, and M. Brezina. Algebraic multigrid by smoothed aggregation for second and fourth order elliptic problems. *Computing*, 56:179–196, 1996. https://doi.org/10.1007/BF02238511.

[31] R. W. C. P. Verstappen and M. Dröge. A symmetry-preserving cartesian grid method for computing a viscous flow past a circular cylinder. *Comptes Rendus Mécanique*, 333:51–57, 2005. https://doi.org/10.1016/j.crme.2004.09.021.

[32] R. W. C. P. Verstappen and A. E. P. Veldman. Symmetry-preserving discretization of turbulent flow. *Journal of Computational Physics*, 187:343–368, 2003. https://doi.org/10.1016/S0021-9991(03)00126-8.

[33] G. D. Weymouth and D. K.-P. Yue. Conservative volume-of-fluid method for free-surface simulations on cartesian-grids. *Journal of Computational Physics*, 229:2853–2865, 2010. https://doi.org/10.1016/j.jcp.2009.12.018.

[34] X. Zeng. A general approach to enhance slope limiters in MUSCL schemes on nonuniform rectilinear grids. *SIAM Journal on Scientific Computing*, 38:A789–A813, 2016. https://doi.org/10.1137/140970185.